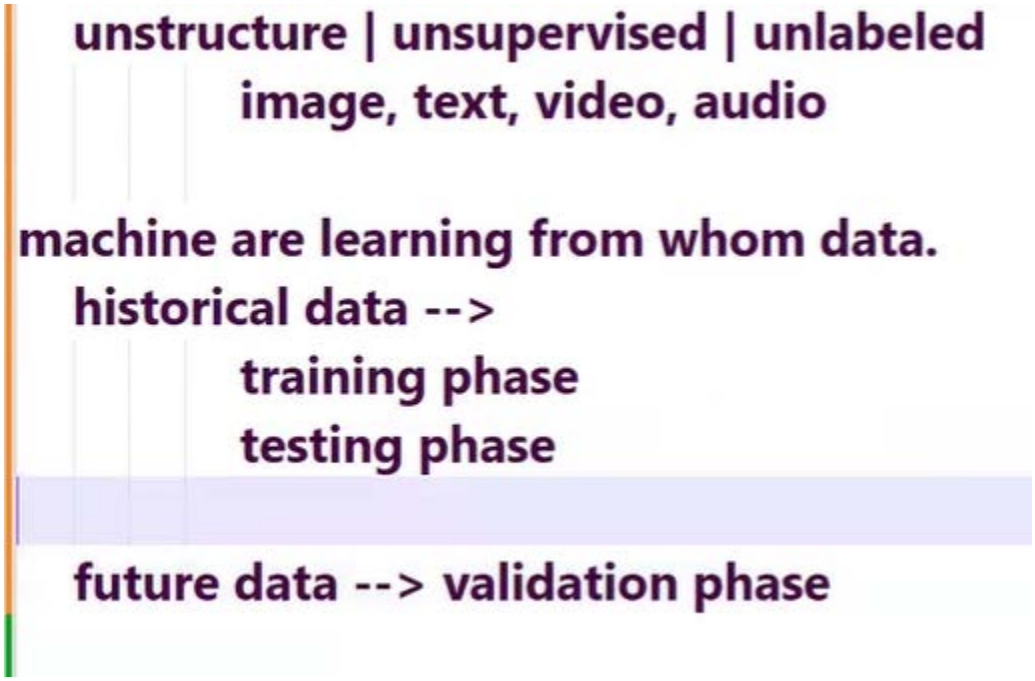
Machine + Learning

Machines are learning from whom from the data that data we called as historical data or future data

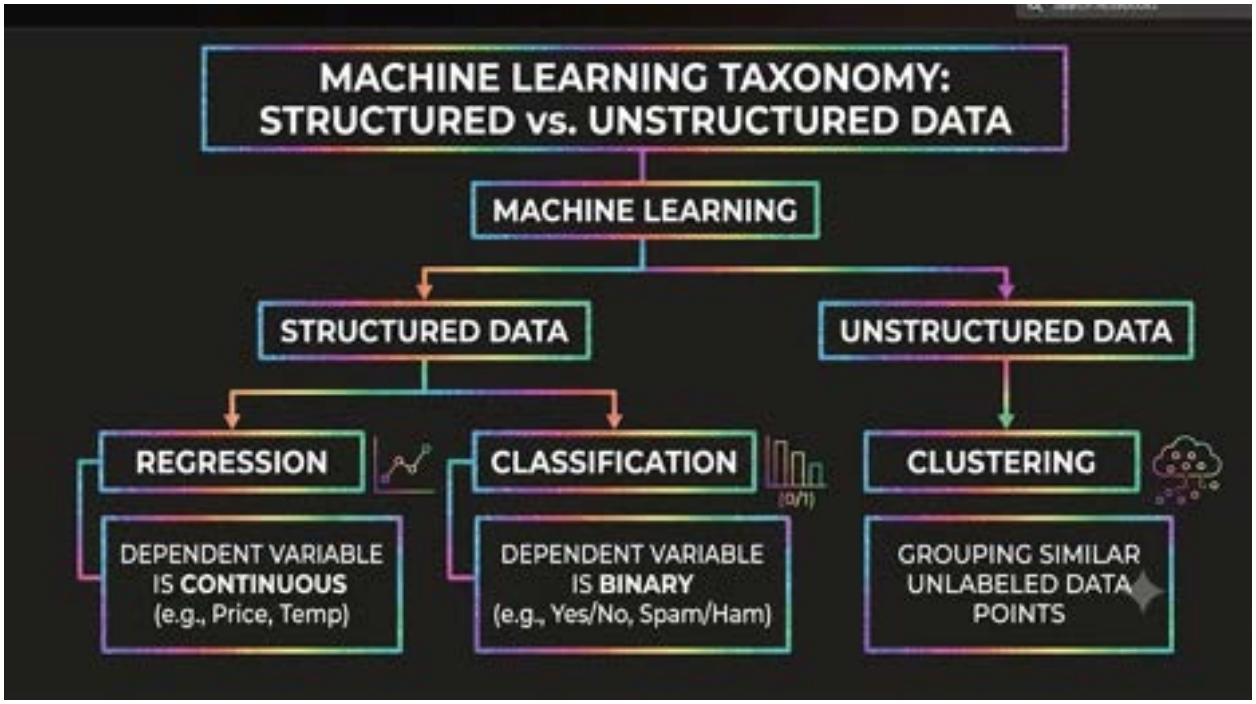
Data is 2 types
1.Structured | supervised | labelled ========Structured means Excel file,csv file
The structured data divided into **regression model** and **classification model**
2.Unstructured | unsupervised | unlabelled ===unstructured means image,video,text,audio

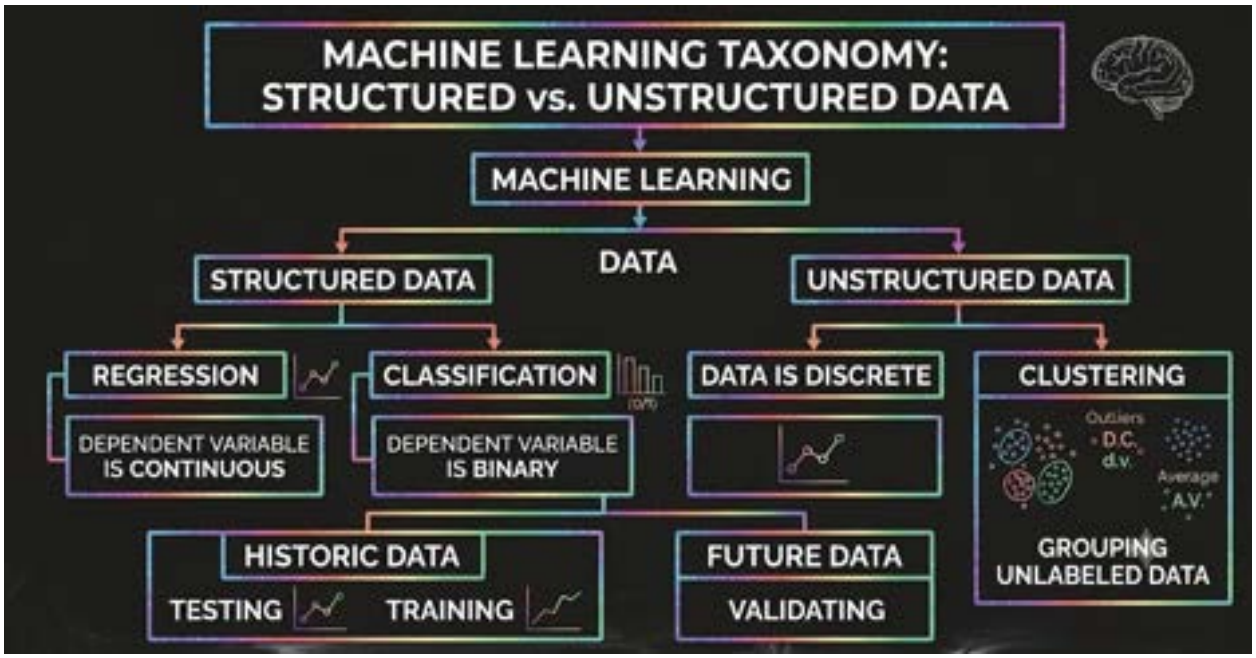
Data is divided into 2 parts
1.Historical data
2.Future data

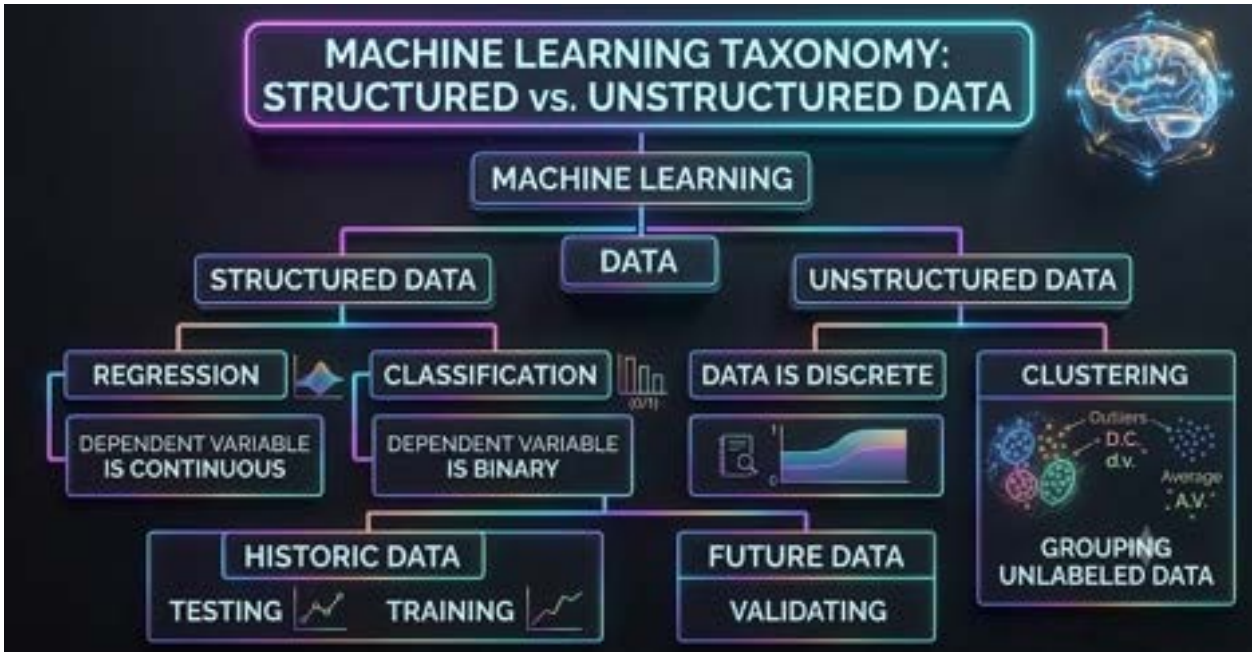
Historic data again divided in to 2 parts 1.training phase 2.testing phase
Future data is called validation phase



The machines are learned from data and the data is fed from me or you or developers .When we fed the data we write the algorithm


MACHINE LEARNING TAXONOMY:
STRUCTURED vs. UNSTRUCTURED DATA
MACHINE LEARNING
STRUCTURED DATA
UNSTRUCTURED DATA
REGRESSION
CLASSIFICATION
CLUSTERING
DEPENDENT VARIABLE IS CONTINUOUS (e.g., Price, Temp)
DEPENDENT VARIABLE IS BINARY (e.g., Yes/No, Spam/Ham)
GROUPING SIMILAR UNLABELED DATA POINTS


MACHINE LEARNING TAXONOMY:
STRUCTURED vs. UNSTRUCTURED DATA
MACHINE LEARNING
DATA
STRUCTURED DATA
UNSTRUCTURED DATA
REGRESSION
CLASSIFICATION
DATA IS DISCRETE
CLUSTERING
DEPENDENT VARIABLE IS CONTINUOUS
DEPENDENT VARIABLE IS BINARY
Outliers
D.C.
d.v.
Average
A.V.
HISTORIC DATA
FUTURE DATA
TESTING
TRAINING
VALIDATING
GROUPING UNLABELED DATA



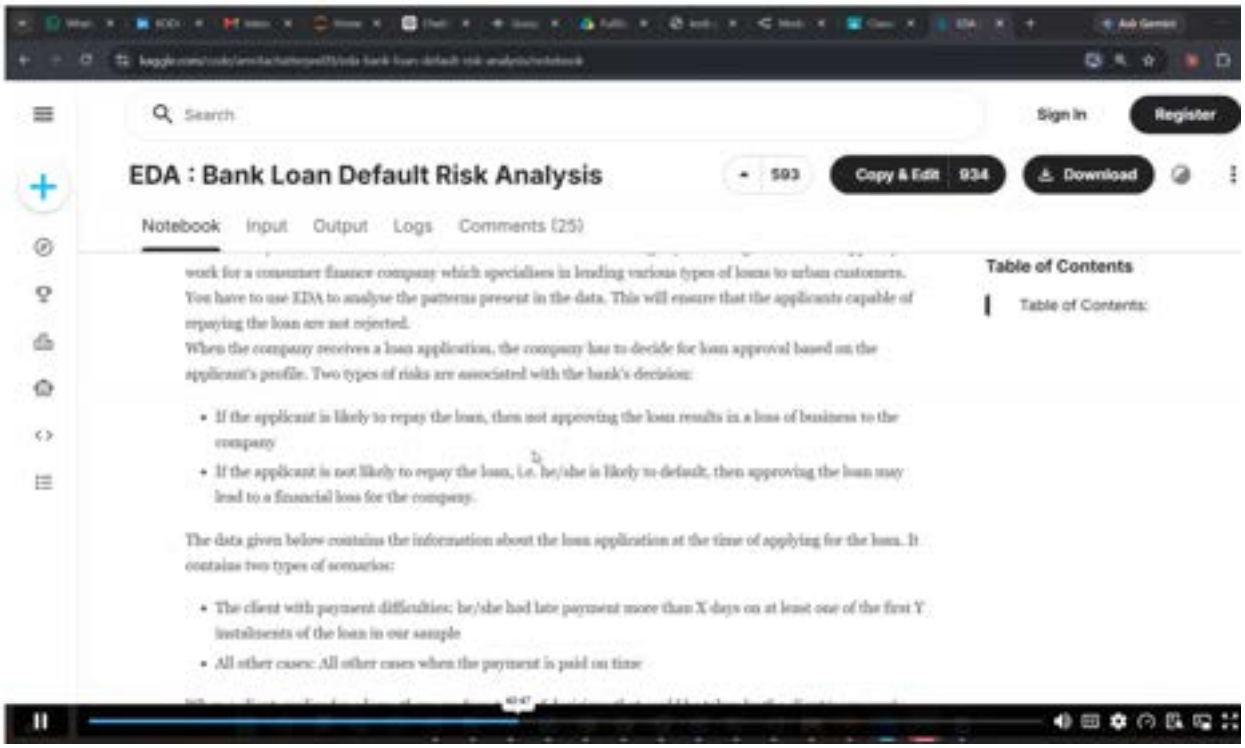
In tenth board exam we read question and answer similarly we read question and we read answer we study input data and we study output data when we read question and answer a model is built in the brain when we write a exam if we train brain well our accuracy is 90%

WHAT IS MEANT BY ML MODEL:

2(Input) + (Logic) 3(Input) = 5 (output) generated this is traditional learning

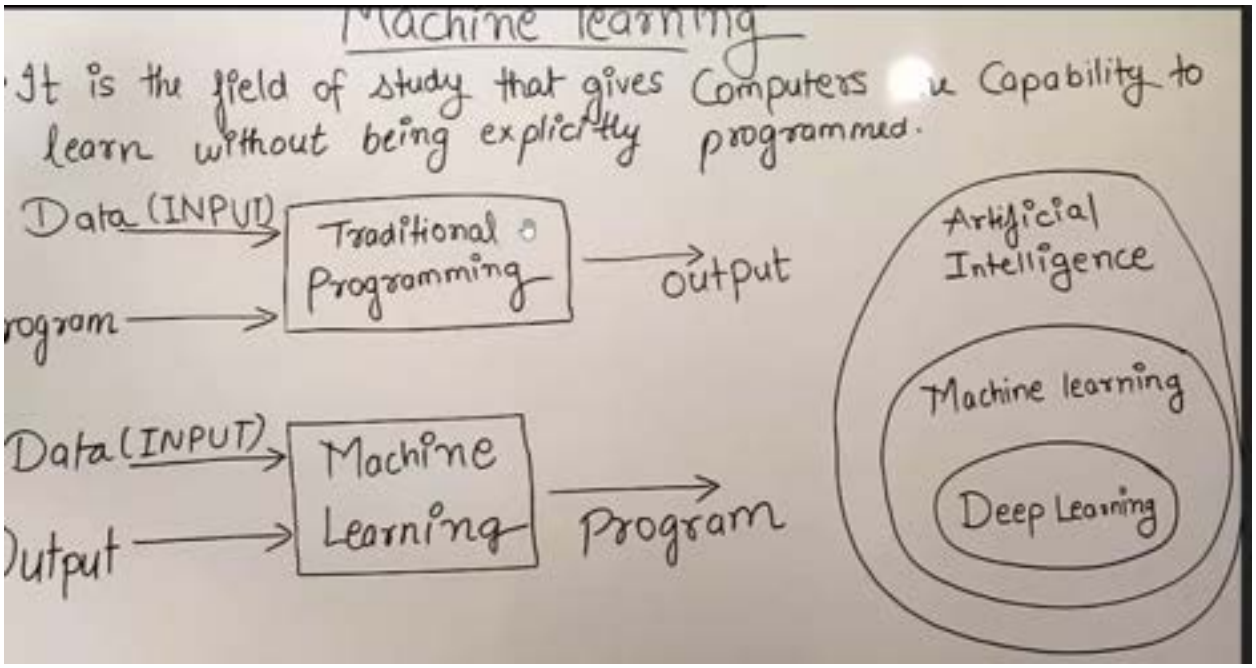
In ML we feed input 2 and 3 as well as we feed output 5 the model will generate the logic this is called ML model using ML model we can predict if we deploy this model and user enter 40 and 41 and ML predict 78 then it is 90 percent accuracy if model predict 81 then it is 100 percent accuracy

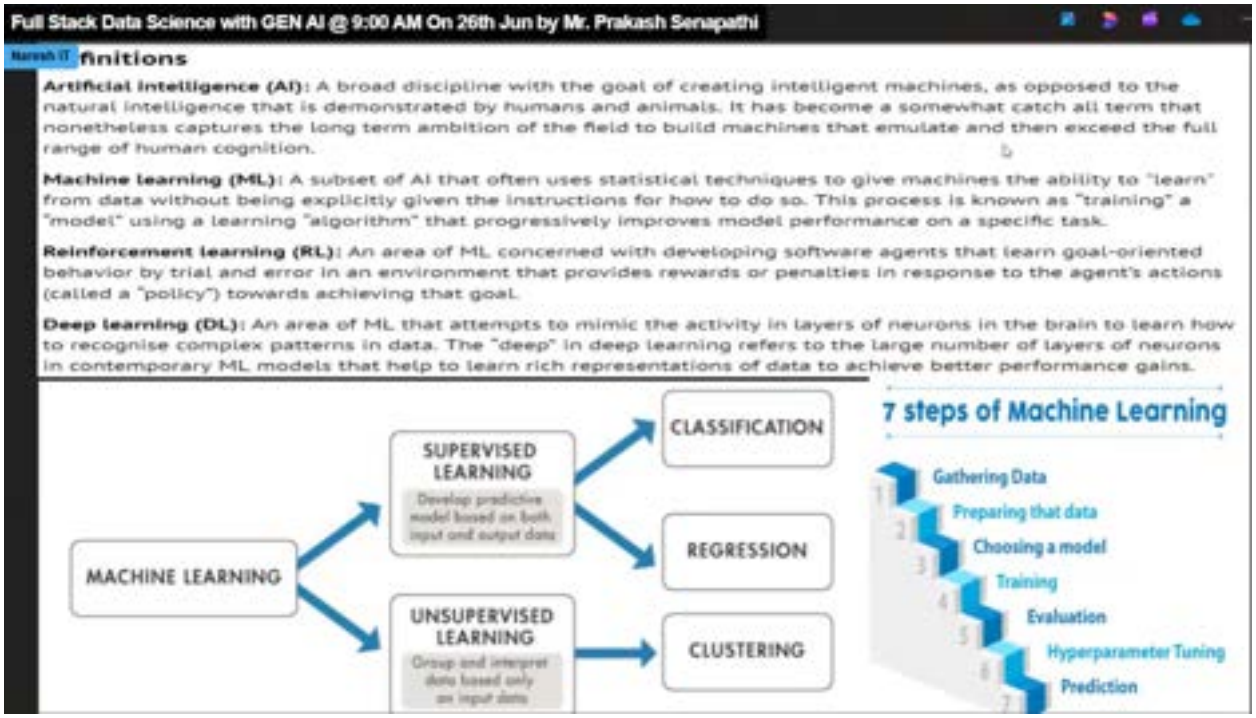
Model is nothing but a logic build by algorithm that algorithm is called regression algorithm or classification algorithm



Bank loan analysis important project

To build ML model we have input data and and output data model will generate





2(Input) + (Logic) 3(Input) = 5 (output) generated this is traditional learning

In ML we feed input 2 and 3 as well as we feed output 5 the model will generate the logic this is called ML model using ML model we can predict if we deploy this model and user enter 40 and 41 and ML predict 78 then it is 90 percent accuracy if model predict 81 then it is 100 percent accuracy

Model is nothing but a logic build by algorithm that algorithm is called regression algorithm or classification algorithm

సార్ బోర్డు మీద రాసిన గణితం **(Maths)** ఉదాహరణ ని చాలా సులభంగా అర్థం చేసుకుందాం:

1. Traditional Programming (సాంప్రదాయ పద్ధతి)

ఇక్కడ సార్ బోర్డు మీద రాసిన అంకెలు మరియు ప్లస్ గుర్తుని చూడండి:

- **Data (INPUT):** 2, 3 **(**అంకెలు**)**
-
- **Program (**మనం ఇచ్చే రూల్/లాజిక్**):** + **(**కూడిక చెయ్యి అని లాజిక్ రాయడం**)**
-
- **Traditional Programming Box:** మనం ఇచ్చిన అంకెలను **(**2, 3**)** అంకగణిత సూత్రం **(**+**)** ప్రకారం లెక్కబెడుతుంది.

-
- **Output:** 5 **(ఫలితం వస్తుంది)**
-

సారాంశం: కంప్యూటర్‌కి అంకెలు **(**$2, 3$**)** మరియు కూడాలి **(**$+$**)** అనే నియమం మనం చెప్తే, అది లెక్కించి ఆన్సర్ **(**5**)** ఇస్తుంది.

2. Machine Learning (మెషిన్ లెర్నింగ్ పద్ధతి)

ఇప్పుడు కింద రాసిన ఉదాహరణ చూడండి:

- **Data (INPUT):** $2, 3$ **(అంకెలు)**
-
- **Output:** 5 **(ఆన్సర్)**
-
- **Machine Learning Box:** కంప్యూటర్ $2, 3$ మరియు 5 ని చూసి ఆలోచిస్తుంది. 2 ని 3 ని ఏం చేస్తే 5 వస్తుంది? $2 + 3 = 5$ అవుతుంది కదా అని గ్రహిస్తుంది.
-
- **Program (**నేర్చుకున్న రూల్**):** $+$ **(**కూడిక అనే రూల్‌ని కంప్యూటర్ స్వయంగా కనుగొంటుంది**)**
-

సారాంశం: కంప్యూటర్‌కి ఇన్‌పుట్ **(**$2, 3$**)** మరియు ఆన్సర్ **(**5**)** ఇచ్చేస్తే, రెండింటి మధ్య ఉన్న లాజిక్ కూడిక **(**$+$**)** అని మిషన్ తనంతట తానే కనుక్కుంటుంది.

ఒక్క ముక్కలో:

- **Traditional: Data (**$2, 3$**) + Logic (**$+$**) → Output (**5**)**
-
-
- **Machine Learning: Data (**$2, 3$**) + Output (**5**) → Logic/Program (**$+$**)**
-

Traditional Programming మరియు **Machine Learning** కి ఉన్న తేడాను ఒక పిల్లవాడికి అర్థమయ్యేలా చాలా చిన్న ఉదాహరణతో చూద్దాం:

ఉదాహరణ: ఆపిల్ **(Apple)** ని గుర్తించడం 🍎

1. Traditional Programming (సాంప్రదాయ పద్ధతి**)**

నువ్వు ఒక రోబోకి ఆపిల్ అంటే ఏంటో చెప్పాలి అనుకున్నావ్. నువ్వే అన్ని నిబంధనలు **(Rules)** రాస్తావ్:

- **Data (**ఇన్‌పుట్**):** పండు రంగు మరియు ఆకారం.
-
- **Program (**మనం ఇచ్చే రూల్స్**):**

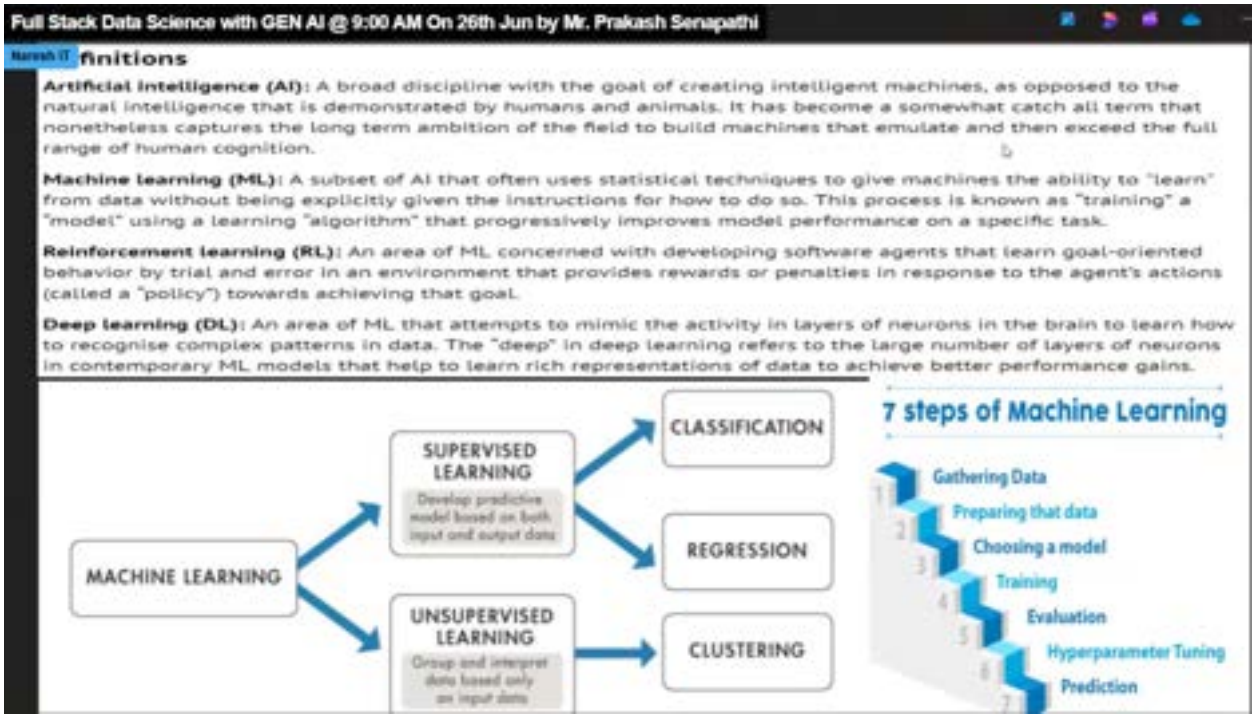
-
- *"పండు ఎర్రగా ఉండాలి, గుండ్రంగా ఉండాలి. అలా ఉంటేనే అది ఆపిల్!"*
-
- **Output (ఫలితం):** రోబో పండును చూసి "ఇది ఆపిల్" అని చెప్తుంది.
-

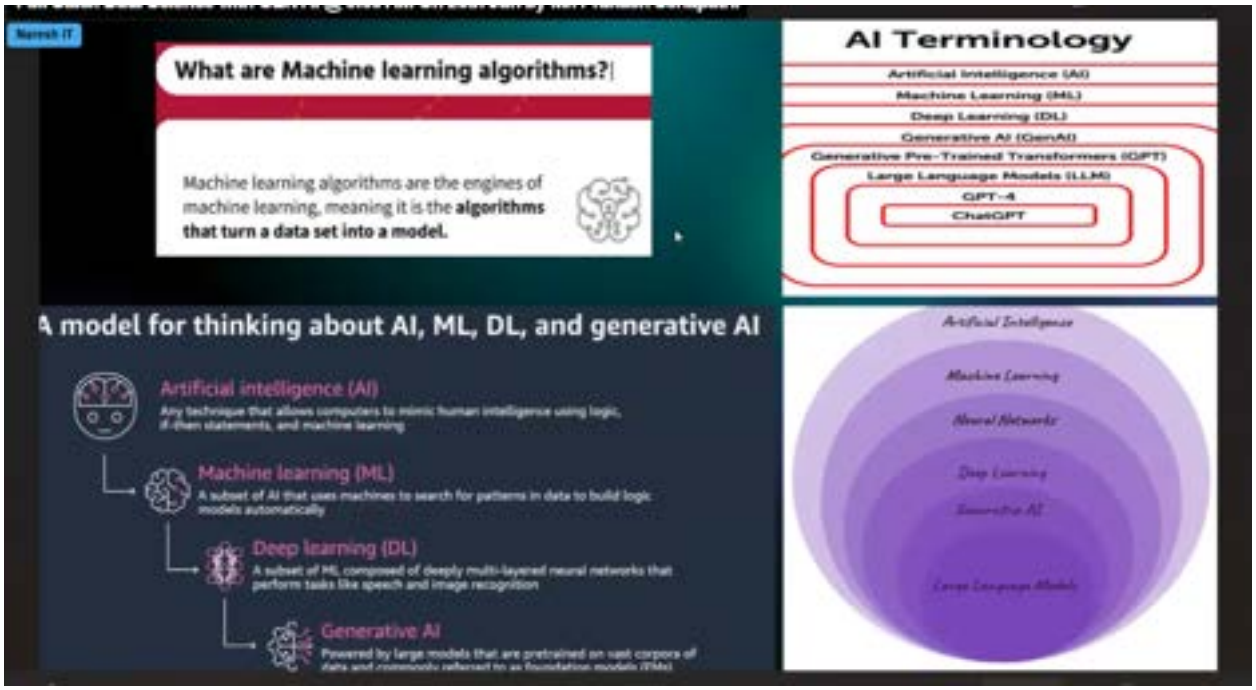
⚠️ సమస్య: ఒకవేళ రోబో ముందు పచ్చటి ఆపిల్ **(Green Apple)** పెడితే, రోబో *"ఇది ఎరుపు రంగులో లేదు కాబట్టి ఆపిల్ కాదు"* అని తప్పుగా చెప్తుంది. ఎందుకంటే మనం ఇచ్చిన రూల్‌లో పచ్చ ఆపిల్ గురించి లేదు!

2. Machine Learning (స్వయంగా నేర్చుకునే పద్ధతి**)**
ఇక్కడ నువ్వు రోబోకి ఎటువంటి రూల్స్ చెప్పవు.

- **Data + Output (**ఇన్‌పుట్**):** నువ్వు రోబోకి **100** ఎర్రటి ఆపిల్స్, **100** పచ్చటి ఆపిల్స్ చూపించి, *"ఇవన్నీ ఆపిల్స్"* అని చెప్తావ్.
-
- **Machine Learning Program (**స్వయం అభ్యాసం**):** రోబో ఆ ఫోటోలన్నీ చూసి, స్వయంగా తన మెదడులో ఓ ఐడియా తెచ్చుకుంటుంది:
-
- *"ఓహో! ఆపిల్స్ ఎర్రగానే కాదు, పచ్చగా కూడా ఉంటాయి. వీటికి పైన ఒక చిన్న తొడెం కూడా ఉంటుంది!"*
-
- ఫలితం: రేపు నువ్వు సరికొత్త రకం ఆపిల్ చూపించినా, రోబో వెంటనే *"ఇది ఆపిల్!"* అని కరెక్ట్‌గా గుర్తించేస్తుంది.
-

ఒక్క ముక్కలో చెప్పాలంటే:
- **Traditional Programming:** అమ్మ వంట తయారు చేయడానికి నీకు రెసిపీ **(Recipe Rules)** ఇస్తుంది. నువ్వు అది చూసి వంట చేస్తావ్.
-
- **Machine Learning:** నువ్వు రోజూ అమ్మ చేసే వంటను, రుచిని **(Data + Output)** గమనించి, నీ అంతట నువ్వే వంట చేయడం నేర్చుకుంటావ్!



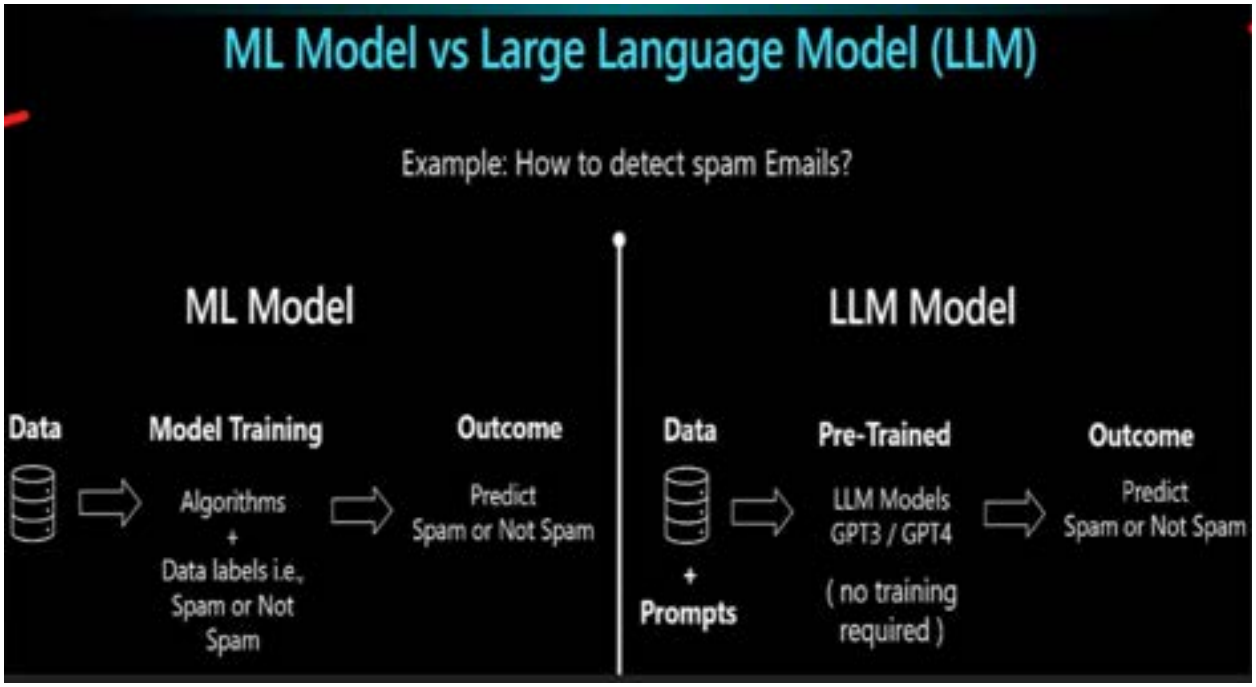


Q)****What is machine learning algorithm**
A)Machine algorithm is the engine of machine learning meaning it is the algorithm that turns a data set into a model.

Q)Can you tell the algorithm to come to home from office

For this the algorithm is first we go for the bus stop we can go by bus or by walk

Algorithm is nothing but step by step approach entire Machine learning is a math



1-dataset (excel sheet)
2- attribute (columns)
3- indepden x & y
x -- indedependent variable
y -- dependent variable

4- x_train & x_test
5- y_train & y_test

First we will get the excel from client or from the DB which contains the attributes(attributes) this attributes contains independent variable and dependent variable

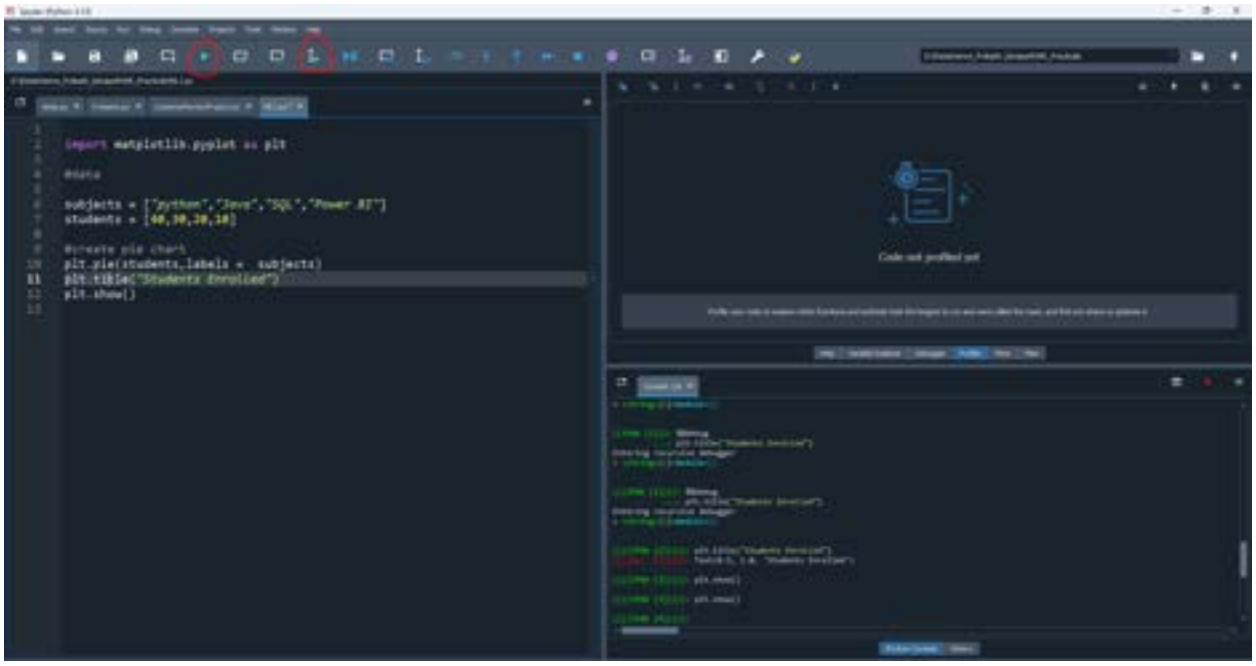
This X is divided into x train and x test and the y divided in to y train and y test

Data preprocessing pipeline

https://learn.nareshit.com/courses/take/new-course-1040/multimedia/76290162-session-50-29th-jun

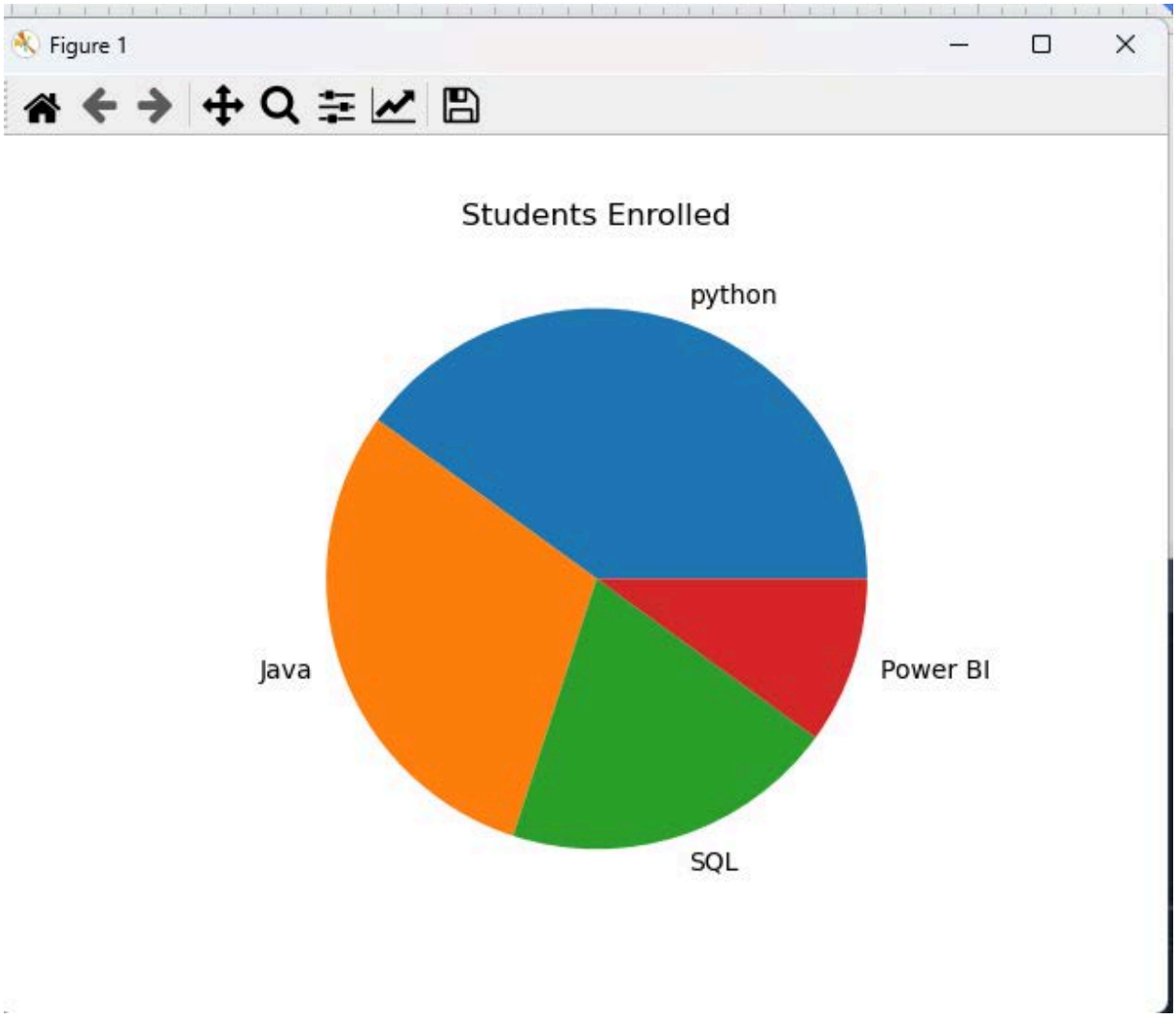
Sir said he will teach how to deploy in AWS,GCP and cloud

First ML program



I run symbol used to execute line by line
Normal run button execute all code at a time

Output :



Second ML program:

A	B	C	D
State	Age	Salary	Purchased
Mumbai	44	72000	No
Bangalore	27	48000	Yes
Hyderabad	30	54000	No
Bangalore	38	61000	No
Hyderabad	40		Yes
Mumbai	35	58000	Yes
Bangalore		52000	No
Mumbai	48	79000	Yes
Hyderabad	50	83000	No
Mumbai	37	67000	Yes

There is missing data

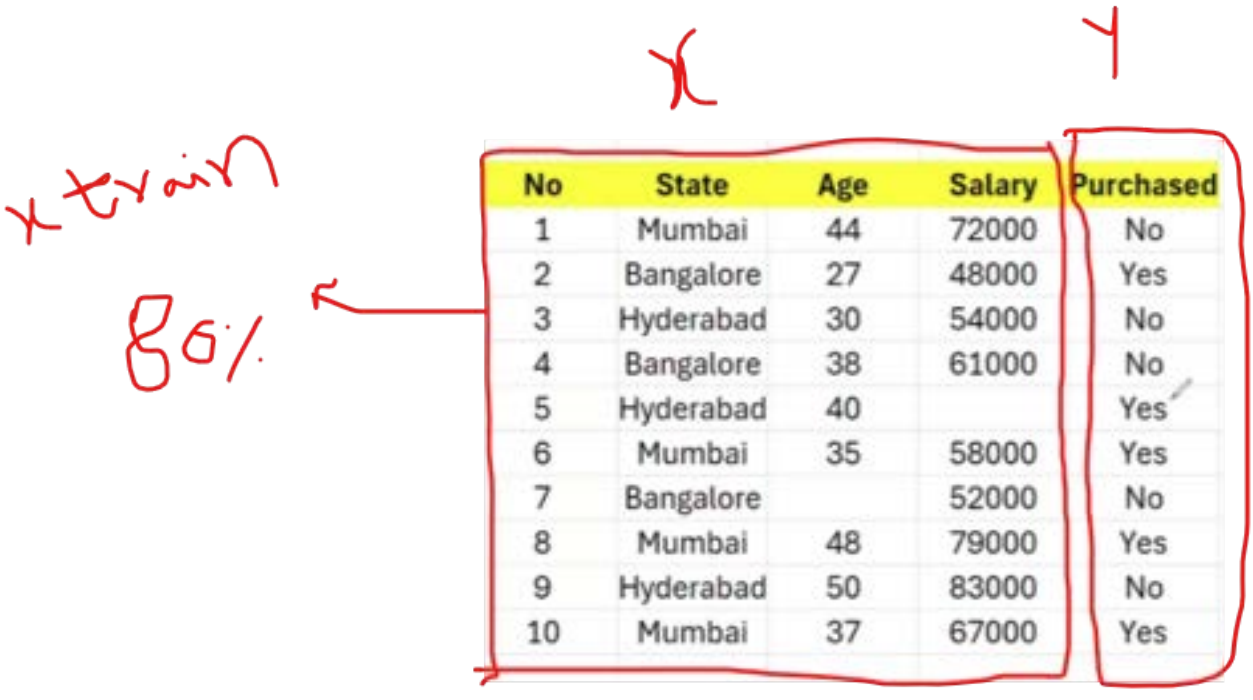
No	State	Age	Salary	Purchased
1	Mumbai	44	72000	No
2	Bangalore	27	48000	Yes
3	Hyderabad	30	54000	No
4	Bangalore	38	61000	No
5	Hyderabad	40		Yes
6	Mumbai	35	58000	Yes
7	Bangalore		52000	No
8	Mumbai	48	79000	Yes
9	Hyderabad	50	83000	No
10	Mumbai	37	67000	Yes

In this there is a dependent variable that is purchase(Y) and it is yes or No so it comes under classification and all other remaining columns are independent variables that come under X.



No	State	Age	Salary	Purchased
1	Mumbai	44	72000	No
2	Bangalore	27	48000	Yes
3	Hyderabad	30	54000	No
4	Bangalore	38	61000	No
5	Hyderabad	40		Yes
6	Mumbai	35	58000	Yes
7	Bangalore		52000	No
8	Mumbai	48	79000	Yes
9	Hyderabad	50	83000	No
10	Mumbai	37	67000	Yes

From the above pic we get to know the data is divided in to x and y



No	State	Age	Salary	Purchased
1	Mumbai	44	72000	No
2	Bangalore	27	48000	Yes
3	Hyderabad	30	54000	No
4	Bangalore	38	61000	No
5	Hyderabad	40		Yes
6	Mumbai	35	58000	Yes
7	Bangalore		52000	No
8	Mumbai	48	79000	Yes
9	Hyderabad	50	83000	No
10	Mumbai	37	67000	Yes

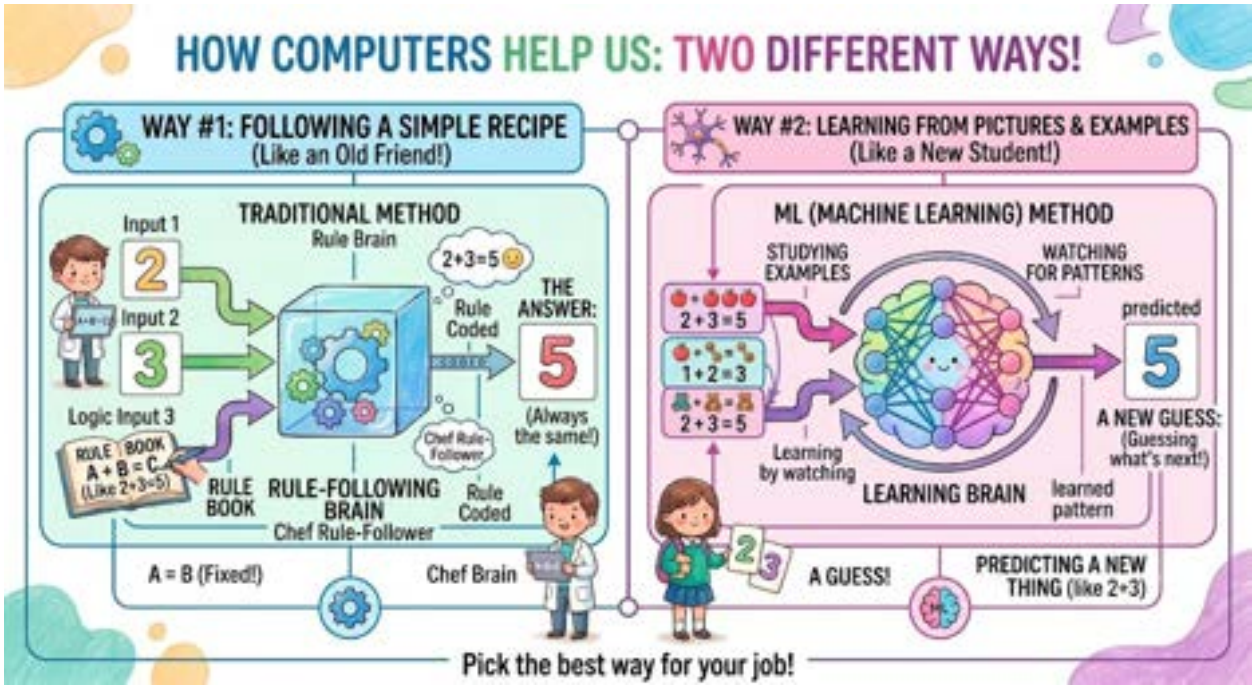
80% records are in to X train



	A	B	F	G	H	I	J
1	No.	State	Age	Salary	Purchased		
2	1	Mumbai	44	72000	No		
2	2	Bangalore	27	48000	Yes		
3	3	Hyderabad	30	54000	No		
4	4	Bangalore	38	61000	No		
5	5	Hyderabad	40	NaN	Yes		
6	6	Mumbai	35	58000	Yes		
7	7	Bangalore	27	52000	No		
8	8	Mumbai	48	79000	Yes		
9	9	Hyderabad	50	83000	No		
10	10	Mumbai	37	67000	Yes		

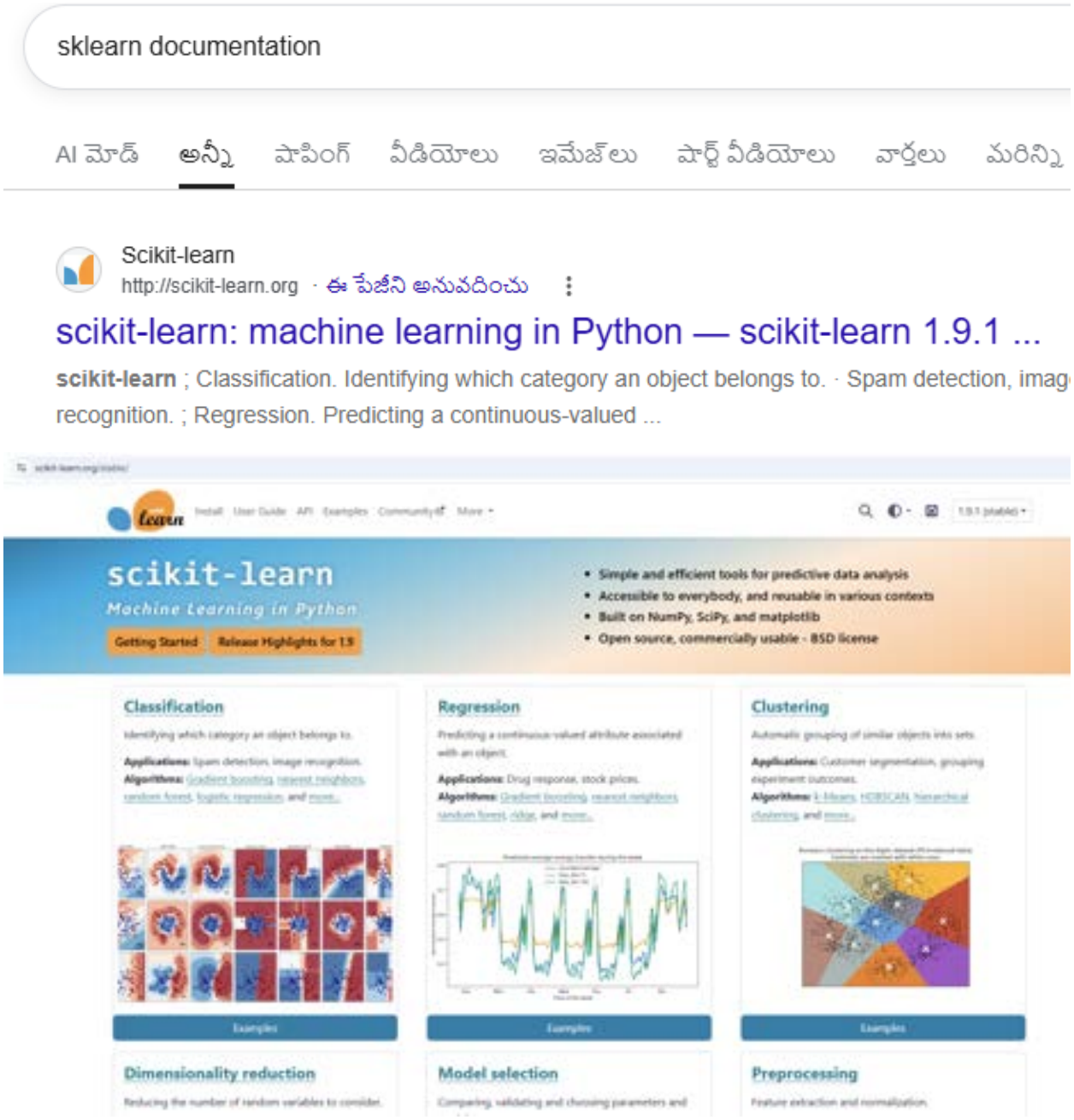


	A	B	F	G	H	I	J
1	No.	State	Age	Salary	Purchased		
2	1	Mumbai	44	72000	No		
2	2	Bangalore	27	48000	Yes		
3	3	Hyderabad	30	54000	No		
4	4	Bangalore	38	61000	No		
5	5	Hyderabad	40	NaN	Yes		
6	6	Mumbai	35	58000	Yes		
7	7	Bangalore	27	52000	No		
8	8	Mumbai	48	79000	Yes		
9	9	Hyderabad	50	83000	No		
10	10	Mumbai	37	67000	Yes		

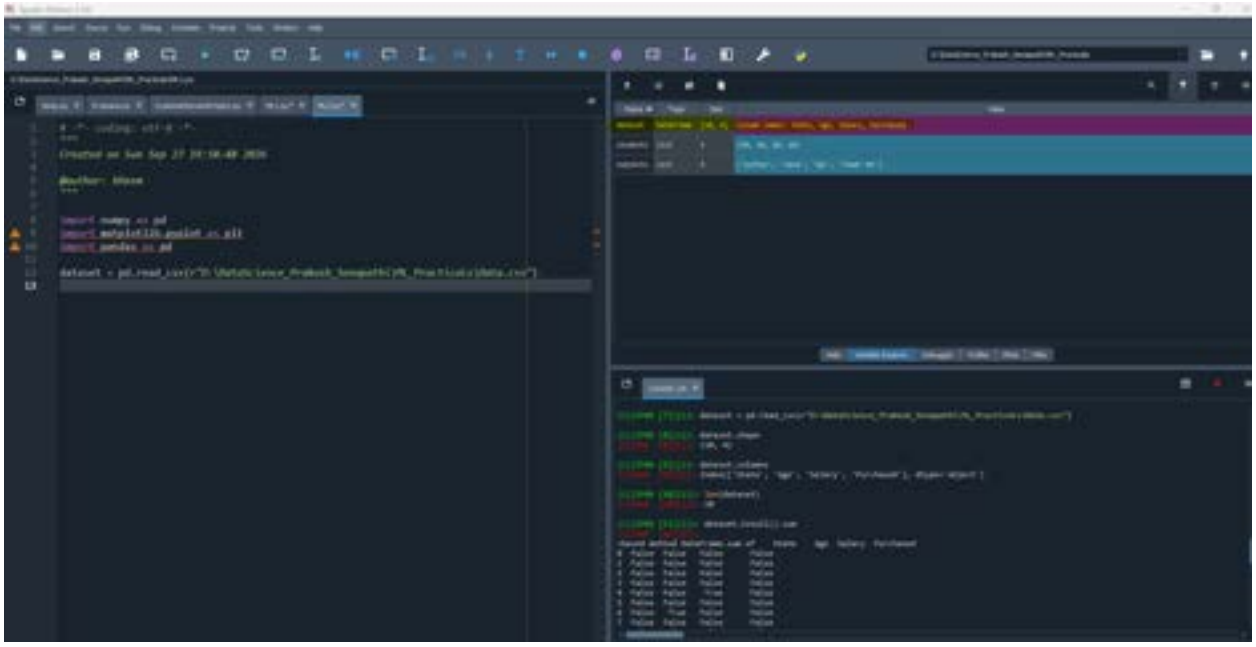


In traditional method we will follow the rule like 2+3 = output generated where as in the ML input(2),Input(2) output also we give 5 so inside it understand there are +

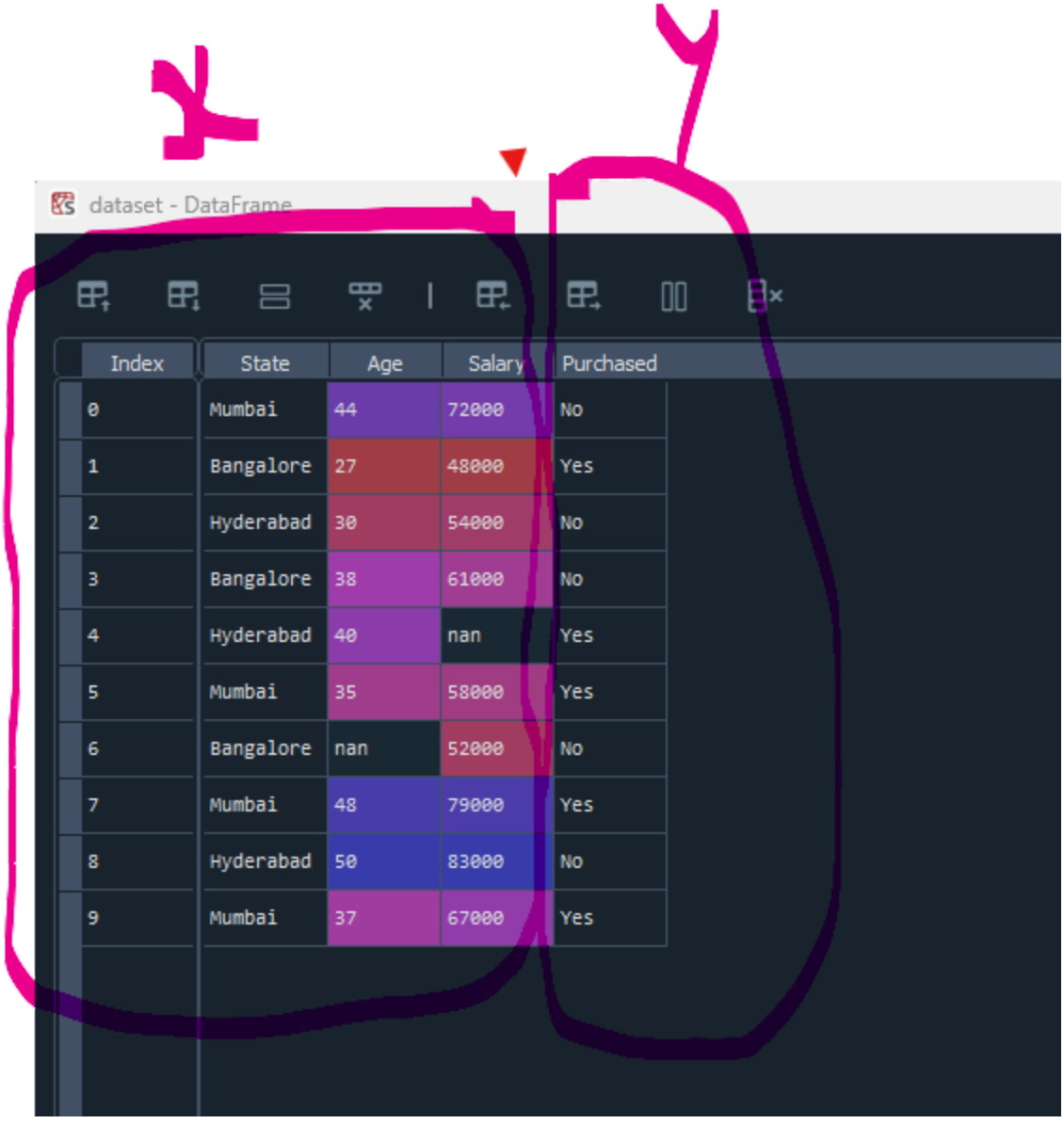
There is a framework to build that logic in the ML that is called scikit learn framework



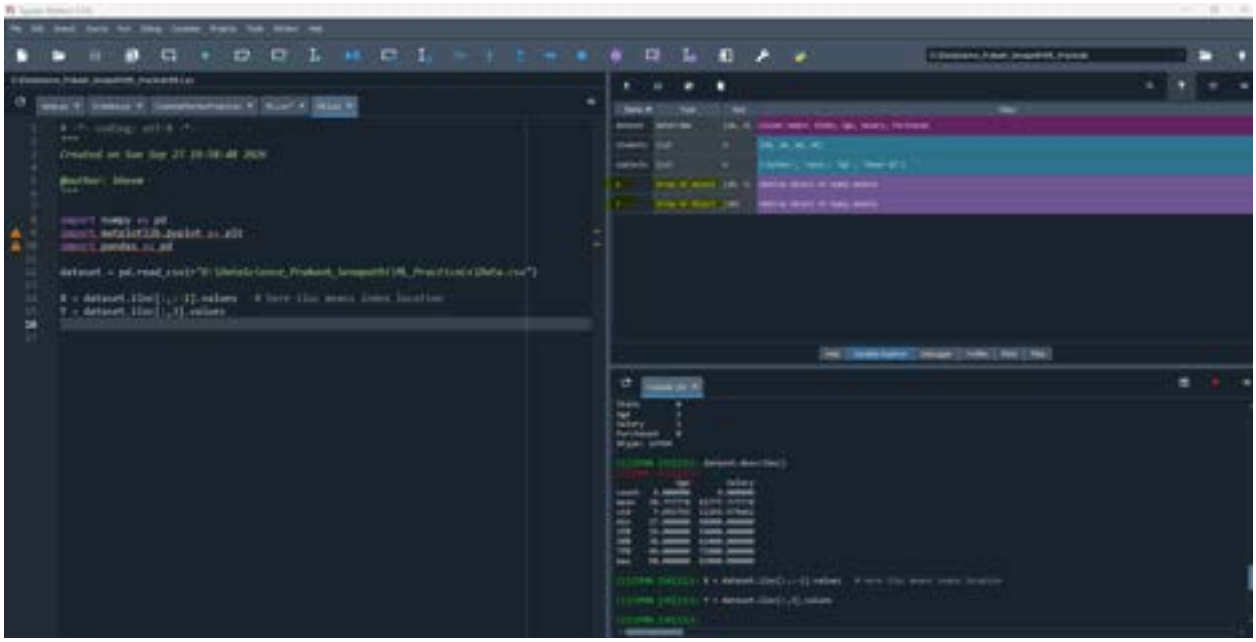
[ML2.py](ML2.py) **practical**



Up to this we are reading the data from excel after this we need to divide the data as X and Y



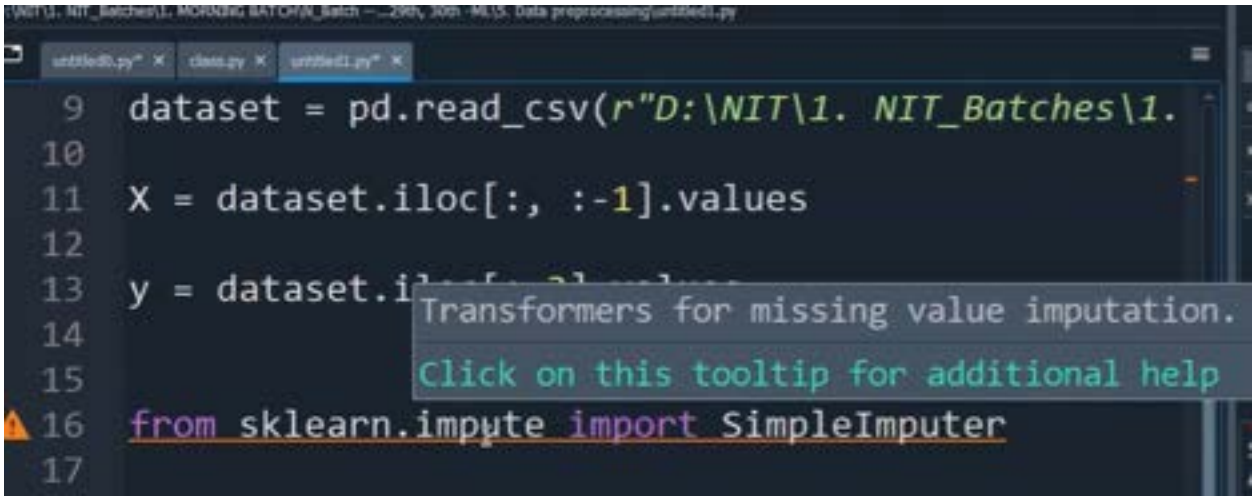
Index	State	Age	Salary	Purchased
0	Mumbai	44	72000	No
1	Bangalore	27	48000	Yes
2	Hyderabad	30	54000	No
3	Bangalore	38	61000	No
4	Hyderabad	40	nan	Yes
5	Mumbai	35	58000	Yes
6	Bangalore	nan	52000	No
7	Mumbai	48	79000	Yes
8	Hyderabad	50	83000	No
9	Mumbai	37	67000	Yes



The age and salary have the missing vales we need to fill it by below procedure

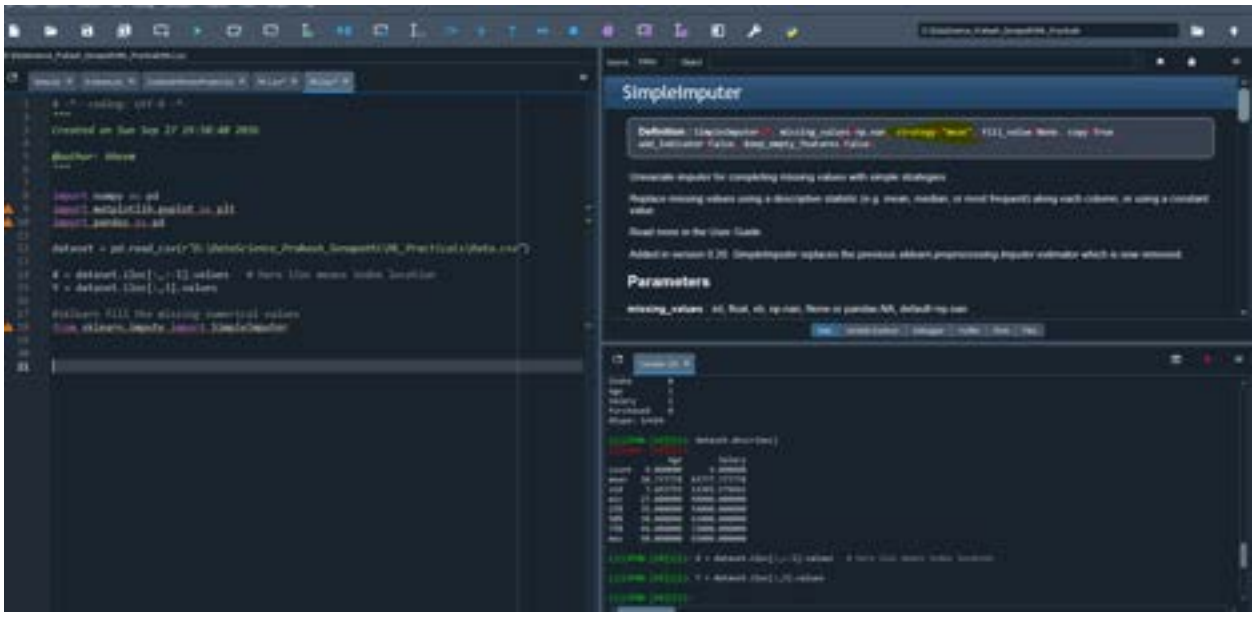
	0	1	2
0	Mumbai	44.0	72000.0
1	Bangalore	27.0	48000.0
2	Hyderabad	30.0	54000.0
3	Bangalore	38.0	61000.0
4	Hyderabad	40.0	nan
5	Mumbai	35.0	58000.0
6	Bangalore	nan	52000.0
7	Mumbai	48.0	79000.0
8	Hyderabad	50.0	83000.0
9	Mumbai	37.0	67000.0

Note : In EDA we have filled the missing values by applying the MEAN strategy in the same way we need to using the sikitlearn library we can fill it



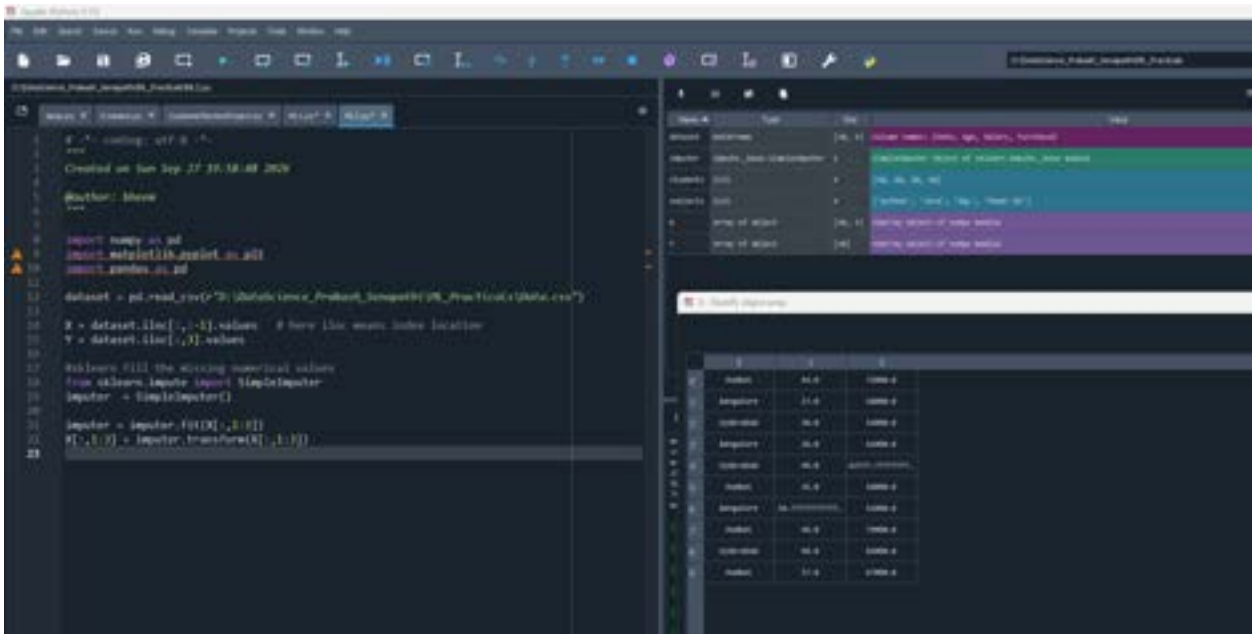
Q)what is machine learning transformer

A) sklearn.impute



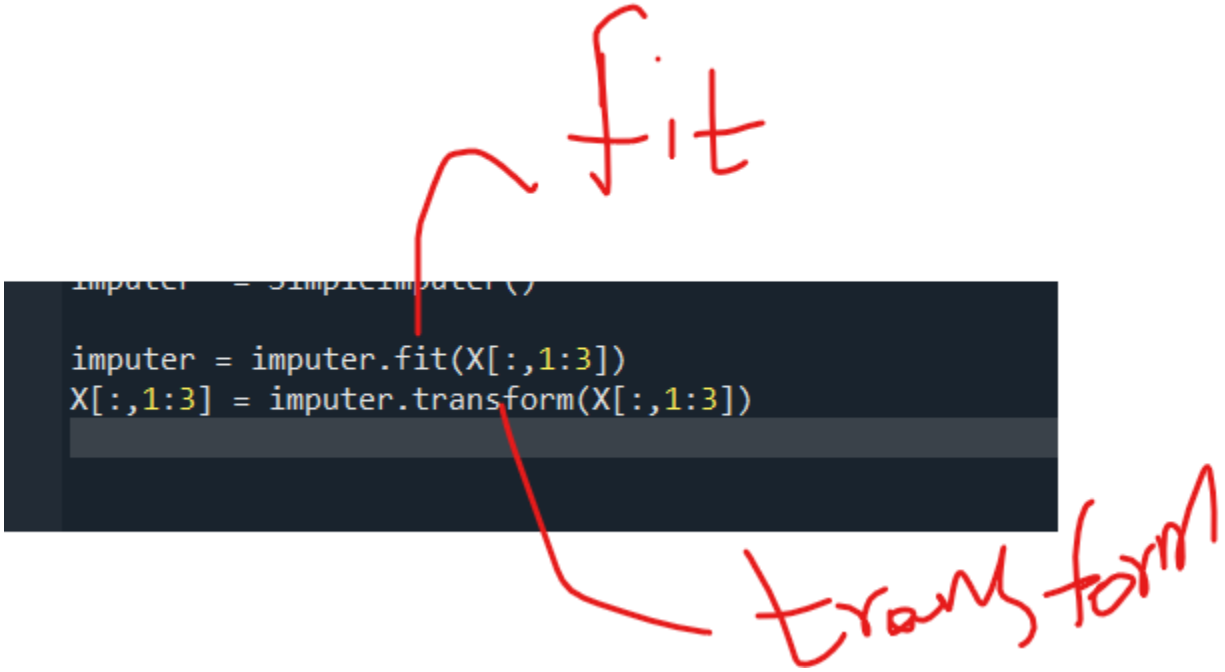
Where ever missing value is there it is applying mean strategy

Simple imputer is another module which is used to fill the missing value



Filled the missing value

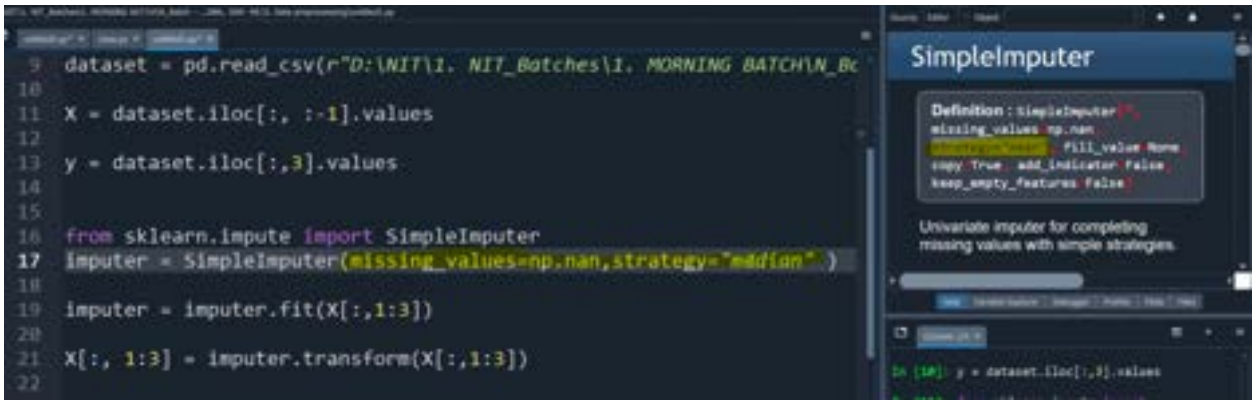
Bydefault system will consider the mean strategy



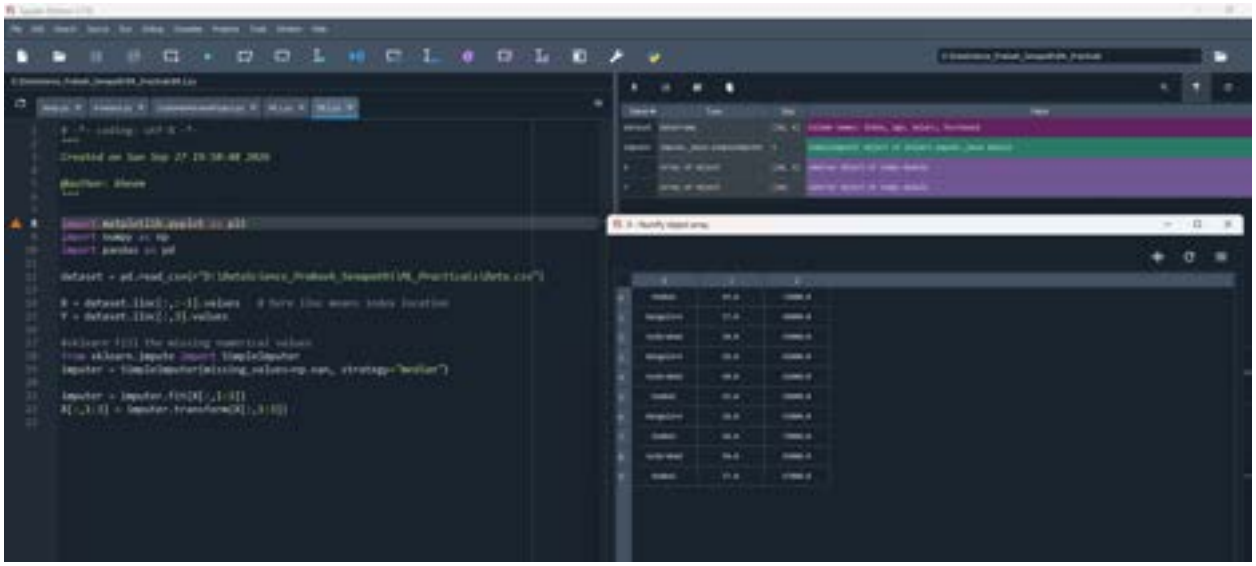
What the fit and transform method will do means the every 6 months we give our vehicle to transform in good condition in the same way the fit and transform will do x part in to clean form this is called parameter tuning

Q)what is parameter tuning
A)In parameter tuning we are not following any own parameter we are following system parameter tuning

If we want to apply median strategy then we will take hepl option and copy the parameter and pass as like below



After applying the median strategy the values changed we can observe



Parameter_tuning means system given own parameter is called system parameter
Hyper Parameter means we change the system parameter (Like from mean to median) is nothing but hyper parameter tuning.
From median to again we changed to mode by below code

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy='most_frequent')

imputer = imputer.fit(X[:,1:3])

X[:, 1:3] = imputer.transform(X[:,1:3])
```

By default system given the mean —>Parameter tuning but we changed from mean to median or mode this is nothing but hyper parameter tuning

Note : When the values are changed the accuracy will fluctuate it means for mean we have around 633777 for median 61000 and for for mode again different value by default system will considered the mean but for the mean if we get very less accuracy then we can go for median or mode depends on our requirement



Step1: Data comes from company
Step2: Cleaning the data
Step3 : Features means below code

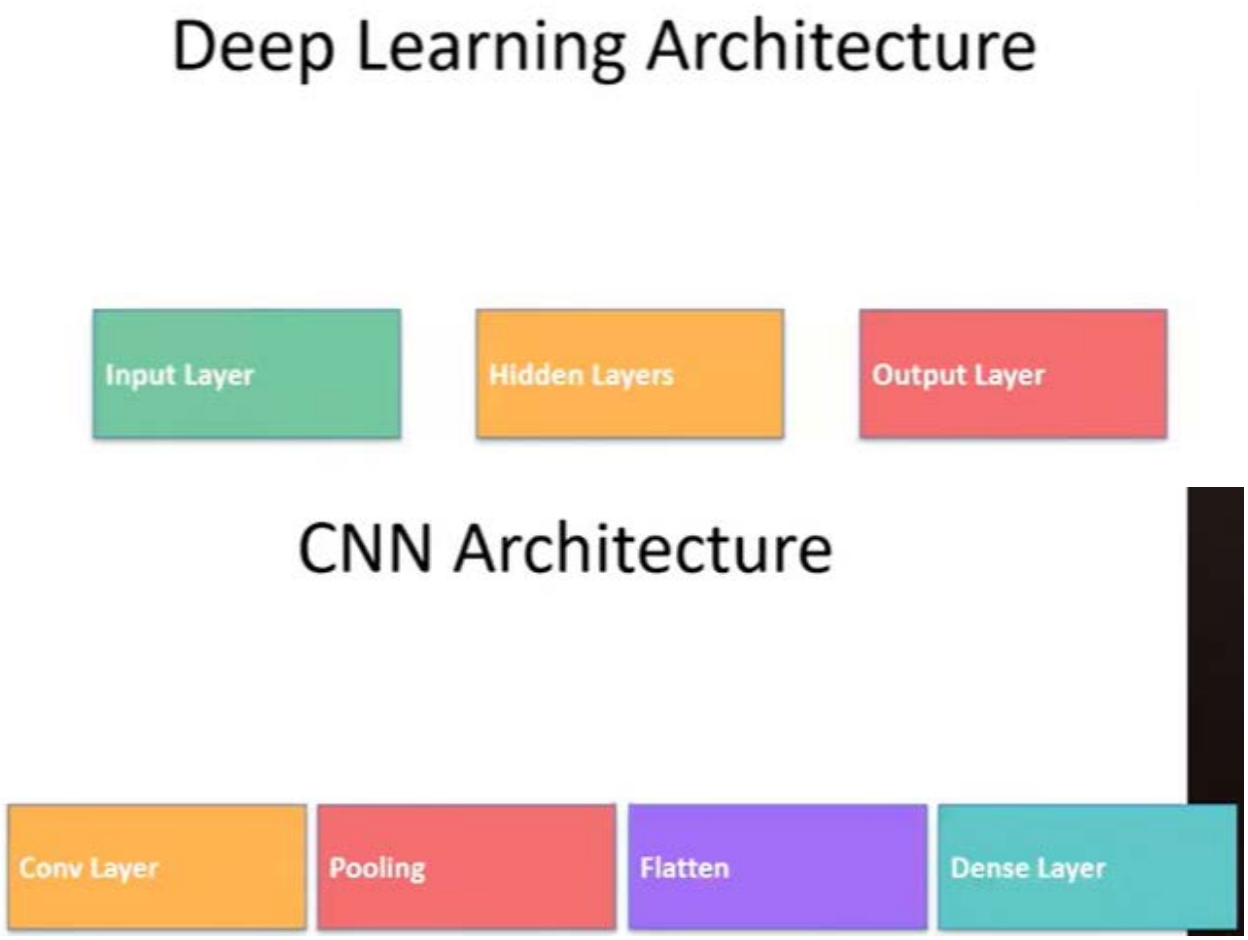
```python
#sklearn fill the missing numerical values
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(missing_values=np.nan, strategy="median")

imputer = imputer.fit(X[:,1:3])
X[:,1:3] = imputer.transform(X[:,1:3])


#sklearn fill the missing wih mode strategy
from sklearn.impute import SimpleImputer
imputer = SimpleImputer(missing_values=np.nan, strategy="most_frequent")

imputer = imputer.fit(X[:,1:3])
X[:,1:3] = imputer.transform(X[:,1:3])
```

Step4: Training phase
Step5: Evaluation phase
Step6: Deployment



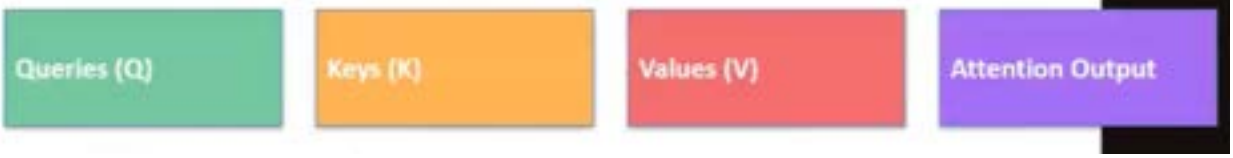
RNN / LSTM Flow



Transformer Architecture



Self-Attention Mechanism



LLM Pipeline



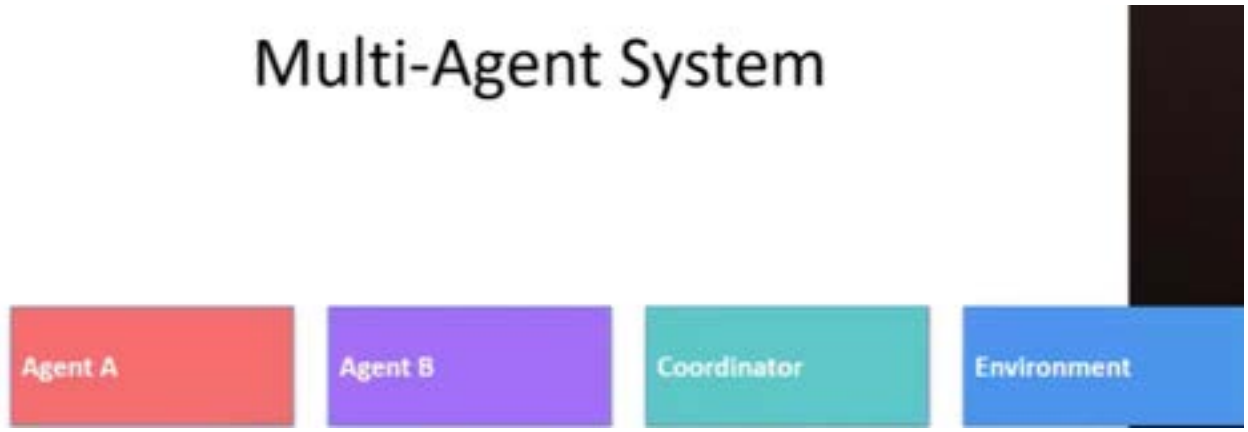
LoRA Fine-Tuning



Agent Workflow





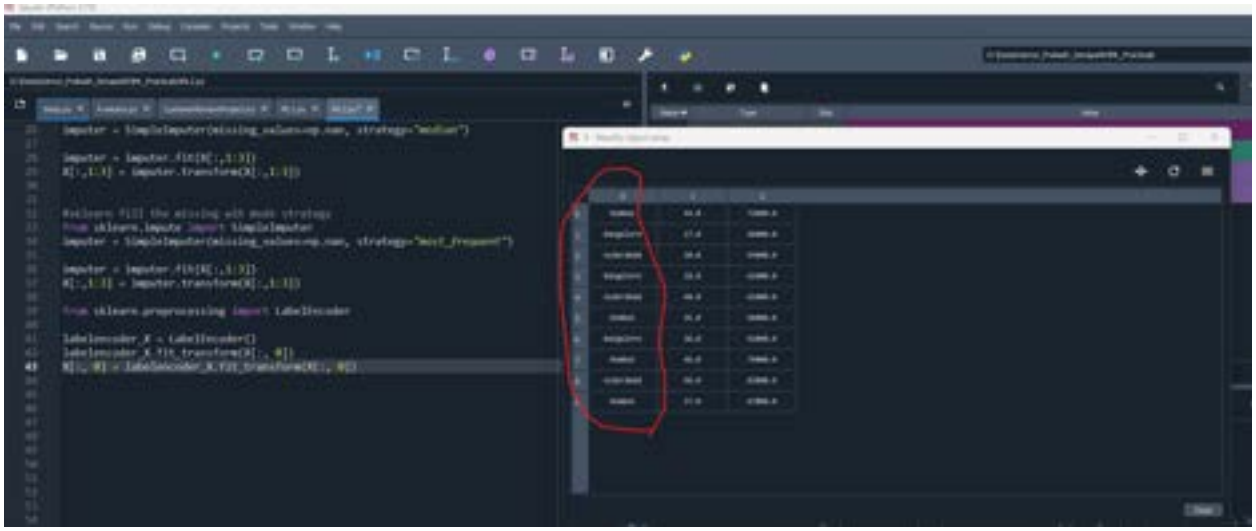


Revision up to now what we have done.

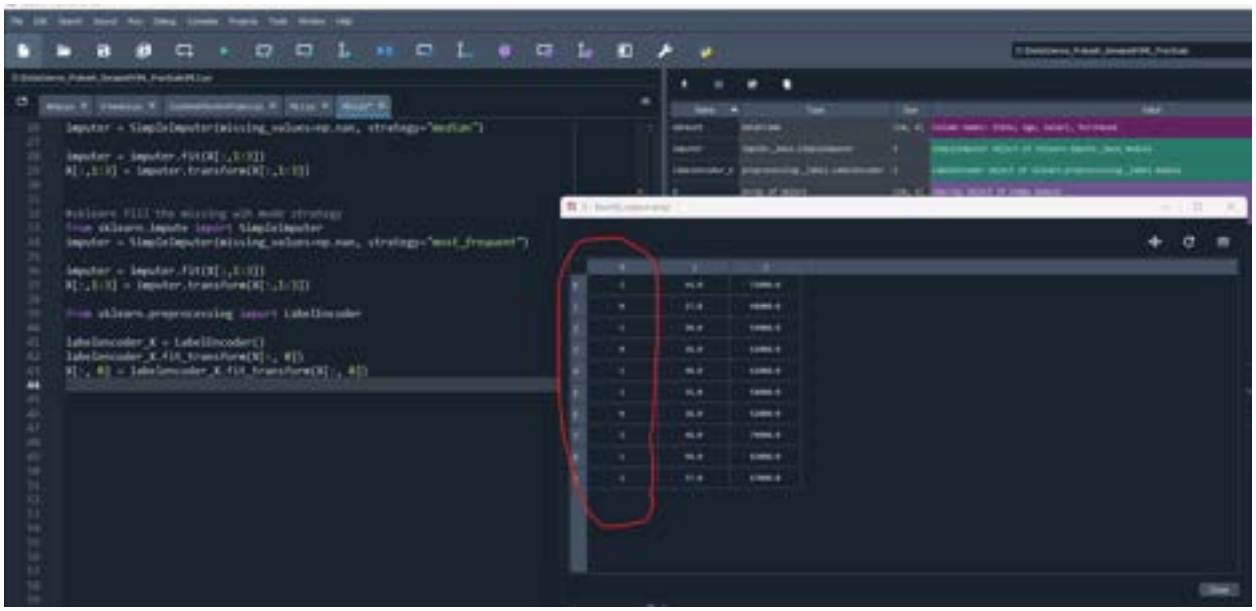
1- dataset
2- split the datainto x & y
3- data cleaning
    to fill missing value using sklearn
    transfomer sklearn.imput import simpleimputer
    simple imputer paramert -- strae -- mean, median, mode
    paramete tun - mean (rest of them hyperparamter tunningt)

Q)How to convert text to number
A)onehotEncoder or label encoder

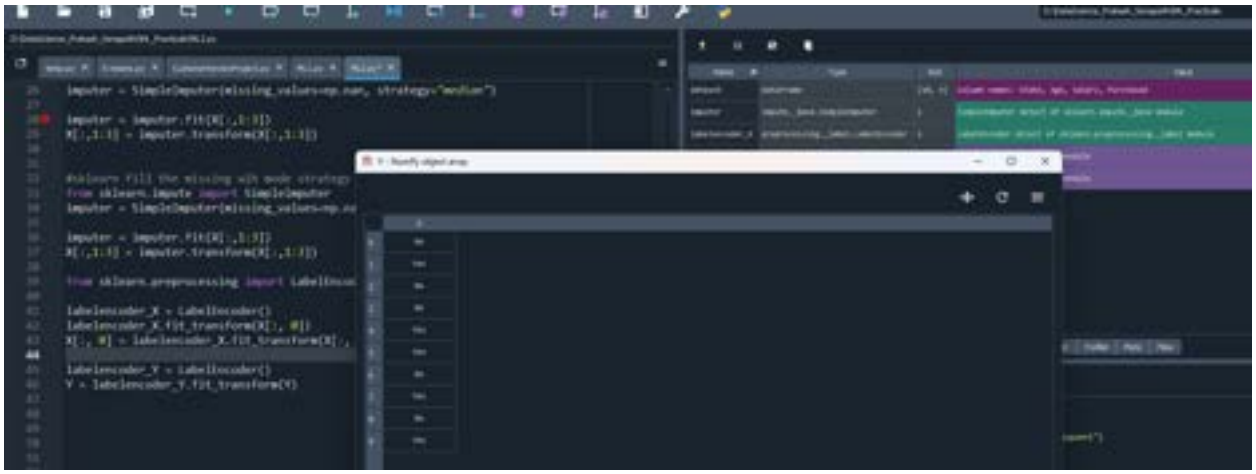
Before running the label code the 0 index of the X is text like mumbai , hyderabad



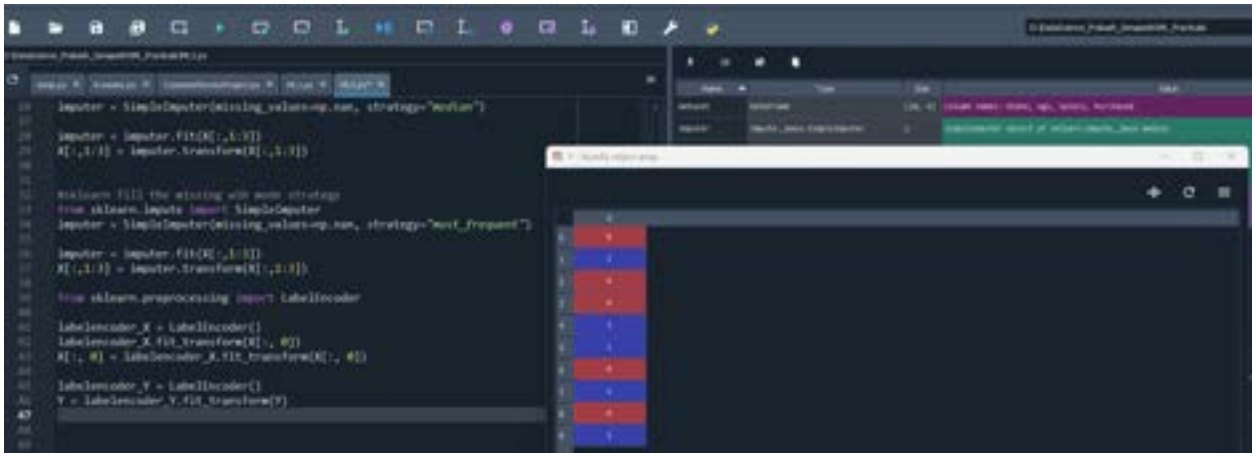
After running the code the output is



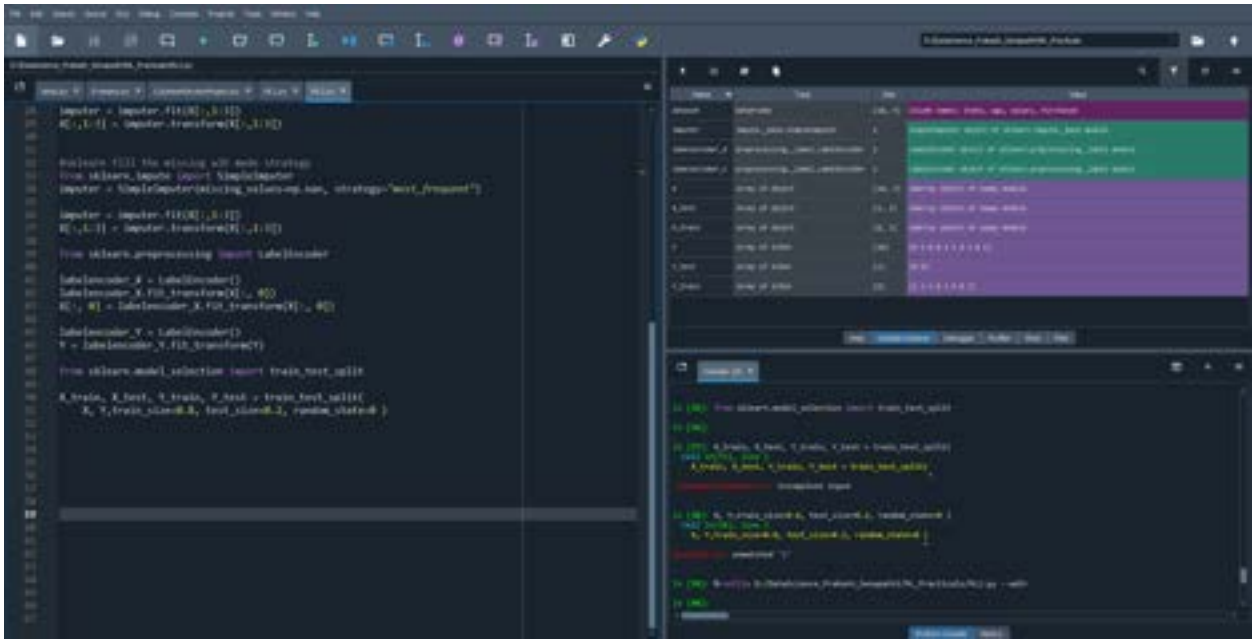
Up to now data cleaning is going on when we apply data machine does not understand what is chennai and bangalore we convert this text to numbers
We have completed for the X so same we are doing for the Y



After the code applying



After this we need to divide the data in to xtrain , xtest and ytrain and ytest



Note : in the code if we do not use random then the values are shuffling(records are shuffling) and everytime when values are changing then the accuracy will change.so the model core not optimized so it will not become constant model when it is not constant model it was not deployed because the accuracy not good so business will impact

Up to now what we have covered

DATA PREPROCESSING PIPELINE -->

1- import library
2- import dataset
**3- split the dataset into x & y **
4- data cleaning (missing value treatment)
5- eda technique (missing value trament)
6- train & test split (80-20 | 70-30)

Next what we need to do

7- train model
8- model building
9- mode evaluation
10- ml model validation
11- ml model testing
12- ml model deployment
13- ml model monitoring -- mlops
14- cicd (continue integration & continue delivery deplomeent)

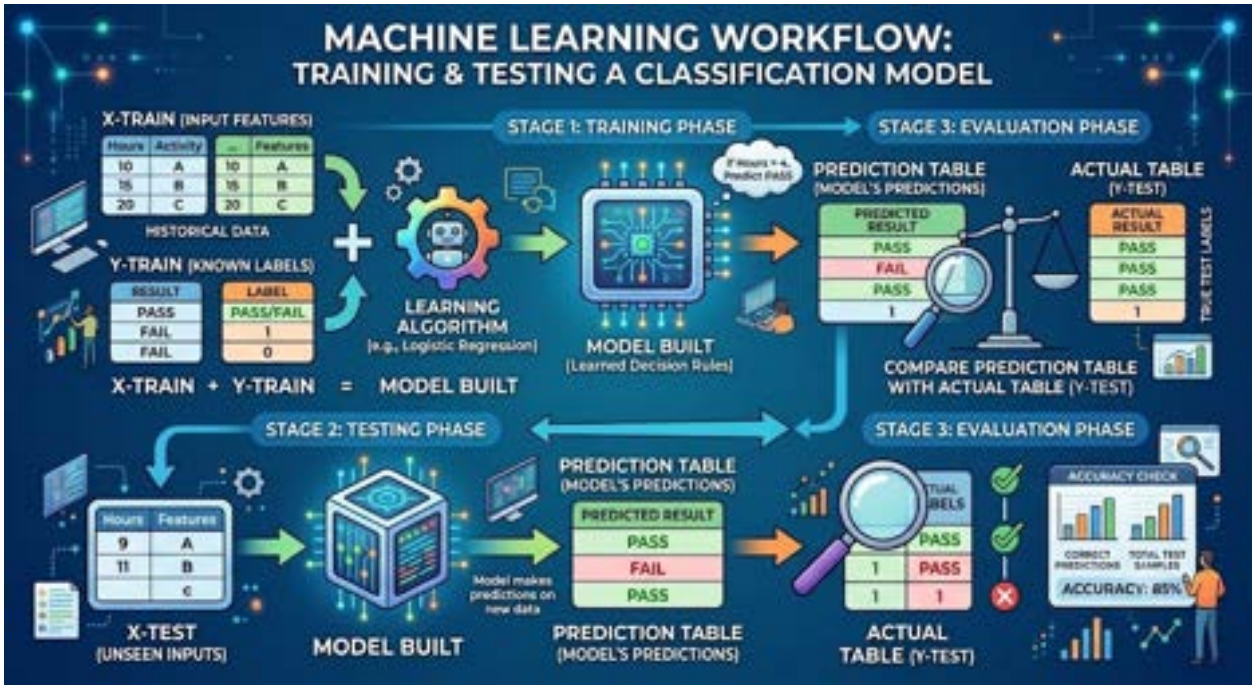
15- same loop will content new data from 1st -14)

we have dataset -- x & y -- data ecleaning using eda
x- train & x-test | y-train & y-test
sklearn.impute import simpleiputer (fill missing value)
sklearn.processing import labelencoder (conver text - number)
random_state = 0 | everytime recods randomly , accuracy impact
introduce to spyder | spyder is advanced ide then jupyter
all mi model we wil build in spyder

We have divided data in to x and y then this X again divided into X-train, X-test,Y-train,Y-test and split ratio is 80 and 20 or 70 and 30 or 75 and 25 and this all are historical model next step we need to train the data

Steps for training the data :
We need to train the X-train and Y-train X-train+Y-train when we train X-train+Y-train the model has built X-train+y-train = Model built like Input+output = modelbuit

After building the model in order to check the model output accurate or not we need to test the model by passing the X-test to the model so the model will generate some prediction table this prediction table need to compare with actual table (y-test)

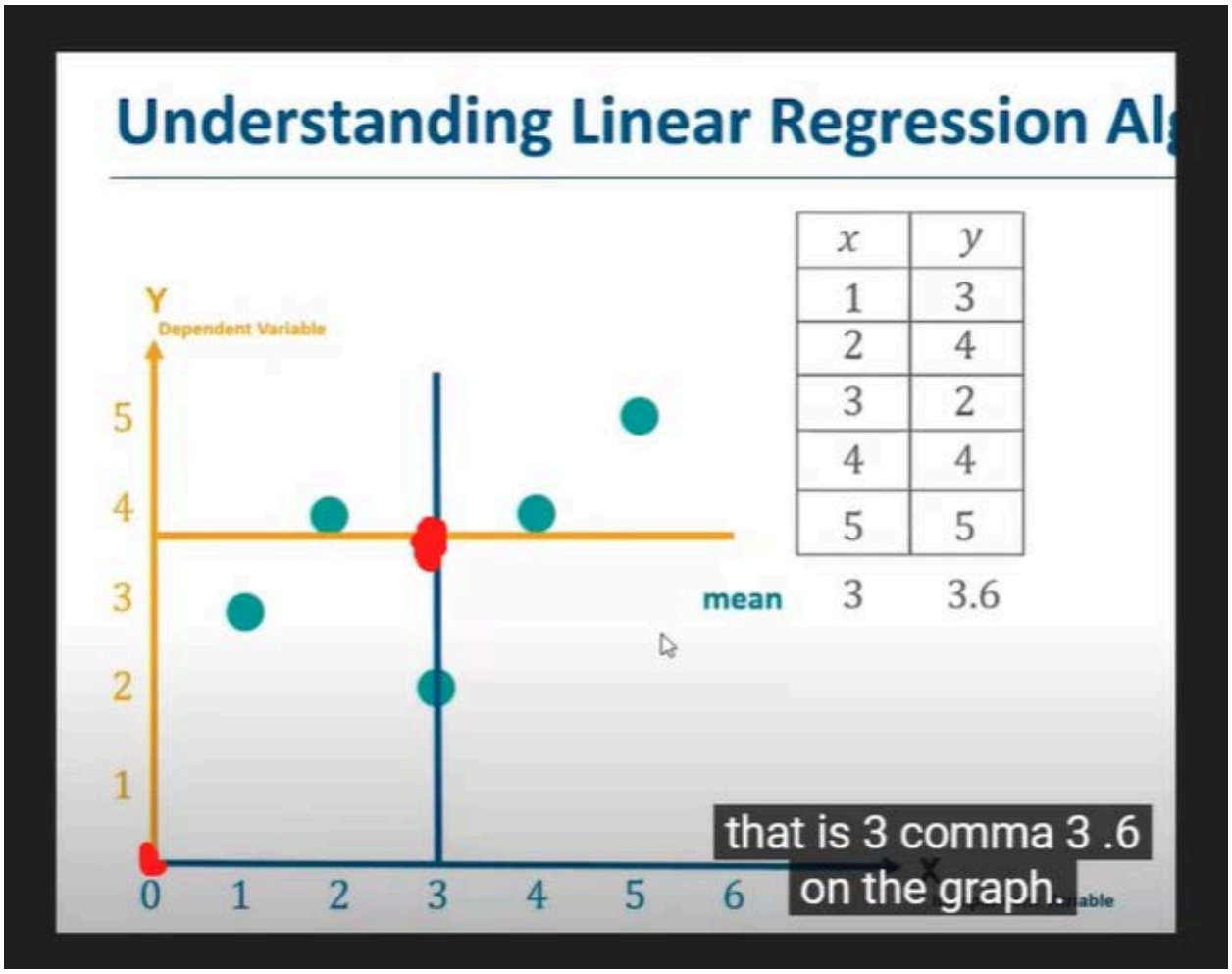


Xtrian+Ytrain = Model will buid
Then we pass the X-test model will generate model prediction table that is Y-pred this prediction compare with Y-test which is actual this is actual vs predicted.

Q)What is absolute error

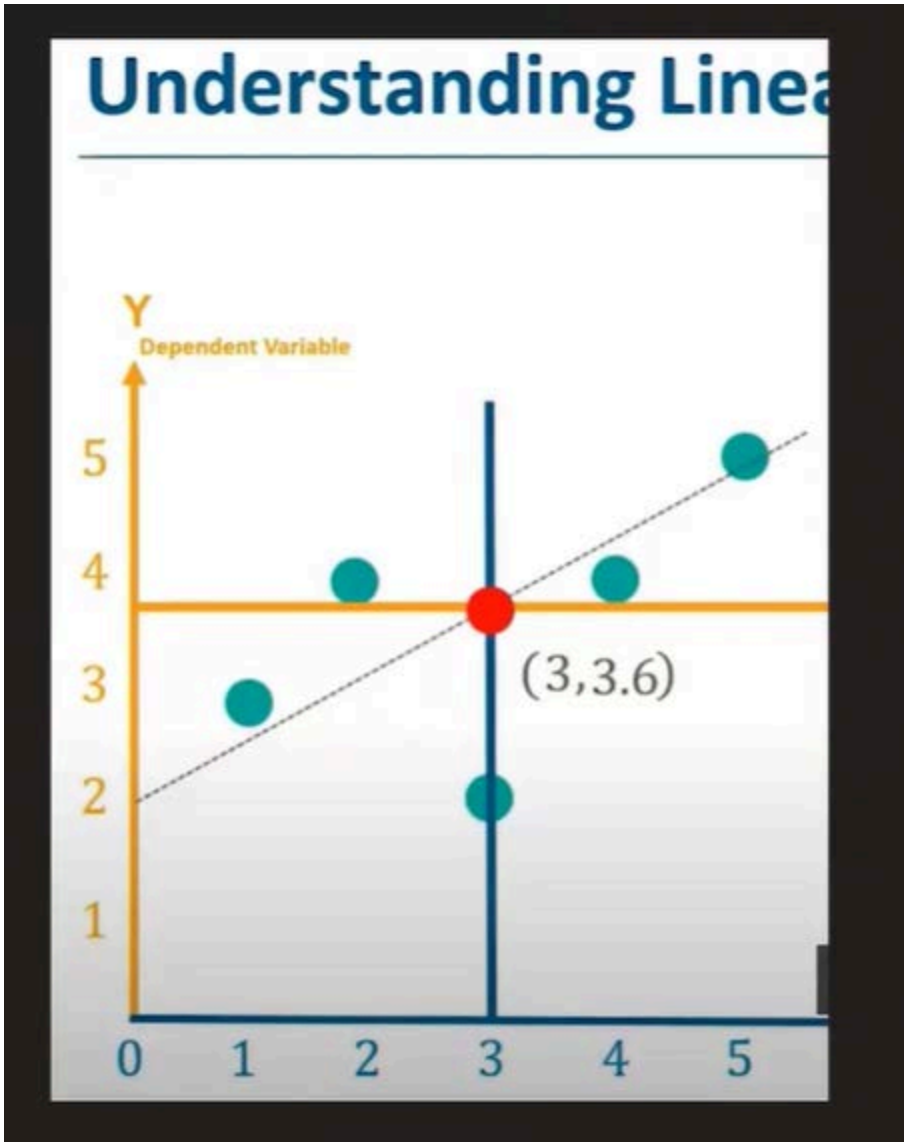
A) Actual-Predicted

Linear Regression :



x	y
1	3
2	4
3	2
4	4
5	5

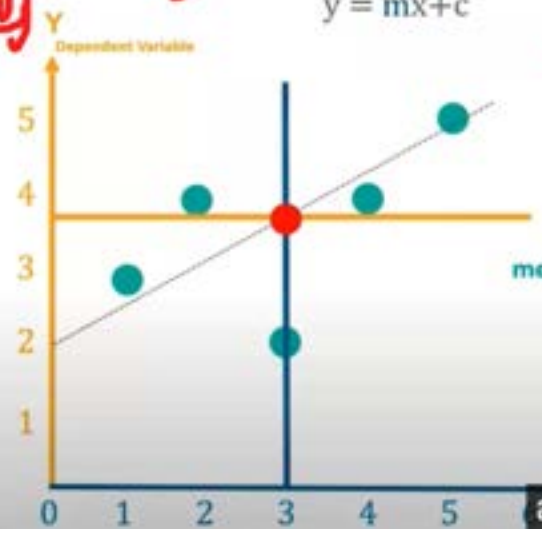
Here the points are according to x and y co-ordinates



The co-ordinates of x and y are meeting here

Understanding Linear Regression Algorithm

$y = mx+c$



x	y
1	3
2	4
3	2
4	4
5	5

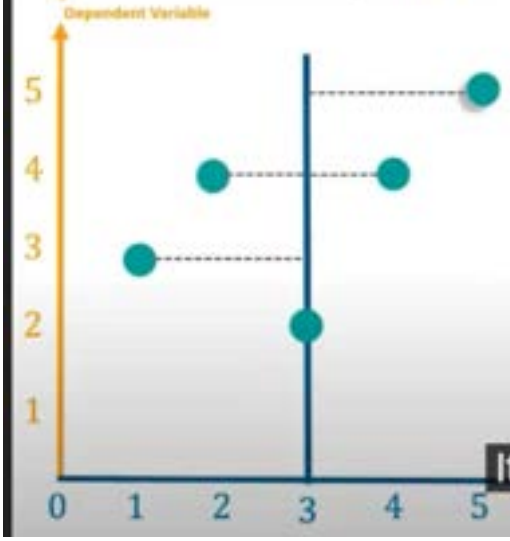
mean

$$m = \frac{\sum (x-\bar{x})(y-\bar{y})}{\sum (x-\bar{x})^2}$$

Now, we have an equation of line.

Understanding Linear Regression Algorithm

$y = mx+c$



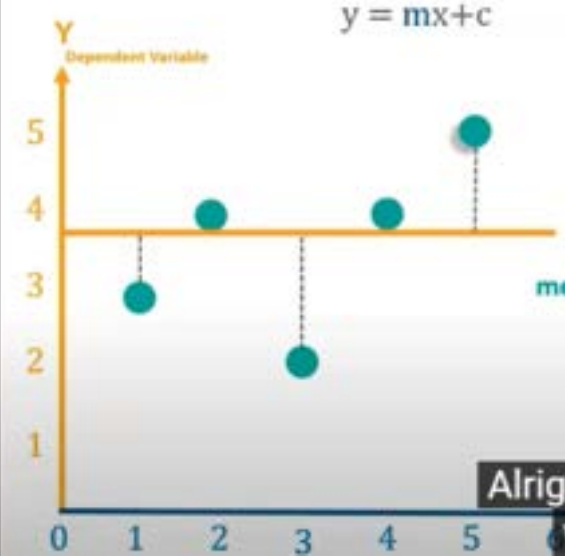
x	y	$x-\bar{x}$
1	3	-2
2	4	-1
3	2	0
4	4	1
5	5	2

mean 3 3.6

$$m = \frac{\sum (x-\bar{x})(y-\bar{y})}{\sum (x-\bar{x})^2}$$

It's nothing but the distance of all the point

Understanding Linear Regression Algorithm

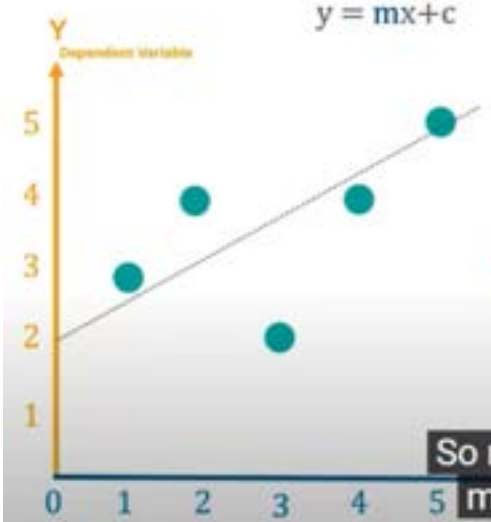


x	y	$x-\bar{x}$	$y-\bar{y}$
1	3	-2	-0.6
2	4	-1	0.4
3	2	0	-1.6
4	4	1	0.4
5	5	2	1.4
mean	3	3.6	

$$m = \frac{\sum (x-\bar{x})(y-\bar{y})}{\sum (x-\bar{x})^2}$$

Alright, so now we are done with Y minus y bar.

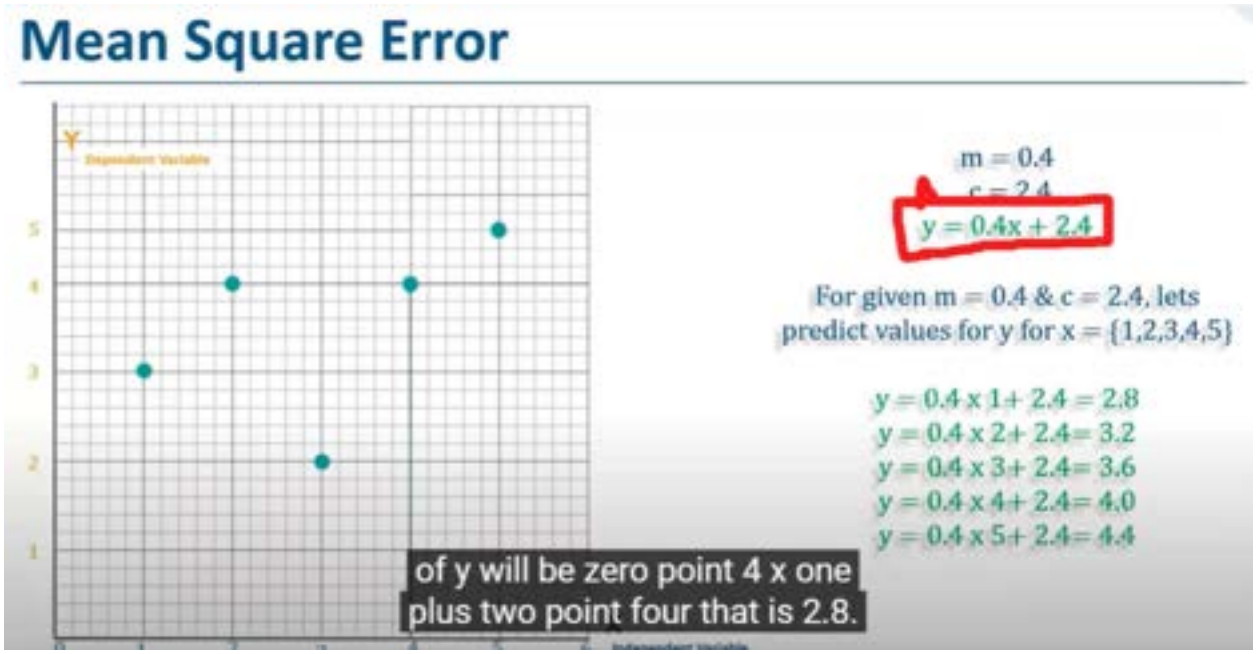
Understanding Linear Regression Algorithm

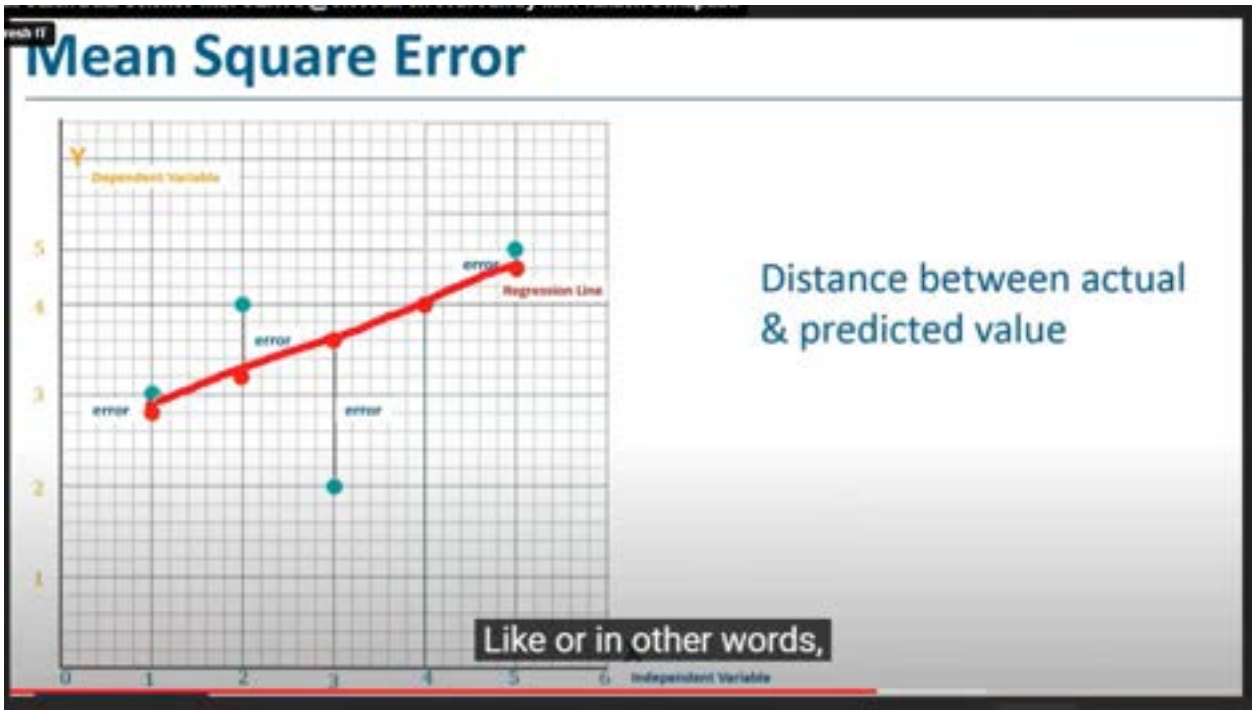


x	y	$x-\bar{x}$	$y-\bar{y}$	$(x-\bar{x})^2$
1	3	-2	-0.6	4
2	4	-1	0.4	1
3	2	0	-1.6	0
4	4	1	0.4	1
5	5	2	1.4	4
mean	3	3.6		

$$m = \frac{\sum (x-\bar{x})(y-\bar{y})}{\sum (x-\bar{x})^2}$$

So now in our table we have x minus X bar y minus y bar



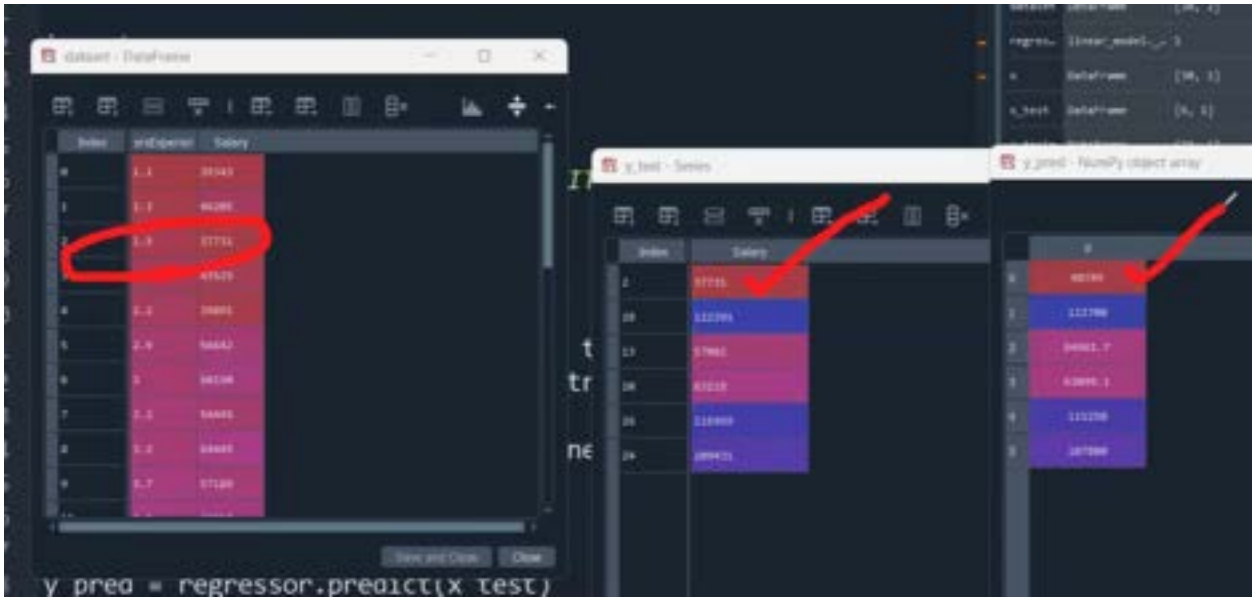


Best fit line

What ever the linear concept explained above will come from the sklearn framework from the LinearRgression libraries

Trianing the model with linear regression
from sklearn.linear_model import LinearRegression

```
regressor = LinearRegression()
regressor.fit(X_train, Y_train)
```



For 1.5year experience the salary is 377 but model predicted 40

Performance Measures of regression is R^2 and Adjusted R^2

Note : In Machine learning the train the data is nothing but X-train and Y-train in Machine learning this is called Bias(All training section is called bias)
We test the data by using the X_test,Y_test is called variance(All testing section is called variance)

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

dataset = pd.read_csv(r"D:\DataScience_Prakash_Senapathi\ML_Practicals\Salary_Data.csv")

X = dataset.iloc[:,:-1]
Y = dataset.iloc[:,-1]

from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y,train_size=0.8, test_size=0.2, random_state=0 )

# Trianing the model with linear regression
from sklearn.linear_model import LinearRegression
```

```
regressor = LinearRegression()
regressor.fit(X_train, Y_train)

#Now we need to test the model accuracy by passing x-test
y_pred = regressor.predict(X_test)

#: Note: We need to compare y-test and Y-pred
plt.scatter(X_test, Y_test, color='red')
plt.plot(X_train,regressor.predict(X_train),color='blue' )
plt.title('Salary vs Experience(Test Set)')
plt.xlabel('Years of experience')
plt.ylabel('Salary')
plt.show()

# in this dataset ends with 10 years per suppose if we want to predict future data
# First we nee to calculate the coefficient and intercept

m_coef = regressor.coef_
print(m_coef)

c_intercept = regressor.intercept_
print(c_intercept)

# Now if we want to predict salry for 12 years experience then
y12 = m_coef*12+c_intercept
print(y12)

# Now if we want to predict salry for 20 years experience then
y20 = m_coef*20+c_intercept
print(y20)
#In Machine learning the train the data is nothing but X-train and Y-train in Machine learning this is called Bias(All training section is called bias)
#Note : When we train the data the bias score will generate
#This is called training score
bias_score = regressor.score(X_train, Y_train)
print(bias_score)
#This si called testing score
variance_score = regressor.score(X_test, Y_test)
print(variance_score)
```

Python 3.13.9 | packaged by Anaconda, Inc. | (main, Oct 21 2025, 19:09:58) [MSC v.1929 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 9.7.0 -- An enhanced Interactive Python. Type '?' for help.

%runfile D:/DataScience_Prakash_Senapathi/ML_Practicals/Linear_RegressionML.py --wdir
[9312.57512673]
26780.099150628186
[138531.00067138]

%runfile D:/DataScience_Prakash_Senapathi/ML_Practicals/Linear_RegressionML.py --wdir
[9312.57512673]
26780.099150628186
[138531.00067138]
[213031.60168521]

%runfile D:/DataScience_Prakash_Senapathi/ML_Practicals/Linear_RegressionML.py --wdir
[9312.57512673]
26780.099150628186
[138531.00067138]
[213031.60168521]
0.9411949620562126

%runfile D:/DataScience_Prakash_Senapathi/ML_Practicals/Linear_RegressionML.py --wdir
[9312.57512673]
26780.099150628186
[138531.00067138]
[213031.60168521]
0.9411949620562126
0.988169515729126

Training score is between 0 to 1(0.9411949620562126)
Testing score is between 0 to 1(0.988169515729126) So this is the best model because the range is between 0 to 1.

Q)*******if the training_score = 0.45 and testing_score= 0.94 here low bias and high variance is called over fitting

Q)Interv_question : What is overfitting
A)The model has the low training score or high test score or low bias high variance is called overfitting

```
best fit line

train the data ( x_train, y_train)  == bias
test teh data (x_test, y_test ) == variance

training_score == 0.94  & testing_score = 0.98 == best model
training_Score == 0.45 & testing_score =0.94 == low bias high variance == overfitting
```

Q)*******if the testing_score = 0.45 and training_score= 0.94 here low variance and high bias is called underfitting**

Q)Interv_question : What is underfitting
A)The model has the low testing score or high training score or high bias low variance is called under fitting

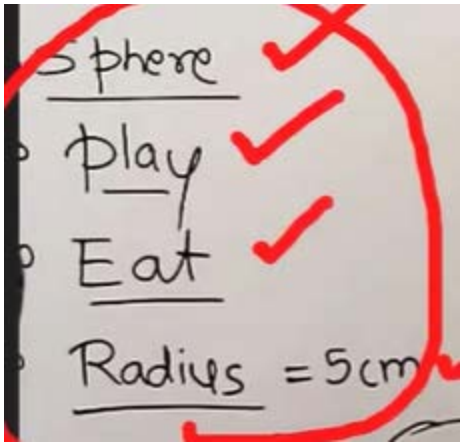
```
best fit line

train the data ( x_train, y_train)  == bias
test teh data (x_test, y_test ) == variance

training_score == 0.94  & testing_score = 0.98 == best model
training_Score == 0.45 & testing_score =0.94 == low bias high variance == overfitting
training_Score = 0.98 & testing_Score = 0.45 == high bias low variance == underfitting
```

overfitting vs underfitting vs best fit everytime we must need to get best fit result before go for deployment

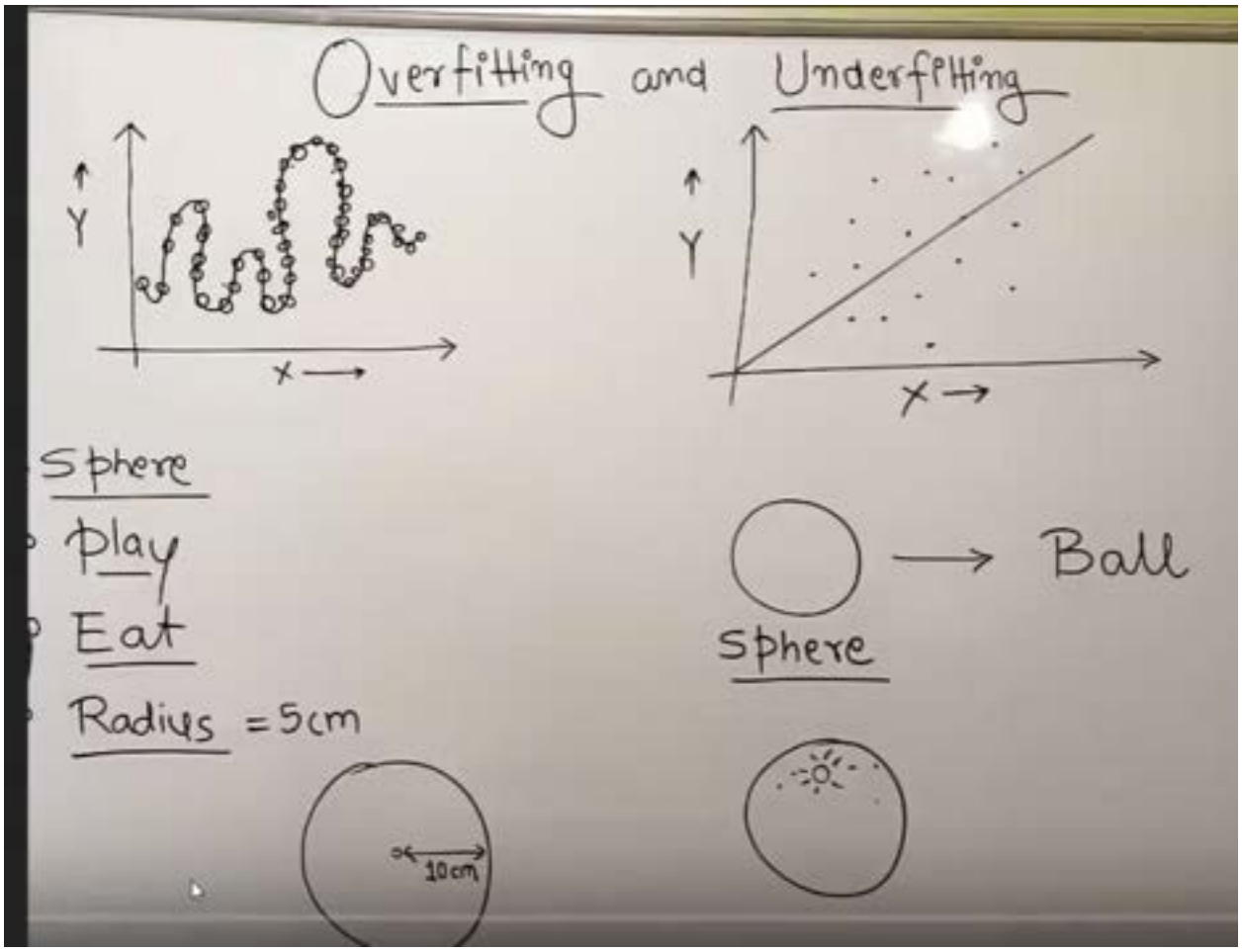
Take a cricket ball it is in the sphere shape and it has the below parameters
1.Ball is sphere shape
2. We can play with the ball
3. You cannot eat
4. Radius of ball is 5 cm



Then we can predict as ball in training phase we will show all the 4 attributes but in testing phase all the first 3 parameters passed but basket ball radius is not 5cms so model predict as not a ball the model is confused and miss classification because of radius the model is overfit and predict it is not a ball

When we train the model with more attributes the overfitting model comes up

> **overfitting -- ml model is train with more attribute -- overfitting problem comes**



If we remove attribute radius then it will predict as ball so the overfitting is gone

overfitting -- ml model is train with more attribute -- overfitting problem comes
if model overfit then model will give more error less accuracy
model cannot deploy
organization ever create any wesbite
customer never clie

If we write the exam our colorr, height , weight does not decide pass or fail exam

height, weight, look, hari color, t-shirt -- model will overfitting
reading hr, learning,

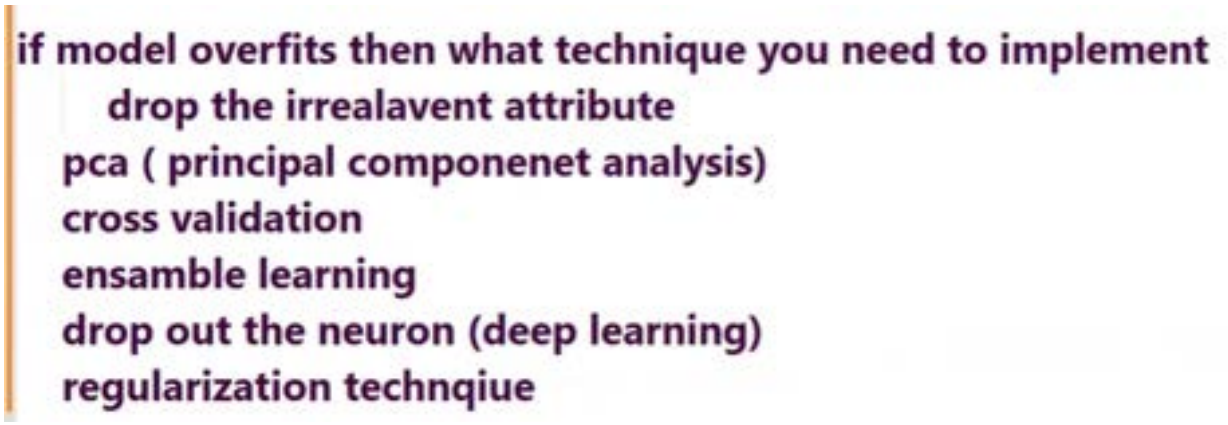
overfitting -- low bias high variance -- bias score -- trainin score

Under fitting: in underfitting if we train the object with only one parameter like a parameter sphere example while testing a sphere model with orange color can be fruit miss classification

Overfitting is too much eating with no gap in stomach underfitting is hungry with no food

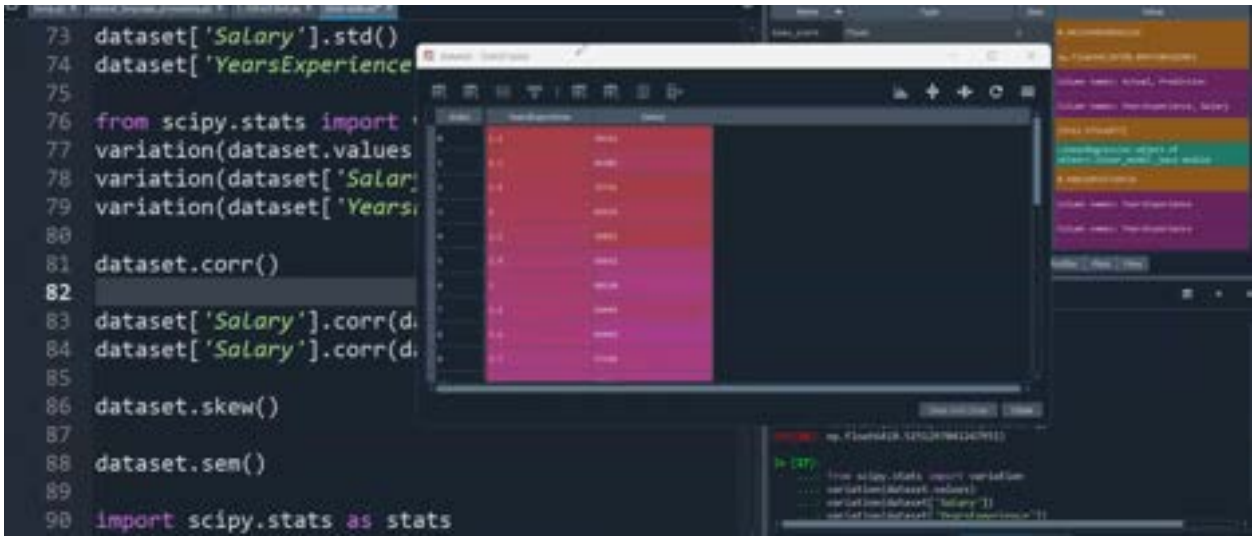
If the model is overfit then we need to implement the below techniques
1.Drop the irrelevant attributes (Ex:Student pass or fail exam in this roll number does not matter because for passing hard work and learning needed) in order to drop we will use the below techniques
1.PCA(principle component analysis)
2.cross validation
3 .Ensemble learning
4.drop out the neuron(In Deep learning)
5.Regularization technique

Q)If your ML model overfits then which technique you will apply A)PCA



If the model underfit which technique to apply A) We need to add more relevant attributes

Bias —->training score
Variance —----->testing score
Both scores should match if there is a difference of 10 or 15 that is fine
How to introduce the statistics to the current data set



In the dataset we have experience and salary when we train this data to the machine it will understand the value 1.1 and the salary is 39343 both have the different values and the 39343 is greater than 1.1 the model is always one side bias when the two numbers are not equal the model predictions will be wrong so we are applying the z score for both 1.1 and 39343 the values will change to

Z-score

In [21]:

```
# for calculating Z-score we have to import a library first
import scipy.stats as stats

df.apply(stats.zscore) # this will give Z-score of entire dataframe
```

Out[21]:

	YearsExperience	Salary
0	-1.510053	-1.360113
1	-1.438373	-1.105527
2	-1.366693	-1.419919
3	-1.187494	-1.204957
4	-1.115814	-1.339781
5	-0.864935	-0.718307

51:49

Complete Practical Code :

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

dataset = pd.read_csv(r"D:\DataScience_Prakash_Senapathi\ML_Practicals\Salary_Data.csv")

X = dataset.iloc[:,:-1]
Y = dataset.iloc[:,-1]

from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y,train_size=0.8, test_size=0.2, random_state=0 )

# Trianing the model with linear regression
from sklearn.linear_model import LinearRegression
regressor = LinearRegression()
regressor.fit(X_train, Y_train)

#Now we need to test the model accuracy by passing x-test
y_pred = regressor.predict(X_test)

#: Note: We need to compare y-test and Y-pred
plt.scatter(X_test, Y_test, color='red')
plt.plot(X_train,regressor.predict(X_train),color='blue' )
plt.title('Salary vs Experience(Test Set)')
plt.xlabel('Years of experience')
plt.ylabel('Salary')
plt.show()

# in this dataset ends with 10 years per suppose if we want to predict future data
# First we nee to calculate the coefficient and intercept

m_coef = regressor.coef_
print(m_coef)

c_intercept = regressor.intercept_
print(c_intercept)

# Now if we want to predict salry for 12 years experience then
y12 = m_coef*12+c_intercept
print(y12)
```

```
# Now if we want to predict salry for 20 years experience then
y20 = m_coef*20+c_intercept
print(y20)
#In Machine learning the train the data is nothing but X-train and Y-train in Machine learning this is called Bias(All training section is called bias)
#Note : When we train the data the bias score will generate
#This is called training score
bias_score = regressor.score(X_train, Y_train)
print(bias_score)
#This si called testing score
variance_score = regressor.score(X_test, Y_test)
print(variance_score)

# Statistics implemetation with this  ML model

# Mean
dataset.mean()
dataset['Salary'].mean()
dataset['YearsExperience'].mean()

# Median
dataset.median()
dataset['Salary'].median()
dataset['YearsExperience'].median()

# Median
dataset.var()
dataset['Salary'].var()
dataset['YearsExperience'].var()

# std
dataset.std()
dataset['Salary'].std()
dataset['YearsExperience'].std()

# Scipy is nothing but scientific python library
#The variation compare the 2 attributes
from scipy.stats import variation
variation(dataset.values)
variation(dataset['Salary'])
variation(dataset['YearsExperience'])

# Relation between 2 attributes employee experience and
#employee slary is a correlation experience increases salary increases
```

```
# The range of correlation is -1 to 1  the -1 to 0 is negative correlation and
# 0 to 1 is positive correlation
dataset.corr()

dataset['Salary'].corr(dataset['YearsExperience'])
dataset['Salary'].corr(dataset['Salary'])

dataset.skew()
dataset.sem()

import scipy.stats as stats


dataset.apply(stats.zscore)

stats.zscore(dataset[['Salary']])

#Annova Frame Work SSR,SSE,SST
# In SSR Actual,Predicted,Average

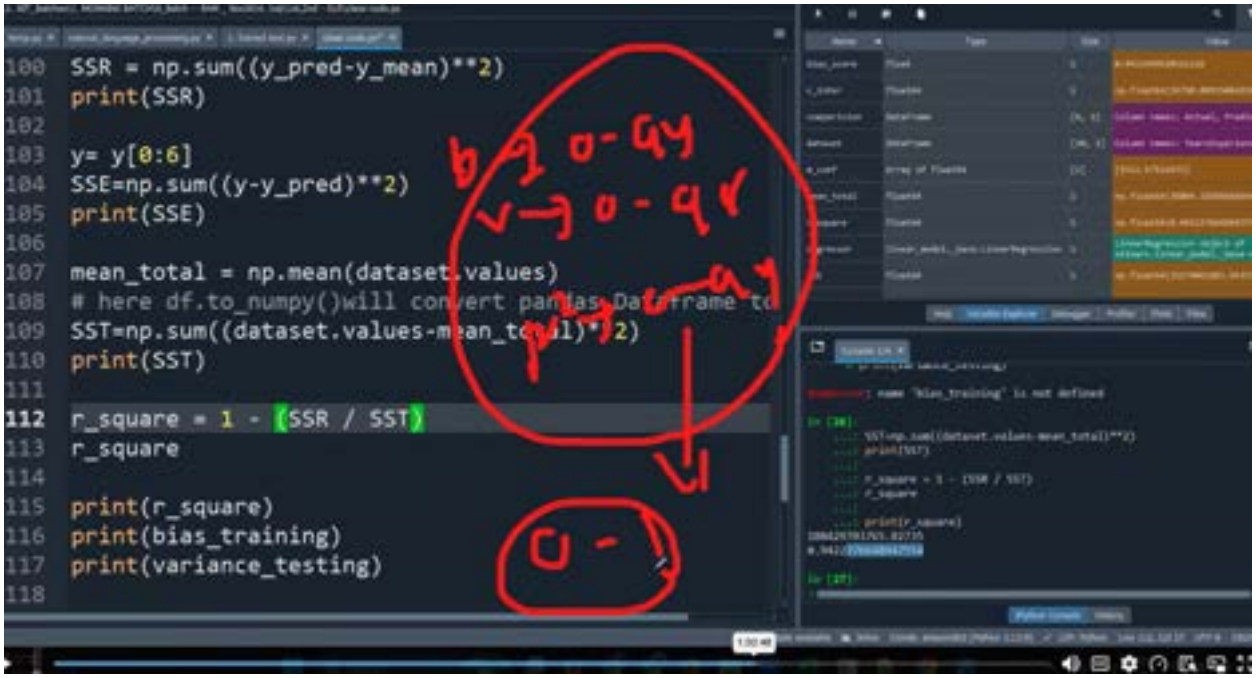
y_mean = np.mean(Y)
SSR = np.sum((y_pred-y_mean)**2)  # predicted-Average SSR
print(SSR)

y = Y[0:6]
SSE = np.sum((y-y_pred)**2) #Actual-predicted
print(SSE)

mean_total = np.mean(dataset.values)
SST = np.sum((dataset.values-mean_total)**2)
print(SST)

r_square = 1-(SSR/SST)
r_square

print(r_square)
print(bias_score)
print(variance_score)
```



All the 3 R_square,bias_training,variance_testing all are between 0 to 1 range so it is good model

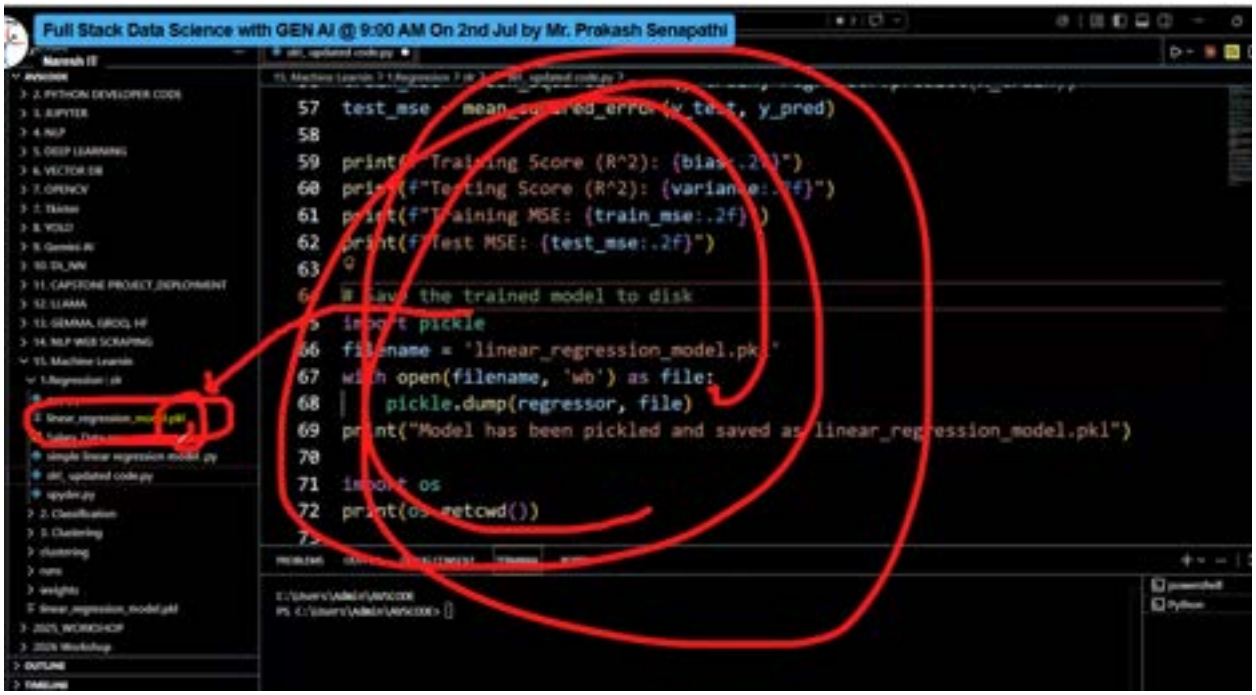


For all the backend code now we are developing the front end
Below are the rules need to be followed for front end
1. We need to keep all the files in one folder
Ex: in VS code create folder machinelearning in that again need to create Regression folder

```
# Save the trained model to disk
import pickle          #Pickle is important library for serialization
filename = 'linear_regression_model.pkl'   # this file is stored in to .pkl it means all the above code is stored in the object
with open(filename, 'wb') as file:
    pickle.dump(regressor, file)
print("Model has been pickled and saved as linear_regression_model.pkl")

import os
print(os.getcwd())
```

Pickle means similar to serialization .pkl this means all the file stored in the form of object



And this object is passed to frontend

This pickle write entire file in to binary files we cannot open
The pickle has two features pickle.dump and pickle.load

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.model_selection import train_test_split
```

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import pickle

# Load the dataset
dataset =
pd.read_csv(r'D:\DataScience_Prakash_Senapathi\Machine_Learning\Salary_Dat
a.csv')

# Split the data into independent and dependent variables
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, 1].values

# Split the dataset into training and testing sets (80-20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20,
random_state=0)

# Train the model
regressor = LinearRegression()
regressor.fit(X_train, y_train)

# Predict the test set
y_pred = regressor.predict(X_test)

#comparision for y_test vs y_pred
comparison = pd.DataFrame({'Actual': y_test, 'Predicted': y_pred})
print(comparison)

# Visualize the training set
plt.scatter(X_train, y_train, color='red')
plt.plot(X_train, regressor.predict(X_train), color='blue')
plt.title('Salary vs Experience (Training set)')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.show()

# Visualize the test set
plt.scatter(X_test, y_test, color='red')
plt.plot(X_train, regressor.predict(X_train), color='blue')
plt.title('Salary vs Experience (Test set)')
```

```
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.show()

# Predict salary for 12 and 20 years of experience using the trained model
y_12 = regressor.predict([[12]])
y_20 = regressor.predict([[20]])
print(f"Predicted salary for 12 years of experience: ${y_12[0]:,.2f}")
print(f"Predicted salary for 20 years of experience: ${y_20[0]:,.2f}")

# Check model performance
bias = regressor.score(X_train, y_train)
variance = regressor.score(X_test, y_test)

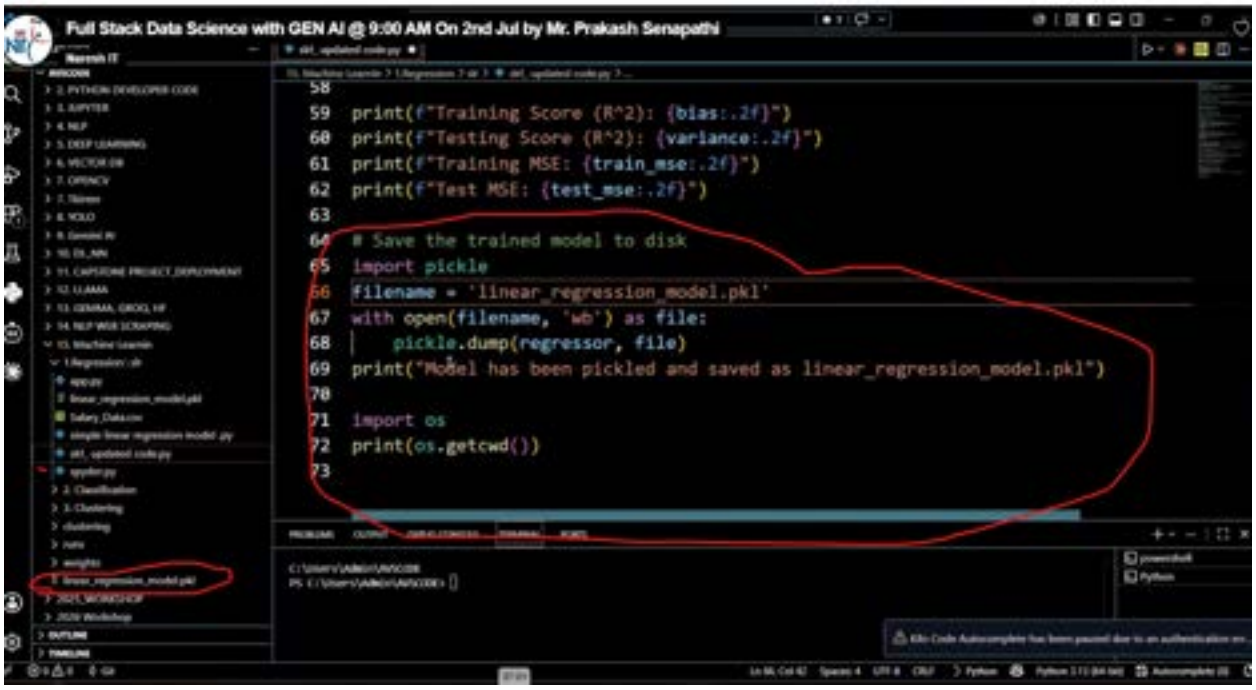
train_mse = mean_squared_error(y_train, regressor.predict(X_train))
test_mse = mean_squared_error(y_test, y_pred)

print(f"Training Score (R^2): {bias:.2f}")
print(f"Testing Score (R^2): {variance:.2f}")
print(f"Training MSE: {train_mse:.2f}")
print(f"Test MSE: {test_mse:.2f}")

# Save the trained model to disk
import pickle           #Pickle is important library for serialization
filename = 'linear_regression_model.pkl'   # this file is stored in to
.pkl it means all the above code is stored in the object
with open(filename, 'wb') as file:
    pickle.dump(regressor, file)
print("Model has been pickled and saved as linear_regression_model.pkl")

import os
print(os.getcwd())
```

All the backend we have stored in pickle file



Up to now pickle file dump is completed(We can see the csv file,code.py file and pickle file) now we need to load the pickle file

Steps for loading the pickle file
Step1: create app.py in the same folder

```
import streamlit as st
import pickle
import numpy as np

# Load the saved model
model =
pickle.load(open(r'D:\DataScience_Prakash_Senapathi\Machine_Learning\linea
r_regression_model.pkl', 'rb')) # Added missing comma before 'rb'

# Set the title of the streamlit app
st.title("Salary prediction app created by Bheemraj")
```

```
# Add a brief description
st.write("This app predicts the salary based on years of experience using
a simple Linear Regression model.")

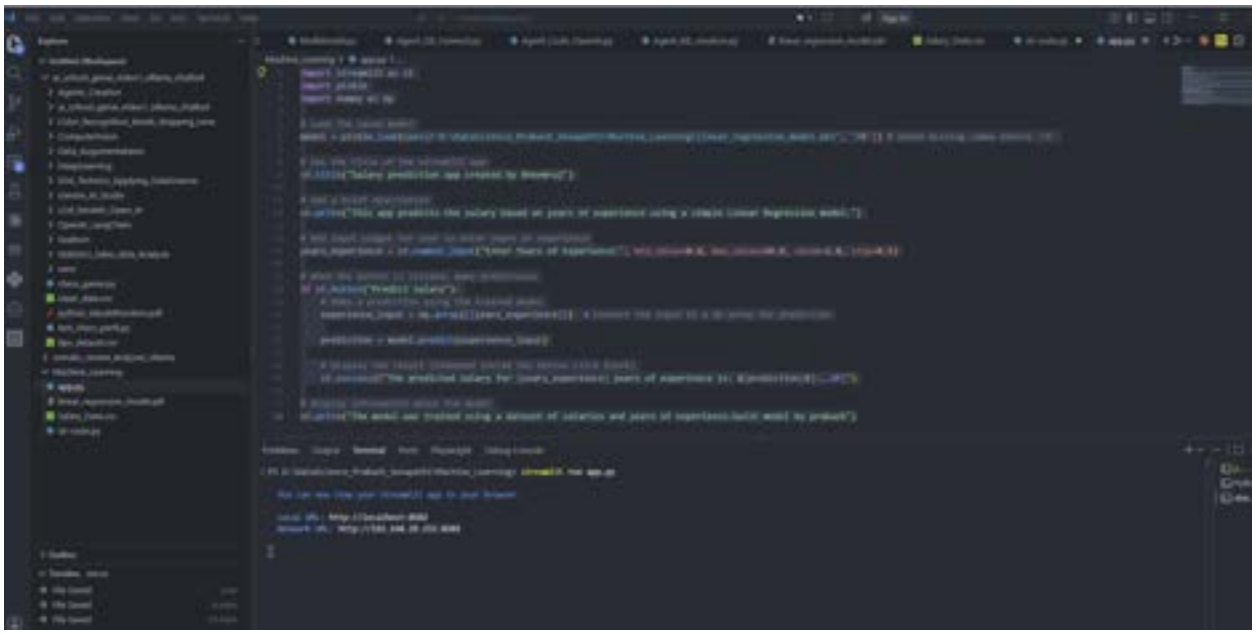
# Add input widget for user to enter years of experience
years_experience = st.number_input("Enter Years of Experience:",
min_value=0.0, max_value=50.0, value=1.0, step=0.5)

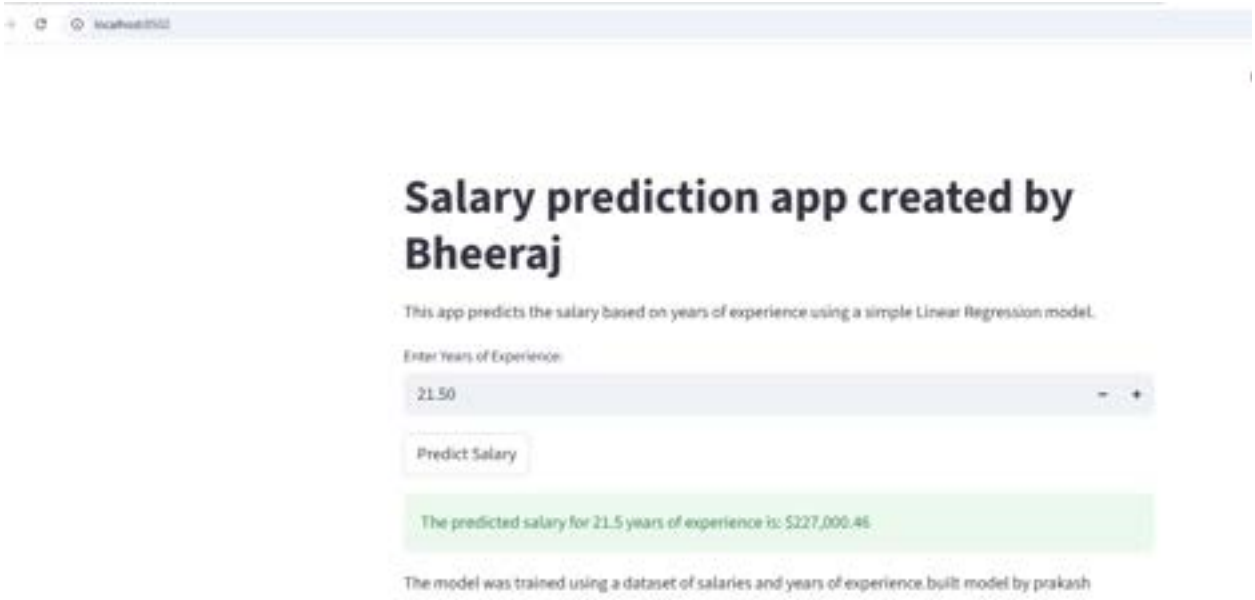
# When the button is clicked, make predictions
if st.button("Predict Salary"):
    # Make a prediction using the trained model
    experience_input = np.array([[years_experience]])  # Convert the input
to a 2D array for prediction

    prediction = model.predict(experience_input)

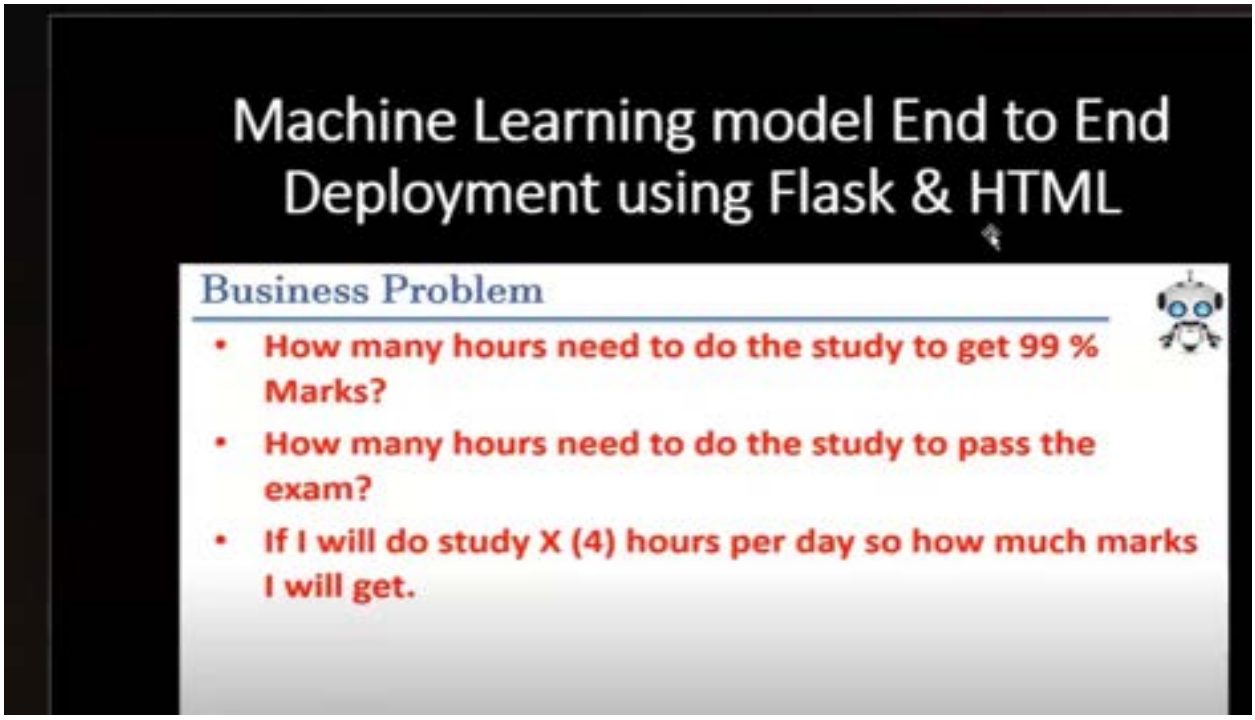
    # Display the result (Indented inside the button click block)
    st.success(f"The predicted salary for {years_experience} years of
experience is: ${prediction[0]:,.2f}")

# Display information about the model
st.write("The model was trained using a dataset of salaries and years of
experience.built model by prakash")
```



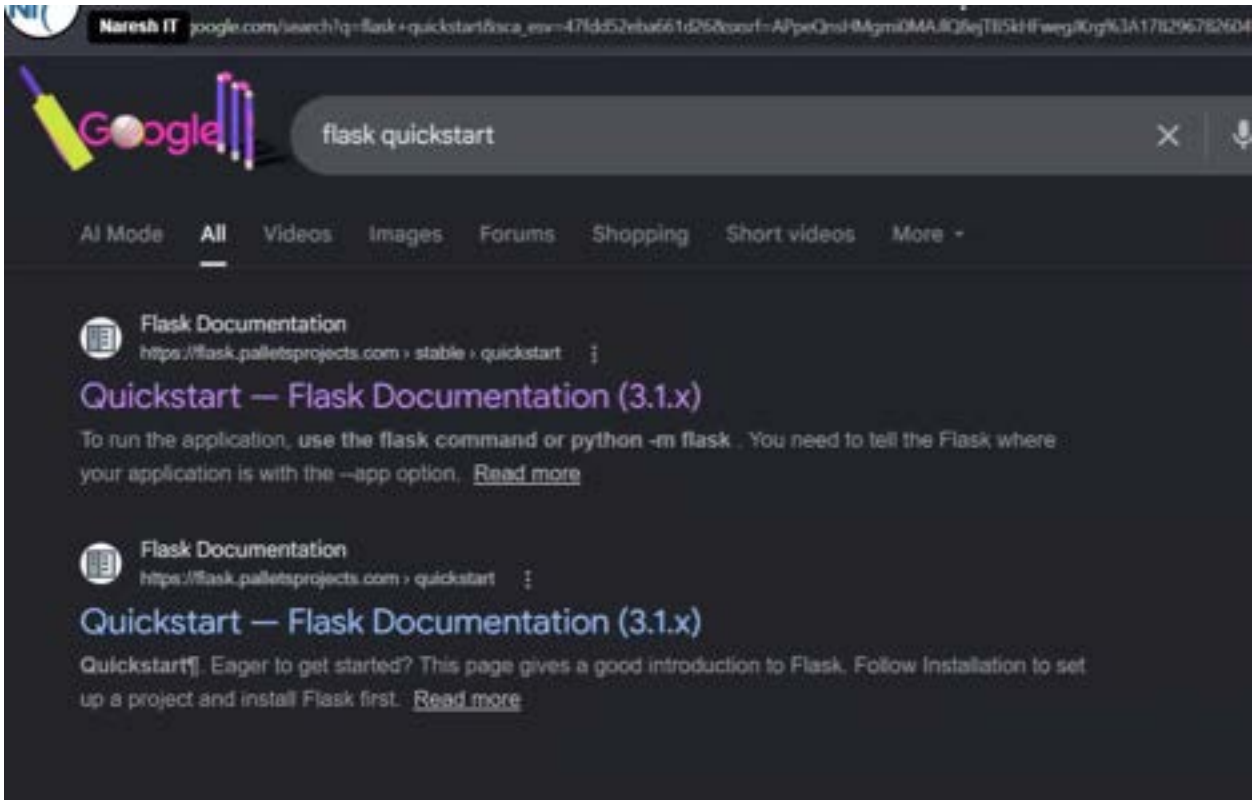


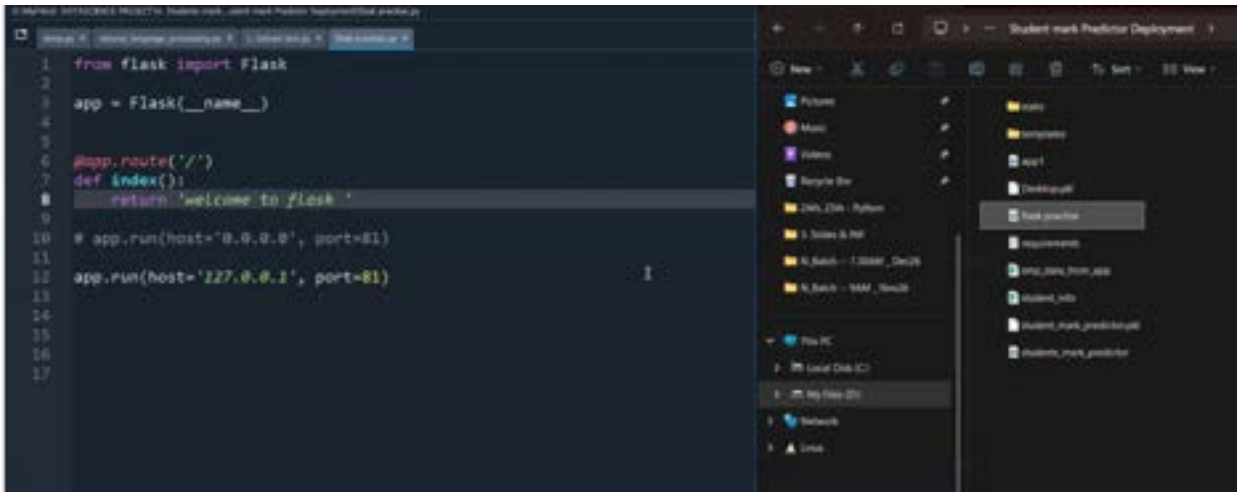
One more project :

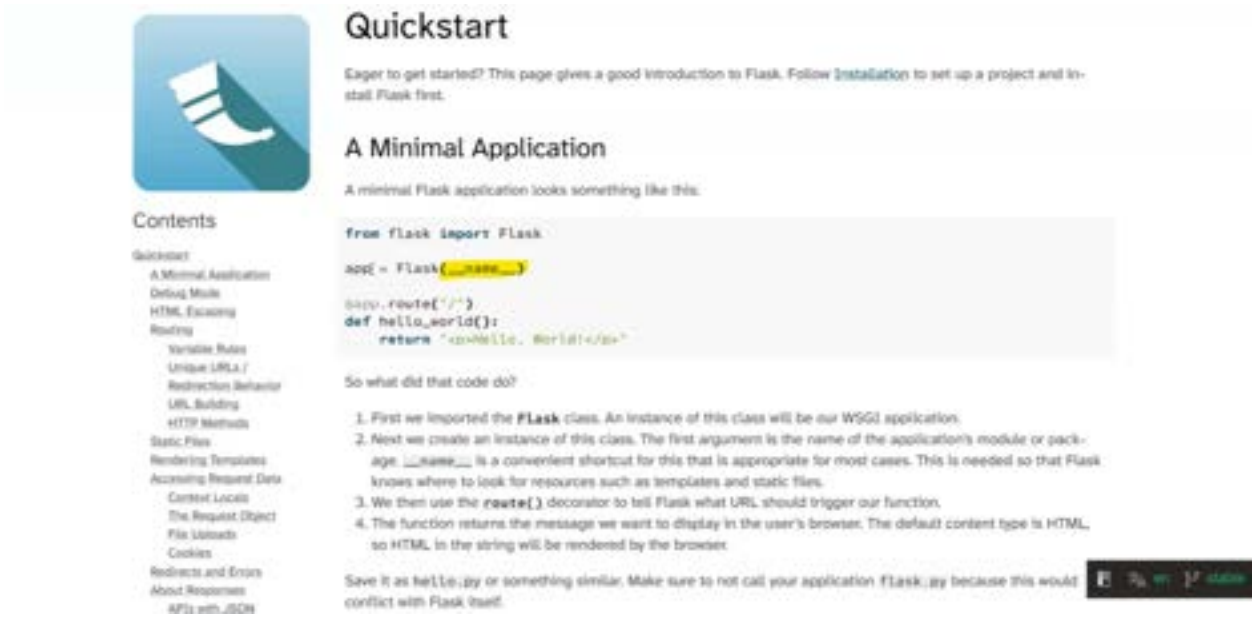


What is flask

The flask, Django and fastAPI this are all powerful frameworks where we can create frontend and all the deployments done by using flask,django,fastapi


Naresh IT
flask quickstart
AI Mode
All
Videos
Images
Forums
Shopping
Short videos
More
Flask Documentation
https://flask.palletsprojects.com › stable › quickstart
Quickstart — Flask Documentation (3.1.x)
To run the application, use the flask command or python -m flask . You need to tell the Flask where your application is with the --app option. Read more
Flask Documentation
https://flask.palletsprojects.com › quickstart
Quickstart — Flask Documentation (3.1.x)
Quickstart¶. Eager to get started? This page gives a good introduction to Flask. Follow Installation to set up a project and install Flask first. Read more


from flask import Flask
app = Flask(__name__)
@app.route('/')
def index():
return 'welcome to flask '
app.run(host='0.0.0.0', port=81)
app.run(host='127.0.0.1', port=81)



Quickstart

Eager to get started? This page gives a good introduction to Flask. Follow Installation to set up a project and install Flask first.

A Minimal Application

A minimal Flask application looks something like this:

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello_world():
    return '<p>Hello, World!</p>'
```

So what did that code do?

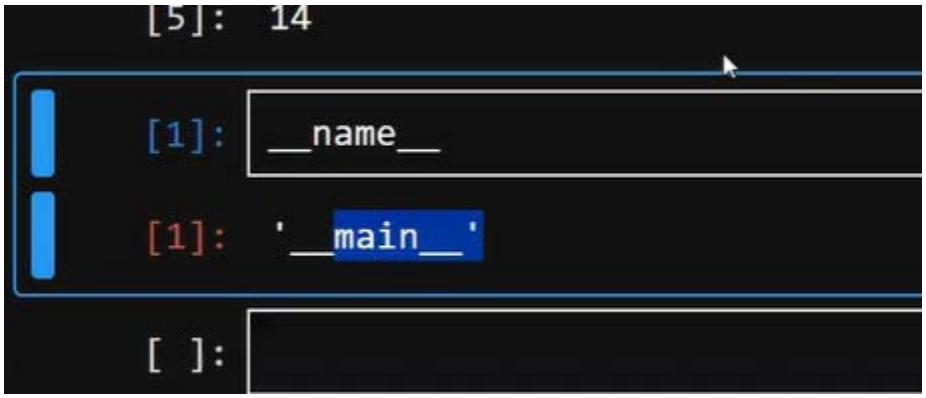
1. First we imported the **Flask** class. An instance of this class will be our WSGI application.
2. Next we create an instance of this class. The first argument is the name of the application's module or package. `__name__` is a convenient shortcut for this that is appropriate for most cases. This is needed so that Flask knows where to look for resources such as templates and static files.
3. We then use the **route()** decorator to tell Flask what URL should trigger our function.
4. The function returns the message we want to display in the user's browser. The default content type is HTML, so HTML in the string will be rendered by the browser.

Save it as `hello.py` or something similar. Make sure to not call your application `flask.py` because this would conflict with Flask itself.

Contents

- Quickstart
  - A Minimal Application
  - Debug Mode
  - HTML Escaping
  - Routing
    - Variable Rules
    - Unique URLs / Redirection Behavior
    - URL Building
    - HTTP Methods
  - Static Files
  - Rendering Templates
  - Accessing Request Data
    - Context Locals
    - The Request Object
    - File Uploads
    - Cookies
  - Redirects and Errors
  - About Responses
    - APIs with JSON

Here the -name- is the main function



All the websites routes with forward slash like below

temp.py | natural_language_processing.py | 1. Extract text.py | flask practise.py

```python
from flask import Flask

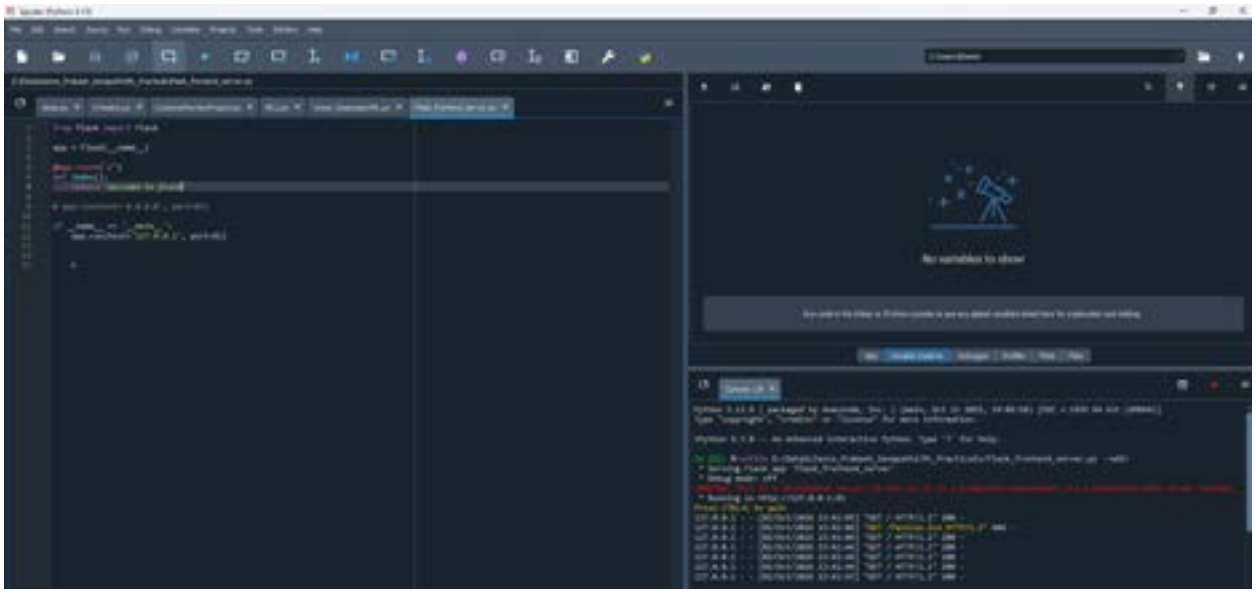
app = Flask(__name__)


@app.route('/')
def index():
    return 'welcome to flask '

# app.run(host='0.0.0.0', port=81)

app.run(host='127.0.0.1', port=81)
```

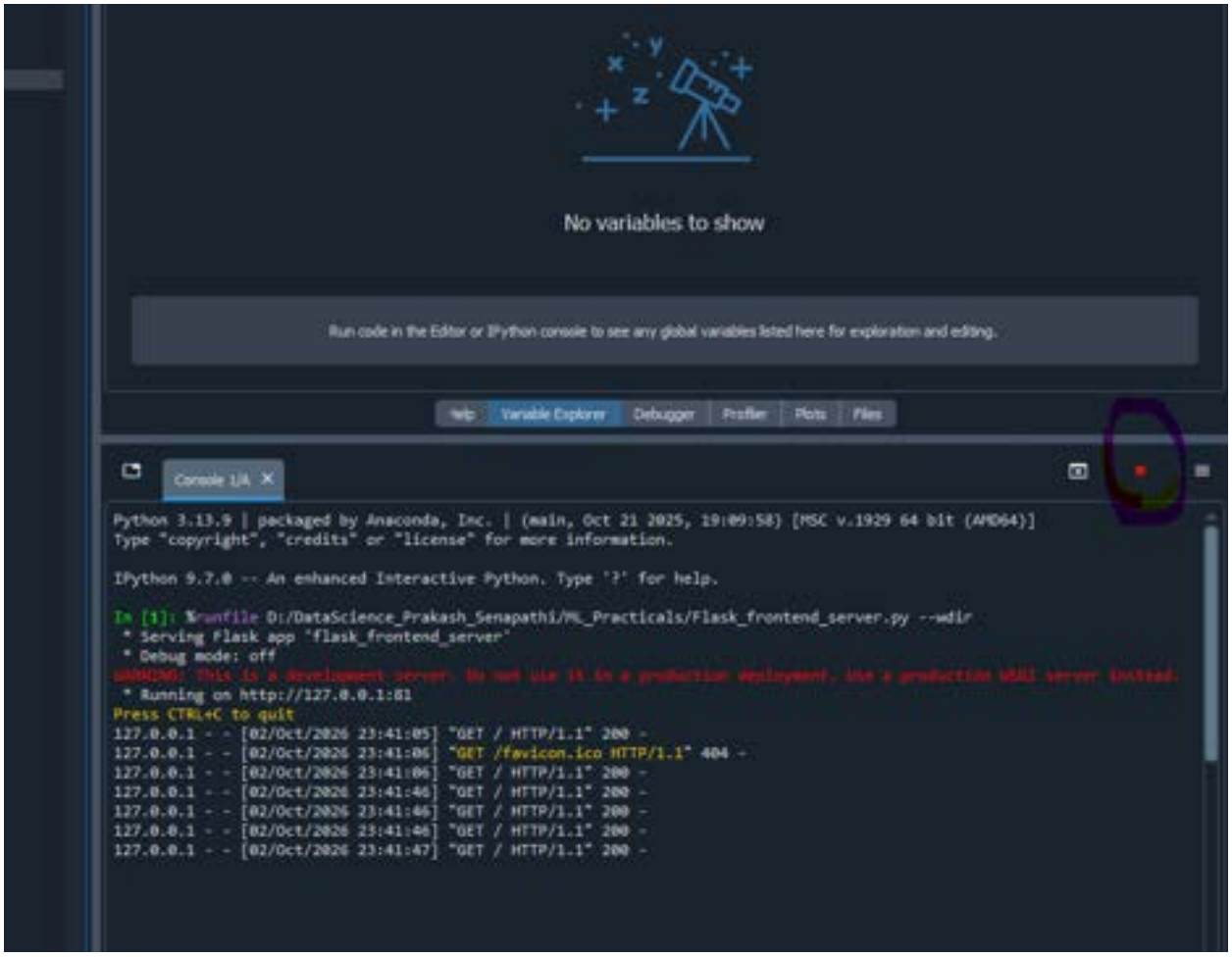
Here the function index calling the welcome to flask





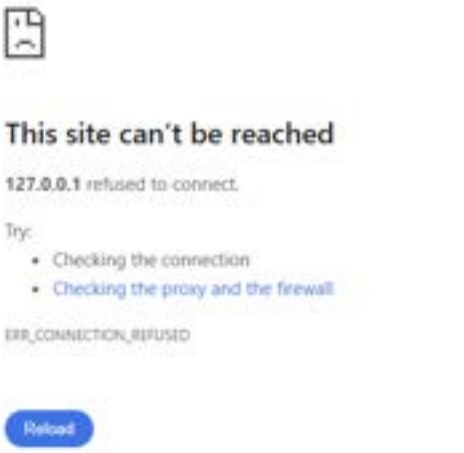
welcome to flask

Now we are stopping the service by hitting red button



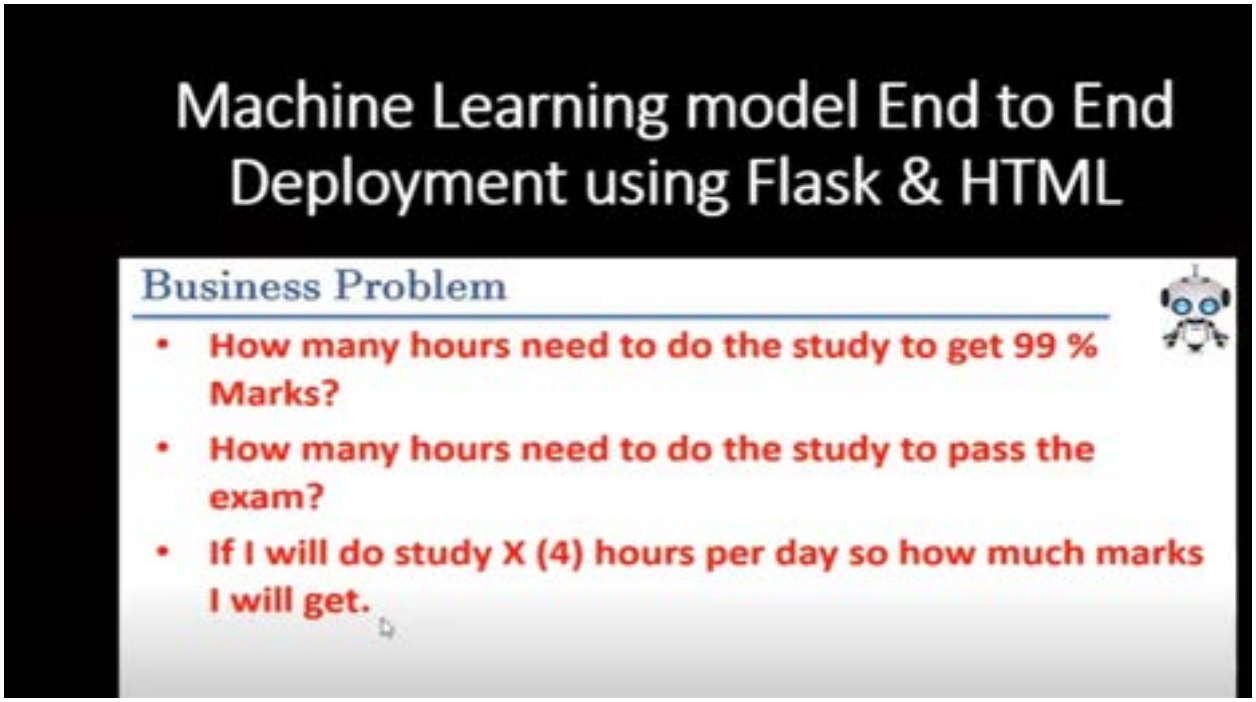
Now in server url we are not able to see





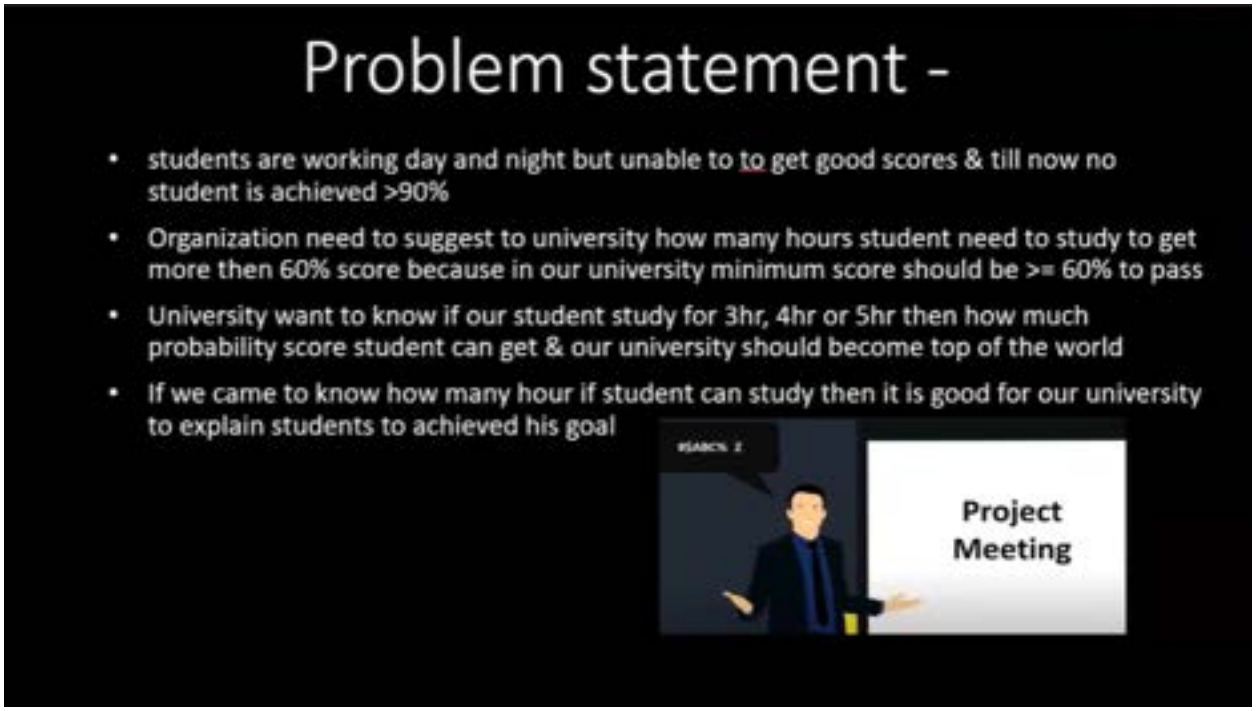
This is called server is down

01-07-2026	simple linear regression -2	train the model using more attribute irrelation overfitting , technique reduce overfiiti pca | cross validation | ensamble learning | regualrization | drop out the neuron train the model less attribute underfitting | high bias low varian .score(x_train, y_train) .score(x_test, y_test) bias - 094, varince - 0.98, score , r2 - 0.9
02-07-2026	simple linear regression -3	frontend pickle -- serialize which convert all ml code to byte file entire python writeen using write binary front end read read the file how to integrate backedn to fronted using pickle file we completed simple linear regression model flask web framework | flash running on host & Port you must need to purchase godaddy subscription
03-07-2026	end to end project	



The flash running on the host

Ex: There is a university and it has departments for this university not even one student scored greater than 90% and minimum pass mark is 60% aand the company want tobuid the tracking system system daily study hours in order to bild they contacted one company called [nit.com](nit.com)



AMX Software pvt Ltd-

- AMX ceo said to solve this problem statement need to discuss with the project team & also need to understand what is the technical requirement to solve this problem
- CEO discussed with project team explaining we got new client called o.x. university & and their requirement is to predict how many need to spend on study to get the percentage
- How to solve this problem and can we build some end to end application
- One team explained we can create using android application or developed a website is the solution for this problem. CEO said then we have to write if else condition and all company is doing this.
- to solve this problem we required Artificial Intelligence system
- CEO had discussion with datascince team one employee explained to required students history data from the university
- To know the prediction we need to students data we need to know last exams average per day how many hours student has spend for studying
- Finally the agreement will sign between university and organization





==== 2nd

how contract sign happend amoung (client - organzation)

cost factor deciden by both

experice -- 60l, 70lpa, you can ask any pakca geeksforgeeks

what if project cost it self 1cr

>50LPA

do you have any budgest limit

After the contract is completed then how the university share the data.
Step1: the university will not give production DB because if it down then it will loss of joining students
Step2: Instead they will clone the data called application server earlier is called production server

every webiste in world connected to -- production server

if in oragnzation 1000 employ are working

only 10 emply got acces to production server

do not touch production server

990 employ can connect to application server

producton server -- prod server

application server -- app server

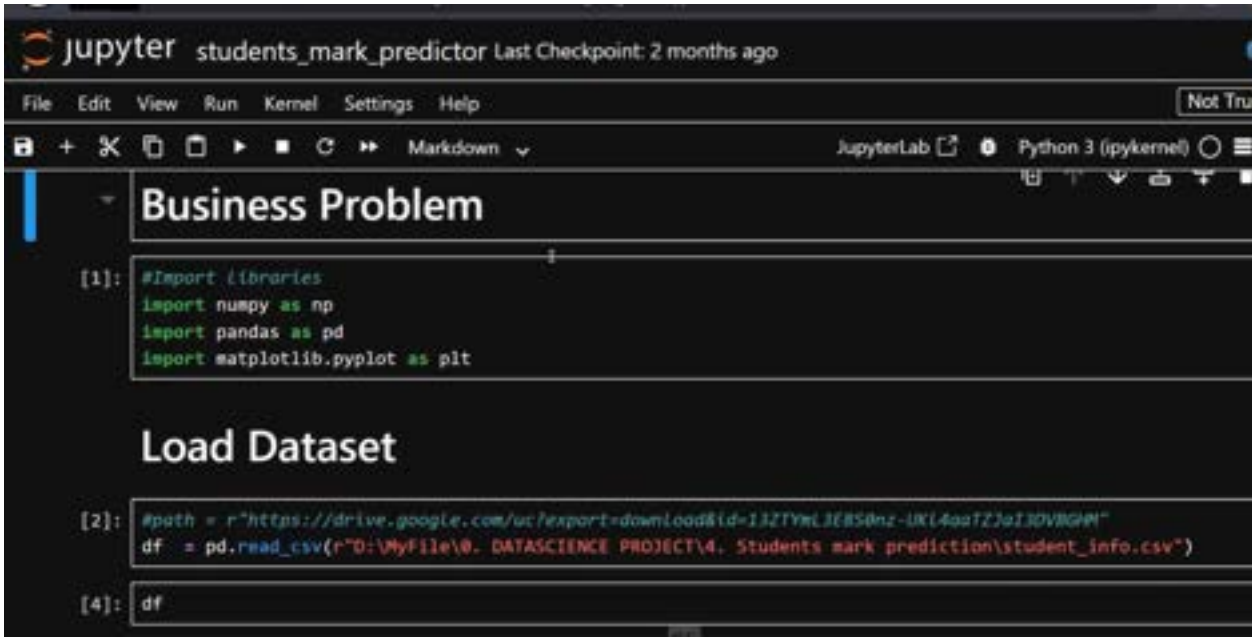
990 employ can connect to application server

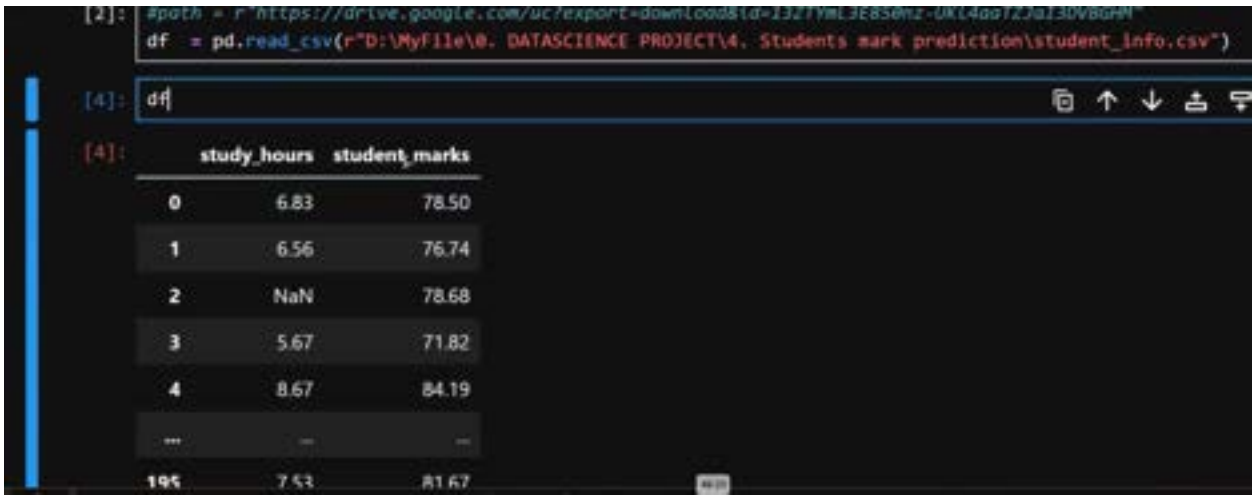
producton server -- prod server (names are difference)
application server -- app server

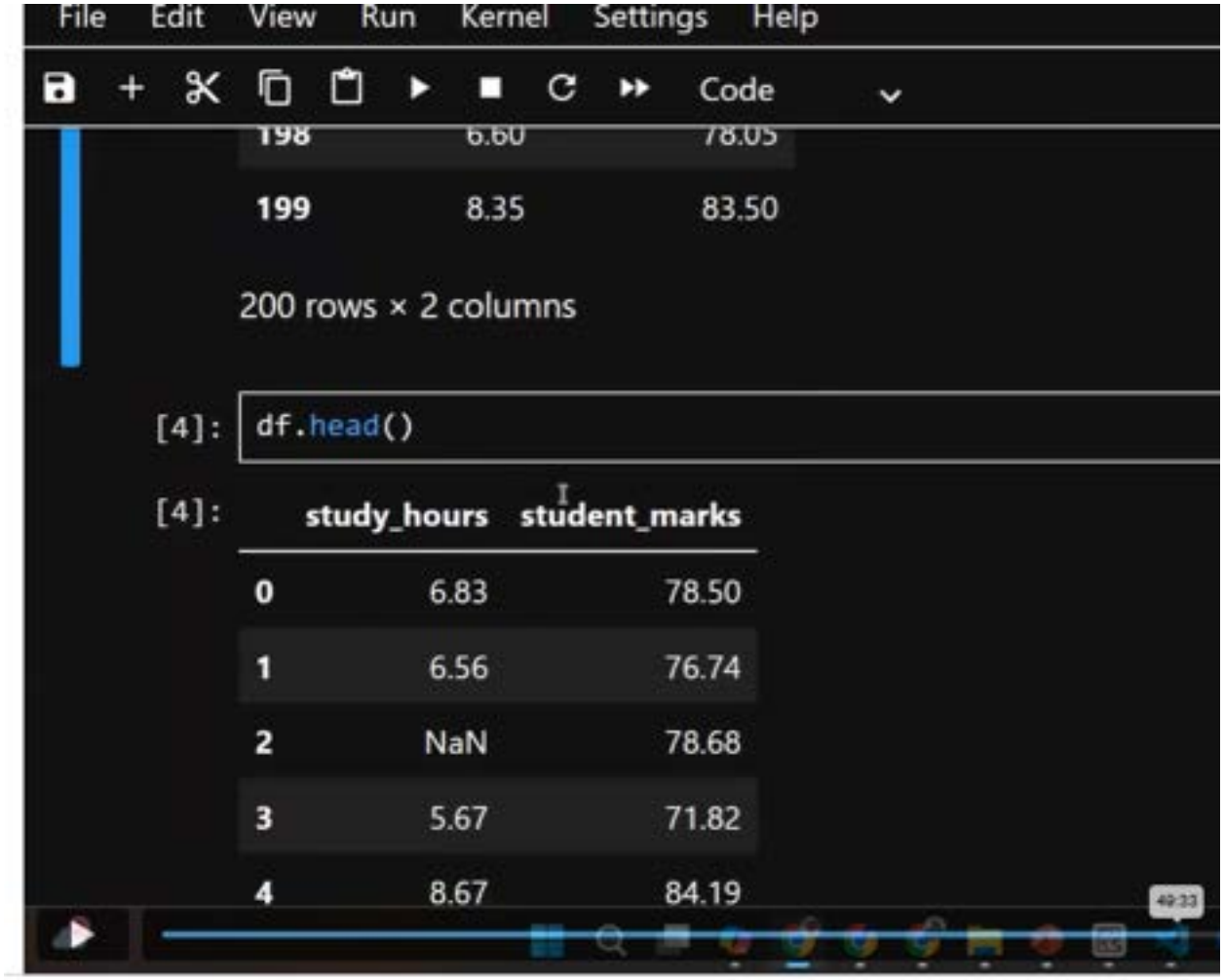
any project for everycompney who ever emplove work 1st they are working application server onl

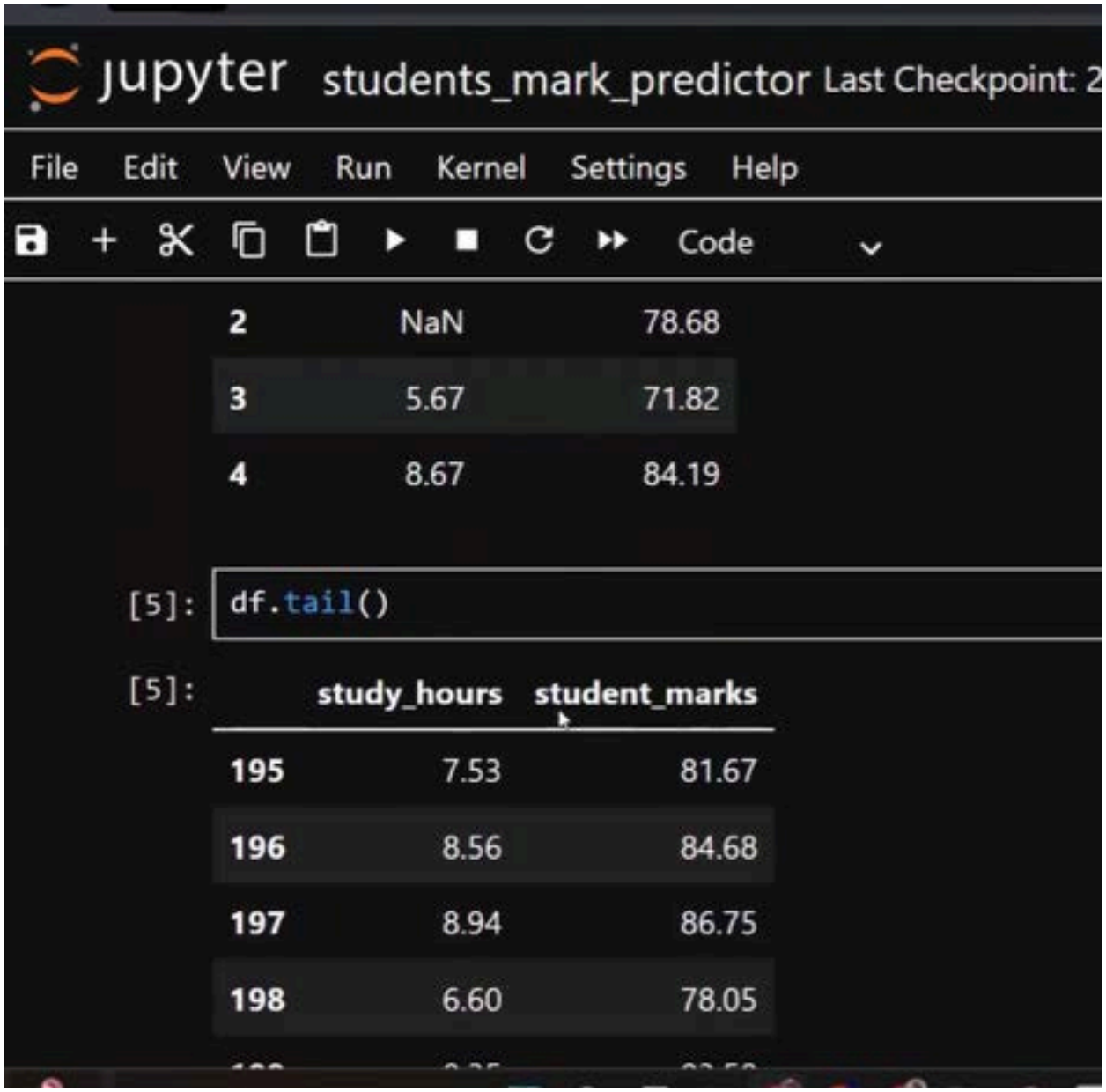
Q)What are the challenges faced while deploying code from application server to production server

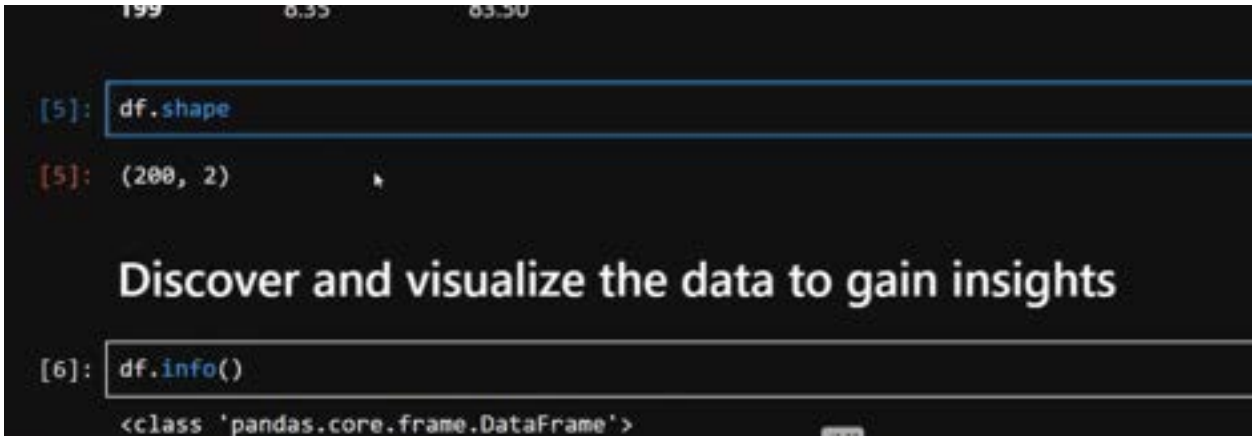
A)






File Edit View Run Kernel Settings Help
Code
198 6.60 78.05
199 8.35 83.50
200 rows × 2 columns
[4]: df.head()
[4]:
study_hours student_marks
0 6.83 78.50
1 6.56 76.74
2 NaN 78.68
3 5.67 71.82
4 8.67 84.19


jupyter students_mark_predictor Last Checkpoint: 2
File Edit View Run Kernel Settings Help
Code
2 NaN 78.68
3 5.67 71.82
4 8.67 84.19
[5]: df.tail()
[5]:
study_hours student_marks
195 7.53 81.67
196 8.56 84.68
197 8.94 86.75
198 6.60 78.05


[5]: df.shape
[5]: (200, 2)
Discover and visualize the data to gain insights
[6]: df.info()
<class 'pandas.core.frame.DataFrame'>

Discover and visualize the data to gain insights

```
[6]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 2 columns):
 #   Column         Non-Null Count  Dtype  
---  ------         --------------  -----  
 0   study_hours    195 non-null    float64
 1   student_marks  200 non-null    float64
dtypes: float64(2)
memory usage: 3.3 KB
```

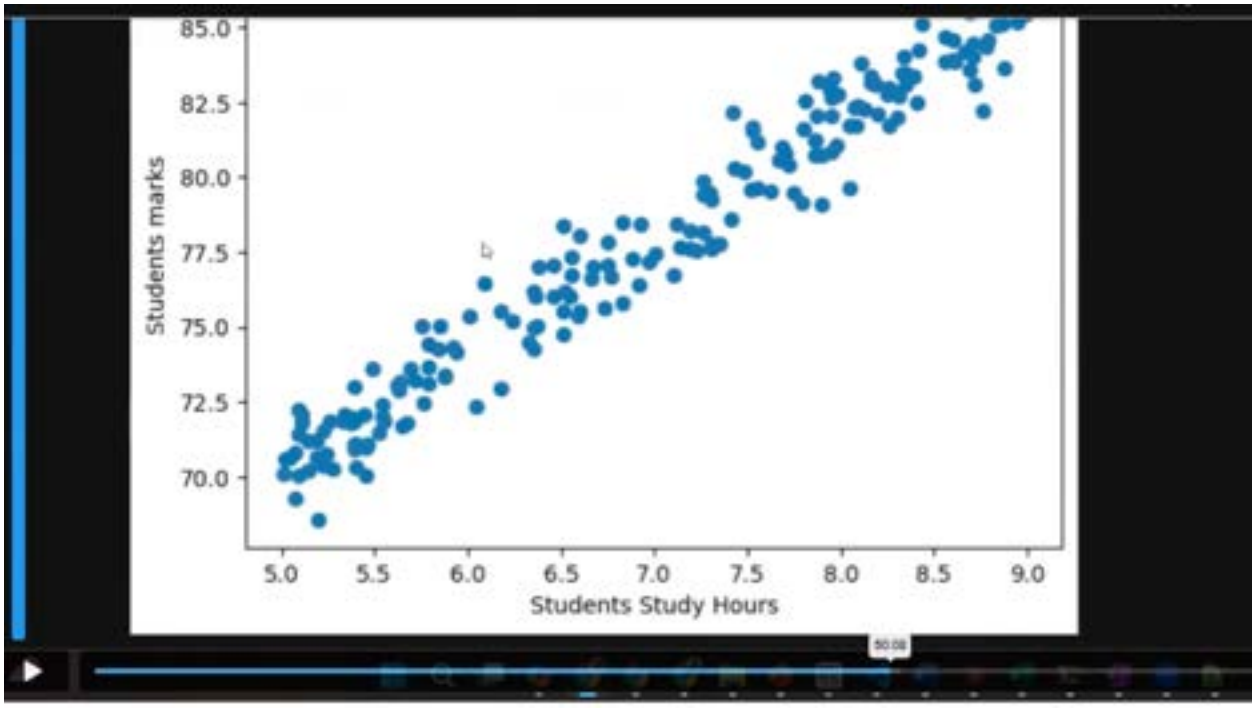
Descriptive statistics

Descriptive statistics

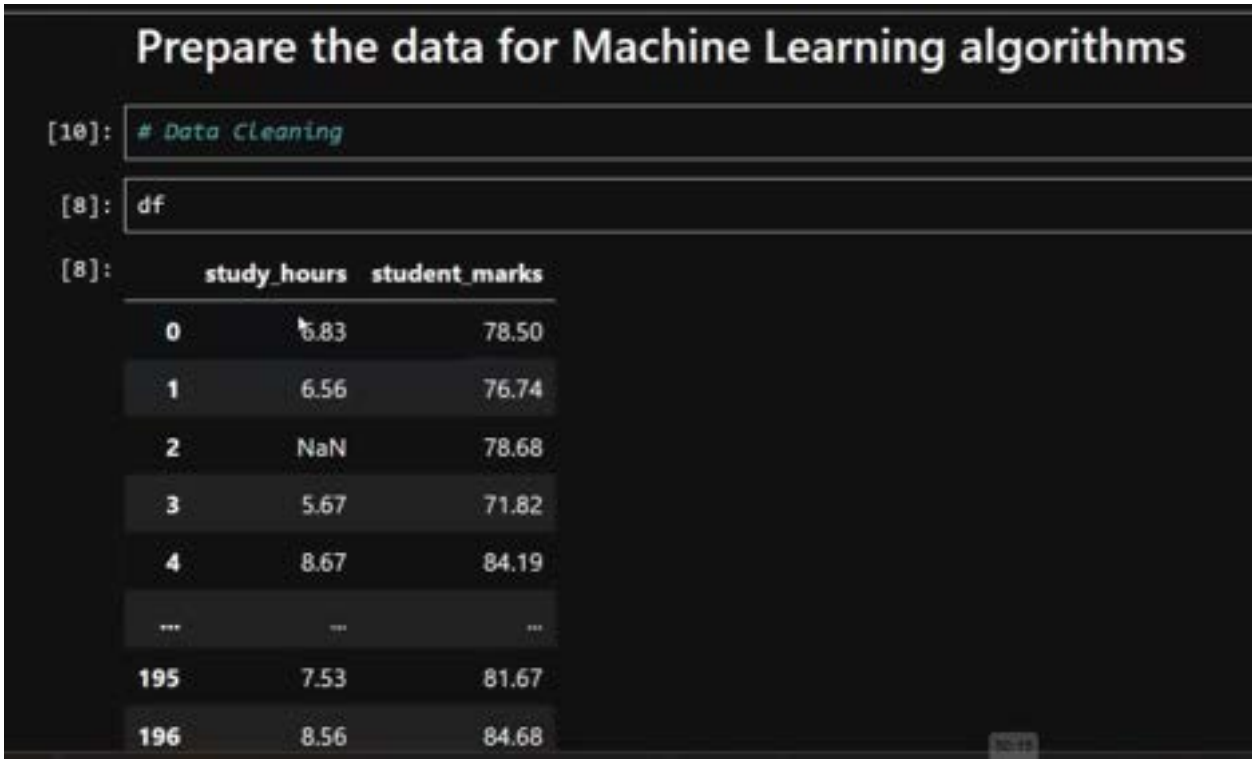
```
[7]: df.describe()
```

[7]:

	study_hours	student_marks
count	195.000000	200.00000
mean	6.995949	77.93375
std	1.253060	4.92570
min	5.010000	68.57000



This is positive correlation



```
[12]: df.isnull().sum()

[12]: study_hours      5
      student_marks    0
      dtype: int64

 [6]: df.mean()

 [6]: study_hours       6.995949
      student_marks    77.933750
      dtype: float64

 [7]: df2 = df.fillna(df.mean())

 [8]: df2.isnull().sum()

 [8]: study_hours      0
      student_marks    0
      dtype: int64

 [9]: df2.head()
```

[9]:

	study_hours	student_marks
0	6.830000	78.50
1	6.560000	76.74
2	6.995949	78.68
3	5.670000	71.82
4	8.670000	84.19

[13]:

```
df.head()
```

[13]:

	study_hours	student_marks

```
 1   student_marks  200 non-null    float64
dtypes: float64(2)
memory usage: 3.3 KB
```

[20]:
```
# split dataset
```

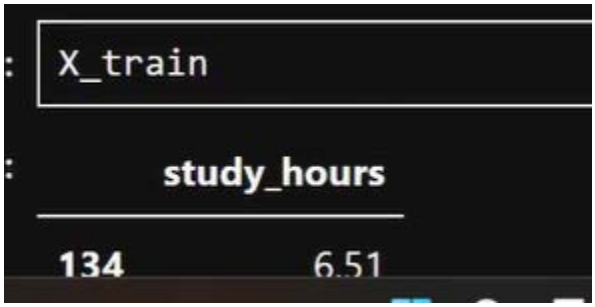
[14]:
```
X = df2.drop("student_marks", axis = "columns")
y = df2.drop("study_hours", axis = "columns")
print("shape of X = ", X.shape)
print("shape of y = ", y.shape)
```

```
shape of X =  (200, 1)
shape of y =  (200, 1)
```

[15]:
```
X
```

[15]:

	study_hours



```
21]: X_test.shape

21]: (40, 1)

22]: y_train
```

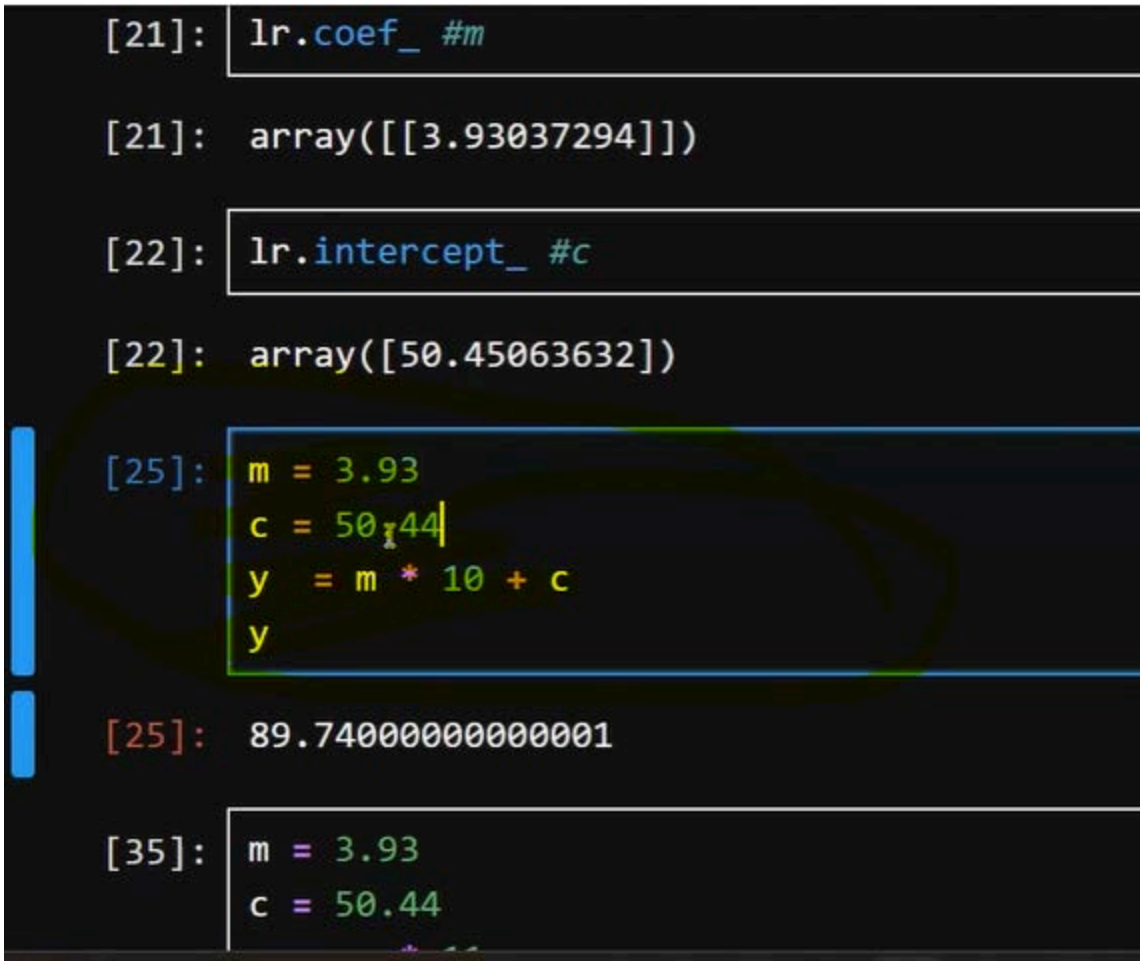
22]:

	student_marks
134	78.39
66	81.25

```
3]: y_test
```

3]:

	student_marks
18	82.50
170	71.18
107	73.25
98	83.64
177	73.64
182	86.99



The highlighted one need to deploy in frontend

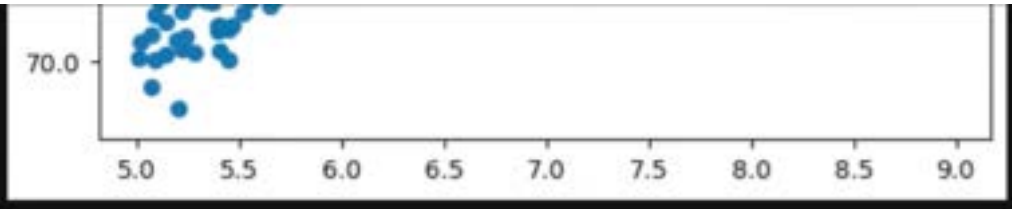
jupyter students_mark_predictor Last Checkpoint: 2 months ago

File Edit View Run Kernel Settings Help

Code JupyterLab

```
[31]: lr.score(X_test,y_test) # variance
[31]: 0.9521841793508594
[32]: lr.score(X_train,y_train) # bias
[32]: 0.9584528455152638
[41]: plt.scatter(X_train,y_train)
[41]: <matplotlib.collections.PathCollection at 0x2c389a8c050>
```






```
3]: plt.scatter(X_test, y_test)
    plt.plot(X_train, lr.predict(X_train), color = "r")
3]: [<matplotlib.lines.Line2D at 0x1926c9c8cd0>]
```



03-07-2026	student mark prediction system	we discussed in oragnziation how the contract signing from organization to customer customer == client = stack holder | use case = how many hr student need to study to get 90% marks simple lineare regression model	joblib	joblib librar which help serialization & deseriztion to pass pickle file to pass to frontend 50 line code -- convert to .pkl format	we wrote code in python but code will binary file we also testing front end using flask	production server & application server every website can connect prod server	all organization employ can connect to app server oragtnzation employ connect to application server

Note : When ever we are deploying we need to put all the files in one folder

organization -- 1project - 1 manager
- multiple team member
  - backend dev
  - froend dev
  - page design
  - html
  - testing

what ever task done be each and every empoy in everyday manager must need to know

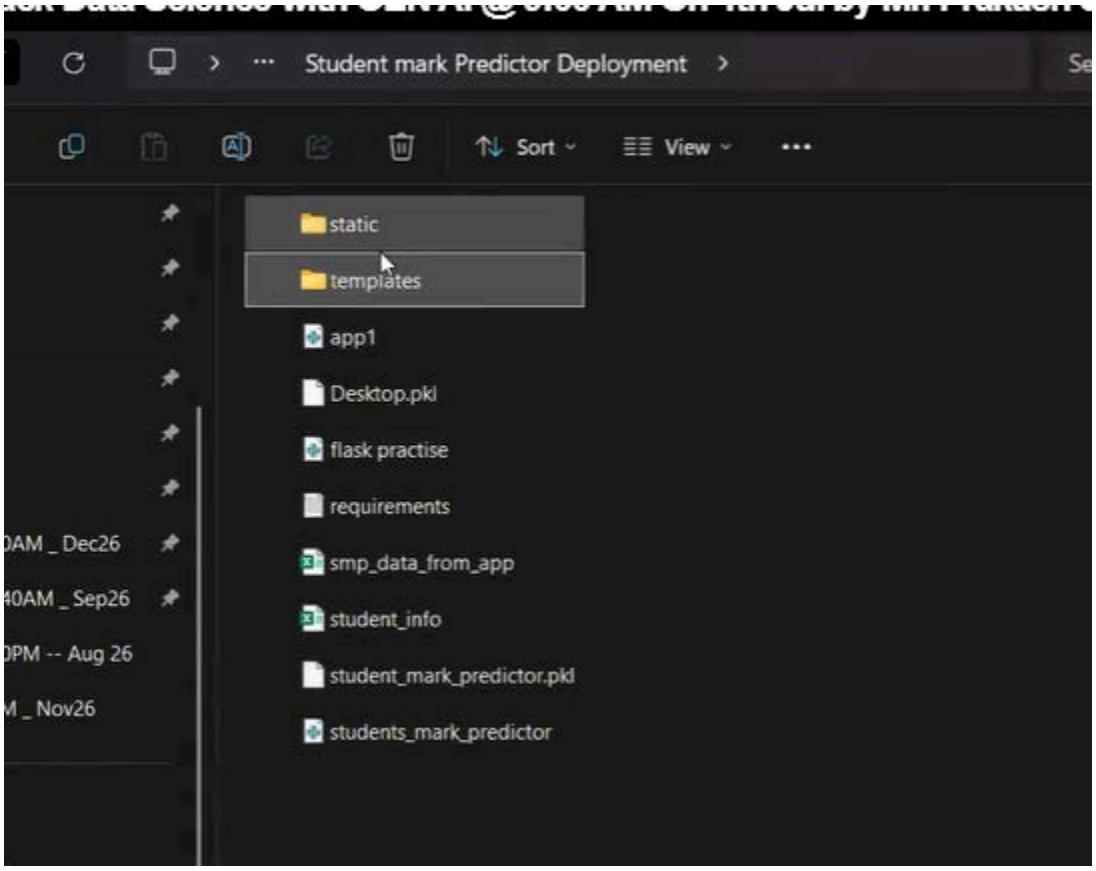
- if you keep all work in one folder
- manager will track every one work
  - github is implment
    - github action using cicd pipeline
    - mlops -- mlflow comes to the picture

In order to view the static pages in front end we need put in static folder like below is the static image



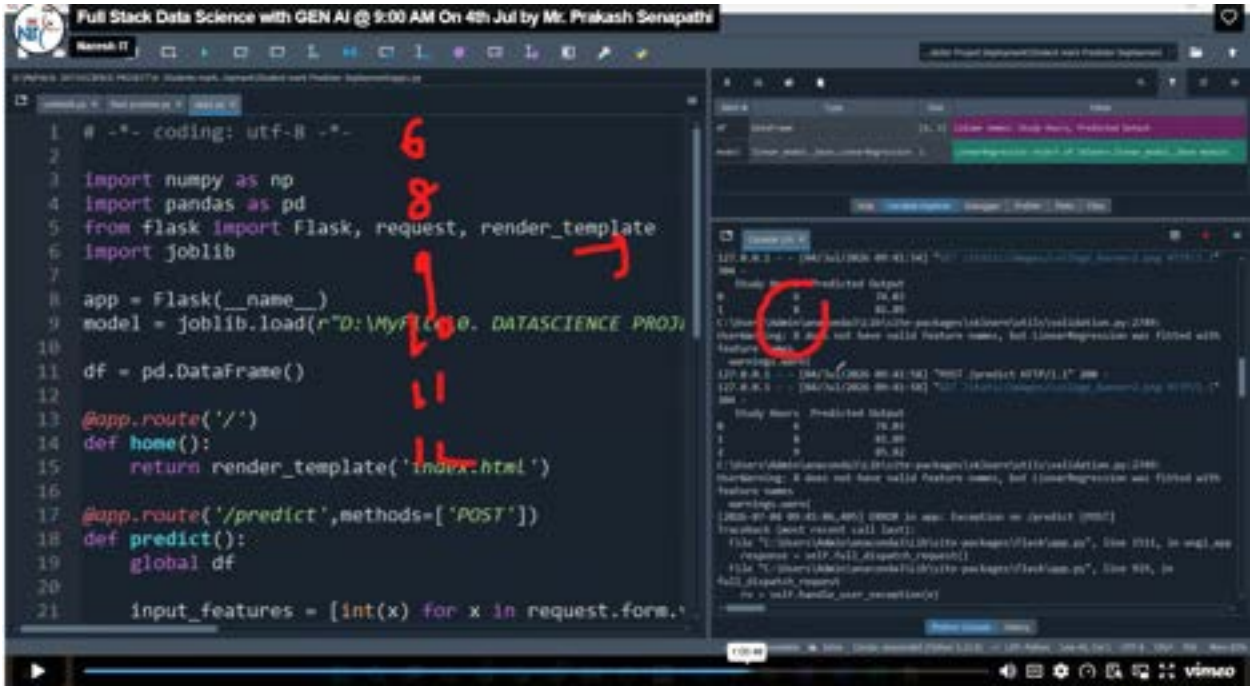
Template





Q)What are the challenges faced while deploying code from application server to production server

A) 1.Library version compatibility (If production server too old and application server is we have updated new version then it will not match)
2. We have to change all the code which act like old code system
3. Fire wall if we activate firewall we cannot open any other applications
4. If the data is too



In frontend any user enter 6 then after that DB is cuttoff

Ex: In banking application we can able to login if we are not log out automatically the session cut will expire because possibility of hacking they are doing
4.Any connection getting from out side that connection should be established for only 1 sec or 2 sec it means all our code must run in 1s or 2s that is why DSA and oops are required and time complexity and space complexity required
6. While deployment we should not apply open source model

1- lib version compatible
2- you have to change all the code which act like old code system
3- firewall & security
4- any connection come from outside that connect should establist only 1ese o r2 sec
5- all your code must should run in
6- while deployment should not apply open source model organztaion paid software

after complet ml -- ml deployment

btech -- everyday bring placment
last sem -- placem t

organzation -- everyday deploymen possible

if you have any html page, do not use stremlit, gradio
alswasy flask or fastapi

If we do not have html pages we have css or javascript then we can go for streamlit

Multiple Linear Regression 👍

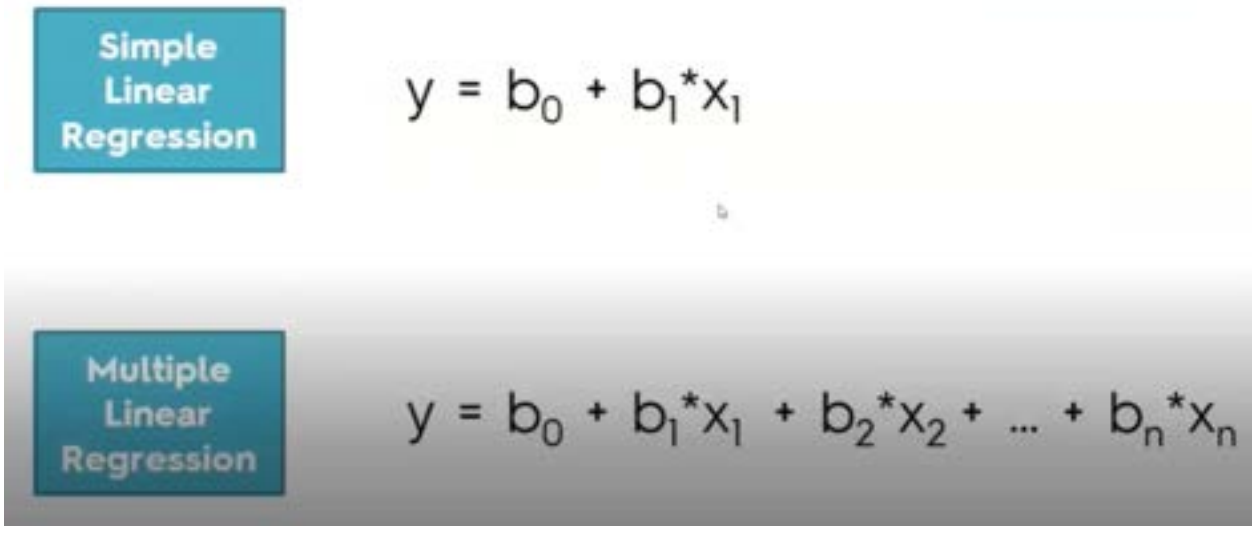
Single linear regression y = mx+c
Multiple linear regression = y = m1x1+m2x2+m3x3+c

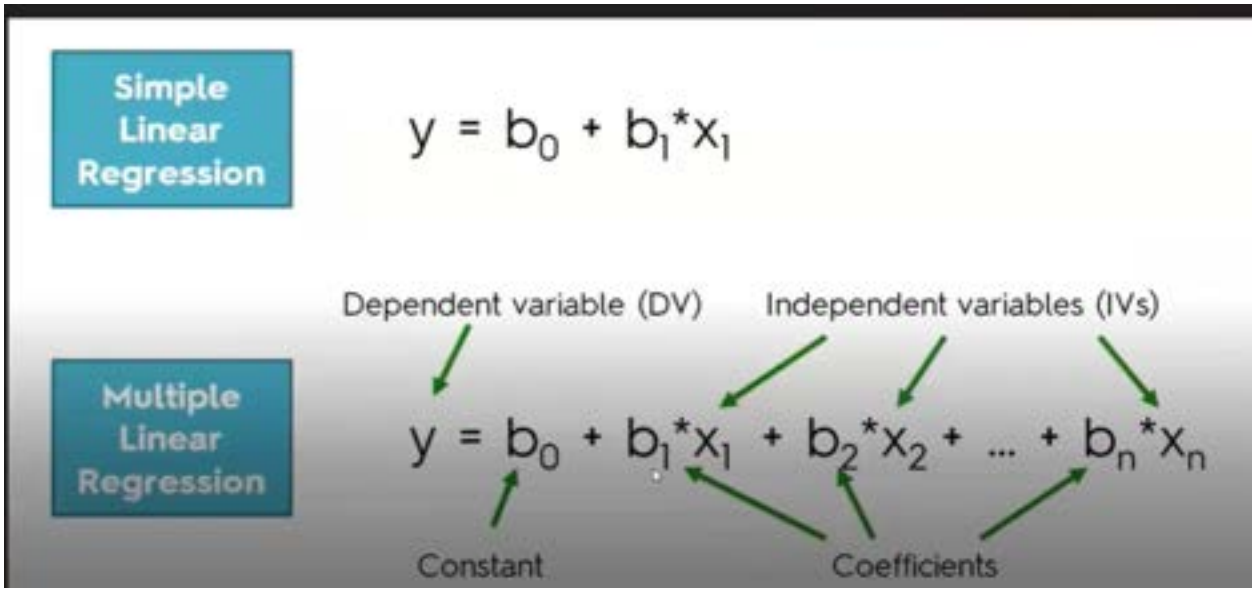
In simple linear regression X is independent variable and Y is dependent variable

Multiple linear module = y = m1x1+m2x2+m3x3+c

Organization cannot build single regression module it will build multiple regression module it means they will use many variables

---





Example : if our friend started digital marketing he spends money on digital marketing, promotion, research, state then friend got this profits

	A	B	C	D	E
1	DigitalMarketing	Promotion	Research	State	Profit
2	165349.2	136897.8	471784.1	Hyderabad	192261.83
3	162597.7	151377.59	443898.53	Bangalore	191792.06
4	153441.51	101145.55	407934.54	Chennai	191050.39
5	144372.41	118671.85	383199.62	Hyderabad	182901.99
6	142107.34	91391.77	366168.42	Chennai	166187.94
7	131876.9	99814.71	362861.36	Hyderabad	156991.12
8	134615.46	147198.87	127716.82	Bangalore	156122.51
9	130298.13	145530.06	323876.68	Chennai	155752.6
10	120542.52	148718.95	311613.29	Hyderabad	152211.77
11	123334.88	108679.17	304981.62	Bangalore	149759.96
12	101913.08	110594.11	229160.95	Chennai	146121.95
13	100671.96	91790.61	249744.55	Bangalore	144259.4
14	93863.75	127320.38	249839.44	Chennai	141585.52
15	91992.39	135495.07	252664.93	Bangalore	134307.35
16	119943.24	156547.42	256512.92	Chennai	132602.65
17	114523.61	122616.84	261776.23	Hyderabad	129917.04
18	78013.11	121597.55	264346.06	Bangalore	126992.93
19	94657.16	145077.58	282574.31	Hyderabad	125370.37
20	91749.16	114175.79	294919.57	Chennai	124266.9
21	86419.7	153514.11	0	Hyderabad	122776.86
22	76253.86	113867.3	298664.47	Bangalore	118474.03

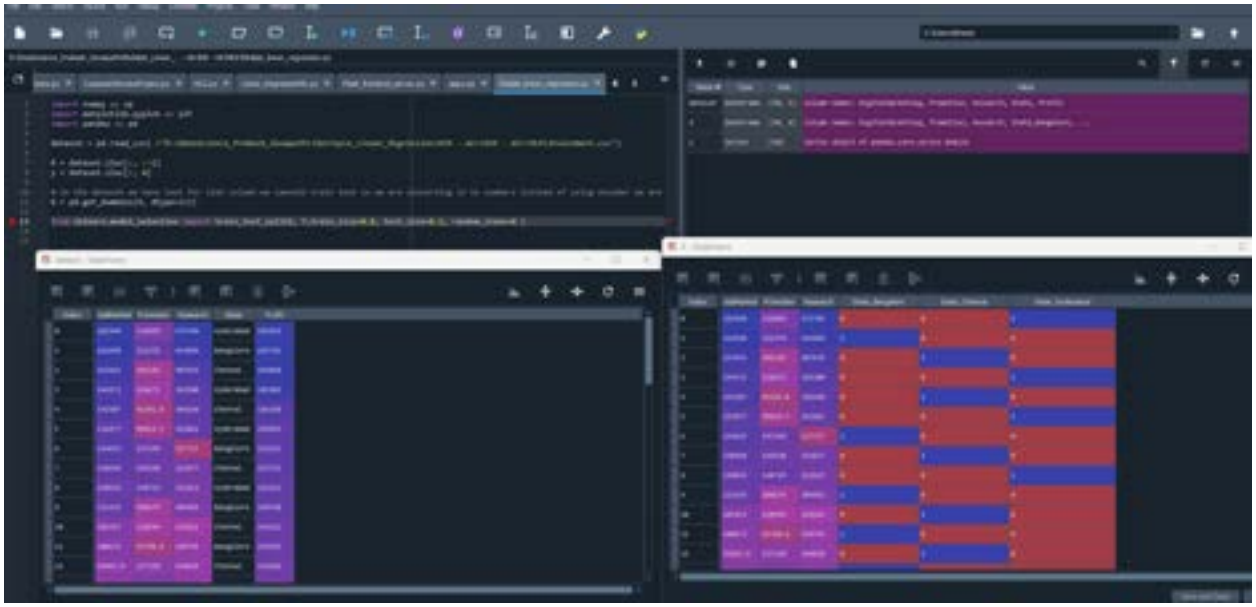
Now in covid h got losses now he is not able to spend money on all above he we spend money only on Digital marketing or any one in order to choose any one he needs data scientist because to know which is performing the best

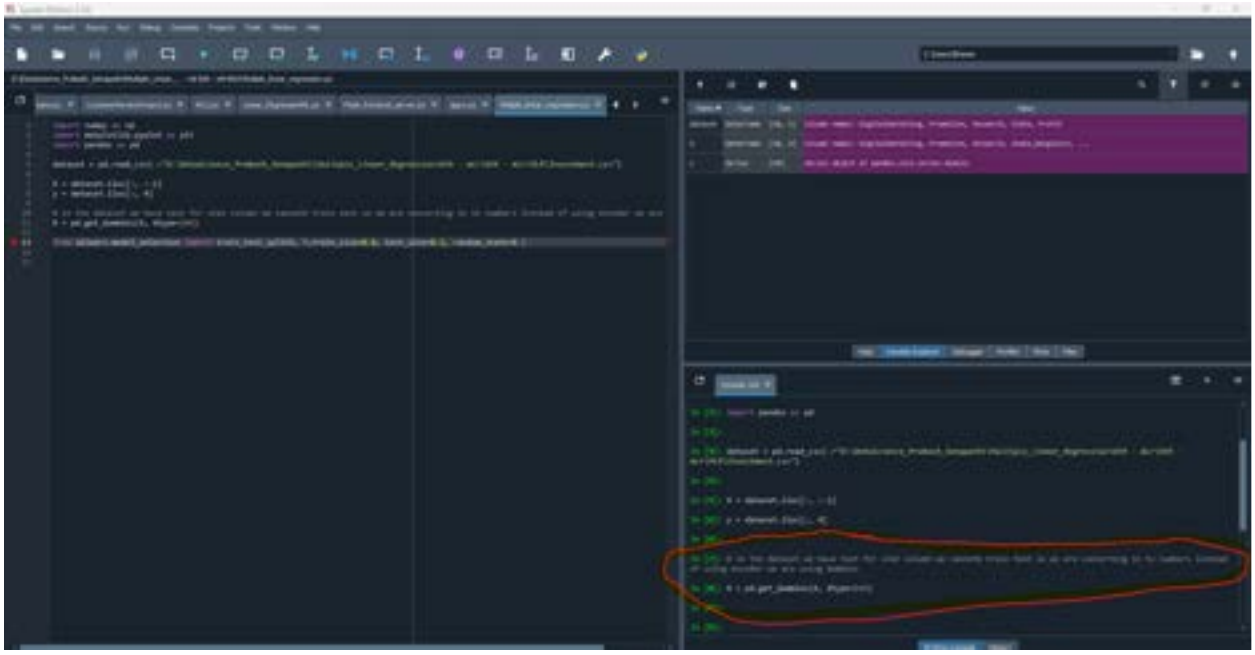
case study -- organzation spend investment on mutliple domain
ml developer or datascientist to suggest rather 3 attribte i dont want to spend the modeny
only 1 attribute -- how do you guide him

"Case Study: An organization invests across multiple domains. Instead of spending money on three attributes, how would you guide an ML developer or data scientist to focus investment on only one key attribute

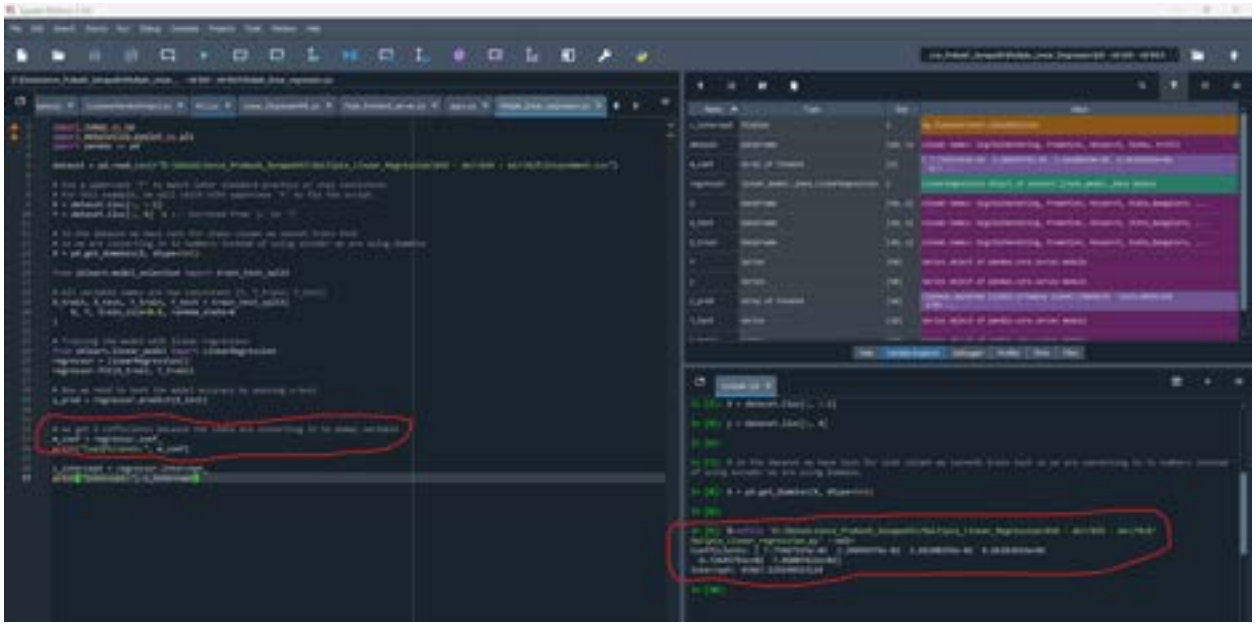
Now we are going to build MLR and statistical backward elimination

As a datascientist we need to define only 1 attribute out of 3 because of losses the client want to spent money on only 1 in order to remove other attributes there is some statistical approach required






Full Stack Data Science with GEN AI @ ₹500 All On 6th Jul by Mr. Prakash Senapathi
$y = x_1 + x_2 + x_3 + x_4$
Profit
Marketing Spend
Promotion Spend
Research Spend
State



1- Mlr model
2- 3attribute you need to information 1 attribtue
statistical approach required
3- understand to the solution

FEATURE ELIMINATION (remove columns to give the best column)

Here is a clean, well-structured set of revision notes based on the concepts we just covered. You can easily copy and paste this into your notebook or markdown notes:

📚 Revision Notes: Multiple Linear Regression (MLR) & Feature Elimination

1. Core Equation of Multiple Linear Regression

In MLR, we predict a target variable (Y) using multiple input features ($x_1, x_2, x_3, \dots$):

$$Y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 x_3 + \beta_4 x_4 + \varepsilon$$

- Y **(Dependent Variable): The output we want to predict (e.g., *Profit*).**
-
- x_1, x_2, x_3, x_4 **(Independent Variables / Features): The input columns influencing the output (e.g., *Digital Marketing, Promotion, Research, State*).**

-
- β_0 **(Intercept / Baseline Profit): The expected value of** Y **when all input features (**x_1 **to** x_4**) are zero.**
-
- $\beta_1, \beta_2, \beta_3, \beta_4$ **(Coefficients): The impact each feature has on the output** Y**.**
-
- ε **(Error Term): The difference between the actual value and the predicted value.**
-

2. Handling Categorical Data (Text to Numbers)

- **Machine learning algorithms cannot process text values directly (e.g.,** ***Hyderabad, Bangalore, Chennai*** **in the** `State` **column).**
-
- **Solution: We use One-Hot Encoding (**`pd.get_dummies()`**) to convert text categories into numerical binary columns (**0 **and** 1**).**
-

3. Case Study Problem: 3 Attributes → 1 Attribute

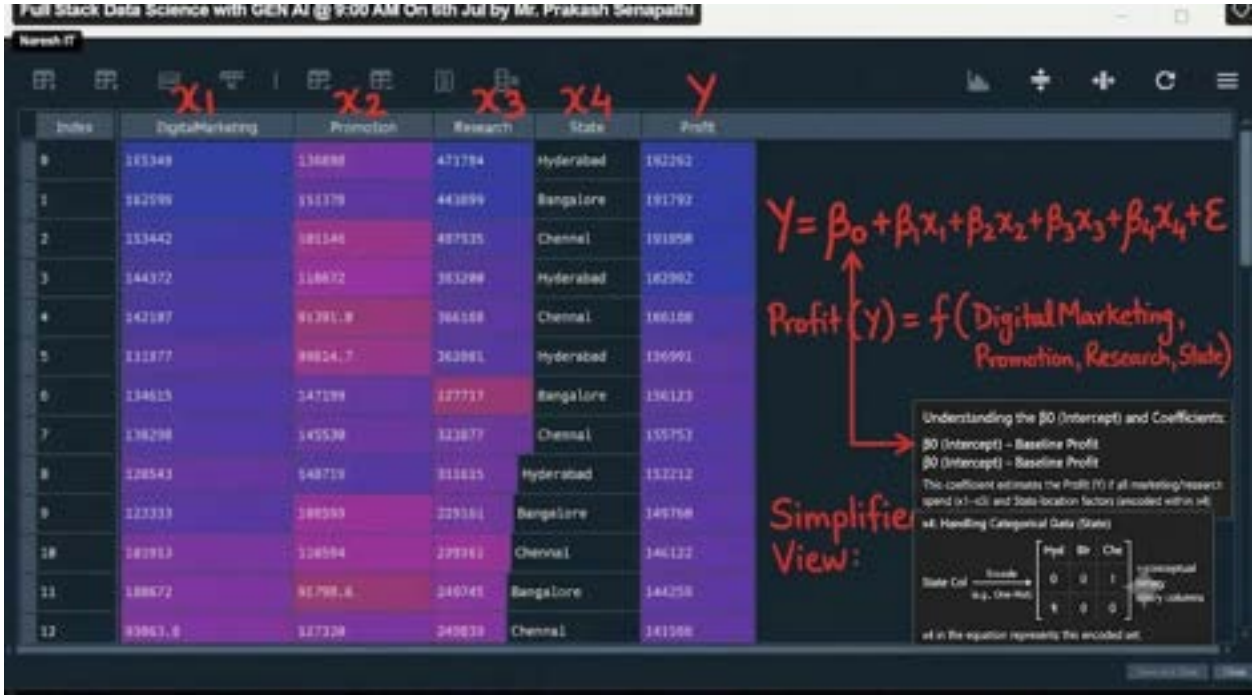
- **The Scenario: An organization is spending money across multiple domains/attributes and wants to optimize costs.**
-
- **The Goal: Instead of spending a budget on 3 attributes, find and keep only 1 key attribute that drives the result.**
-
- **The Approach: A statistical approach is required—we cannot simply guess. We use Feature Elimination to remove columns scientifically until the best one remains.**
-

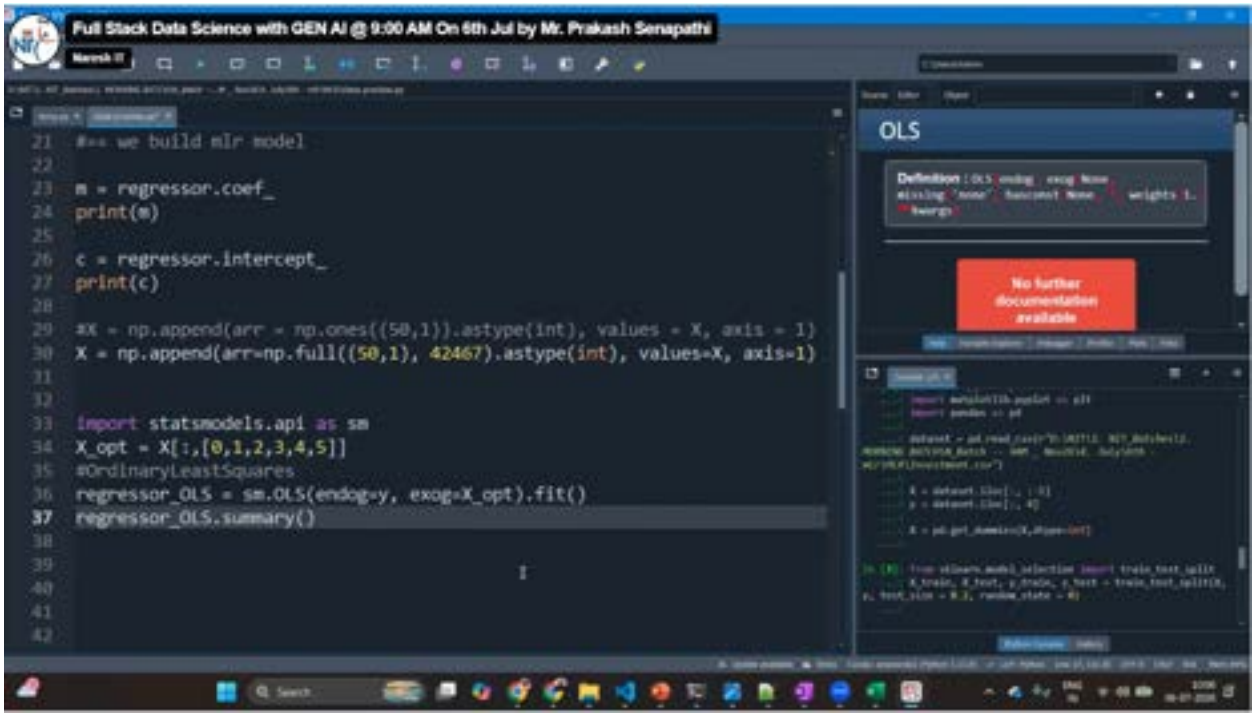
4. Key Solution Concept: Feature Elimination

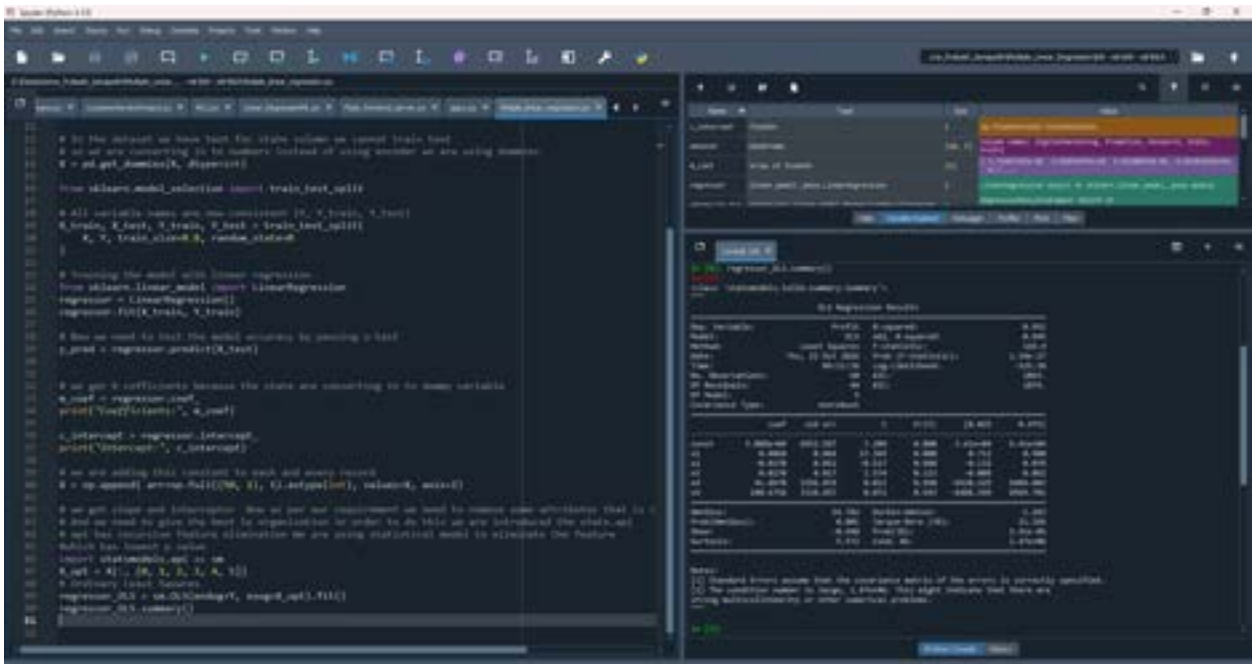
- **Definition: The process of systematically removing columns (features) one by one to find the most significant predictor.**
-
- **How it works:**
-

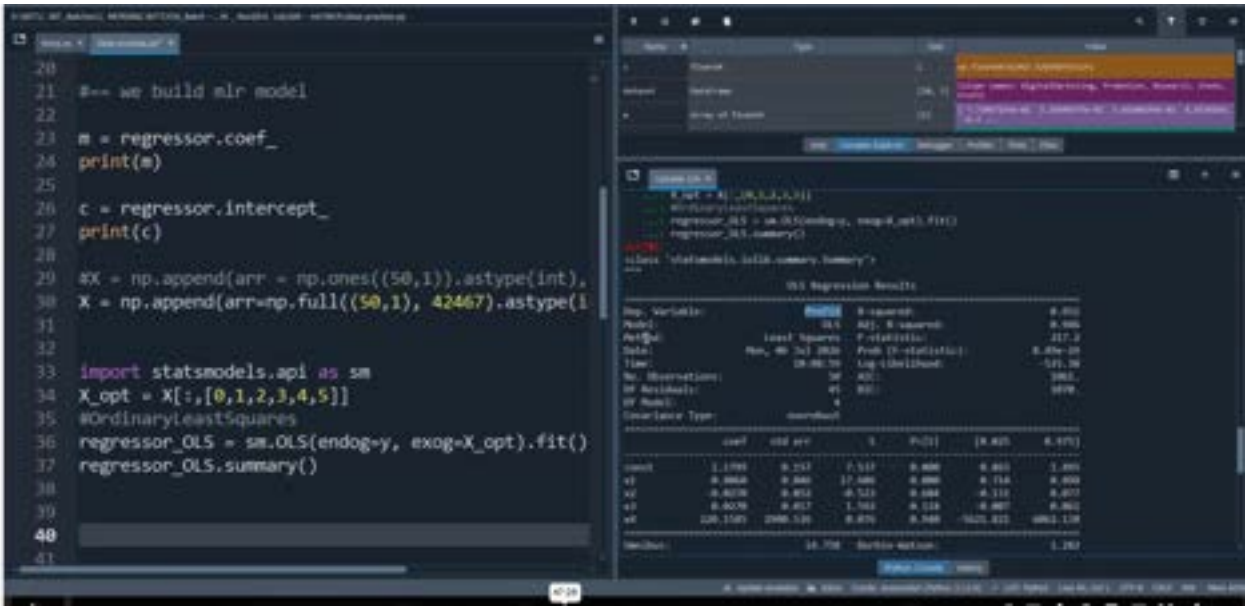
    1. **Train the model using all features.**
    2.
    3. **Check statistical metrics like p-values.**
    4.
    5. **Remove the feature with the highest p-value (least significance).**
    6.
    7. **Repeat until you narrow it down to the single best-performing attribute.**

-

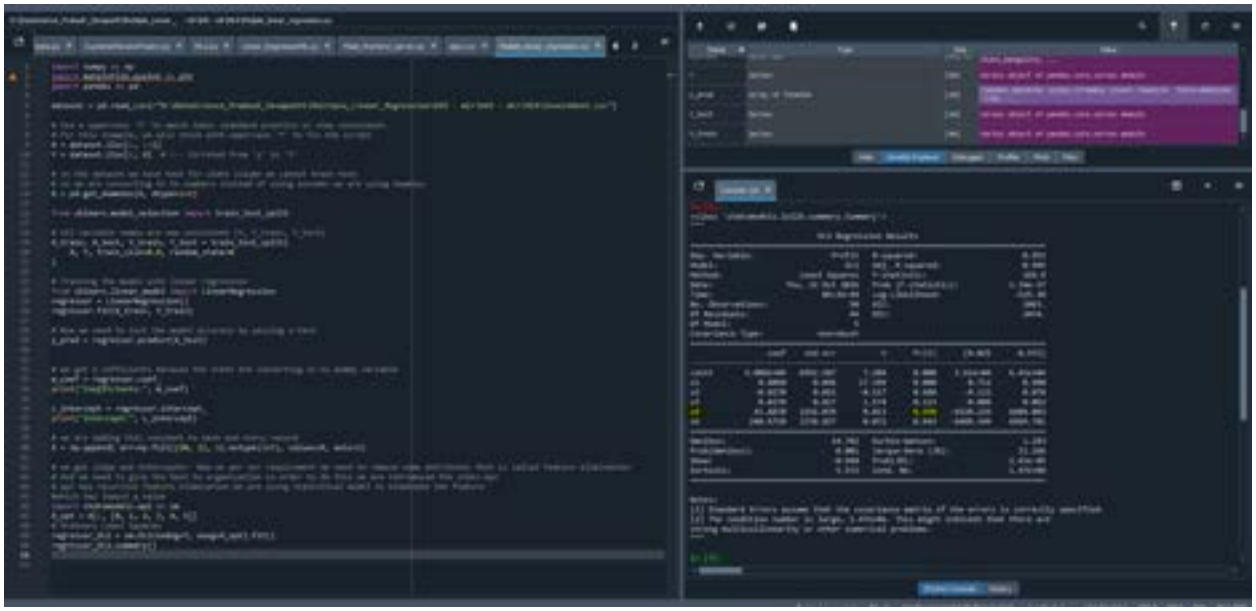

Full Stack Data Science with GEN AI @ 9:00 AM On 6th Jul by Mr. Prakash Senapathi
Naresh IT
X1
X2
X3
X4
Y
Index
DigitalMarketing
Promotion
Research
State
Profit
0 165349 136898 471784 Hyderabad 192262
1 162598 151378 443899 Bangalore 191792
2 153442 101546 407935 Chennai 191058
3 144372 118672 383200 Hyderabad 182902
4 142107 91391.8 366168 Chennai 166188
5 131877 99814.7 362861 Hyderabad 156991
6 134615 147199 127717 Bangalore 156123
7 130298 145530 323877 Chennai 155752
8 120543 148719 311835 Hyderabad 152212
9 123333 108599 229161 Bangalore 149768
10 101913 110594 299469 Chennai 146122
11 100672 91790.6 249745 Bangalore 144259
12 93863.8 127320 249839 Chennai 141586
Y = β0 + β1x1 + β2x2 + β3x3 + β4x4 + ε
Profit (Y) = f(Digital Marketing, Promotion, Research, State)
Understanding the β0 (Intercept) and Coefficients:
β0 (Intercept) – Baseline Profit
β0 (Intercept) – Baseline Profit
Simplified View:


Full Stack Data Science with GEN AI @ 9:00 AM On 6th Jul by Mr. Prakash Senapathi
Naresh IT
#== we build mlr model
m = regressor.coef_
print(m)
c = regressor.intercept_
print(c)
#X = np.append(arr = np.ones((50,1)).astype(int), values = X, axis = 1)
X = np.append(arr=np.full((50,1), 42467).astype(int), values=X, axis=1)
import statsmodels.api as sm
X_opt = X[:,[0,1,2,3,4,5]]
#OrdinaryLeastSquares
regressor_OLS = sm.OLS(endog=y, exog=X_opt).fit()
regressor_OLS.summary()
OLS
Definition : OLS(endog, exog=None, missing='none', hasconst=None, weights=1., **kwargs)
No further documentation available

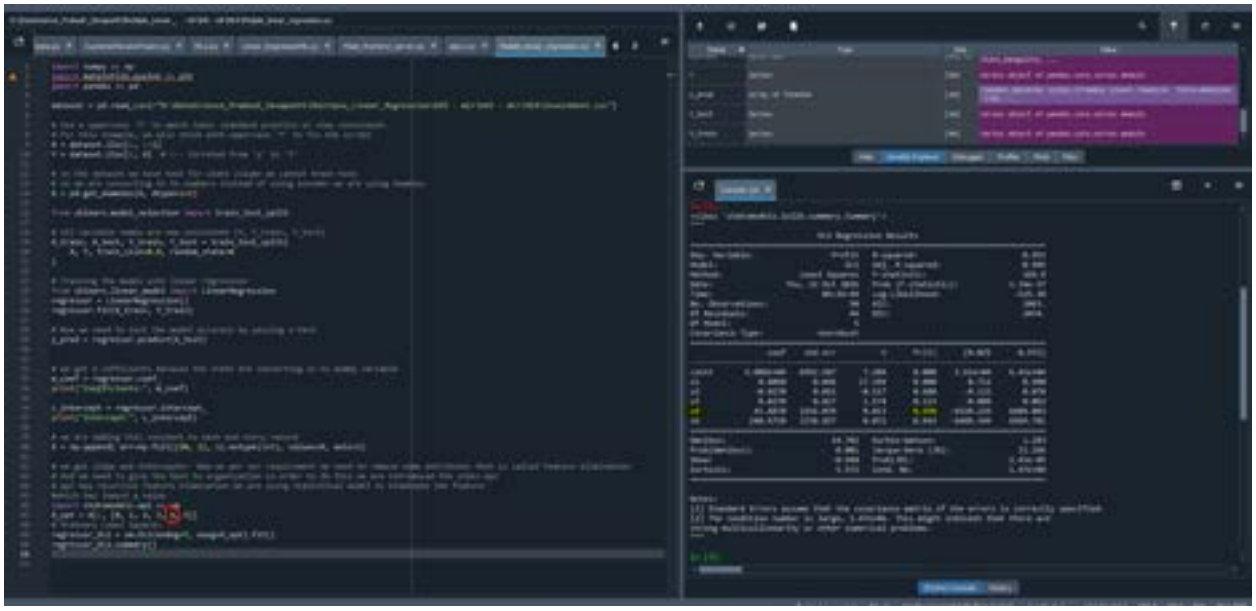




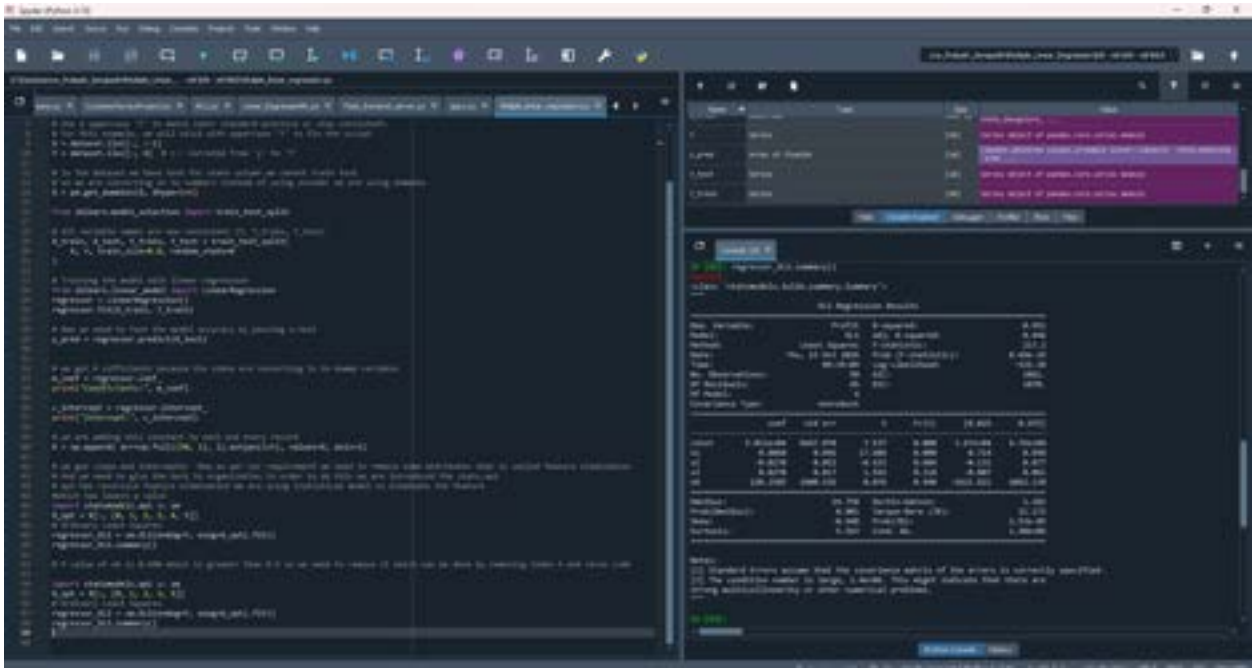
This sis a good model because Rsquare is greater than adjusted RSquare

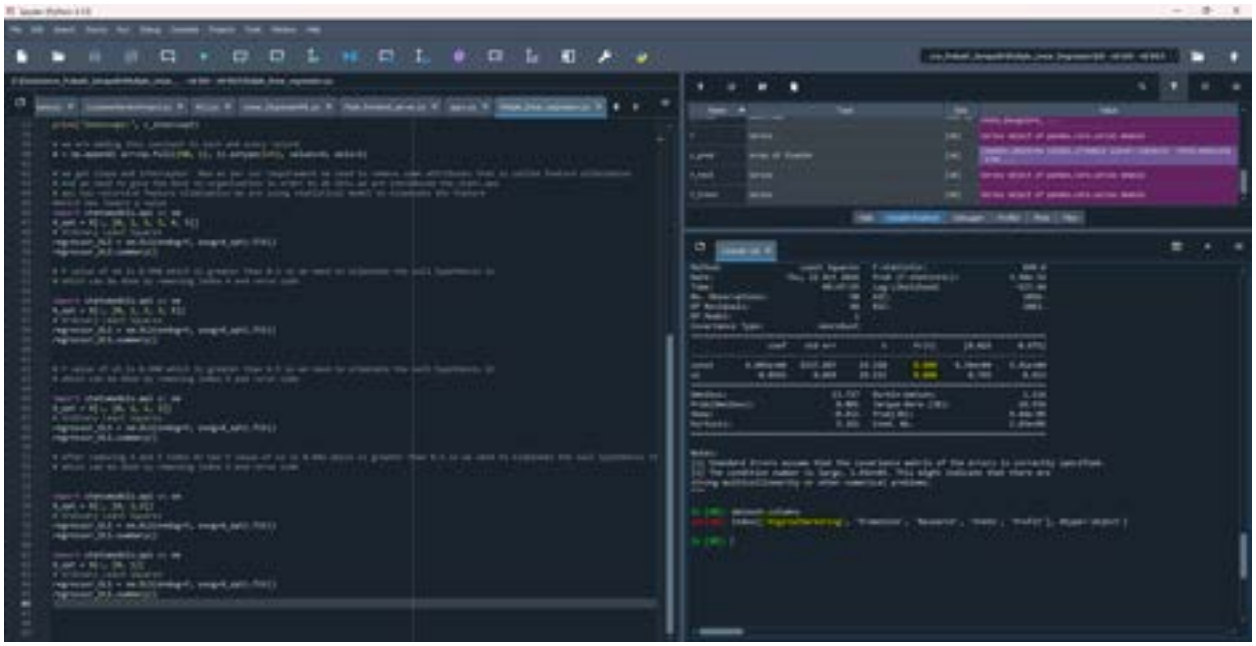


Here the X4 has the high p value
The X4 value 0.990 is greater than 0.5 p value so we are eliminating how we rae eliminating means by eliminating the 4 index as highlighted below



and copy paste the same code after removing index 4 related to x4 and rerun





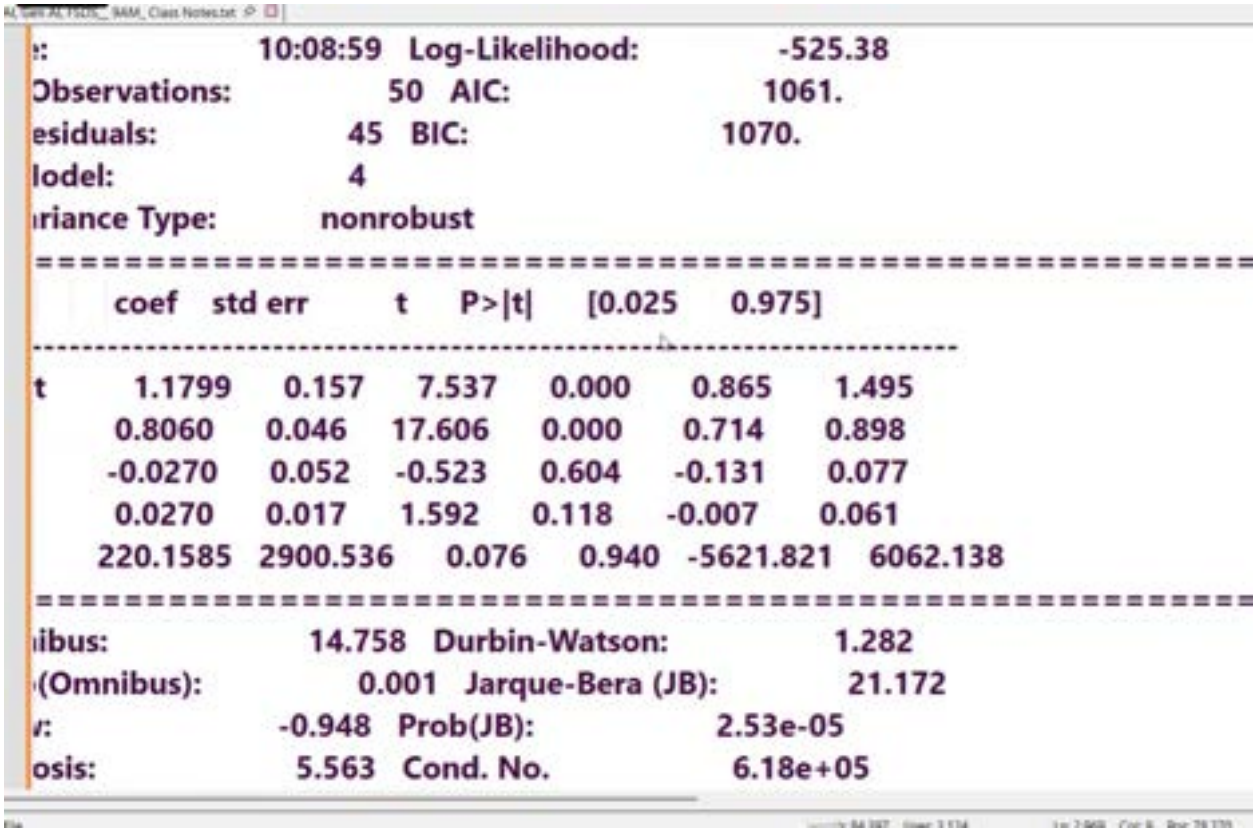
```
(Omnibus):              0.001   Jarque-Bera (JB):          21.172
r:                     -0.948   Prob(JB):                2.53e-05
osis:                   5.563   Cond. No.                 6.18e+05
```

Feature ellimation
Recursive ellimination in machine leanring

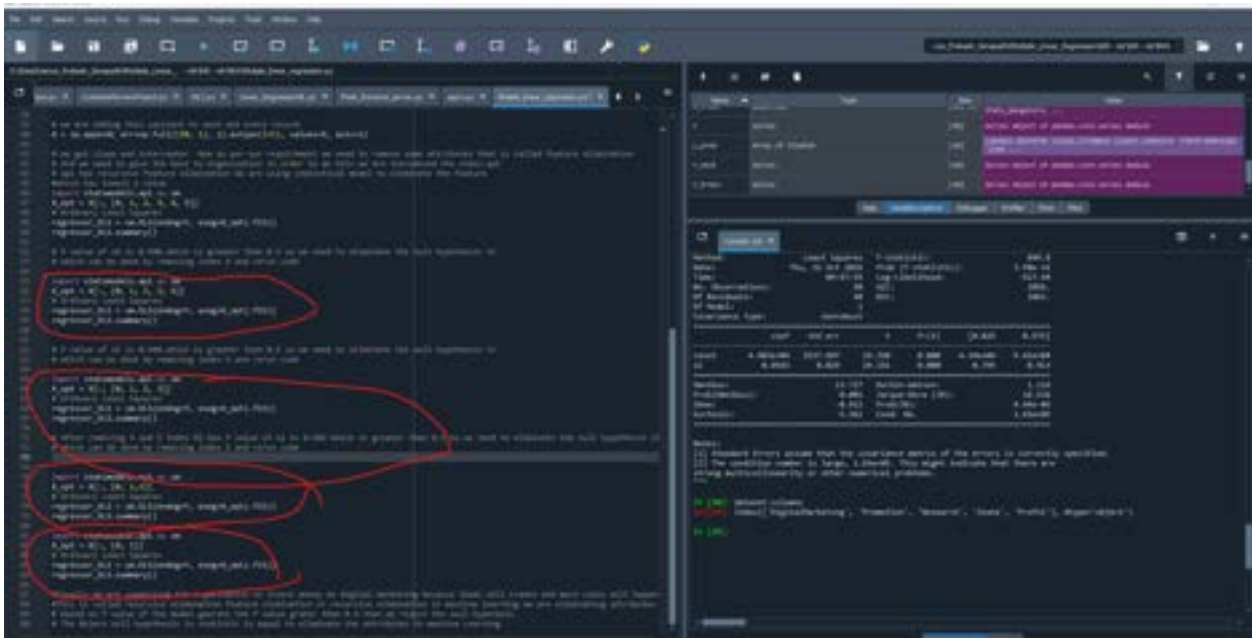
we are tying to eliminat atttibute base on p-value

model geneated value > 0.05 -- we reject the null hypothesis
reject null hypot statsion === eliminat the attribute in machine learning

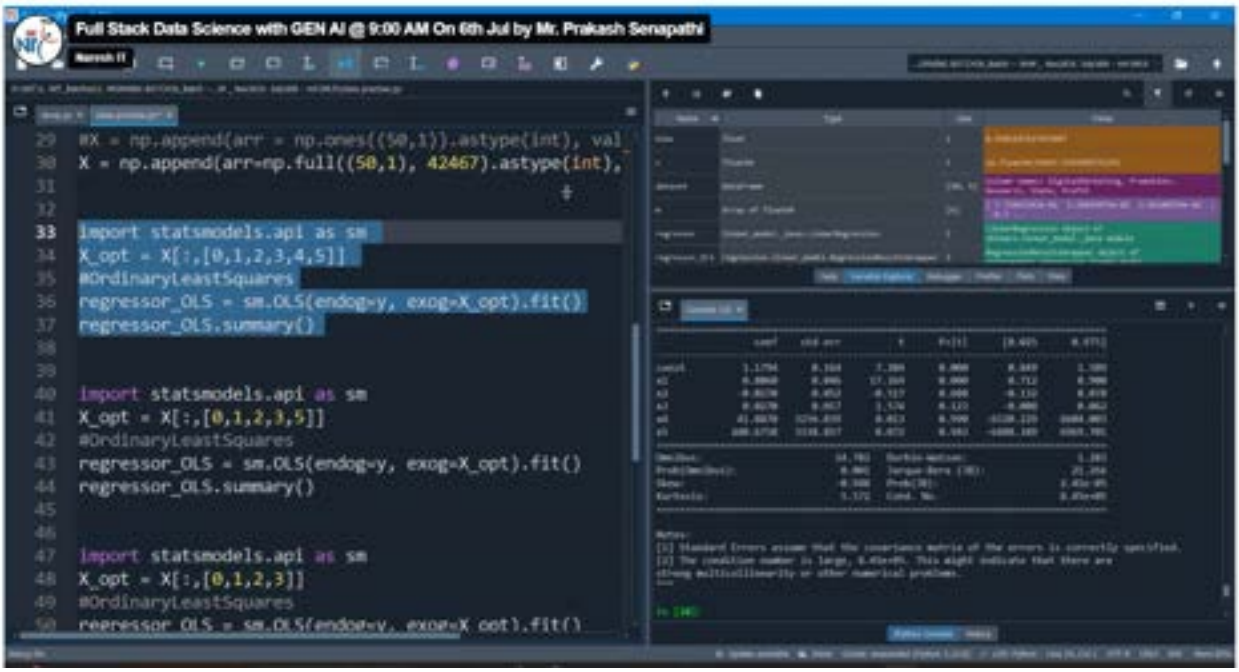
#Finally we are suggesting the organization to invest money on digital marketing because leads will create and more sales will happen
#This is called recursive elimination feature elimination or recursive elimination in machine learning we are eliminating attributes
based on P value if the model geerate the P value grater than 0.5 then we reject the null hypotesis
The Reject null hypothesis in statistic is equal to eliminate the attributes in machine Learning



```
:                  10:08:59   Log-Likelihood:          -525.38
Observations:            50   AIC:                     1061.
esiduals:                45   BIC:                     1070.
lodel:                   4
riance Type:         nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
t             1.1799      0.157      7.537      0.000       0.865       1.495
              0.8060      0.046     17.606      0.000       0.714       0.898
             -0.0270      0.052     -0.523      0.604      -0.131       0.077
              0.0270      0.017      1.592      0.118      -0.007       0.061
            220.1585   2900.536      0.076      0.940   -5621.821    6062.138
==============================================================================
ibus:                  14.758   Durbin-Watson:                   1.282
(Omnibus):              0.001   Jarque-Bera (JB):               21.172
v:                     -0.948   Prob(JB):                     2.53e-05
osis:                   5.563   Cond. No.                     6.18e+05
```



We did in 4 steps but by seeing we can remove required indexes and can be done in single step



Note : In simple linear regression we will predict feature where as in the multiple linear regression we are target which is right attribute

Assume we are working in a stock market in world multiple stock is there which stock we want to buy which stock performance is good we can do multiple linear regression by calling recursive elimination or feature elimination

Note: Backward elimination comes under RFE

Multiple linear model	**multiple linear regression algorithm stats.ols import stats.api | what is rfe (recursive feature elimination) we eliminatee the feature based on backward elimination using p-value house price prediction model | model genereate significance value > 0.05 -- we rejet the null hypothesis | 1. pd.get_dummies() – Converts categorical (text) data into binary (0/1) because ML models cannot understand text directly. 2. Backward Elimination – Removes features with P-value > 0.05 using Statsmodels OLS. 3. Recursive Feature Elimination (RFE) – Selects the best features by recursively removing less important ones using a machine learning model. Unlike Backward Elimination (P-value based), RFE is model-based and uses feature importance/ranking.**

```
overfitting -- train the model with more attribute
   overfitting -- low bias high variance
   bias -- training & varianace -- testing part
to reduce overfitting we need to remove some attribute

to reduce overfitting we have some technique
      regularization technique
      cross validation
      ensamble learning
      pca
      drop out the neurons
```

1.Regularization : This regularization technique reduce overfitting

```
Regularization tehcnique -->
   regularziation technqiue reduce overfitting
   how this technique reduce overfitting

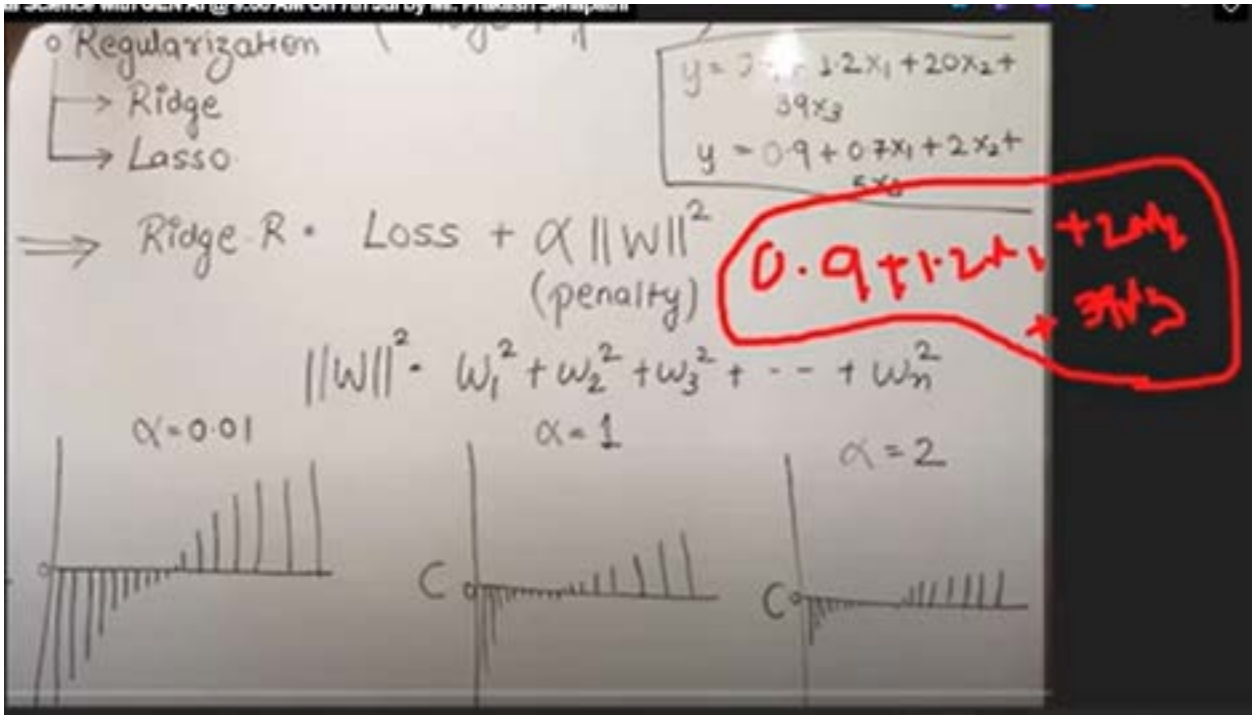
ridge regularization == l2 regularization | l2 regression
lasso regularization == l1 regularization | l1 regression
elasticnet = l1 + l2
```

ML model overfits because of two reasons
1.Train the model with more attributes
2.Train model with high coefficient
Ex: A student pass or fail his rollnumber, height or weight not matters when we feed with these attributes model will overfit if we remove this type of attributes then model perform well and model will be best one.

Train model with high coefficient

Ex: 1023x1+2395x2 here x1 coefficient is 1023 and x2 coefficient is 2395 these coefficients are very high when we train with these high coefficients the model will overfits to adjust these coefficients from high magnitude to low magnitude that is called

regularize we have below equation



0.9 + 1.2x1 + 20x2 + 39x3
For this equation we need to apply below formula

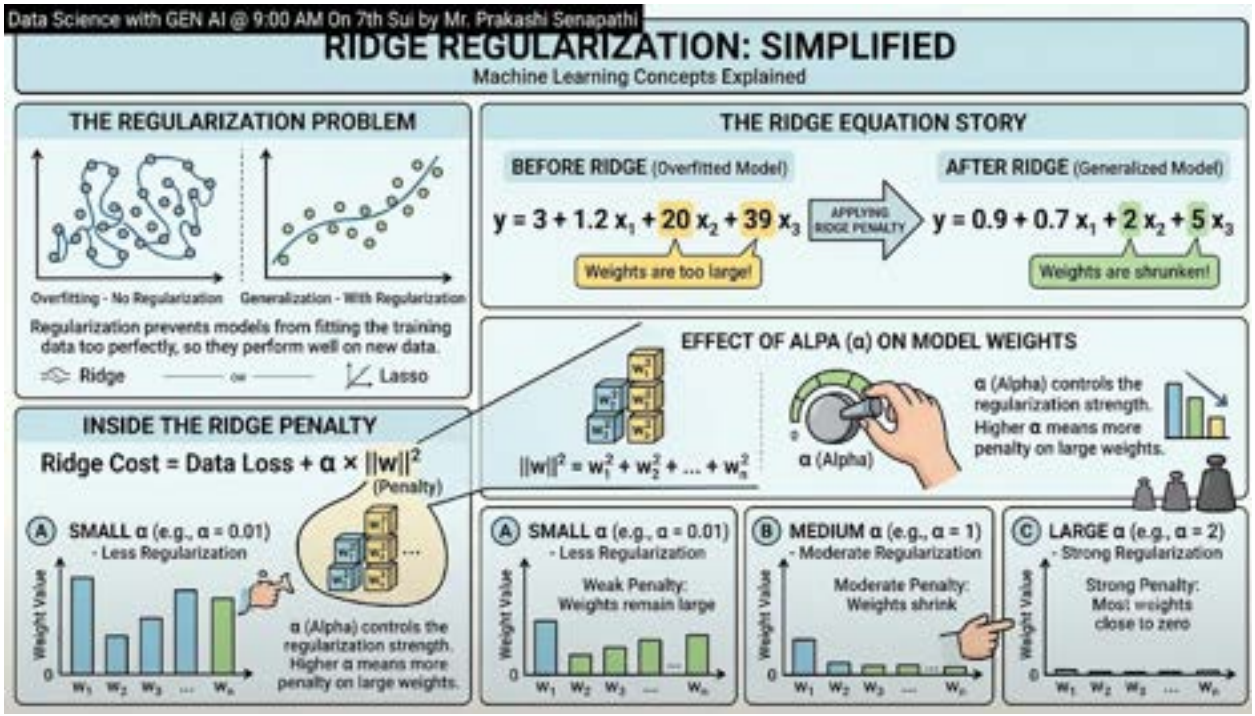
Ridge R = Loss + alpha ||W||^2 (penalty)

Loss = y-y^
Penalty means weight w = weight1+weight2+weight3

||W||^2 = w_1^2 + w_2^2 + w_3^2 + ... + w_n^2

Every dataset has weights associated with it when we apply this formula Ridge R = Loss + alpha ||W||^2 (penalty)
To this equation 0.9 + 1.2x1 + 20x2 + 39x3 the coefficient is reduced now the equation to this y = 0.9 + 0.7x1 + 2x2 + 5x3 in this the coefficient 1.2 is reduced 0.7 and the 20 is reduced to 2 and the coefficient 39 is reduced 5

Regularization technique is one of the technique which is reducing the coefficient of each and every independent variable



Regularization technique is one of the technique which is reduce the overfitting by scale down high coefficient to low coefficient is called Ridge

L2 = 0.9 + 0.7x1 + 2x2 + 5x3

Lasso.R = Loss + α || W || (penalty)

The difference between lasso and ridge is in lasso the square is absent

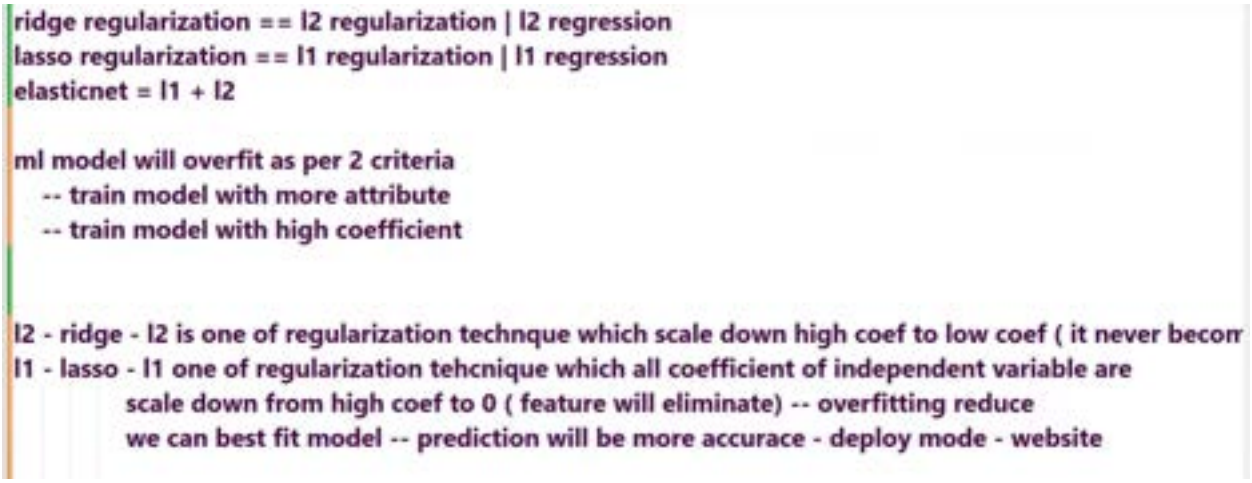
Note : The Lasso is L1 and ridge was L2

Y = 0.9+0+0+5x2
y= 0.9+5x2 here we are eliminated 2 features it means we are reducing the overfitting in Lasso and ridge

Q)could you please speak about lasso and Ridge

A) **Ridge - L2 is one of the regularization techniques which scales down high coefficients to low coefficients (it never becomes 0).**

L1 - Lasso - L1 is one of the regularization techniques in which coefficients of independent variables are scaled down from high coefficients to 0 (features will be eliminated) -- overfitting is reduced, we get a best-fit model -- prediction will be more accurate - ready for deployment in production/website.

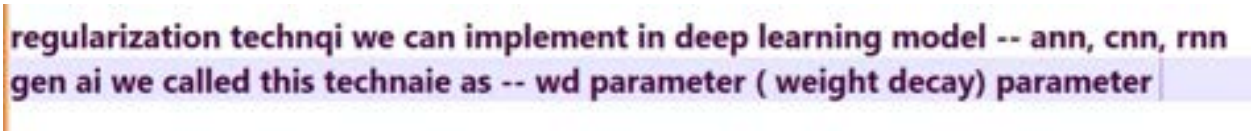


L1 act as a feature elimination

Elasticnet = L1+L2

L1 —---->it coefficient to 0
L2—-----> it scaledown high coefficient to low coefficient

The elastic net will inherent coefficient to 0 and as well as it scaledown high coefficient to low coefficient
It reduce model complexity



regularization technique we can implement in deep learning model -- ann, cnn, rnn
gen ai we called this technique as -- wd parameter (weight decay) parameter
In this every model and every algorithm implement this

*** **200 attributes = 200 coefficients:** In machine learning, if your dataset has 200 independent features (attributes, like columns or inputs), your linear model will learn **200 separate weights (coefficients)** to multiply with those attributes. * **sklearn model regularization module:** Scikit-Learn (`sklearn`) provides built-in modules (like `Ridge` and `Lasso`) to handle regularization easily for these models.**

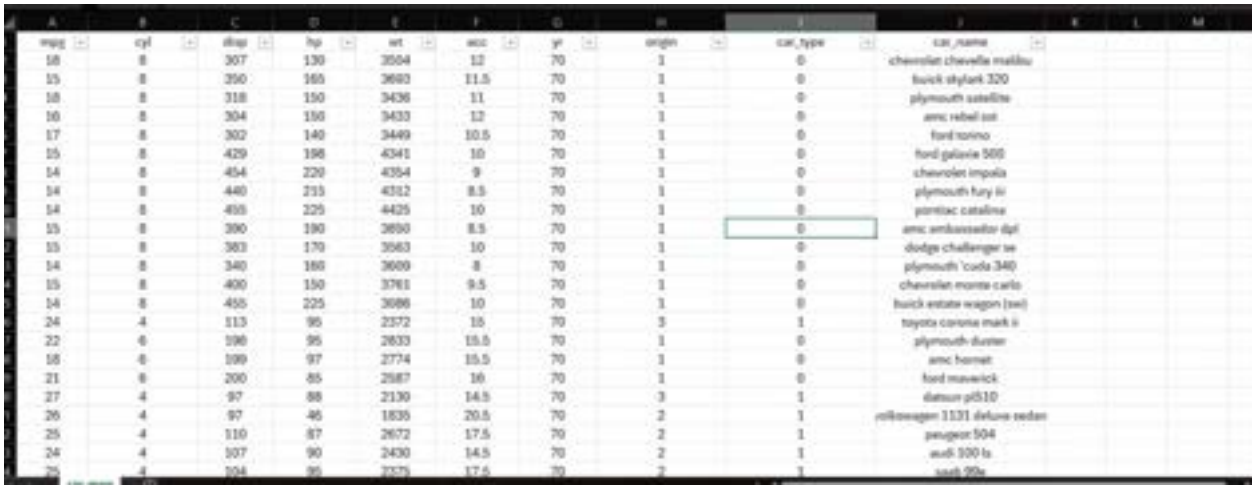
l1 will act as feature elimination

elasticnet = l1 + l2

regularization technqi we can implement in deep learning model -- ann, cnn, rnn
gen ai we called this technaie as -- wd parameter (weight decay) parameter

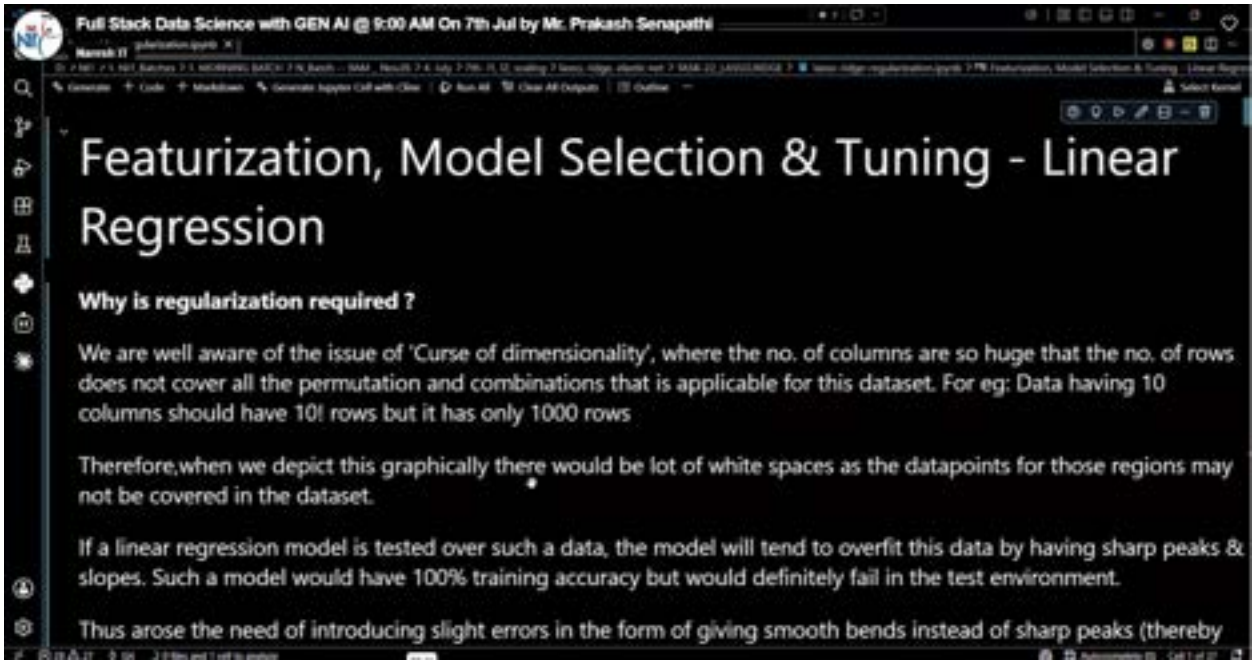
everymodel, every algorithm

200 attribute -- 200 coefficaine
thats skleanr model reguarlization moudle



mpg	cyl	disp	hp	wt	acc	yr	origin	car_type	car_name
18	8	307	130	3504	12	70	1	0	chevrolet chevelle malibu
15	8	350	165	3693	11.5	70	1	0	buick skylark 320
18	8	318	150	3436	11	70	1	0	plymouth satellite
16	8	304	150	3433	12	70	1	0	amc rebel sst
17	8	302	140	3449	10.5	70	1	0	ford torino
15	8	429	198	4341	10	70	1	0	ford galaxie 500
14	8	454	220	4354	9	70	1	0	chevrolet impala
14	8	440	215	4312	8.5	70	1	0	plymouth fury iii
14	8	455	225	4425	10	70	1	0	pontiac catalina
15	8	390	190	3850	8.5	70	1	0	amc ambassador dpl
15	8	383	170	3563	10	70	1	0	dodge challenger se
14	8	340	160	3609	8	70	1	0	plymouth 'cuda 340
15	8	400	150	3761	9.5	70	1	0	chevrolet monte carlo
14	8	455	225	3086	10	70	1	0	buick estate wagon (sw)
24	4	113	95	2372	15	70	3	1	toyota corona mark ii
22	6	198	95	2833	15.5	70	1	0	plymouth duster
18	6	199	97	2774	15.5	70	1	0	amc hornet
21	6	200	85	2587	16	70	1	0	ford maverick
27	4	97	88	2130	14.5	70	3	1	datsun pl510
26	4	97	46	1835	20.5	70	2	1	volkswagen 1131 deluxe sedan
25	4	110	87	2672	17.5	70	2	1	peugeot 504
24	4	107	90	2430	14.5	70	2	1	audi 100 ls
25	4	104	95	2375	17.5	70	2	1	saab 99e

In this dataset dependent variable is mileage when ever if we buy a car we see the millage when ever if a person want to buy a car he will see the mileage



Agenda

- Perform basic EDA
- Scale data and apply Linear, Ridge & Lasso Regression with Regularization
- Compare the r^2 score to determine which of the above regression methods gives the highest score
- Compute Root mean squared error (RMSE) which inturn gives a better score than r^2
- Finally use a scatter plot to graphically depict the correlation between actual and predicted mpg values

1. Import packages and observe dataset

```
#Import numerical libraries
import pandas as pd
import numpy as np
```

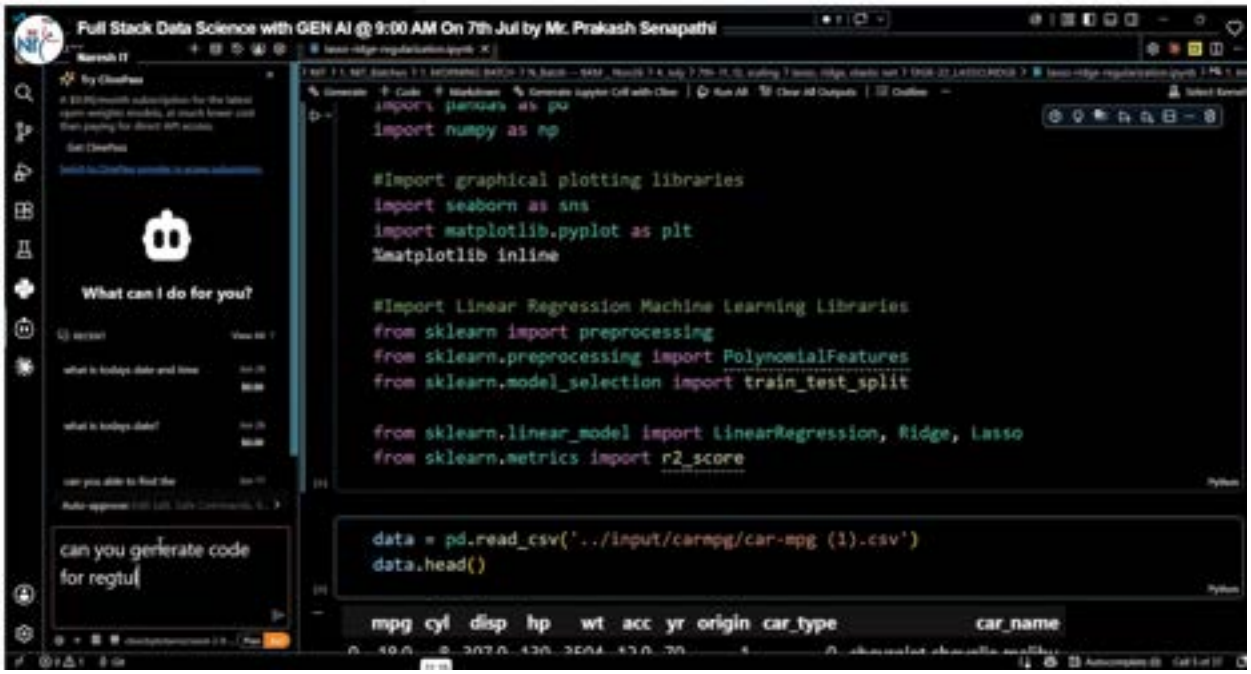
1. Import packages and observe dataset

```
#Import numerical libraries
import pandas as pd
import numpy as np

#Import graphical plotting libraries
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

#Import Linear Regression Machine Learning Libraries
from sklearn import preprocessing
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score
```



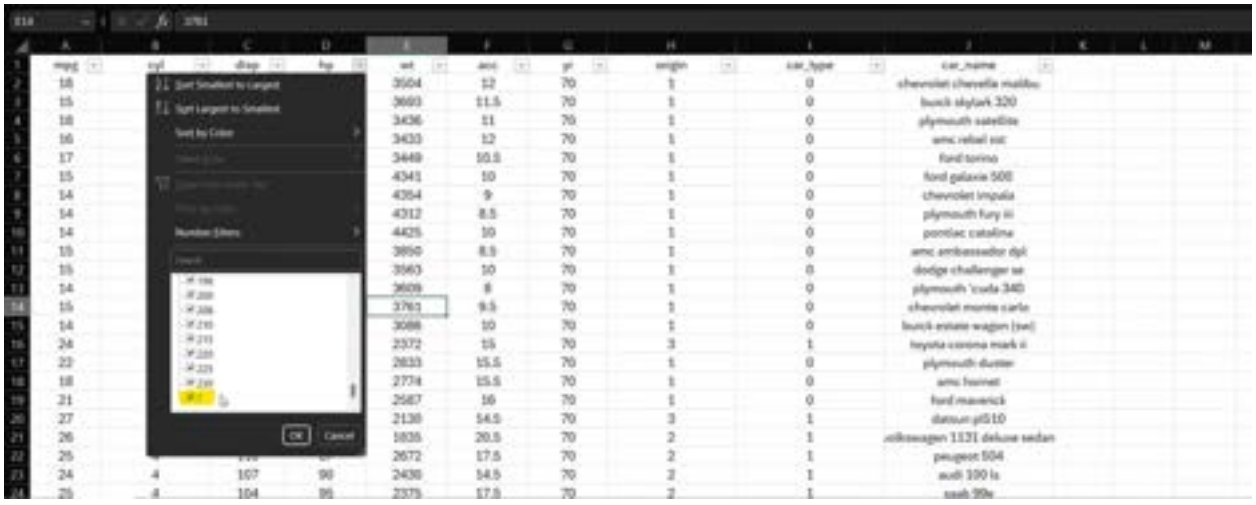
```
#Import Linear Regression Machine Learning Libraries
from sklearn import preprocessing
from sklearn.preprocessing import PolynomialFeatures
from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score
```

```
data = pd.read_csv('../input/carmpg/car-mpg (1).csv')
data.head()
```

	mpg	cyl	disp	hp	wt	acc	yr	origin	car_type	car_name
0	18.0	8	307.0	130	3504	12.0	70	1	0	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	0	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	0	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	0	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	0	ford torino

Note : ? will not considered as a null



mpg	cyl	disp	hp	wt	acc	yr	origin	car_type	car_name
25	4	98	?	2046	19	71	1	1	ford pinto
21	6	200	?	2875	17	74	1	0	ford maverick
40.9	4	85	?	1835	17.3	80	2	1	renault lecar deluxe
23.6	4	140	?	2905	14.3	80	1	1	ford mustang cobra
34.5	4	100	?	2320	15.8	81	2	1	renault 18i
23	4	151	?	3035	20.5	82	1	1	amc concord dl

Note : ? will not considered to fill nan in order to do that first we need to convert it in to nan thin all comes under data cleaning so where ever if we find the ? then replace it with nan value

```
#Drop car name
#Replace origin into 1,2,3.. dont forget get_dummies
#Replace ? with nan
#Replace all nan with median
Q
data = data.drop(['car_name'], axis = 1)
data['origin'] = data['origin'].replace({1: 'america', 2: 'europe', 3: 'asia'})
data = pd.get_dummies(data,columns = ['origin'])
data = data.replace('?', np.nan)
data = data.apply(lambda x: x.fillna(x.median()), axis = 0)
```

```
data.head()
```

	mpg	cyl	disp	hp	wt	acc	yr	car_type	origin_america	origin_asia	origin_europe
0	18.0	8	307.0	130	3504	12.0	70	0	1	0	0
1	15.0	8	350.0	165	3693	11.5	70	0	1	0	0
2	18.0	8	318.0	150	3436	11.0	70	0	1	0	0

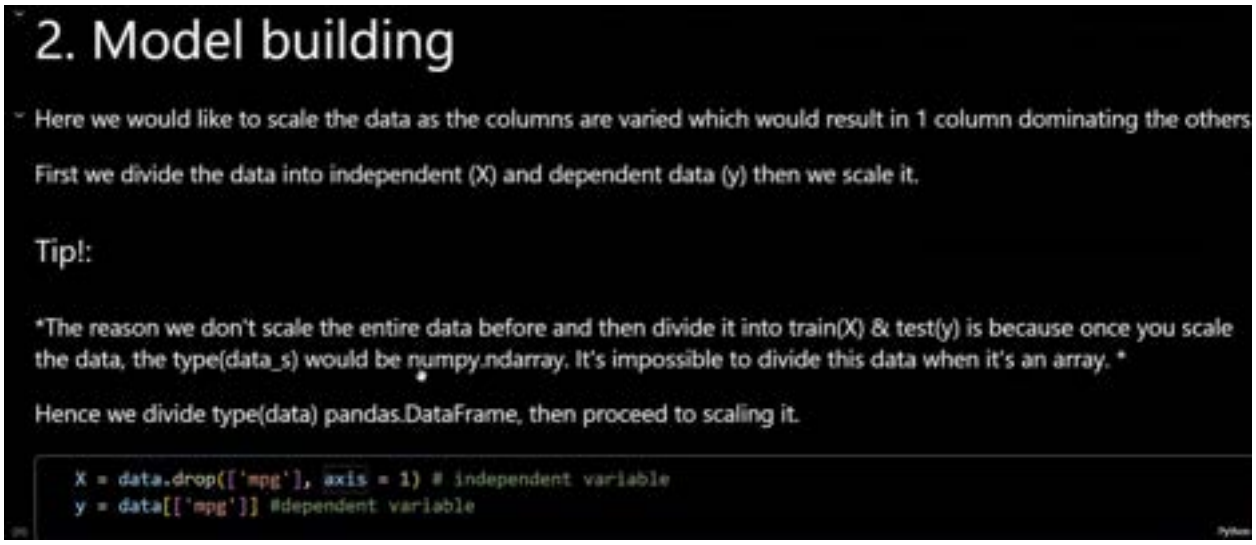

```
data['origin'] = data['origin'].replace({1: 'america', 2: 'europe', 3: 'asia'})
data = pd.get_dummies(data,columns = ['origin'])
data = data.replace('?', np.nan)
data = data.apply(lambda x: x.fillna(x.median()), axis = 0)
```

```
data.head()
```

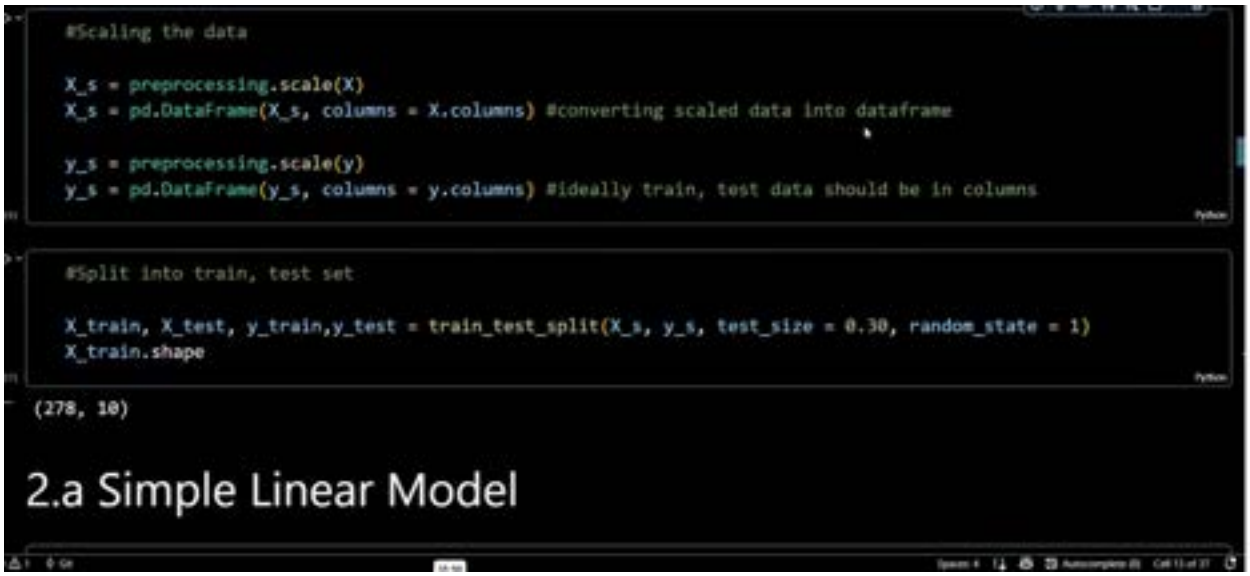

	mpg	cyl	disp	hp	wt	acc	yr	car_type	origin_america	origin_asia	origin_europe
0	18.0	8	307.0	130	3504	12.0	70	0	1	0	0
1	15.0	8	350.0	165	3693	11.5	70	0	1	0	0
2	18.0	8	318.0	150	3436	11.0	70	0	1	0	0
3	16.0	8	304.0	150	3433	12.0	70	0	1	0	0
4	17.0	8	302.0	140	3449	10.5	70	0	1	0	0

We have to predict the mpg column given the features.

2. Model building







మీరు చదివిన నోట్స్ మరియు క్లాస్ యొక్క ముఖ్యమైన విషయాలను చాలా సింపుల్ తెలుగులో వివరిస్తాను చూడండి:

1. **Overfitting** అంటే ఏమిటి? (మోడల్ ఎందుకు ఓవర్‌ఫిట్ అవుతుంది?)

ఒక మెషిన్ లెర్నింగ్ మోడల్ ట్రైనింగ్ అప్పుడే అన్ని లెక్కలు బై బై హార్ట్ చేసి, కొత్త డేటా ఇచ్చేసరికి సరిగ్గా ఆన్సర్ చేయలేకపోవడాన్ని **Overfitting** అంటారు. ఇది ప్రధానంగా రెండు కారణాల వల్ల జరుగుతుంది:

1. ఎక్కువ అట్రిబ్యూట్స్ **(Columns)** ఉండటం:
2.
    - ఉదాహరణకు: ఒక విద్యార్థి పాస్ అవుతాడా లేదా అని ప్రెడిక్ట్ చేయడానికి వాడి రోల్ నంబర్, హైట్, వెయిట్ లాంటివి అసలు సంబంధం లేని విషయాలు. కానీ మనం ఆ చెత్త డేటా అంతా మోడల్‌కి ఇస్తే అది కన్ఫ్యూజ్ అయి ఓవర్‌ఫిట్ అవుతుంది. అలాంటి పనికిరాని ఆట్రిబ్యూట్స్ తీసేస్తే మోడల్ చాలా బాగా పనిచేస్తుంది.
    -
3. పెద్ద కోఎఫిషియంట్స్ **(High Coefficients)** ఉండటం:
4.
    - ఉదాహరణకు $1023x_1 + 2395x_2$ లాంటి పెద్ద పెద్ద నంబర్లు **(weights)** వస్తే మోడల్ ఓవర్‌ఫిట్ అవుతుంది. ఈ పెద్ద నంబర్లను చిన్నవిగా మార్చడాన్నే **Regularization** అంటారు.
    -

2. Ridge Regularization (L2 Regularization)

- ఇది ఏం చేస్తుంది? మోడల్‌లో ఉన్న పెద్ద కోఎఫిషియంట్ విలువలను చిన్నవిగా **Scale Down** చేస్తుంది **(**తగ్గిస్తుంది**).**
-
- ఉదాహరణకు: $20x_2$ కాస్తా $2x_2$ లాగా తగ్గిపోతుంది.
-
- ముఖ్యమైన పాయింట్: రిడ్జ్ **(Ridge)** లో కోఎఫిషియంట్ విలువలు తగ్గుతాయి కానీ ఎప్పటికీ సున్నా **(0)** అవ్వవు.
-
- ఫార్ములా: $Ridge = Loss + \alpha \sum W^2$ **(**ఇక్కడ స్క్వేర్ ఉంటుంది కాబట్టి దీన్ని **L2** అంటారు**).**
-

3. Lasso Regularization (L1 Regularization)

- ఇది ఏం చేస్తుంది? లాసో కూడా కోఎఫిషియంట్‌లను తగ్గిస్తుంది, కానీ అవసరం లేని ఫీచర్ల **(columns)** కోఎఫిషియంట్‌లను పూర్తిగా సున్నా **(0)** చేసేస్తుంది.
-
- ఫీచర్ ఎలిమినేషన్: ఏ ఫీచర్ అయితే మోడల్‌కి అవసరం లేదో దాని కోఎఫిషియంట్‌ని **0** చేసి ఆ కాలమ్‌ని బయటికి తోసేస్తుంది **(Feature Elimination).**
-
- ఫార్ములా: **$Lasso = Loss + \alpha \sum \vert{}W\vert{}$ (**ఇక్కడ స్క్వేర్ ఉండదు, దీన్ని **L1** అంటారు**).**
-

4. ElasticNet (L1 + L2)

- ఇది **Lasso (L1)** మరియు **Ridge (L2)** రెండింటి కలయిక (ElasticNet = L1 + L2**).**
-
- ఇది కొన్ని ఫీచర్ల కోఎఫిషియంట్‌లను సున్నా **(0)** చేస్తుంది **(L1** లాగా**)** మరియు మిగతా వాటిని తగ్గించి బ్యాలెన్స్ చేస్తుంది **(L2** లాగా**).** దీనివల్ల మోడల్ కాంప్లెక్సిటీ తగ్గుతుంది.
-

చిన్న సారాంశం (Quick Summary):

- **Overfitting:** ఎక్కువ కాలమ్స్ లేదా పెద్ద కోఎఫిషియంట్స్ ఉంటే వస్తుంది. దీన్ని తగ్గించడానికి **Regularization** వాడతాం.
-
- **Ridge (L2):** కోఎఫిషియంట్‌లను తగ్గిస్తుంది కానీ సున్నా చేయదు.
-
- **Lasso (L1):** అనవసరమైన ఫీచర్ల కోఎఫిషియంట్‌లను సున్నా **(0)** చేసి తొలగిస్తుంది **(Feature Elimination).**
-
- **ElasticNet: L1** మరియు **L2** రెండూ కలిపి పనిచేస్తాయి.
-

ఇది మీకు స్పష్టంగా అర్థమైందా? ఇందులో ఇంకేదైనా పాయింట్ సింపుల్ కావాలంటే అడగండి!

Machine Learning లో **Weights** (లేదా **Coefficients**) మరియు **Regularization** గురించిన కాన్సెప్ట్‌ను చిన్న పిల్లలకు చెప్పే ఉదాహరణతో, చాలా సులభమైన తెలుగులో అర్థమయ్యేలా కింద వివరించాను.

👶 చిన్నపిల్లల ఉదాహరణ (Child-Understanding Example)

మీరు ఒక స్కూల్ పిక్నిక్ బ్యాగ్ **(School Picnic Bag)** సర్దుతున్నారు అనుకుందాం:

1. **Weights** (ప్రాముఖ్యత/ప్రాధాన్యత)**:**
2.
    - బ్యాగ్‌లో వేసే ప్రతి వస్తువుకి ఒక విలువ **(Weight)** ఉంటుంది.
    -
    - వాటర్ బాటిల్ **&** లంచ్ బాక్స్: చాలా ముఖ్యం → వీటికి **High Weight** ఇస్తాం.
    -
    - స్నాక్స్ **&** ఆడుకునే బంతి: కొద్దిగా ముఖ్యం → వీటికి **Medium Weight** ఇస్తాం.
    -
    - ఇంటి కీలు, రోల్ నంబర్ స్టిక్కర్, పాత కాగితాలు: పిక్నిక్‌కి అసలు అవసరమే లేదు → **Zero/No Weight.**
    -
3. **Overfitting** (భారం ఎక్కువ అవ్వడం)**:**
4.
    - పిక్నిక్‌కి అవసరం లేని రోల్ నంబర్ కార్డ్, పాత పుస్తకాలు, పెద్ద పెద్ద రాళ్ళు కూడా బ్యాగ్‌లో వేసుకుంటే... బ్యాగ్ చాలా బరువైపోయి నడవలేరు! మోడల్‌లో అనవసరమైన **Attributes** (కాలమ్స్) మరియు పెద్ద **Coefficients (High Weights)** ఉన్నప్పుడు కూడా ఇలాగే మోడల్ బరువైపోయి సరిగ్గా పనిచేయదు **(Overfit** అవుతుంది**).**
    -
5. **Regularization** (టీచర్ చెప్పే రూల్)**:**
6.

- టీచర్ వచ్చి బ్యాగ్ చెక్ చేసి: **"బ్యాగ్ చాలా బరువుగా ఉంది, అనవసరమైన బరువులు తగ్గించండి!"** అని చెప్తారు. ఈ పద్ధతినే **Regularization** అంటారు.
-

📘 బోర్డు చిత్రంలోని ముఖ్య అంశాలు - తెలుగు వివరణ

1. Weights (w) అంటే ఏమిటి?

- **Formula: $y = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$**
-
- అర్థం: ఇక్కడ x_1, x_2 అనేవి మనం ఇచ్చే డేటా **(Attributes)**. w_1, w_2 అనేవి ఆ డేటాకి మోడల్ ఇచ్చే **Weights** (లేదా **Coefficients**).
-
- ఒక ఫీచర్ ఎంత ముఖ్యమో చెప్పేదే **Weight.**
-

2. High Weights vs Low Weights

- **High Coefficients** ($1023x_1, 2395x_2$)**:** మోడల్ ఒక విషయాన్ని ఎక్కువగా పట్టించుకుని **Overfit** అయిపోతుంది (రెడ్ ట్యూబ్ లాగా డెంజర్).
-
- **Low Coefficients** ($0.7x_1, 2x_2$)**:** మోడల్ సరిగ్గా, బ్యాలెన్స్‌డ్‌గా పనిచేస్తుంది **(Best Fit).**
-

3. Regularization రకాలు (Lasso vs Ridge)

- **L1 - Lasso Regularization (Feature Elimination):**
-
    - చేసే పని: అవసరం లేని వస్తువులను పూర్తిగా బ్యాగ్‌లోంచి తీసి బయటపడేస్తుంది.
    -
    - ఉదాహరణ: విద్యార్థి మార్కులను అంచనా వేయడానికి **"హైట్ (Height)"** లేదా **"రోల్ నంబర్"** అవసరం లేదు. కాబట్టి లాసో వాటి **Weight** ను పూర్తిగా **0** ($w_2 \to 0$) చేసేసి ఆ కాలమ్‌ను డిలీట్ చేస్తుంది.
    -
    - ఫార్ములా: **$L1 = Loss + \alpha \sum \vert{}w_i\vert{}$**
    -
- **L2 - Ridge Regularization (Scaling Down):**
-
    - చేసే పని: ఏ వస్తువునీ పడేయదు, కానీ ప్రతీ వస్తువు సైజును చిన్నదిగా తగ్గిస్తుంది **(Scale Down).**
    -
    - ఉదాహరణ: $20x_2$ ని $2x_2$ గా తగ్గిస్తుంది. కానీ **Weight** ని ఎప్పటికీ **0** చేయదు.
    -
    - ఫార్ములా: $L2 = Loss + \alpha \sum w_i^2$
    -

4. ElasticNet

- **L1 (Lasso) + L2 (Ridge)** రెండింటినీ కలిపి వాడటమే **ElasticNet.**
-
- ఇది కొన్ని పనికిరాని ఫీచర్లను పూర్తిగా **0** చేస్తుంది **(L1),** మిగిలిన ఫీచర్ల **Weight** ను తగ్గిస్తుంది **(L2).**
-

 ఒకే మాటలో గుర్తుంచుకోవడానికి **(Quick Summary):**

కాన్సెప్ట్	పని చేసే విధానం	చిన్న ఉదాహరణ
High Weights	మోడల్‌ను **Overfit** చేస్తాయి	బ్యాగ్ బరువైపోవడం
Ridge (L2)	**Weights** ని తగ్గిస్తుంది **(0** చేయదు**)**	పెద్ద బాటిల్ తీసి చిన్న బాటిల్ పెట్టడం
Lasso (L1)	అనవసరమైన **Weight** ను **0** చేసి డిలీట్ చేస్తుంది	అవసరం లేని రాయిని బ్యాగ్‌లోంచి బయటపడేయడం

ఈ వివరణతో మీకు కాన్సెప్ట్ సులభంగా అర్థమైందా? ఇంకేదైనా ఉదాహరణ కావాలా?

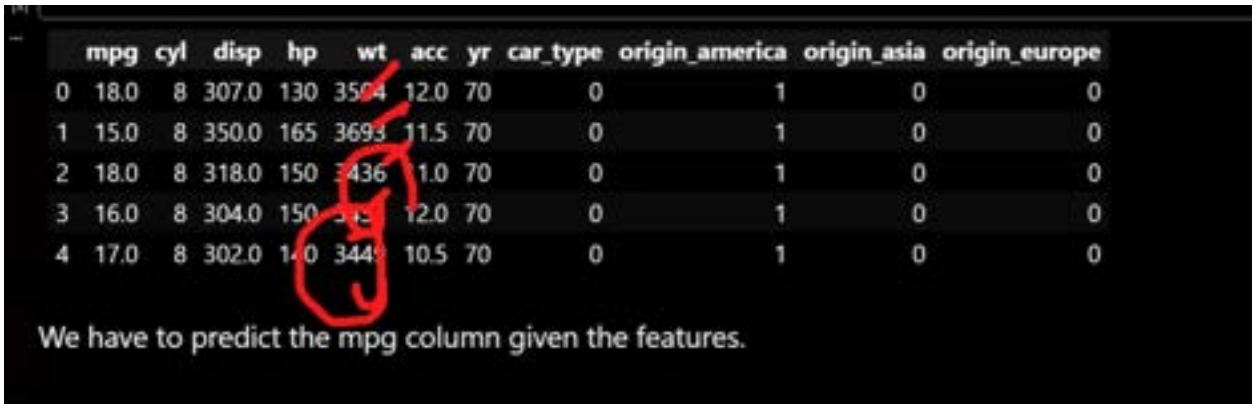
Lasso will reduce to zero
Ridge will not reduce to zero

Feature scaling technique :
Standardization technique(Z-score) = standard scaler
Normalization technique | normalizer = = min max scaler

What is feature scaling
Ex: Suppose to day is our birthday and there are 5 kids for the first kid we have given 10 choclatesfor second kid given 20 chocolate for 3rd kid 15 chocolates and for 4th kid given 6 chocolates immediately fight will start if we given in the ratio as first kid in the ratio of 5 to 10 for second kid 5 to 10 for third also 5 to 10 for the fourth as well 5 to 10 The feature scaling will also do the same

	mpg	cyl	disp	hp	wt	acc	yr	car_type	origin_america	origin_asia	origin_europe
0	18.0	8	307.0	130	3504	12.0	70	0	1	0	0
1	15.0	8	350.0	165	3693	11.5	70	0	1	0	0
2	18.0	8	318.0	150	3436	11.0	70	0	1	0	0
3	16.0	8	304.0	150	3433	12.0	70	0	1	0	0
4	17.0	8	302.0	140	3449	10.5	70	0	1	0	0

When we feed this numbers to machine will understand 130 is less than 3504 and 3504 is greater than 0 1 0 0 when we build the model with numbers then model will bias(bias means one side ex : in a match 10 boys and 1 girl play then winner is boys this is bias because 10>1) similarly below highlighted numbers in red are high numbers



When we applñ scaling to each number by applying pre processing scaler

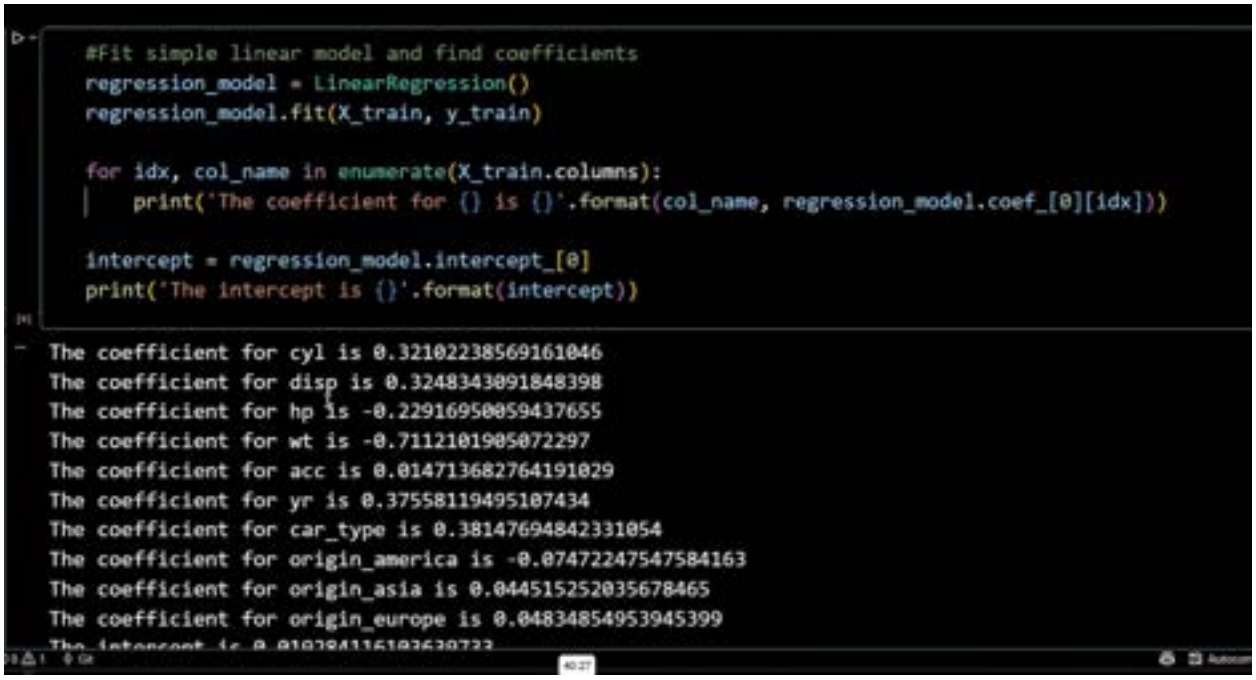


```
#Scaling the data

X_s = preprocessing.scale(X)
X_s = pd.DataFrame(X_s, columns = X.columns) #converting scaled data into dataframe

y_s = preprocessing.scale(y)
y_s = pd.DataFrame(y_s, columns = y.columns) #ideally train, test data should be in columns
```

After applying scaling we need to divide data in to X-train, Y-train when we buld the model



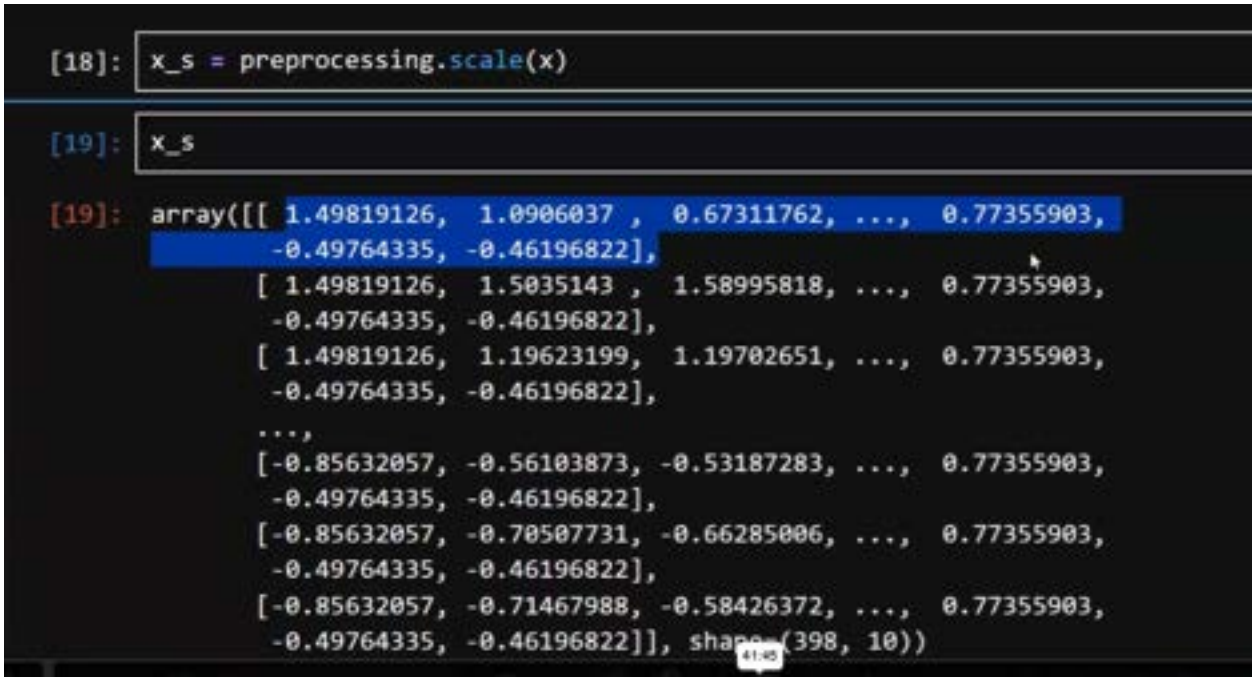
```
#Fit simple linear model and find coefficients
regression_model = LinearRegression()
regression_model.fit(X_train, y_train)

for idx, col_name in enumerate(X_train.columns):
    print('The coefficient for {} is {}'.format(col_name, regression_model.coef_[0][idx]))

intercept = regression_model.intercept_[0]
print('The intercept is {}'.format(intercept))
```

```
The coefficient for cyl is 0.32102238569161046
The coefficient for disp is 0.3248343091848398
The coefficient for hp is -0.22916950059437655
The coefficient for wt is -0.7112101905072297
The coefficient for acc is 0.014713682764191029
The coefficient for yr is 0.37558119495107434
The coefficient for car_type is 0.38147694842331054
The coefficient for origin_america is -0.07472247547584163
The coefficient for origin_asia is 0.044515252035678465
The coefficient for origin_europe is 0.04834854953945399
```

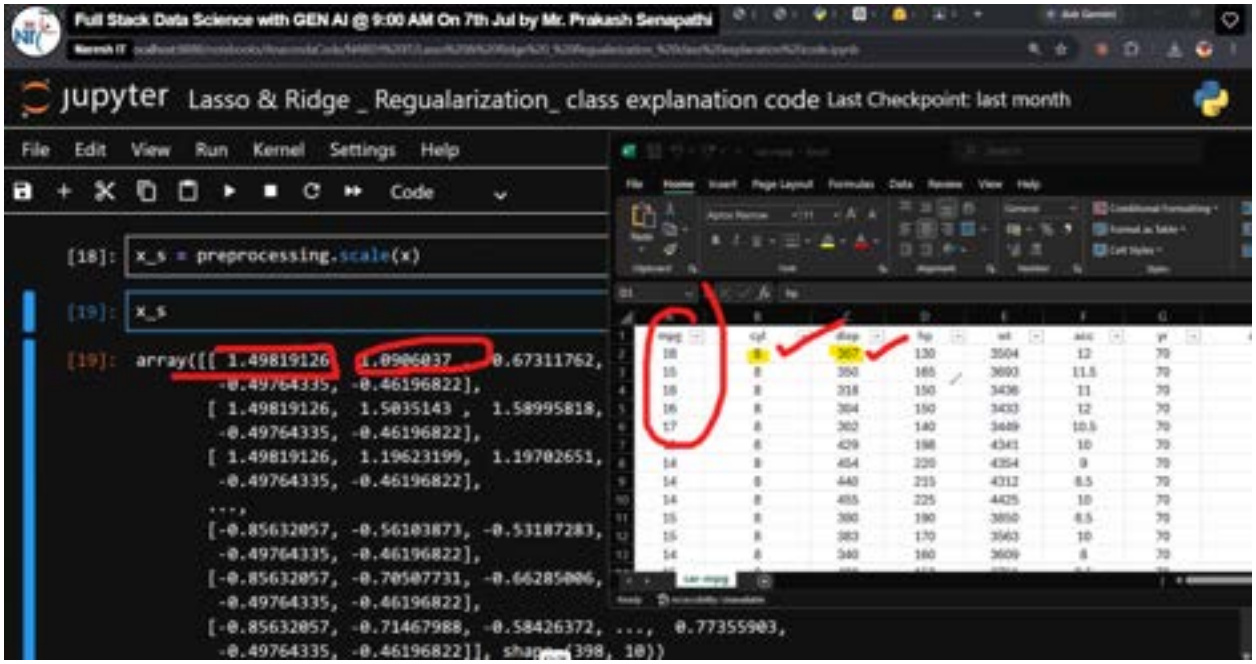
When we apply feature scaling to each and every number



```
[18]: x_s = preprocessing.scale(x)

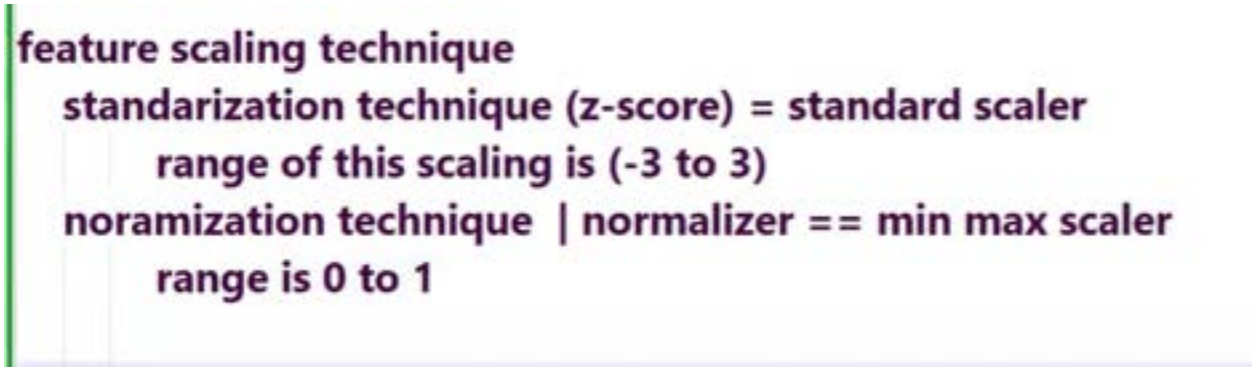
[19]: x_s

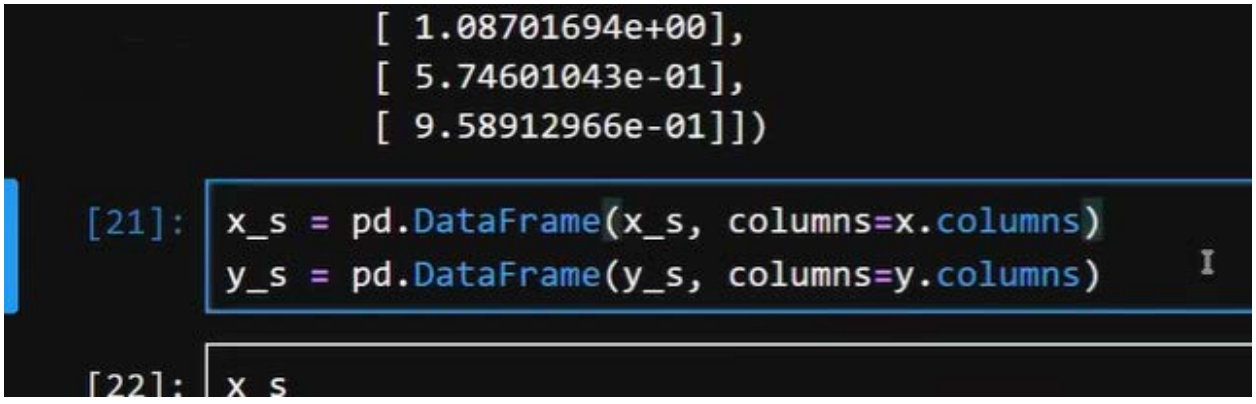
[19]: array([[ 1.49819126,  1.0906037 ,  0.67311762, ...,  0.77355903,
               -0.49764335, -0.46196822],
             [ 1.49819126,  1.5035143 ,  1.58995818, ...,  0.77355903,
              -0.49764335, -0.46196822],
             [ 1.49819126,  1.19623199,  1.19702651, ...,  0.77355903,
              -0.49764335, -0.46196822],
             ...,
             [-0.85632057, -0.56103873, -0.53187283, ...,  0.77355903,
              -0.49764335, -0.46196822],
             [-0.85632057, -0.70507731, -0.66285006, ...,  0.77355903,
              -0.49764335, -0.46196822],
             [-0.85632057, -0.71467988, -0.58426372, ...,  0.77355903,
              -0.49764335, -0.46196822]], shape=(398, 10))
```

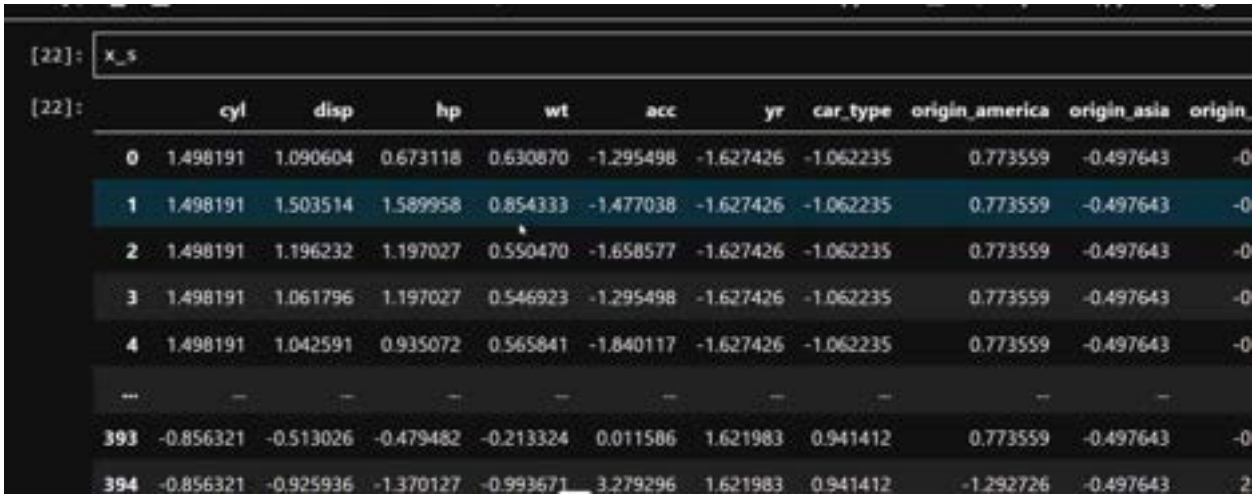


8 is scaled down to 1.49819126 and 307 scale down to 1.0906037 and 130 scale down to 0.67311762

For this reason ML needs scaling if we do not do scaling then accuracy is inconsistent
Standardization of this scaling range of z-score is -3 to 3
Normalization range is 0 to 1







```
[22]: x_s

[22]:
```

	cyl	disp	hp	wt	acc	yr	car_type	origin_america	origin_asia	origin_
0	1.498191	1.090604	0.673118	0.630870	-1.295498	-1.627426	-1.062235	0.773559	-0.497643	-0
1	1.498191	1.503514	1.589958	0.854333	-1.477038	-1.627426	-1.062235	0.773559	-0.497643	-0
2	1.498191	1.196232	1.197027	0.550470	-1.658577	-1.627426	-1.062235	0.773559	-0.497643	-0
3	1.498191	1.061796	1.197027	0.546923	-1.295498	-1.627426	-1.062235	0.773559	-0.497643	-0
4	1.498191	1.042591	0.935072	0.565841	-1.840117	-1.627426	-1.062235	0.773559	-0.497643	-0
...	...	...	...	...	...	...	...	...	...	
393	-0.856321	-0.513026	-0.479482	-0.213324	0.011586	1.621983	0.941412	0.773559	-0.497643	-0
394	-0.856321	-0.925936	-1.370127	-0.993671	3.279296	1.621983	0.941412	-1.292726	-0.497643	2



```
398 rows × 1 columns

[24]: x_train, x_test, y_train, y_test = train_test_split(x_s, y_s, test_size=0.3, random_state=0)

[27]: print(x_train.shape)
      print(x_test.shape)
      print(y_train.shape)
      print(y_test.shape)

      (278, 10)
      (120, 10)
      (278, 1)
      (120, 1)

[28]: len(data)
```



```
      (278, 1)
      (120, 1)

[28]: len(data)

[28]: 398

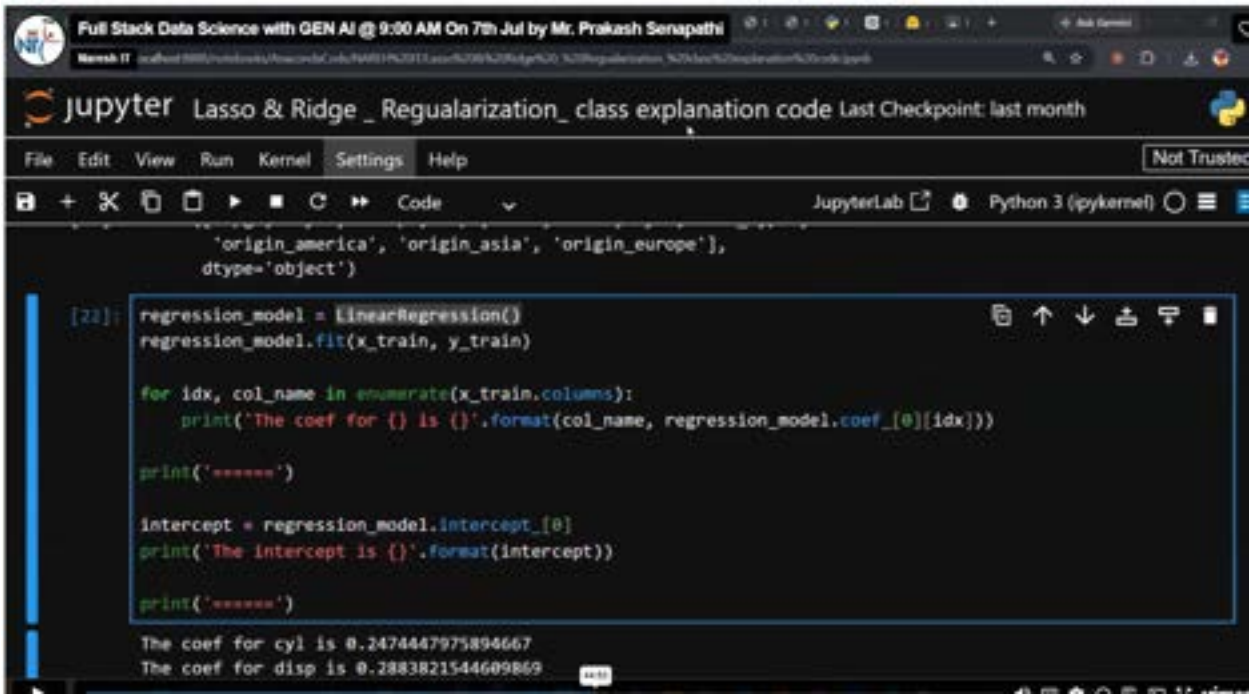
[29]: data.columns

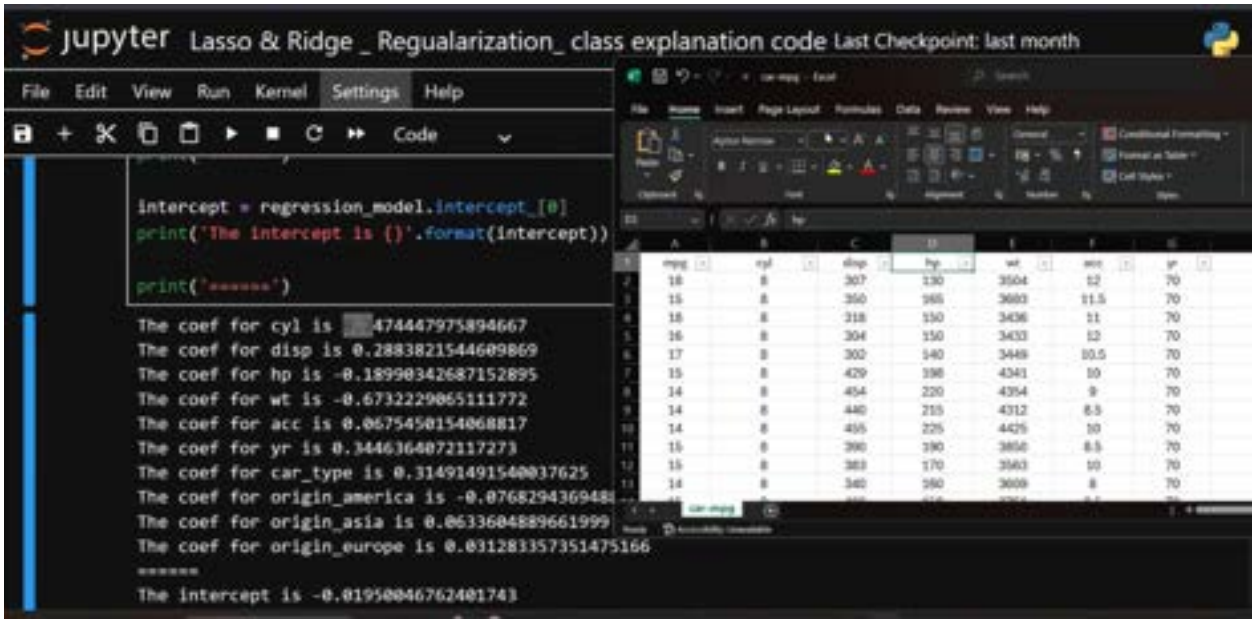
[29]: Index(['mpg', 'cyl', 'disp', 'hp', 'wt', 'acc', 'yr', 'car_type',
             'origin_america', 'origin_asia', 'origin_europe'],
            dtype='object')

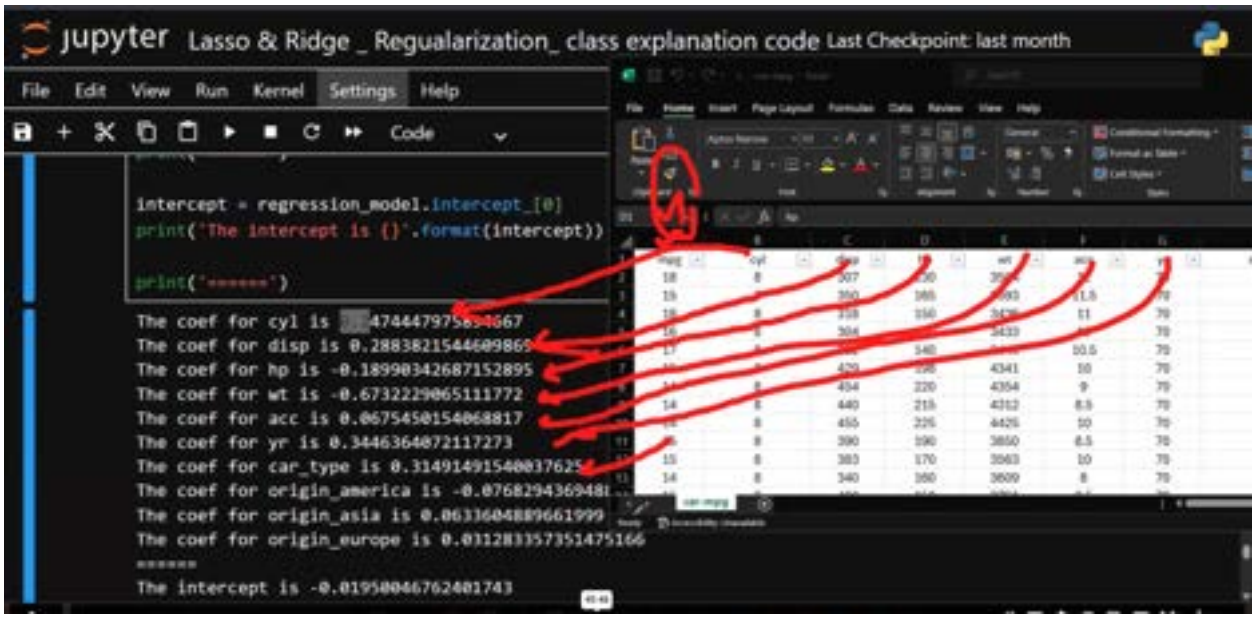
[22]: regression_model = LinearRegression()
      regression_model.fit(x_train, y_train)

      for idx, col_name in enumerate(x_train.columns):
          print('The coef for {} is {}'.format(col_name, regression_model.coef_[0][idx]))
```

Wer are building linear regression because for the coefficient


Full Stack Data Science with GEN AI @ 9:00 AM On 7th Jul by Mr. Prakash Senapathi
jupyter Lasso & Ridge _ Regualarization_ class explanation code Last Checkpoint: last month
File Edit View Run Kernel Settings Help
Not Trusted
Code
JupyterLab
Python 3 (ipykernel)
'origin_america', 'origin_asia', 'origin_europe'],
dtype='object')
[22]:
regression_model = LinearRegression()
regression_model.fit(x_train, y_train)
for idx, col_name in enumerate(x_train.columns):
print('The coef for {} is {}'.format(col_name, regression_model.coef_[0][idx]))
print('======')
intercept = regression_model.intercept_[0]
print('The intercept is {}'.format(intercept))
print('======')
The coef for cyl is 0.2474447975894667
The coef for disp is 0.2883821544609869


jupyter Lasso & Ridge _ Regualarization_ class explanation code Last Checkpoint: last month
File Edit View Run Kernel Settings Help
Code
intercept = regression_model.intercept_[0]
print('The intercept is {}'.format(intercept))
print('======')
The coef for cyl is 0.2474447975894667
The coef for disp is 0.2883821544609869
The coef for hp is -0.18990342687152895
The coef for wt is -0.6732229065111772
The coef for acc is 0.0675450154068817
The coef for yr is 0.3446364072117273
The coef for car_type is 0.31491491540037625
The coef for origin_america is -0.076829436948
The coef for origin_asia is 0.0633604889661999
The coef for origin_europe is 0.031283357351475166
======
The intercept is -0.01950046762401743



This value scale down to

```
intercept = regression_model.intercept_[0]
print('The intercept is {}'.format(intercept))

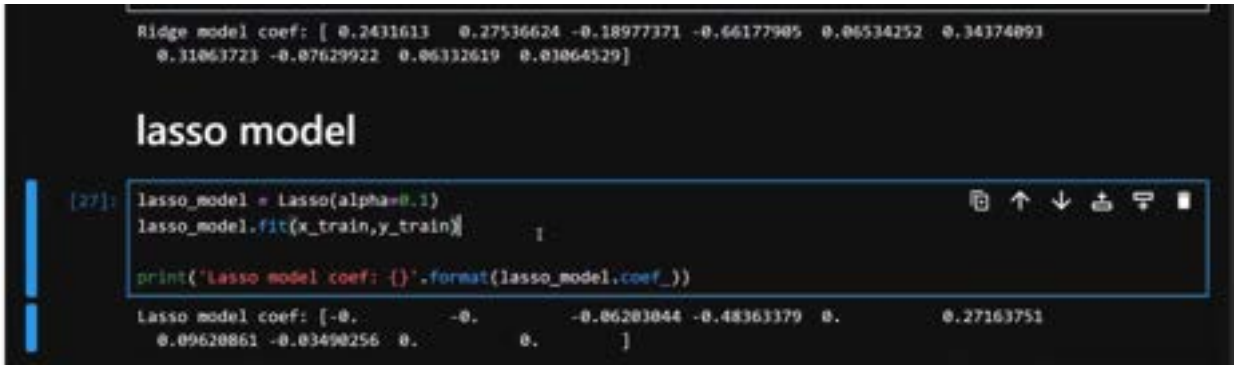
print('======')
```

```
The coef for cyl is 0.2474447975894667
The coef for disp is 0.2883821544609869
The coef for hp is -0.18990342687152895
The coef for wt is -0.6732229065111772
The coef for acc is 0.0675450154068817
The coef for yr is 0.3446364072117273
The coef for car_type is 0.31491491540037625
The coef for origin_america is -0.07682943694882868
The coef for origin_asia is 0.0633604889661999
The coef for origin_europe is 0.031283357351475166
======
The intercept is -0.01950046762401743
```

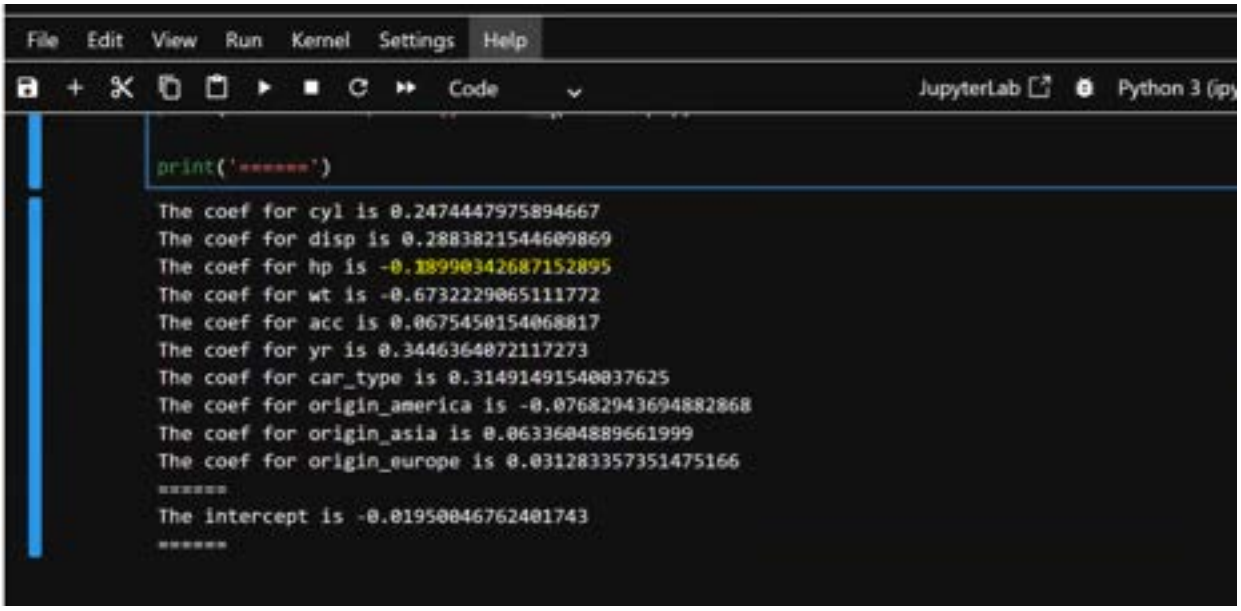


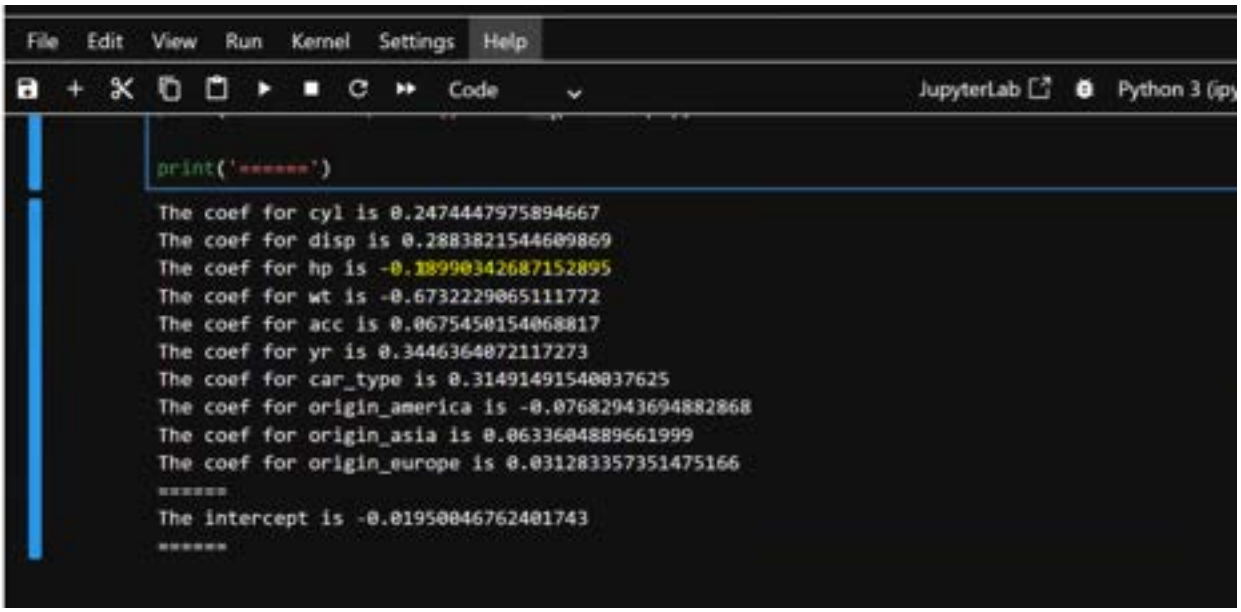
Here by applying ridge regression we are reducing coefficients

The moment when we apply lasso model

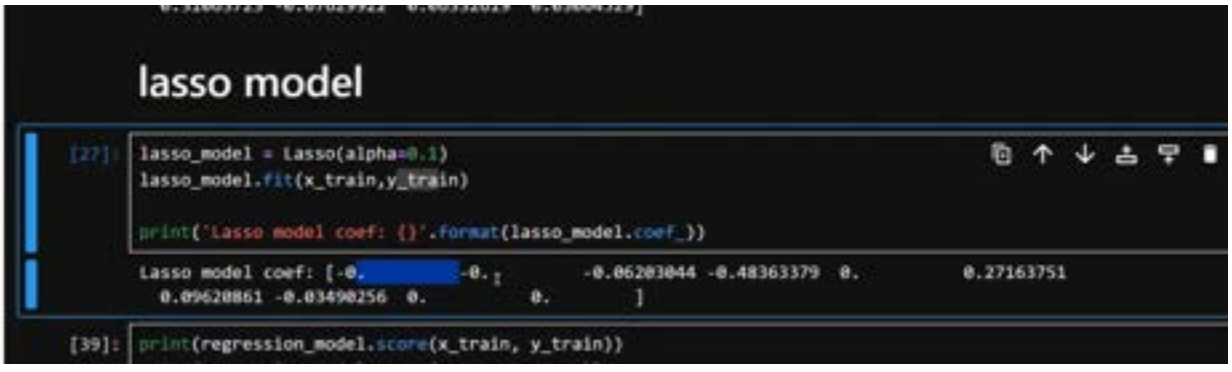


The 0.24 scale down to zero

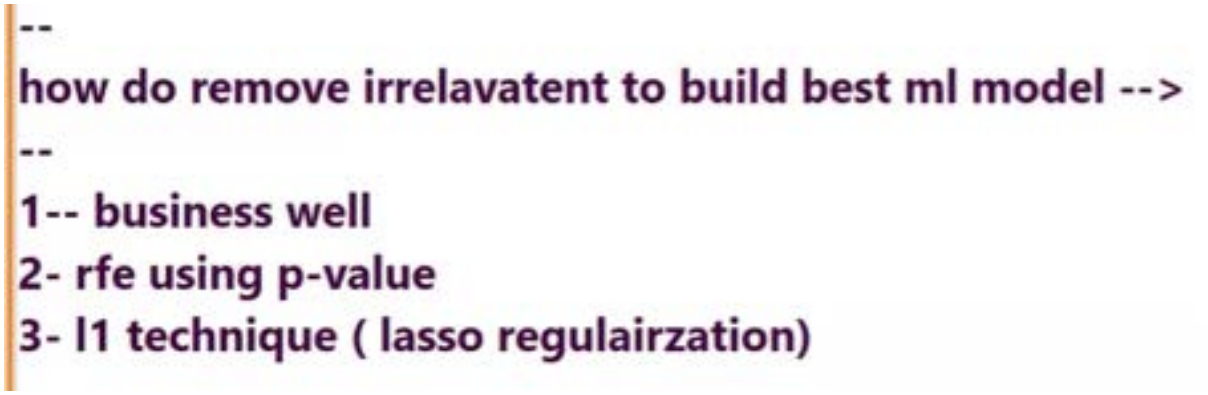




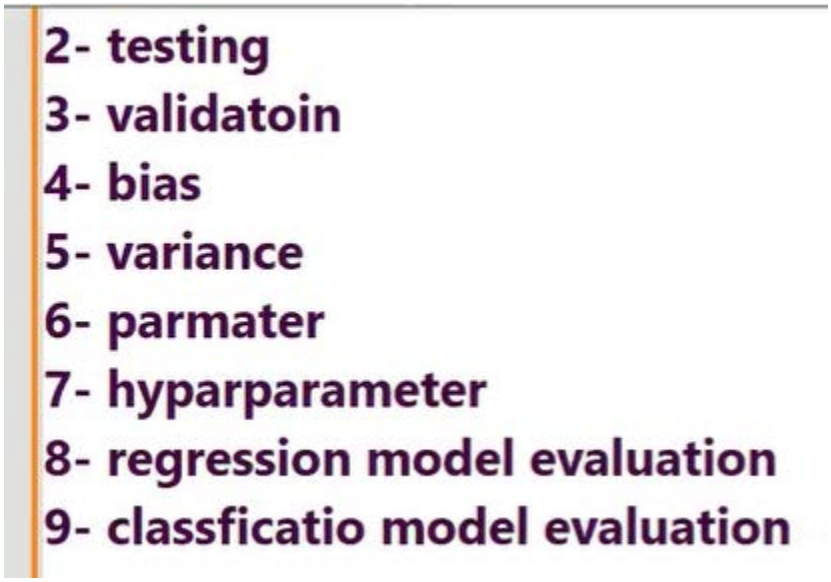
Out of many attributes some becomes zero this is called feature elimination



How to remove irrelevant features to build best ML model







Best resume

Session 57 - 7th Jul



TECHNICAL SKILLS

Agentic AI / LLM	LangGraph (StateGraph, Multi-Agent, Tool Nodes), CrewAI (Agent, Task, Crew, Process), LangChain (LCEL), LangSmith (Tracing & Observability), ChatPromptTemplate, StrOutputParser, OpenAI GPT, Gemini 2.5 Flash, Ollama (Gemma2:2b), Hugging Face Transformers, OpenRouter API
RAG & Vector DB	RAG Pipeline Design, FAISS, ChromaDB, Document Loaders, Text Splitters, Embedding Models, Retrieval QA Chains, Gradio UI Integration, PDF Ingestion, Semantic Search
ML / AI	Scikit-learn, Pandas, NumPy, Matplotlib, Seaborn, Statistical Modeling, Feature Engineering, Predictive Analytics, Supervised & Unsupervised Learning, Customer Segmentation (K-Means, DBSCAN), Hypothesis Testing
Computer Vision	OpenCV, CNN (TensorFlow/Keras), Image Classification, Object Detection, Vehicle Tracking, Background Subtraction, Haar Cascade, EuclideanDistTracker, Video Stream Processing
Deep Learning	TensorFlow, Keras, CNN (Conv2D, MaxPooling2D, Flatten, Dense, Dropout), Sequential API, Data Augmentation, Transfer Learning, TFLite Export
NLP	Text Preprocessing, Tokenization, TF-IDF, Sentiment Analysis, Customer Feedback Analysis, VADER, TextBlob, Word Embeddings (Word2Vec/GloVe), Hugging Face Transformers, Prompt Engineering
App Frameworks	Gradio, Streamlit, FastAPI (in progress), REST APIs, ASP.NET Core (.NET 8/6)
LLMOps/MLOps / CI-CD	LLMOps/ LangSmith (LLM Observability & Trace Monitoring), GitHub Actions, MLflow, Experiment Tracking, Model Versioning
Data	PostgreSQL, MS SQL Server
Programming	Python (primary), SQL

57:45

CORE COMPETENCIES

Agentic AI Architecture	Multi-Agent Orchestration	LangGraph Workflows	CrewAI Framework
RAG Pipeline Design	Vector DB & Semantic Search	LLM App Development	Prompt Engineering

Gen AI / LLM Engineering	LangChain & LangSmith	Customer Feedback NLP	Customer Segmentation
Image Classification / CV	Deep Learning / CNN	NLP & Transformers	MLOps & CI/CD Pipelines
Machine Learning (ML)	Feature Engineering	LLMOps	Predictive Analytics
OpenAI / Gemini / Ollama	Gradio & Streamlit Apps	SQL & DB Optimization	Agile / Scrum

GEN AI, AGENTIC AI & ML PROJECTS

GEN AI, AGENTIC AI & ML PROJECTS

RAG Application with Gradio — Document Intelligence Chatbot

Python • LangChain • FAISS / ChromaDB • HuggingFace Embeddings • Gradio • OpenAI / Gemini • PDF Loader • RecursiveCharacterTextSplitter

- Engineered an end-to-end Retrieval-Augmented Generation (RAG) application with a Gradio chat interface — enabling users to upload domain-specific documents (PDF, TXT) and ask natural language questions answered with grounded, context-aware responses from the uploaded knowledge base.
- Built a modular document ingestion pipeline using LangChain's PyPDFLoader and RecursiveCharacterTextSplitter — chunking documents into semantically coherent segments with configurable chunk size (512 tokens) and overlap (64 tokens) to maximise retrieval precision.
- Integrated FAISS vector store with HuggingFace sentence-transformer embeddings (all-MiniLM-L6-v2) for fast, dense similarity search — enabling sub-second semantic retrieval of the top-k most relevant document chunks for every user query.
- Implemented RetrievalQA chain with a custom ChatPromptTemplate injecting retrieved context directly into the LLM prompt — ensuring factual, document-grounded responses while preventing hallucination on out-of-scope questions.
- Delivered interactive Gradio UI with file upload, conversational history, and streaming LLM response rendering — providing a polished, user-friendly RAG demonstration deployable on HuggingFace Spaces or local environments.

LangGraph Multi-Agent Orchestration System

Python • LangGraph • LangChain • StateGraph • Tool Nodes • OpenAI / Gemini • LangSmith • Conditional Edges

- Architected a stateful multi-agent workflow using LangGraph's StateGraph — defining discrete agent nodes (Researcher, Planner, Executor, Reviewer) with typed state management via TypedDict, enabling clean, inspectable agent-to-agent message passing.

RAG Application with Gradio — Document Intelligence Chatbot

Python • LangChain • FAISS / ChromaDB • HuggingFace Embeddings • Gradio • OpenAI / Gemini • PDF Loader • RecursiveCharacterTextSplitter

- Engineered an end-to-end Retrieval-Augmented Generation (RAG) application with a Gradio chat interface — enabling users to upload domain-specific documents (PDF, TXT) and ask natural language questions answered with grounded, context-aware responses from the uploaded knowledge base.
- Built a modular document ingestion pipeline using LangChain's PyPDFLoader and RecursiveCharacterTextSplitter — chunking documents into semantically coherent segments with configurable chunk size (512 tokens) and overlap (64 tokens) to maximise retrieval precision.
- Integrated FAISS vector store with HuggingFace sentence-transformer embeddings (all-MiniLM-L6-v2) for fast, dense similarity search — enabling sub-second semantic retrieval of the top-k most relevant document chunks for every user query.
- Implemented RetrievalQA chain with a custom ChatPromptTemplate injecting retrieved context directly into the LLM prompt — ensuring factual, document-grounded responses while preventing hallucination on out-of-scope questions.
- Delivered interactive Gradio UI with file upload, conversational history, and streaming LLM response rendering — providing a polished, user-friendly RAG demonstration deployable on HuggingFace Spaces or local environments.

LangGraph Multi-Agent Orchestration System

Python • LangGraph • LangChain • StateGraph • Tool Nodes • OpenAI / Gemini • LangSmith • Conditional Edges

- Architected a stateful multi-agent workflow using LangGraph's StateGraph — defining discrete agent nodes (Researcher, Planner, Executor, Reviewer) with typed state management via TypedDict, enabling clean, inspectable agent-to-agent message passing.
- Implemented conditional routing logic using LangGraph's conditional_edges — dynamically directing task flow based on agent output, enabling adaptive decision trees where the Reviewer node loops back to Executor for refinement or routes to END upon task completion.
- Equipped each agent node with specialised tools (web search, code execution, document retrieval) via LangChain's ToolNode — enabling tool-augmented agent reasoning with automatic tool invocation, argument parsing, and result injection back into the graph state.
- Integrated LangSmith full-pipeline tracing across the entire LangGraph execution — capturing per-node latency, state transitions, tool call inputs/outputs, and LLM token usage for comprehensive agentic workflow observability and debugging.
- Visualised the agent graph topology using LangGraph's Mermaid diagram export — providing clear architectural documentation of agent roles, edge conditions, and workflow states for team review and iterative design refinement.

Python • LangGraph • LangChain • StateGraph • Tool Nodes • OpenAI / Gemini • LangSmith • Conditional Edges

- Architected a stateful multi-agent workflow using LangGraph's StateGraph — defining discrete agent nodes (Researcher, Planner, Executor, Reviewer) with typed state management via TypedDict, enabling clean, inspectable agent-to-agent message passing.
- Implemented conditional routing logic using LangGraph's conditional_edges — dynamically directing task flow based on agent output, enabling adaptive decision trees where the Reviewer node loops back to Executor for refinement or routes to END upon task completion.
- Equipped each agent node with specialised tools (web search, code execution, document retrieval) via LangChain's ToolNode — enabling tool-augmented agent reasoning with automatic tool invocation, argument parsing, and result injection back into the graph state.
- Integrated LangSmith full-pipeline tracing across the entire LangGraph execution — capturing per-node latency, state transitions, tool call inputs/outputs, and LLM token usage for comprehensive agentic workflow observability and debugging.
- Visualised the agent graph topology using LangGraph's Mermaid diagram export — providing clear architectural documentation of agent roles, edge conditions, and workflow states for team review and iterative design refinement.

Multi-Agent AI System — CrewAI Framework

Python • CrewAI • LangChain • OpenAI GPT / Gemini • CrewAI Tools • Agent • Task • Crew • Process

- Designed and deployed a collaborative multi-agent system using CrewAI's Agent-Task-Crew architecture — defining specialised agents (Research Agent, Analysis Agent, Writing Agent, QA Agent) with distinct roles, goals, backstories, and tool access for autonomous task delegation.
- Configured CrewAI's sequential and hierarchical Process modes — enabling both linear task pipelines (each agent output feeds the next) and manager-led hierarchical delegation where a manager agent dynamically assigns subtasks to worker agents based on task complexity.
- Equipped agents with custom tools (SerperDevTool for web search, FileReadTool for document access, custom Python functions) — extending agent capabilities beyond base LLM reasoning into real-world action execution with structured tool outputs.
- Implemented CrewAI's memory system (short-term, long-term, entity memory) — enabling agents to persist knowledge across task execution, recall prior research findings, and maintain coherent context in extended multi-step workflows.

- Benchmarked CrewAI vs LangGraph for agentic task execution — documenting tradeoffs: CrewAI excels at role-based collaboration with minimal setup; LangGraph provides finer-grained control for complex state machines and conditional agent routing.

Customer Feedback Intelligence System — NLP Pipeline

Python • NLTK • VADER • TextBlob • Scikit-learn • Transformers • TF-IDF • Pandas • Matplotlib • Streamlit

- Built an end-to-end NLP pipeline for customer feedback analysis — ingesting raw review text, applying multi-stage preprocessing (lowercasing, punctuation removal, stopword filtering, lemmatization) and extracting actionable sentiment signals at scale.
- Implemented multi-model sentiment scoring pipeline combining VADER (rule-based, excels on short social text), TextBlob (lexicon-based polarity), and a fine-tuned Hugging Face transformer (distilbert-base-uncased-finetuned-sst-2-english) — ensemble voting achieving ~88% accuracy on held-out customer review test set.
- Engineered TF-IDF based topic extraction to identify the most frequently co-occurring terms in negative feedback segments — surfacing actionable product/service pain points (e.g., "delivery delay", "poor packaging") for direct product team intervention.
- Designed automated feedback categorisation using Logistic Regression classifier (trained on labelled feedback categories: Product Quality, Delivery, Customer Service, Pricing) with TF-IDF features — routing incoming feedback to the correct department without manual triage.
- Packaged the entire analytics pipeline into an interactive Streamlit dashboard — enabling non-technical stakeholders to paste review batches, receive real-time sentiment breakdowns, topic clouds, and trend visualisations without code.

Customer Segmentation Engine — Unsupervised ML

Python • Scikit-learn • K-Means • DBSCAN • PCA • Pandas • Seaborn • Matplotlib • RFM Analysis • StandardScaler

- Engineered a customer segmentation pipeline on transactional e-commerce data — constructing RFM (Recency, Frequency, Monetary) features from raw order history, applying StandardScaler normalisation, and using the Elbow Method + Silhouette Score to determine the optimal cluster count (k=4).
- Implemented K-Means clustering to segment customers into behavioural cohorts (Champions, Loyal Customers, At-Risk, Lost) — validating cluster quality with inertia curves, Davies-Bouldin index, and Silhouette coefficient (achieved 0.62 for k=4).
- Applied DBSCAN as a complementary density-based clustering approach — [illegible]ssfully isolating high-value outlier

- Engineered a customer segmentation pipeline on transactional e-commerce data — constructing RFM (Recency, Frequency, Monetary) features from raw order history, applying StandardScaler normalisation, and using the Elbow Method + Silhouette Score to determine the optimal cluster count (k=4).
- Implemented K-Means clustering to segment customers into behavioural cohorts (Champions, Loyal Customers, At-Risk, Lost) — validating cluster quality with inertia curves, Davies-Bouldin index, and Silhouette coefficient (achieved 0.62 for k=4).
- Applied DBSCAN as a complementary density-based clustering approach — successfully isolating high-value outlier customer clusters and noise points that K-Means incorrectly merged due to its sensitivity to spherical cluster assumptions.
- Dimensionality-reduced feature space to 2D using PCA for cluster visualisation — producing interpretable scatter plots revealing distinct customer segments with minimal inter-cluster overlap, supporting executive-level business presentations.
- Delivered segment-specific business recommendations (personalised offers for Champions, win-back campaigns for At-Risk) backed by statistical cluster profiling — translating ML output directly into actionable CRM and marketing strategy insights.

Image Classification System — Deep CNN with Transfer Learning

Python • TensorFlow • Keras • CNN (Conv2D / MaxPooling2D / Dropout) • MobileNetV2 (Transfer Learning) • ImageDataGenerator • OpenCV • Gradio

- Designed and trained a multi-class image classification CNN from scratch using Keras Sequential API — architecture: Conv2D(32) → MaxPooling2D → Conv2D(64) → MaxPooling2D → Conv2D(128) → Flatten → Dense(256) → Dropout(0.5) → Softmax output layer achieving 91% test accuracy.
- Implemented Transfer Learning using MobileNetV2 pre-trained on ImageNet as the feature extractor backbone — freezing convolutional layers, replacing the classification head, and fine-tuning the top 20 layers with a low learning rate (1e-5) to achieve 94% accuracy with ~5× faster convergence vs training from scratch.
- Applied comprehensive data augmentation pipeline via ImageDataGenerator (rotation_range=20, zoom_range=0.2, horizontal_flip=True, width/height_shift_range=0.1) — synthetically expanding training set diversity and reducing overfitting gap between training and validation accuracy.
- Monitored training via accuracy/loss curves, confusion matrix heatmaps, and per-class precision-recall analysis — identifying underperforming classes and informing targeted data collection to address class imbalance (class weights applied via compute_class_weight).
- Deployed trained model behind an interactive Gradio image upload interface — enabling real-time classification inference with confidence scores, accessible via browser without any code, demonstrating end-to-end ML productionisation from notebook to deployed UI.

Custom GPT Chatbot — OpenAI GPT with LangSmith Observability

Python • LangChain • ChatOpenAI • ChatPromptTemplate • StrOutputParser • Lan... • Streamlit • dotenv

58:54

- Architected a production-grade LLM chatbot using LangChain LCEL pipeline (ChatPromptTemplate → ChatOpenAI → StrOutputParser) with LangSmith full-pipeline tracing capturing prompt inputs, token usage, latency, and chain execution metadata.
- Built interactive Streamlit frontend with real-time LLM response rendering; implemented secure API key management via python-dotenv following production-grade secrets management best practices.

Gemini 2.5 Flash Chatbot — Google GenAI with LangSmith Integration

Python • LangChain • langchain-google-genai • Gemini 2.5 Flash • LangSmith • Streamlit • LCEL

- Engineered multimodal LLM chatbot leveraging Gemini 2.5 Flash via langchain-google-genai with LCEL pipeline architecture — configured model parameters (temperature=0.2, max_output_tokens=1024) for deterministic production response generation with LangSmith observability.

Toll-Route Traffic Intelligence System — Live Vehicle Tracking

Python • OpenCV • NumPy • EuclideanDistTracker • White Mask • findContours • BoundingRect

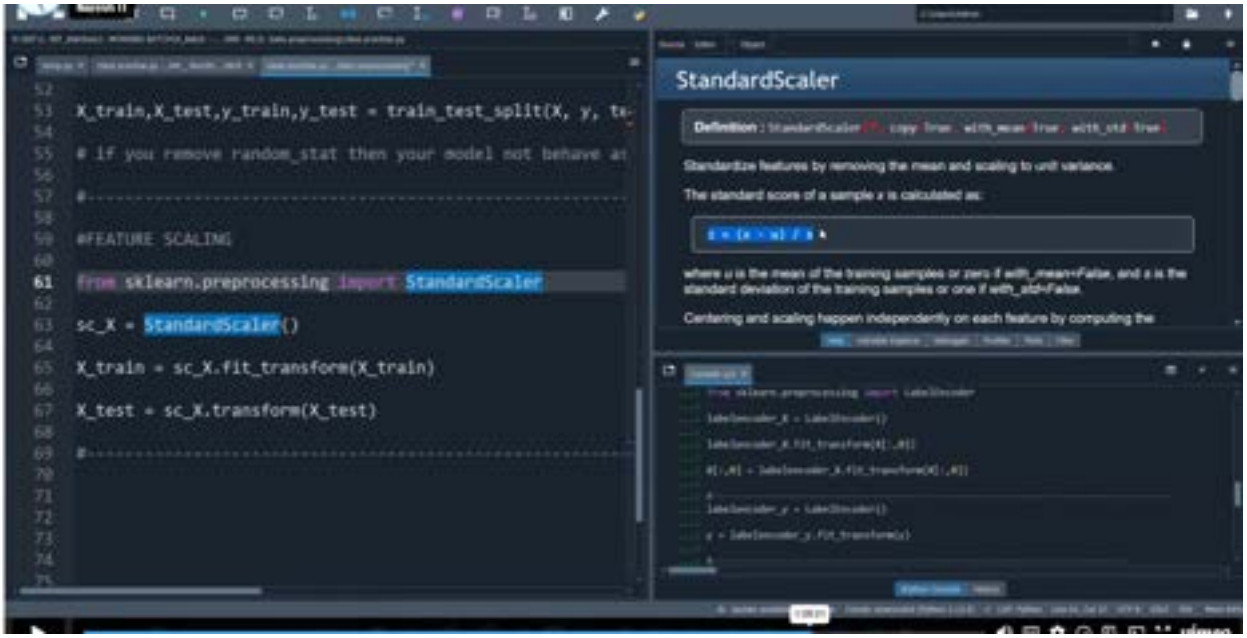
- Engineered real-time vehicle detection and tracking system processing live MP4 video streams using cv2.VideoCapture with custom EuclideanDistTracker maintaining persistent object identities via centroid-distance association and White Mask background segmentation.

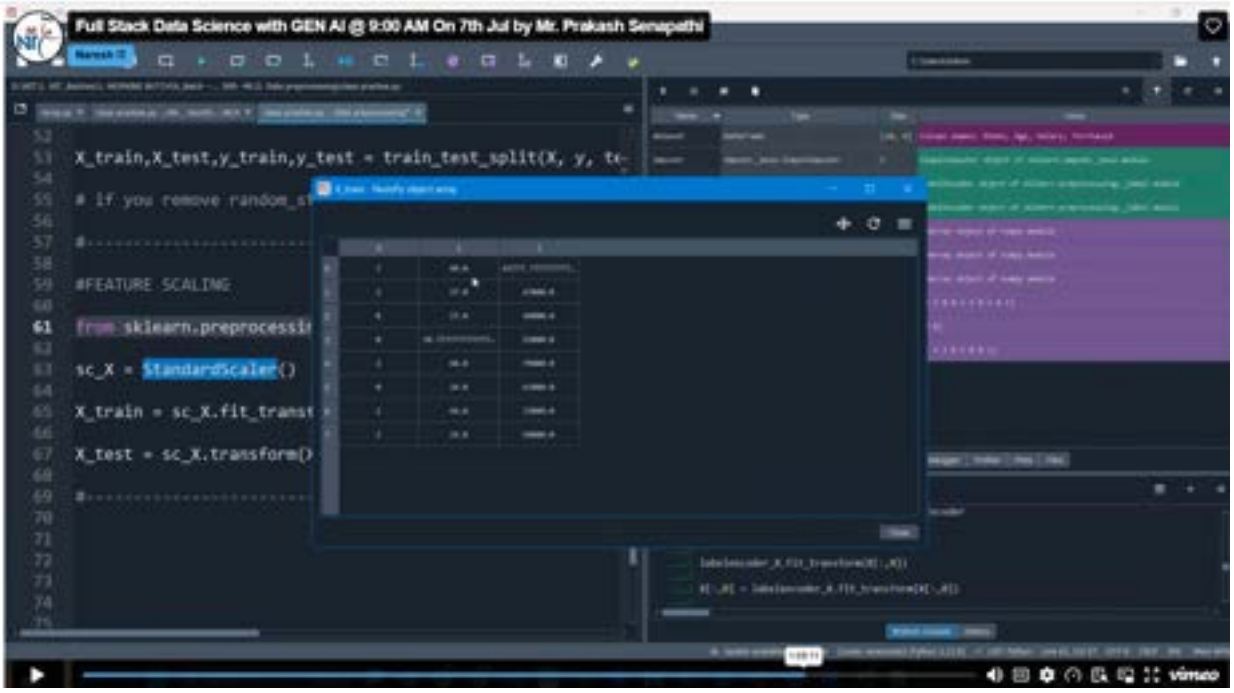
Facial Emotion Recognition System — Binary Expression Classifier

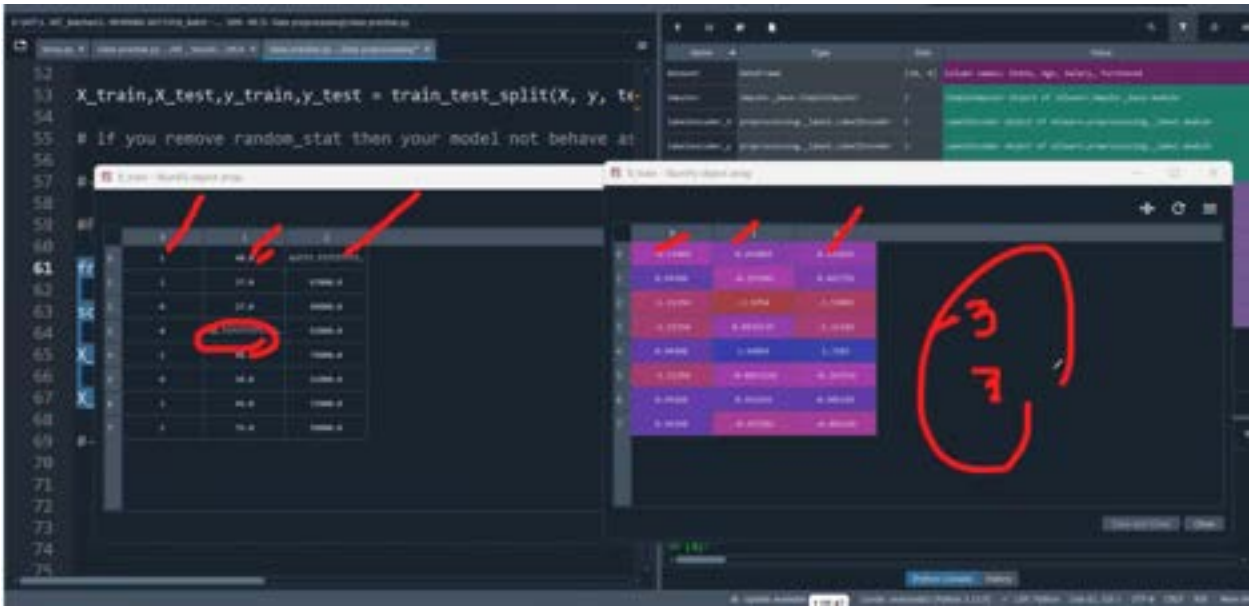
Python • TensorFlow • Keras • OpenCV • CNN: Conv2D / MaxPooling2D / Dropout / Dense • ImageDataGenerator

- Designed and trained a deep CNN image classifier (Conv2D → MaxPooling2D → Dropout → Flatten → Dense) with data augmentation for facial expression classification (Happy/Sad) with real-time OpenCV integration and TFLite export for edge deployment.

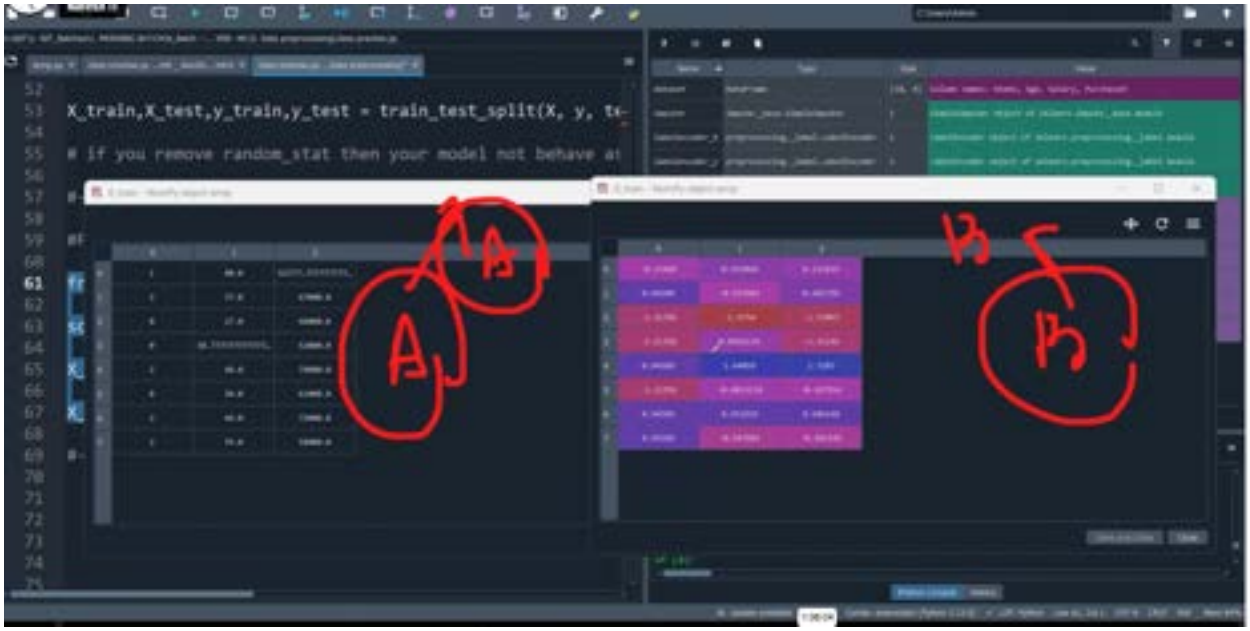
59:03


X_train,X_test,y_train,y_test = train_test_split(X, y, te
if you remove random_stat then your model not behave as
#FEATURE SCALING
from sklearn.preprocessing import StandardScaler
sc_X = StandardScaler()
X_train = sc_X.fit_transform(X_train)
X_test = sc_X.transform(X_test)
StandardScaler
Definition : StandardScaler(*, copy: True, with_mean: True, with_std: True)
Standardize features by removing the mean and scaling to unit variance
The standard score of a sample x is calculated as:
z = (x - u) / s
where u is the mean of the training samples or zero if with_mean=False, and s is the standard deviation of the training samples or one if with_std=False.
Centering and scaling happen independently on each feature by computing the

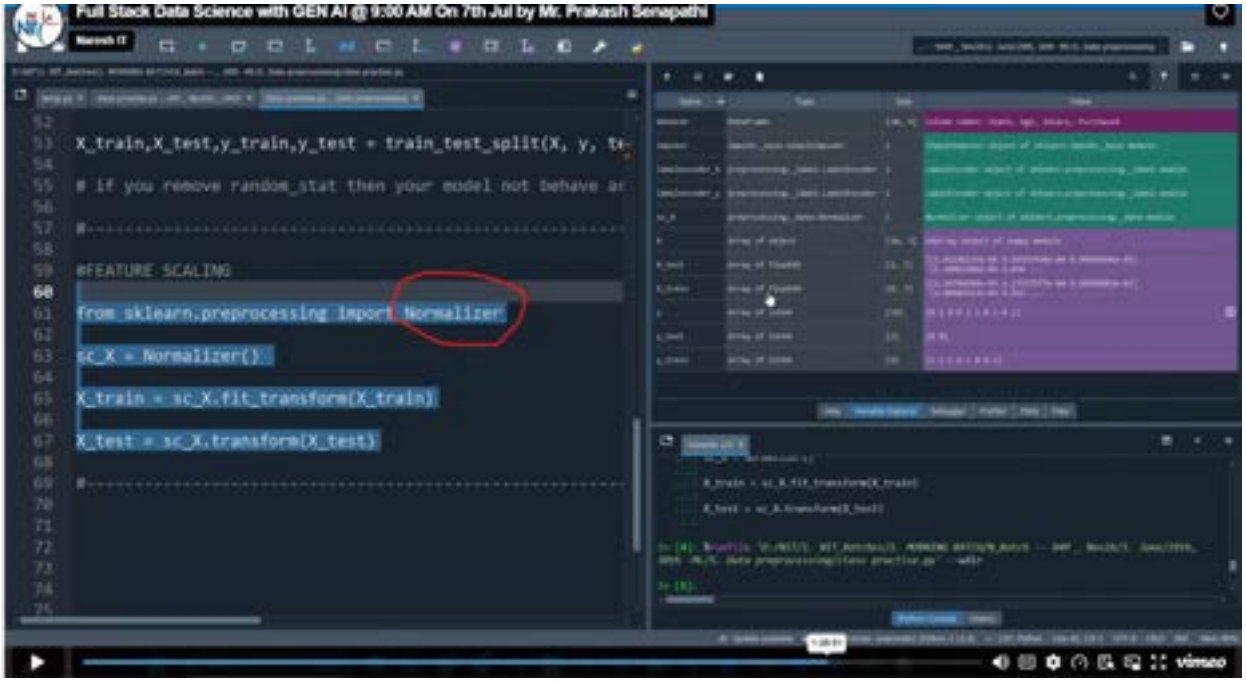

Full Stack Data Science with GEN AI @ 9:00 AM On 7th Jul by Mr. Prakash Senapathi
X_train,X_test,y_train,y_test = train_test_split(X, y, te
#FEATURE SCALING
sc_X = StandardScaler()
vimeo



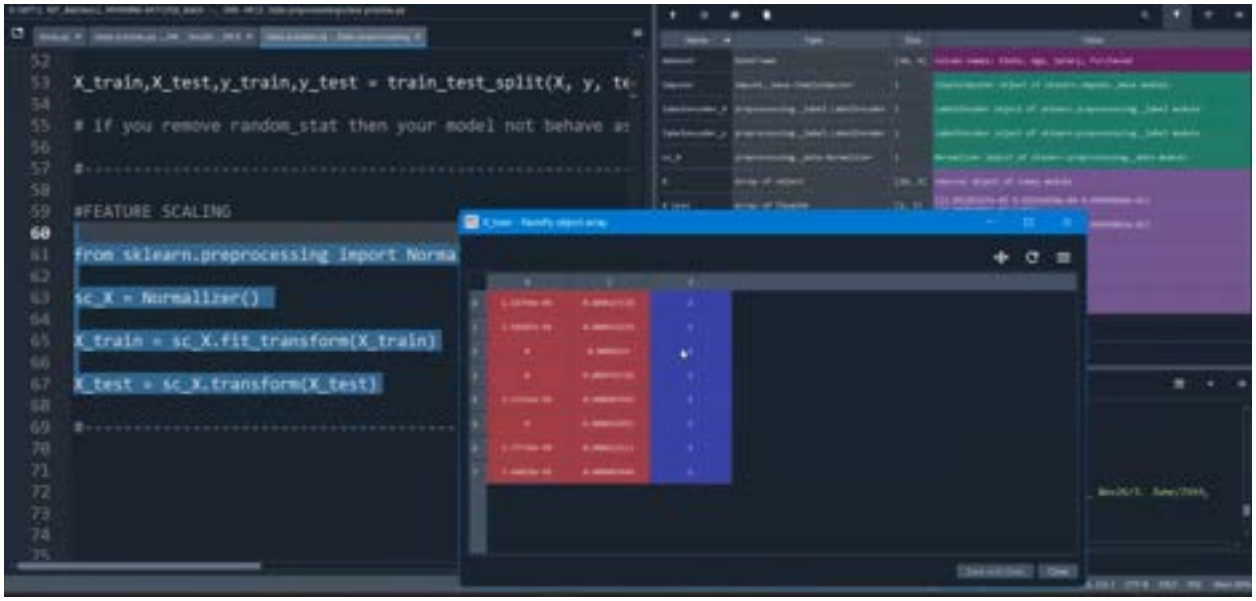
All high numbers reduced to -3 to 3



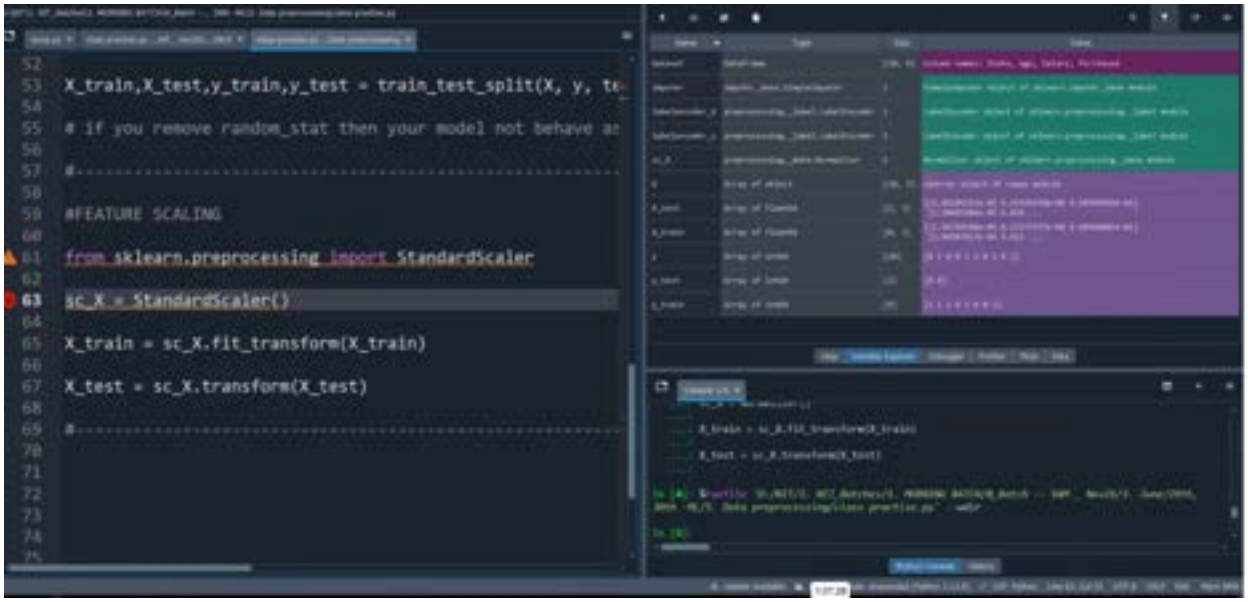
If we buld ML model by using A and B which is more accuracy the answer is B because the numbers in B are in equal scale this is called feature scaling here feature means attributes or columns

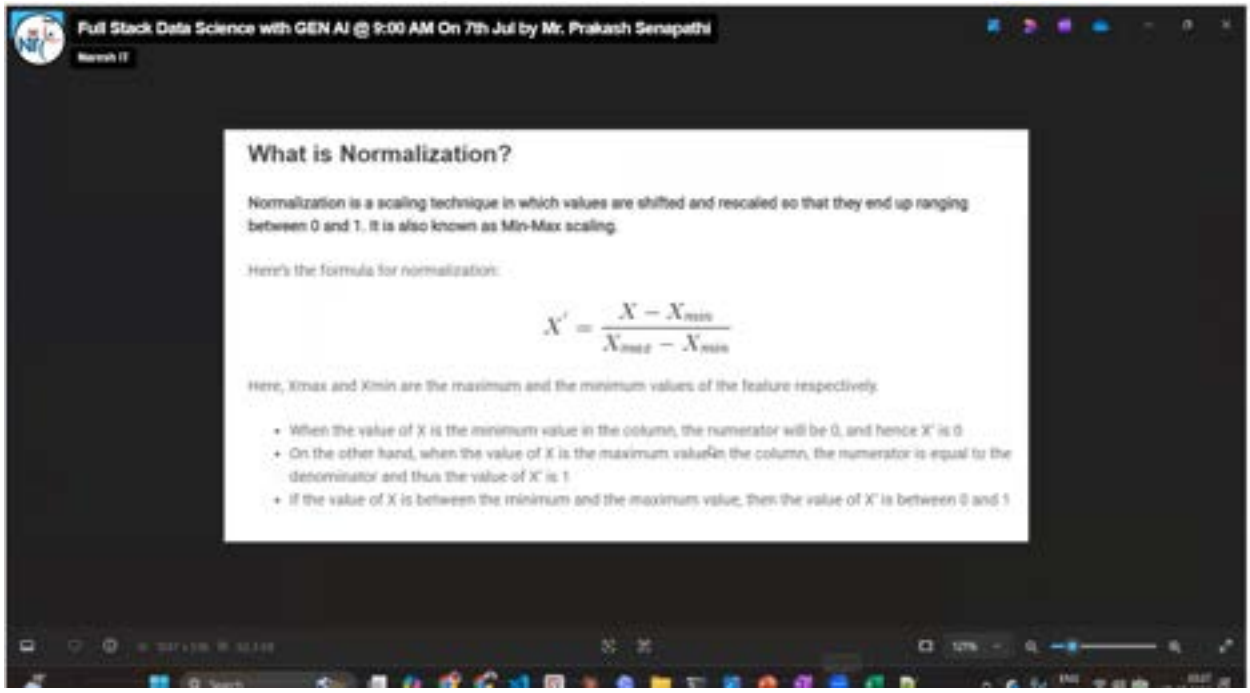


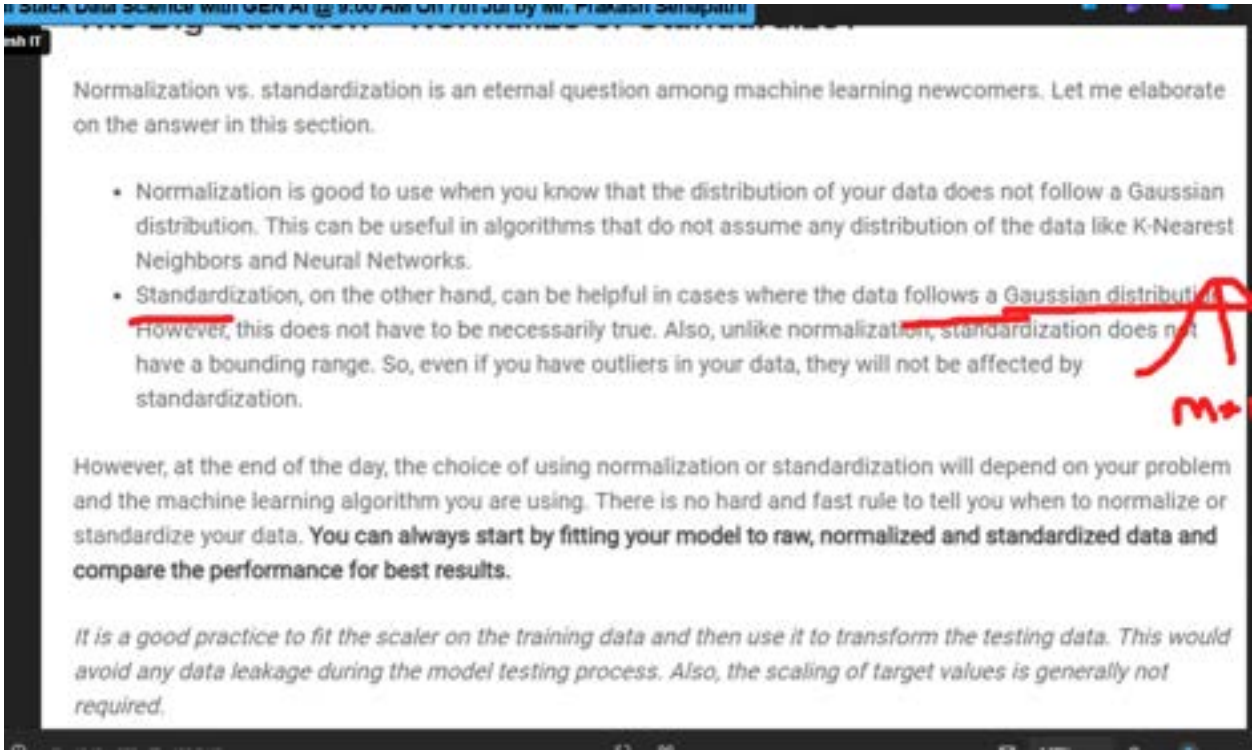
If we change to normalizer we will get the below values 1 or 0 like that



Note : In normalization technique we never get negative value out of standardization and normalization first better is standardization why standardization means it follows gaussian distribution

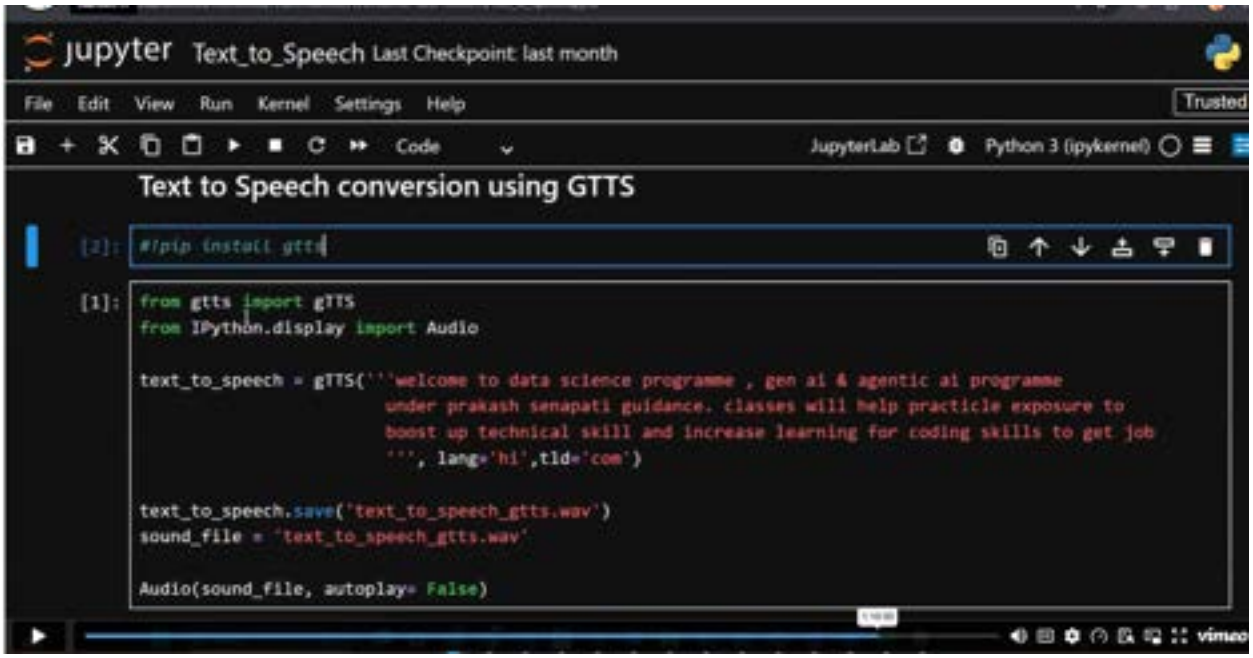

X_train,X_test,y_train,y_test = train_test_split(X, y, te
if you remove random_stat then your model not behave as
#FEATURE SCALING
from sklearn.preprocessing import StandardScaler
sc_X = StandardScaler()
X_train = sc_X.fit_transform(X_train)
X_test = sc_X.transform(X_test)


Full Stack Data Science with GEN AI @ 9:00 AM On 7th Jul by Mr. Prakash Senapathi
Naresh IT
What is Normalization?
Normalization is a scaling technique in which values are shifted and rescaled so that they end up ranging between 0 and 1. It is also known as Min-Max scaling.
Here's the formula for normalization:
$$X' = \frac{X - X_{min}}{X_{max} - X_{min}}$$
Here, Xmax and Xmin are the maximum and the minimum values of the feature respectively.
When the value of X is the minimum value in the column, the numerator will be 0, and hence X' is 0
On the other hand, when the value of X is the maximum value in the column, the numerator is equal to the denominator and thus the value of X' is 1
If the value of X is between the minimum and the maximum value, then the value of X' is between 0 and 1



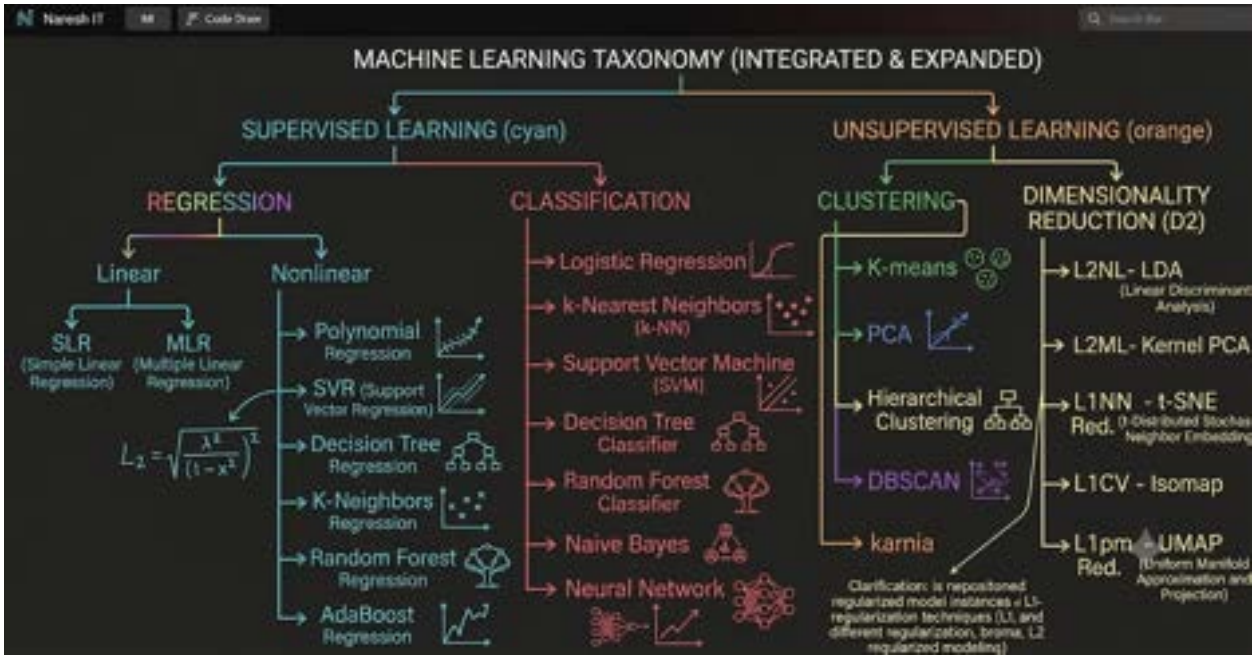
Gaussian distribution is bell curve because mean = median = mode this followed by standardization.

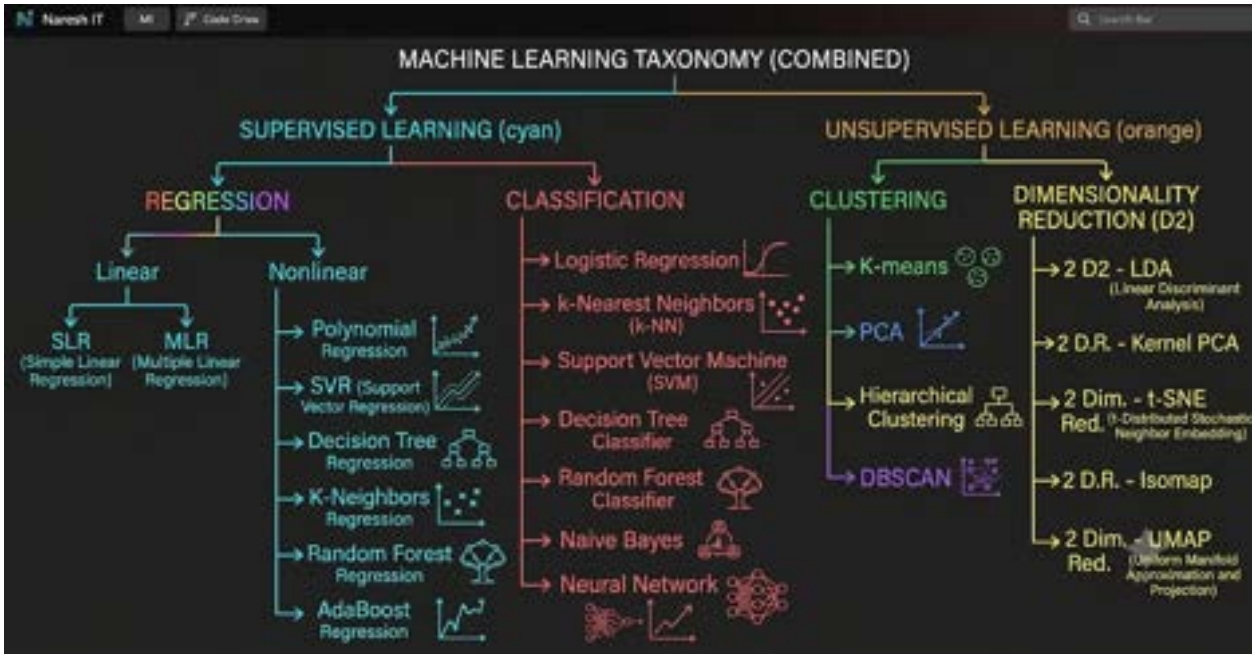
Normalization does not follow gaussian distribution so standardization is first priority and normalization is second priority



07-07-2026	Regularization | feature scaling	Regularization technqie -- l1 , l2 regularize high coefficient to low coefficinet -- regularization ridge l2 - reduce high coef to low coef | lasso l1 -- reduce high coef to 0 we are eliminat the attribute | featureing scaling technique feature enginnering technique(eda) vs feature scaling technique(standarization & normalization) , feature selection technique(rfe, l1, business)

06-06-2026	Healthcare domain -- CAPSTONE LIVE DOAMIN	project -- heart diseas analysis
06-06-2026	IMDB Data analysis	project- Movie rating analysis
08-06-2026	llm using ollama	ollam with streamlit
19-06-2026	data anlaysis using sql vs python	how to extract raw data from the database
22-06-2026	descriptive stats	Income expenses dataset
23-06-2026	Inferential stats	Sales retial analysis
29-06-2026	data processing pipeline	classification data -- vechicle is purchased
30-06-2026	bank risk anlaysi s	RESUME PROJECT
02-07-2026	Simple linear regression model	Employe salary prediction
04-07-2026	Simple linear regression model	Student mark prediction using flask & html
06-07-2026	multiple linear regression model	investor data nalysis to find opuimal attribute
07-07-2026	regularization	l1 & l2 model
08-07-2026		





ml -- regression

linear --

slr, mlr, l1, l2, gd, sgd, time series

non linear

svr, poly, knn regression model, dtr, random forest

Gradident descend -->

stocastic graident descend

batch gradient descend

model geneated value > 0.05 -- we reject the null hypothesis
reject null hypot statsion === eliminat the attribute in machine learning
==== 7th

overfitting -- train the model with more attribute
overfitting -- low bias high variance
bias -- training & varianace -- testing part
to reduce overfitting we need to remove some attribute

to reduce overfitting we have some technique
regularization technique
cross validation
ensamble learning
pca
drop out the neurons

Bias means —--------Training part
Variance means —---Testing part

Feature ellimation
Recursive ellimination in machine leanring

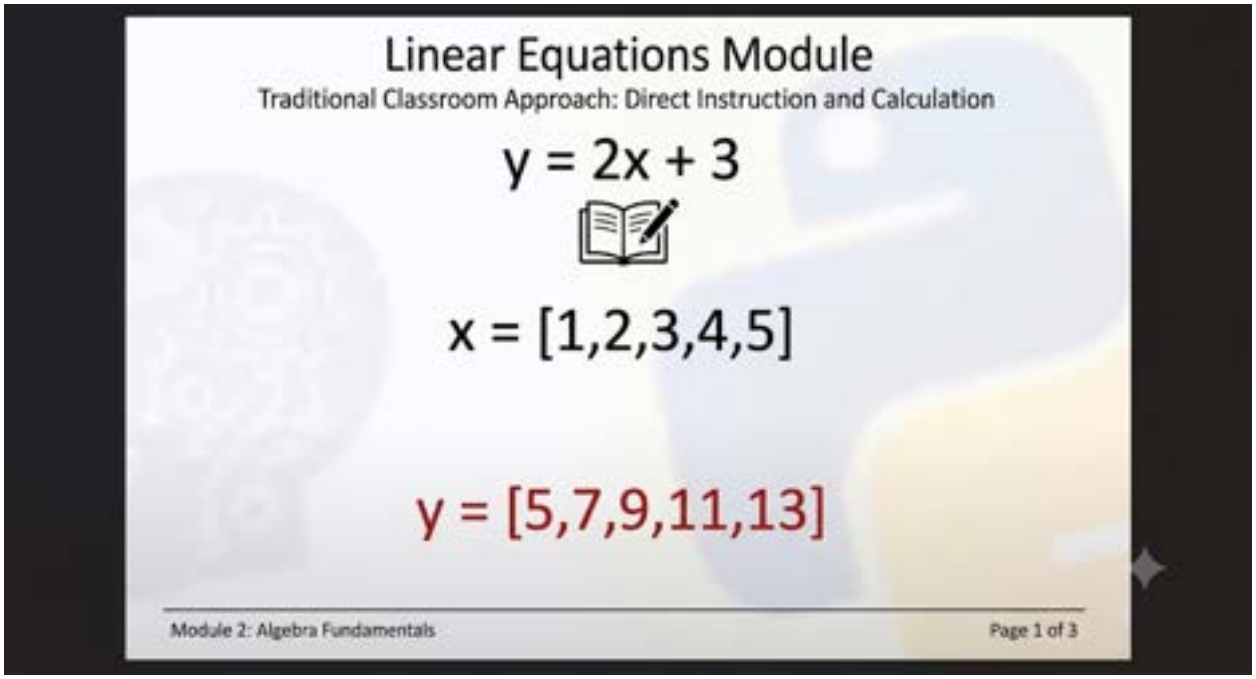
we are tying to eliminat atttibute base on p-value

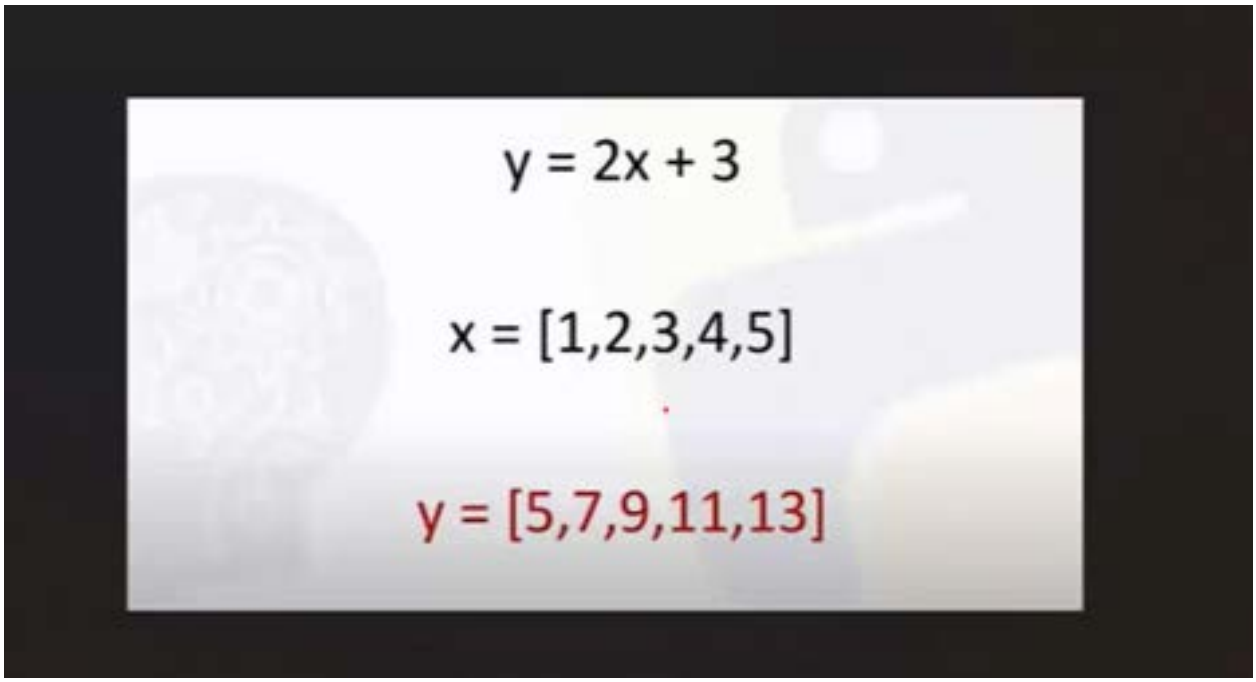
model geneated value > 0.05 -- we reject the null hypothesis
reject null hypot statsion === eliminat the attribute in machine learning
==== 7th

overfitting -- train the model with more attribute
overfitting -- low bias high variance
bias -- training & varianace -- testing part
to reduce overfitting we need to remove some attribute

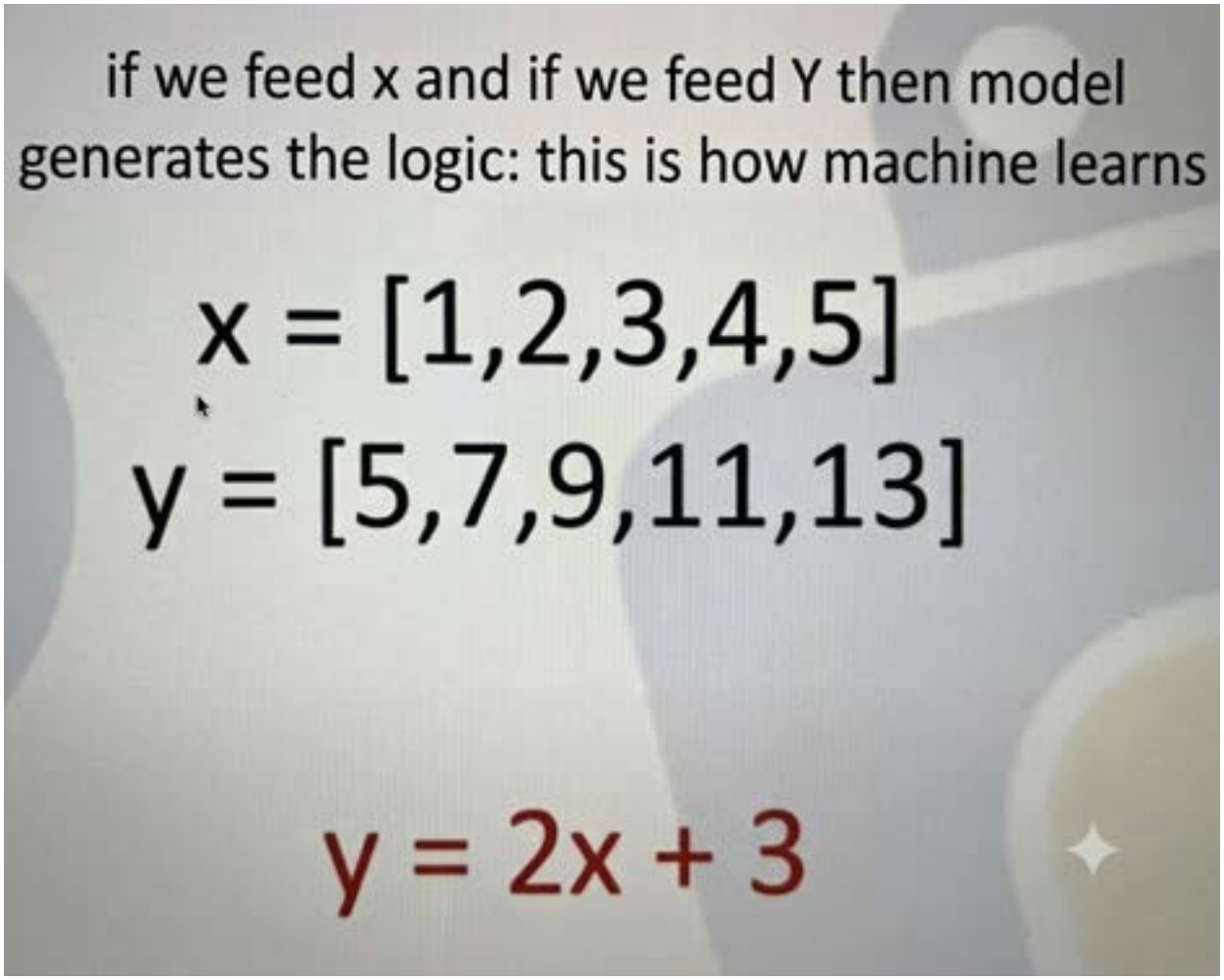
to reduce overfitting we have some technique

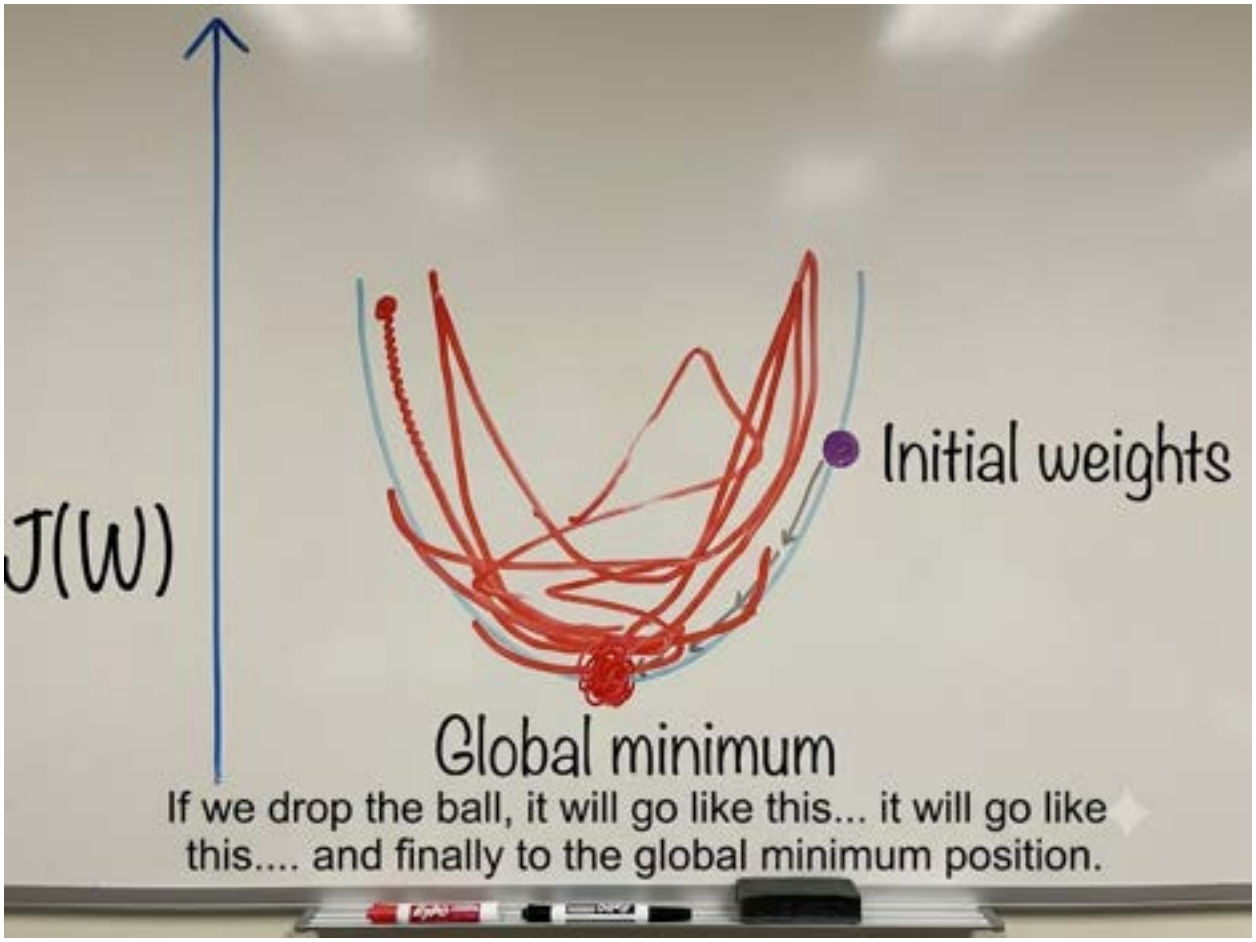
Gradident descend -->
stocastic graident descend
batch gradient descend



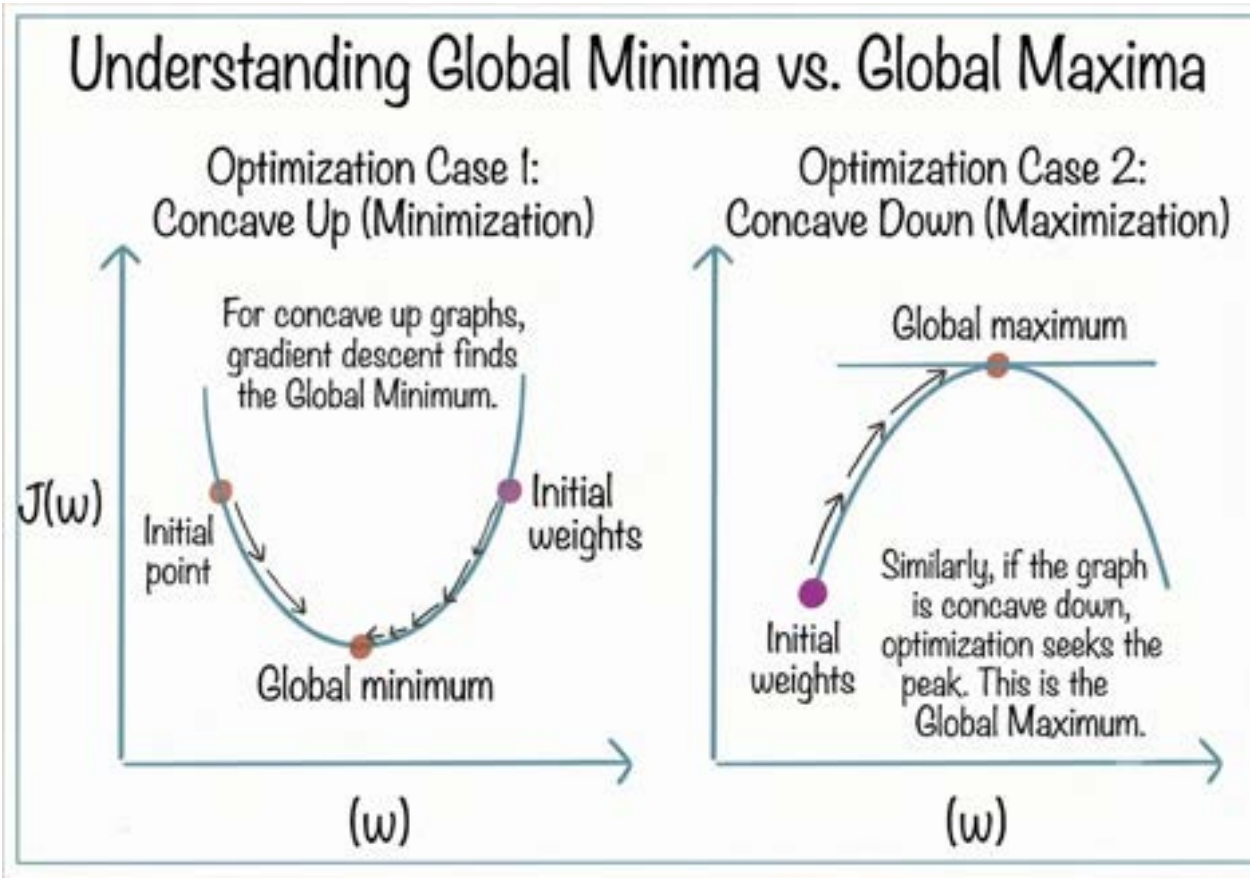


If we feed X and if we feed Y then model generate a logic

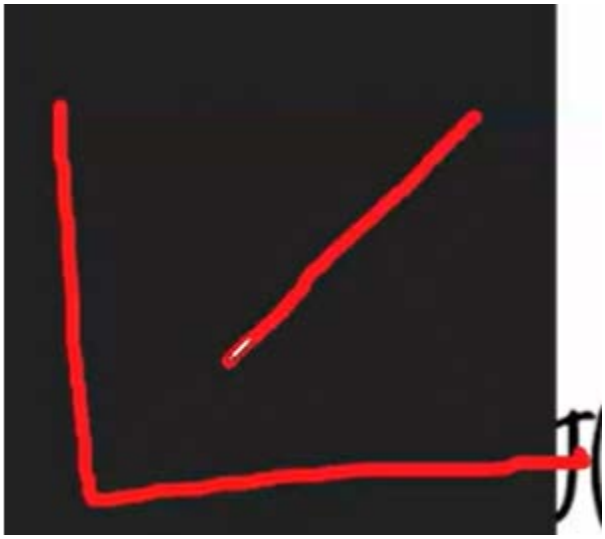

if we feed x and if we feed Y then model generates the logic: this is how machine learns
x = [1,2,3,4,5]
y = [5,7,9,11,13]
y = 2x + 3



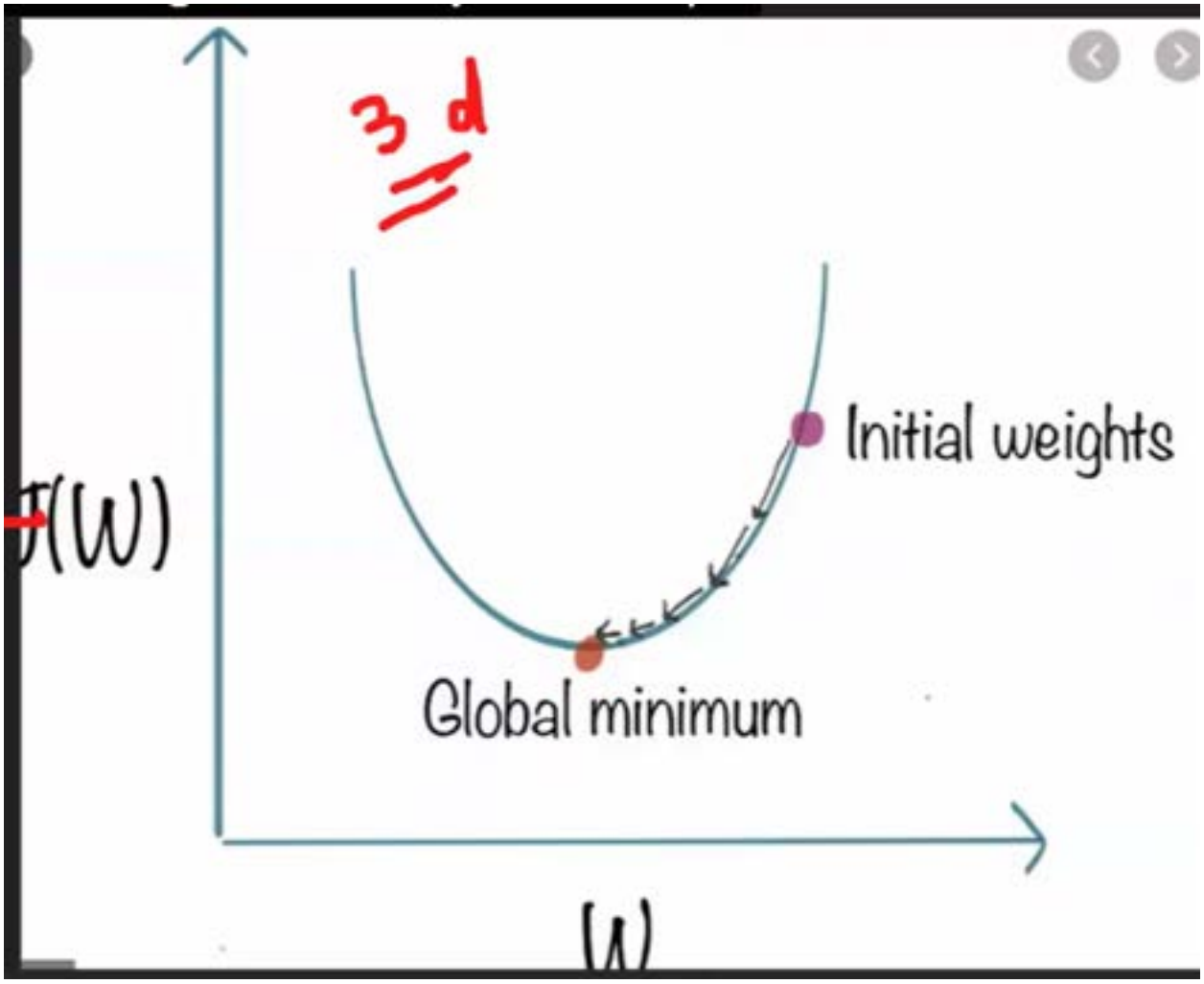
This concept is called global minimum

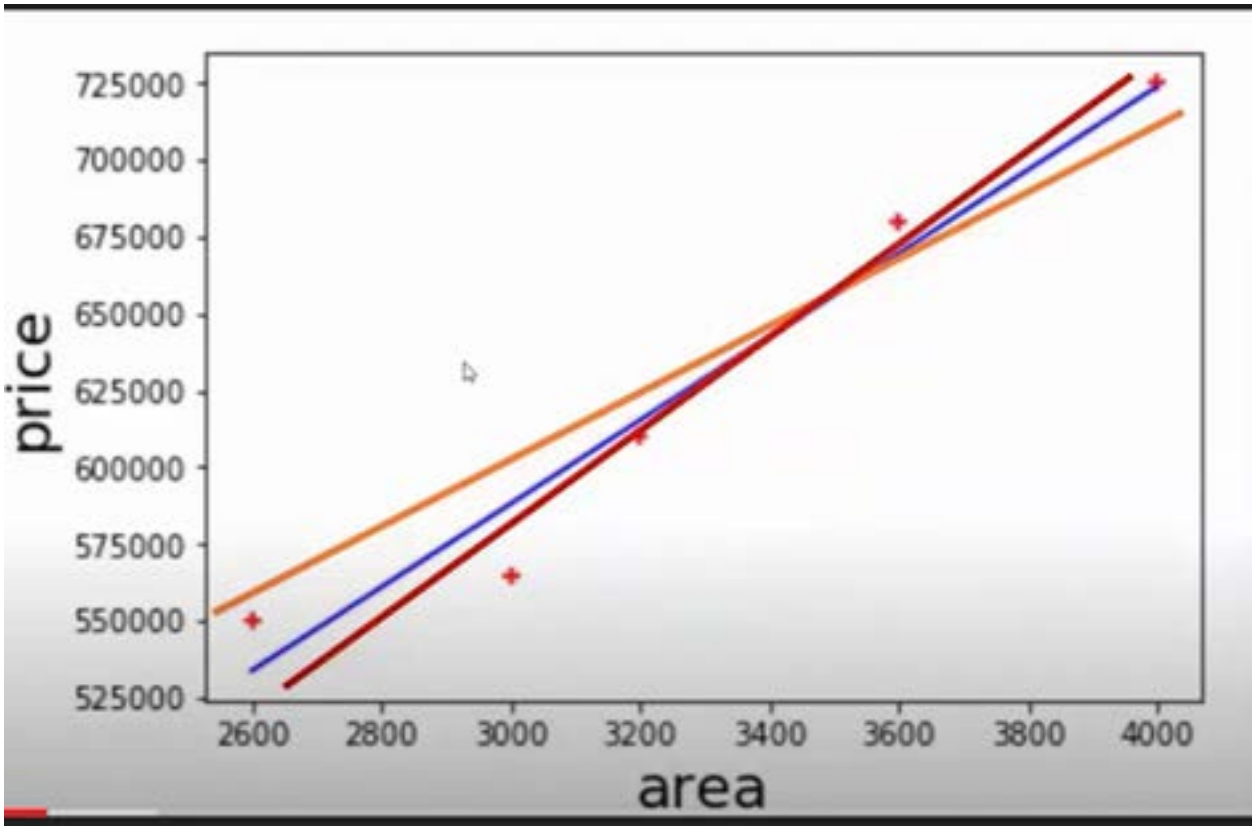


This gradient decent always plot in 3d graph
The below is 2d graph

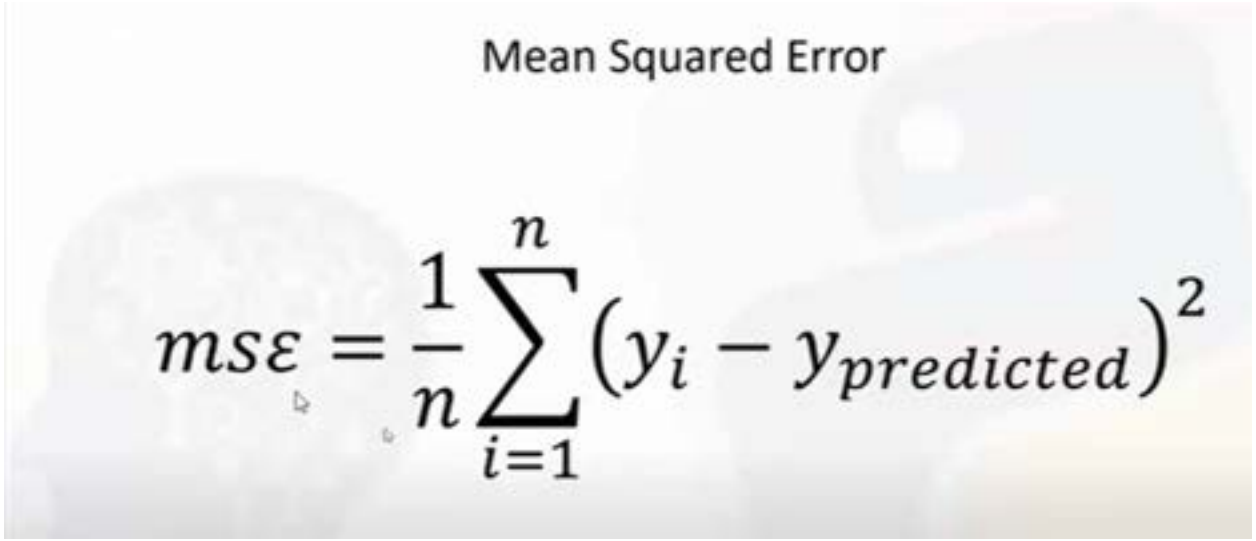


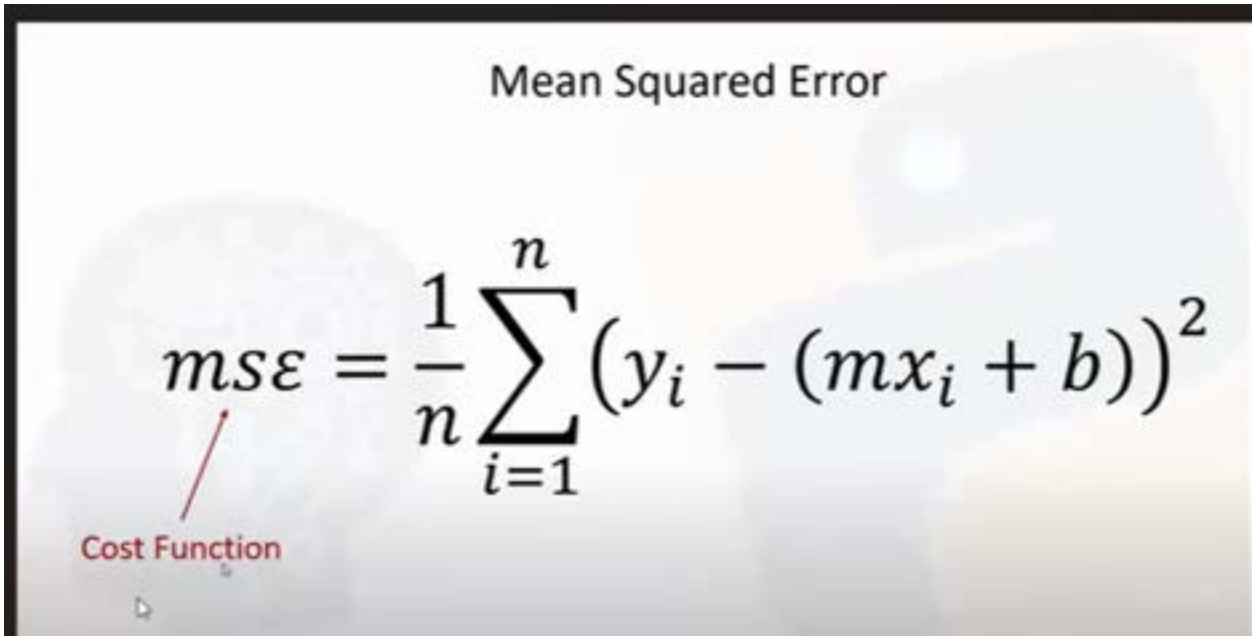
Behind it is like 3d graph


3 d
J(W)
Initial weights
Global minimum
W



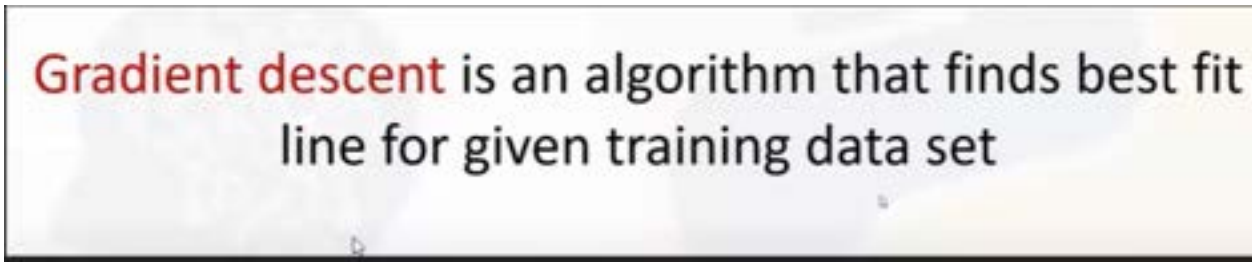
In above graph the line with minimum error is called best fit line





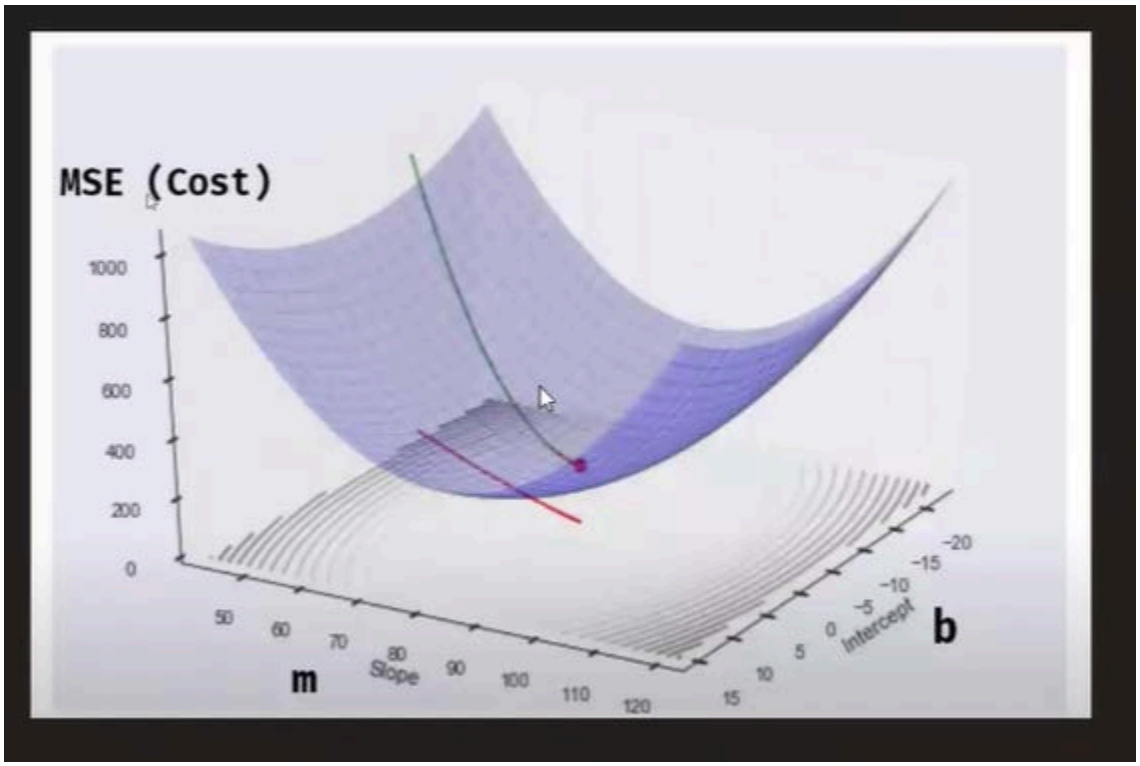
loss function == actula - preidcted

cost function == mse (mean squerd error)



gradient descend is an optimization algorithm to find the best fit line from the training data

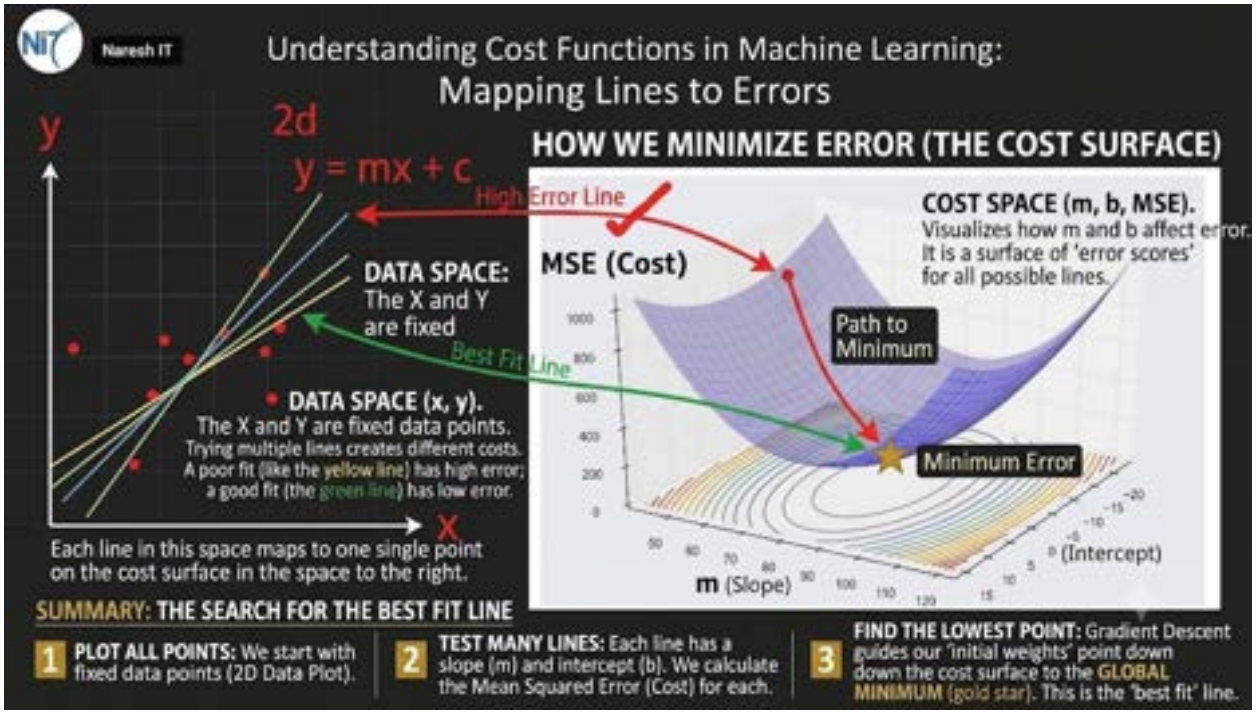
What is meant by optimize algorithm



Below is the 2d graph has the data points we can plot multiple lines here and this 2d graph represents 3d graph we plot the graph using cost function slope m and intercept C

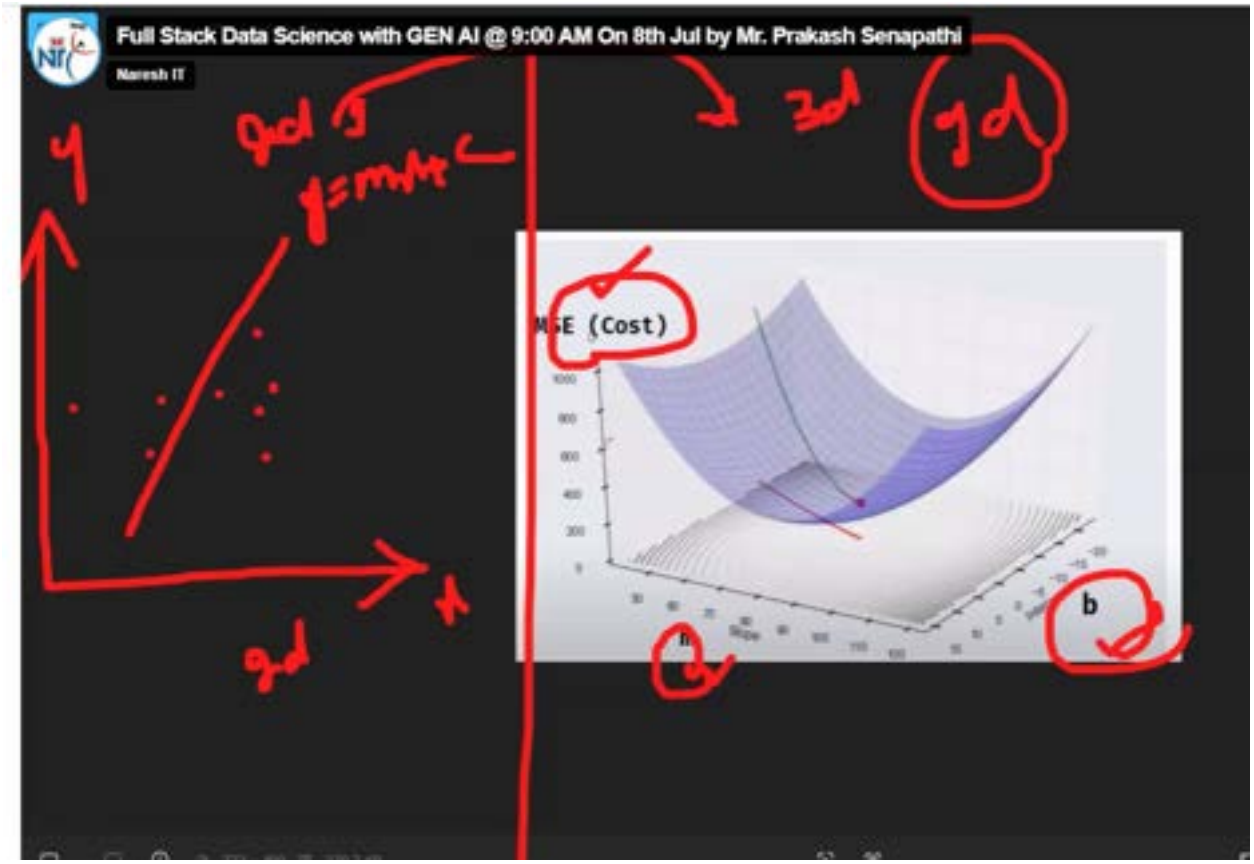
- **2D Data Space (Left Graph): The graph shows how trying multiple possible lines against the fixed data points creates different error amounts, shown as cost points on the right-hand graph.**
- **3D Cost Space (Right Graph): The 3D surface visualizes the Mean Squared Error (MSE) based on the combined choice of both slope (m) and intercept (b).**
- **Connecting the Concepts: Added arrows and text boxes explicitly state that one line in the 2D space corresponds to one specific point in the 3D cost space.**
- **The Best Fit (Golden Star): The ultimate goal is identified as the golden star: the Global Minimum, which represents the unique combination of slope (m) and intercept (b) that yields the lowest possible error, or the "best fit" line.**

This detailed layout connects the intuition of fitting a line to a data graph with the rigorous process of finding that line through cost function minimization.

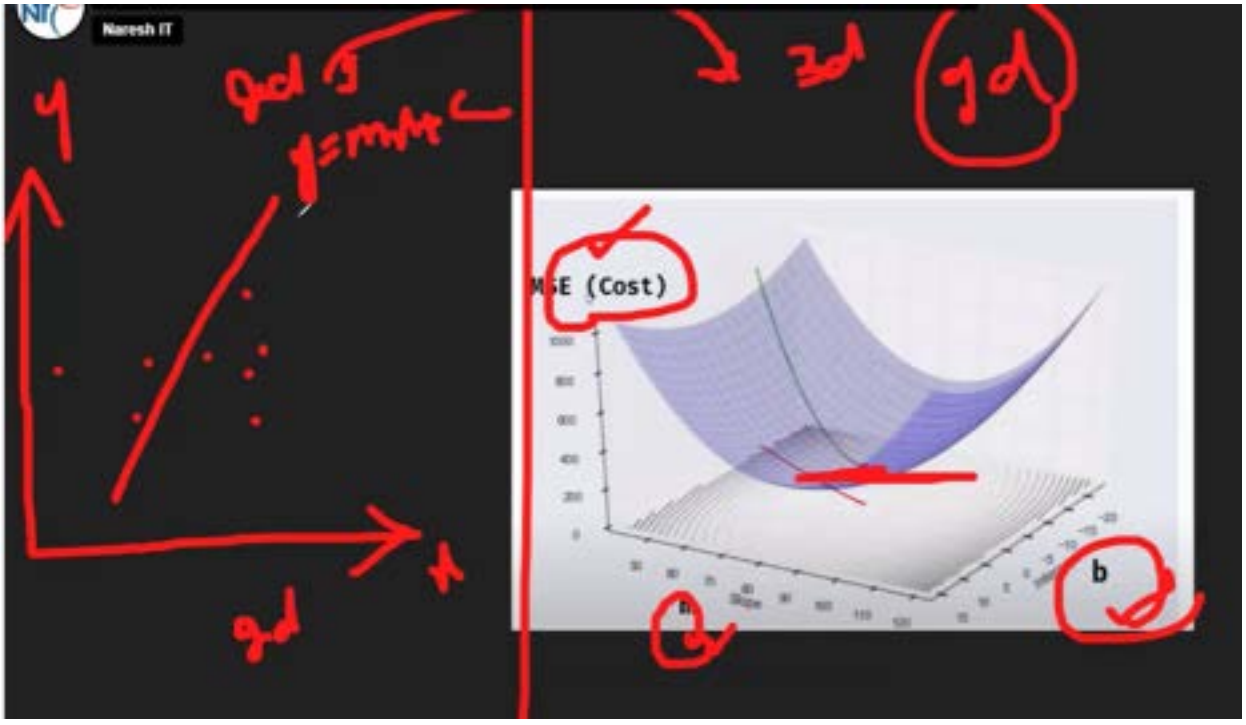


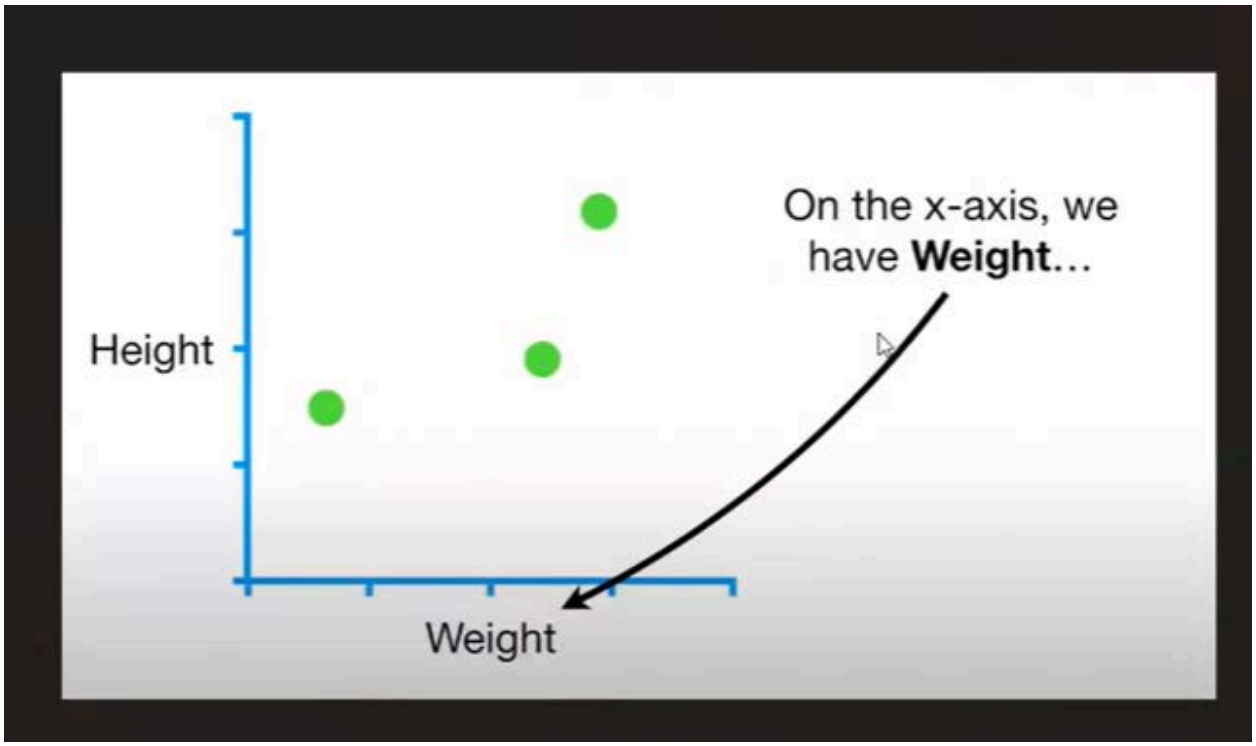
Below is the 2d graph has the data points we can plot multiple lines here and this 2d graph represents 3d graph we plot the graph using cost function slope m and intercept C

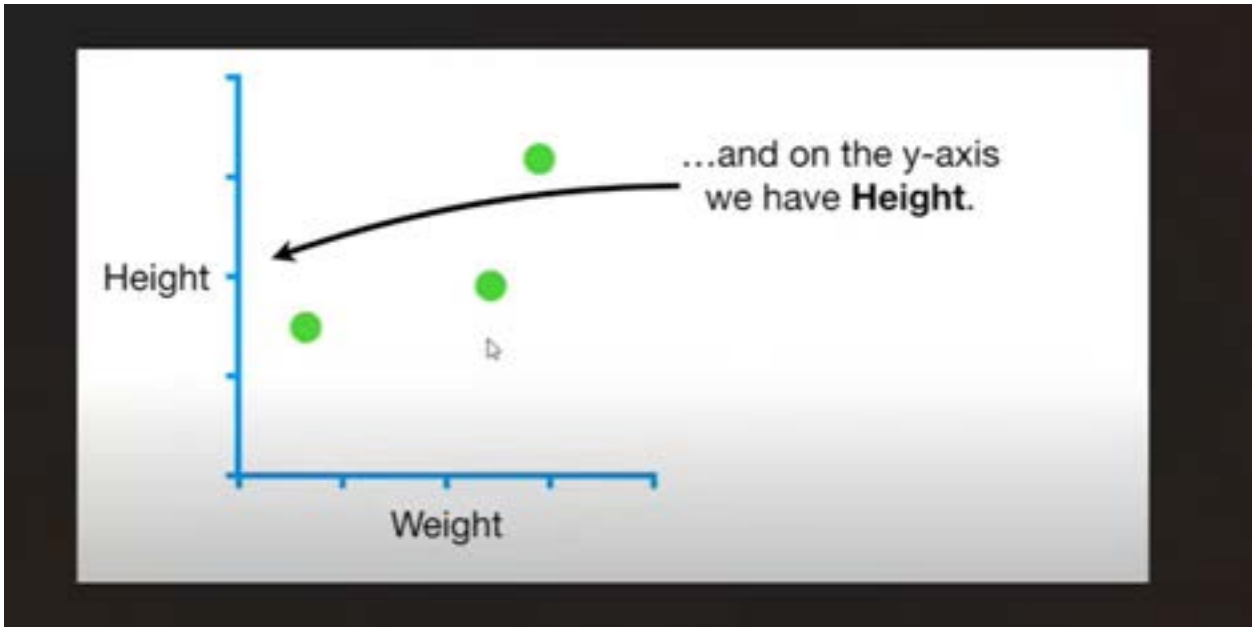
Gradient descent graph based on slope intercept and cost function

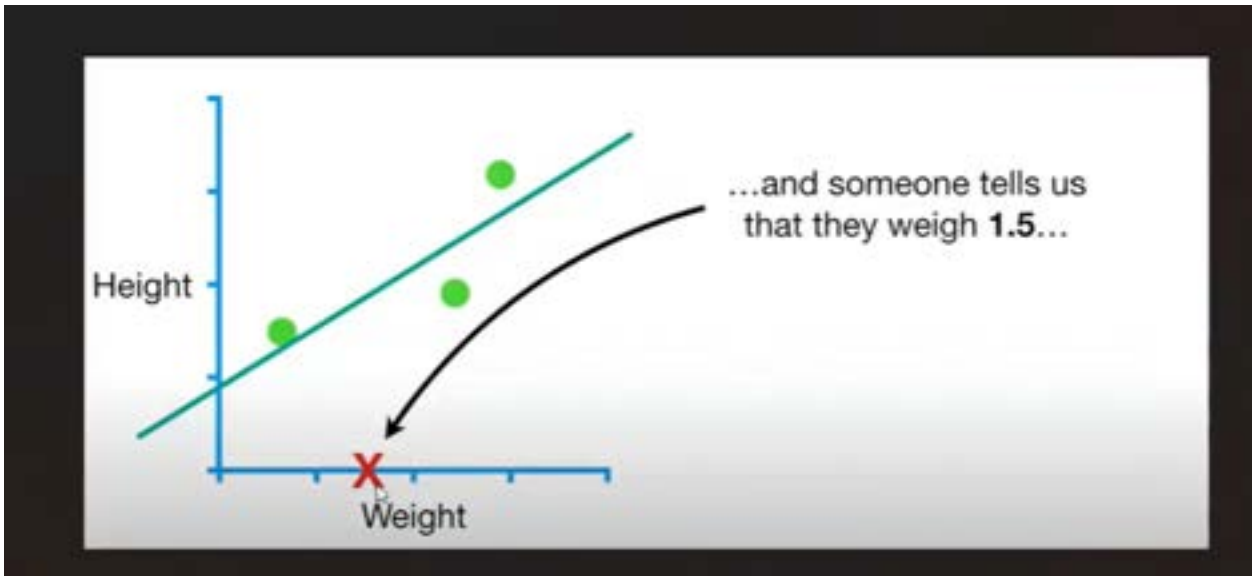


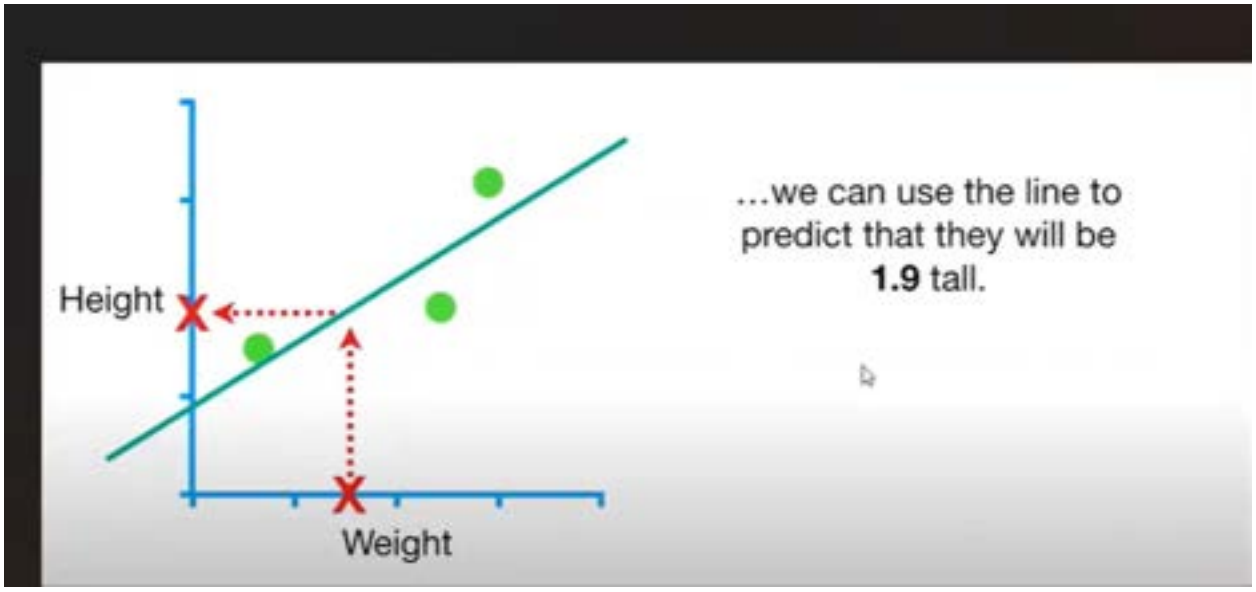
What ever the point touch to the global minima that would be the best fit line

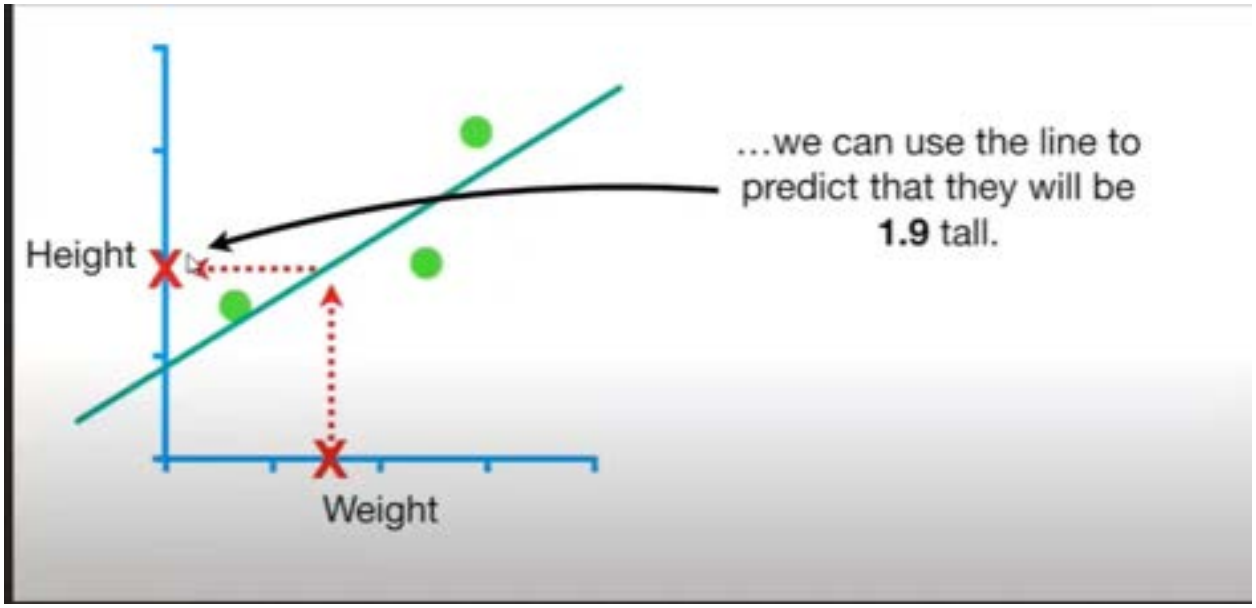


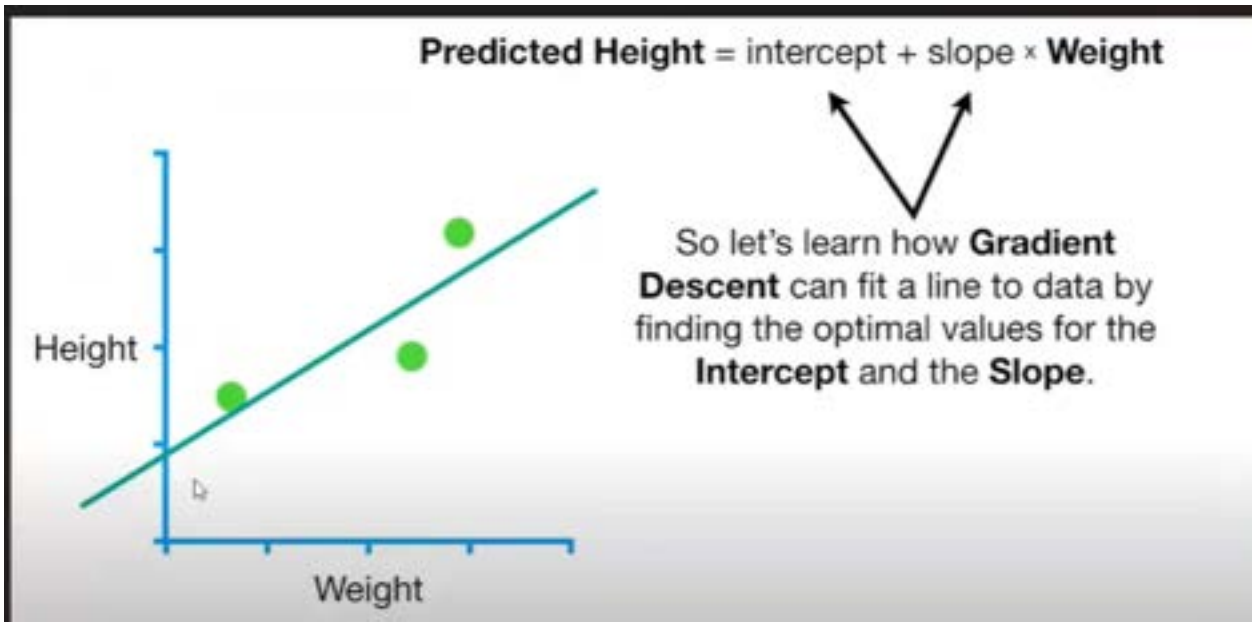

On the x-axis, we have **Weight**...
Height
Weight

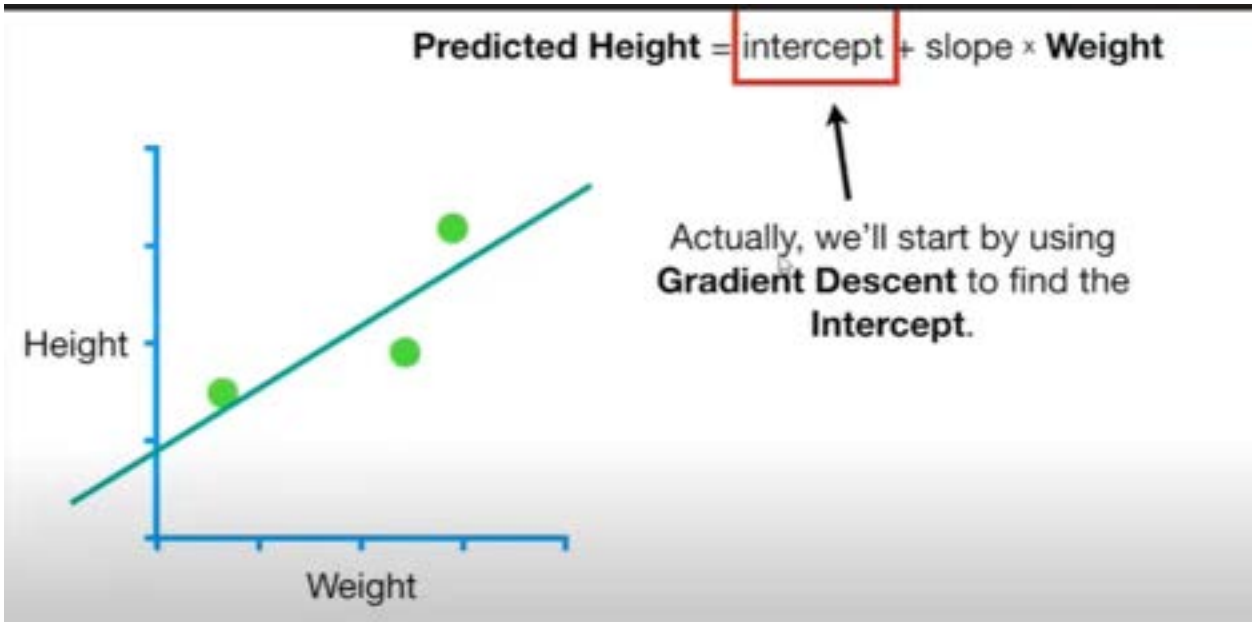

...and on the y-axis we have **Height**.
Height
Weight

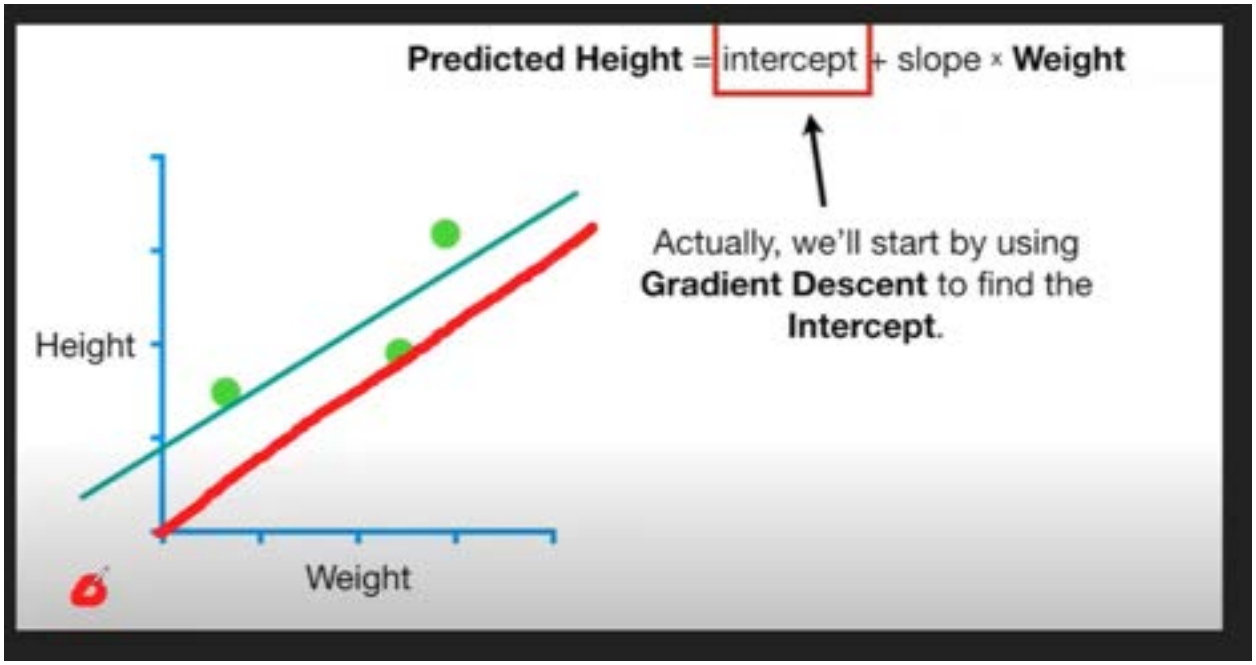

Height
Weight
...and someone tells us that they weigh **1.5**...


Height
Weight
...we can use the line to predict that they will be **1.9** tall.

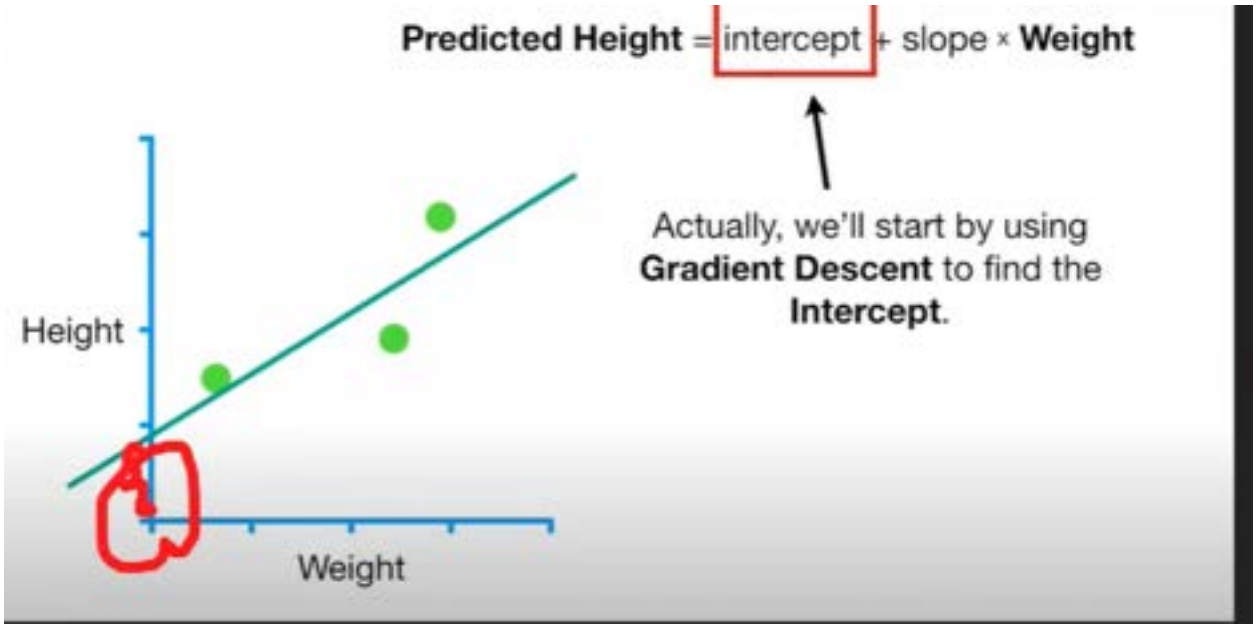

Height
Weight
...we can use the line to predict that they will be **1.9** tall.


Predicted Height = intercept + slope × **Weight**
Height
Weight
So let's learn how **Gradient Descent** can fit a line to data by finding the optimal values for the **Intercept** and the **Slope**.

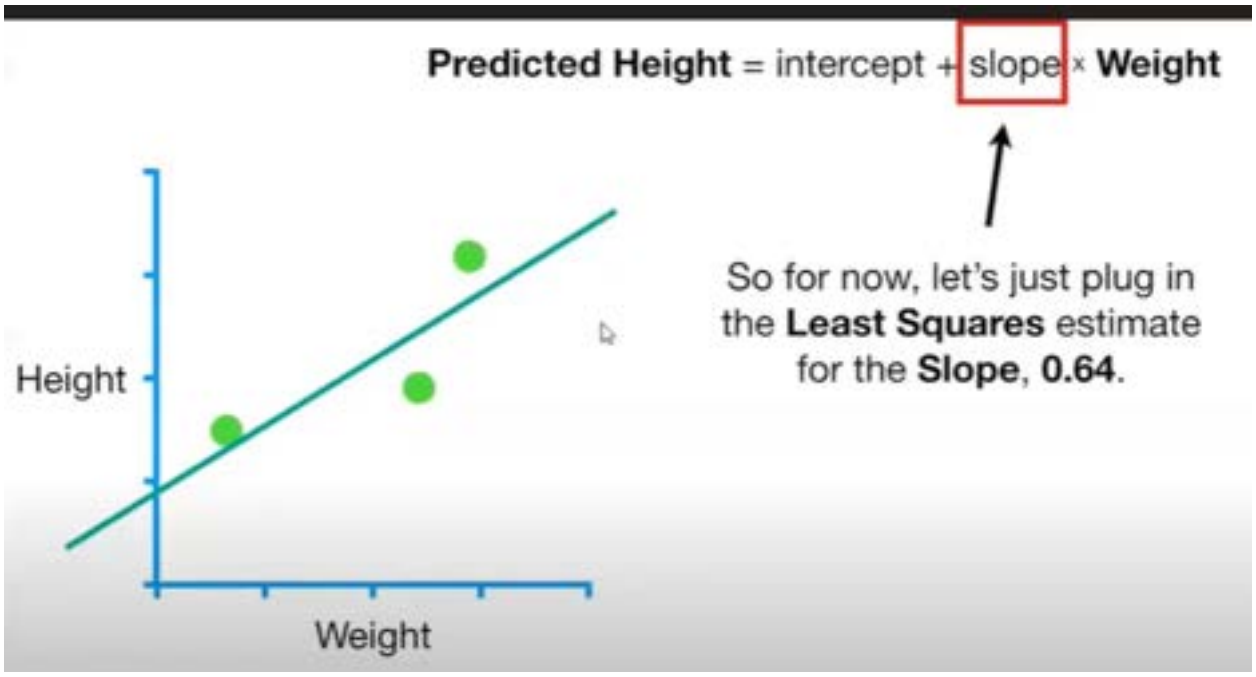




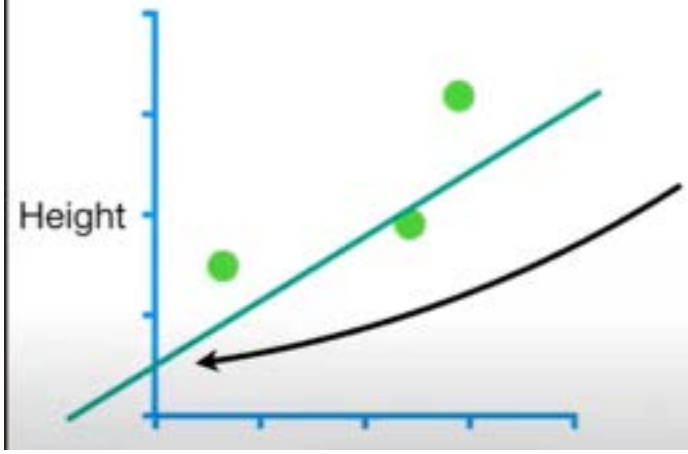
If the intercept line touches the origin then the intercept is zero.



Here we are getting some intercept value for this calculations

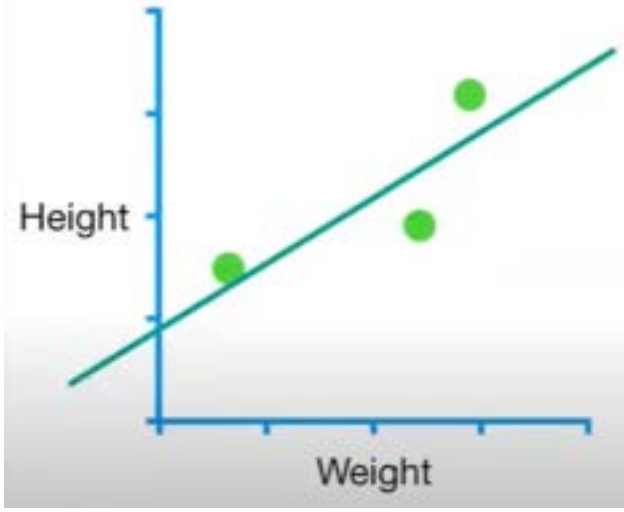


Predicted Height = intercept + 0.64 × **Weight**

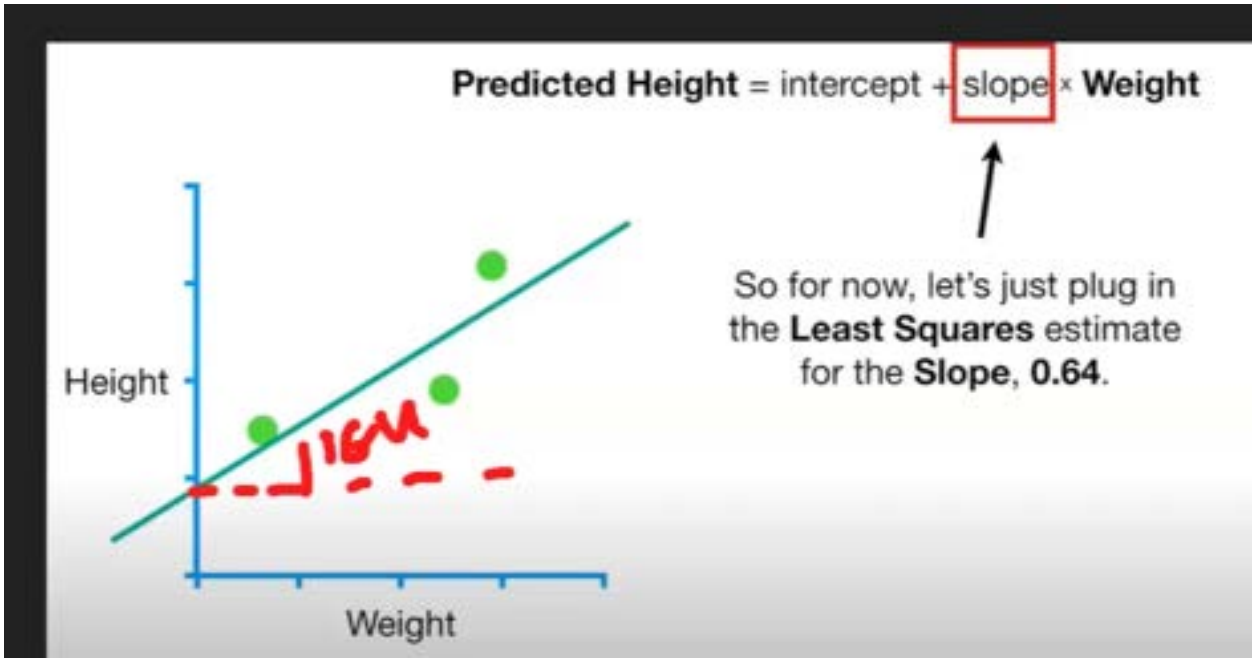


…and we'll use **Gradient Descent** to find the the optimal value for the Intercept.

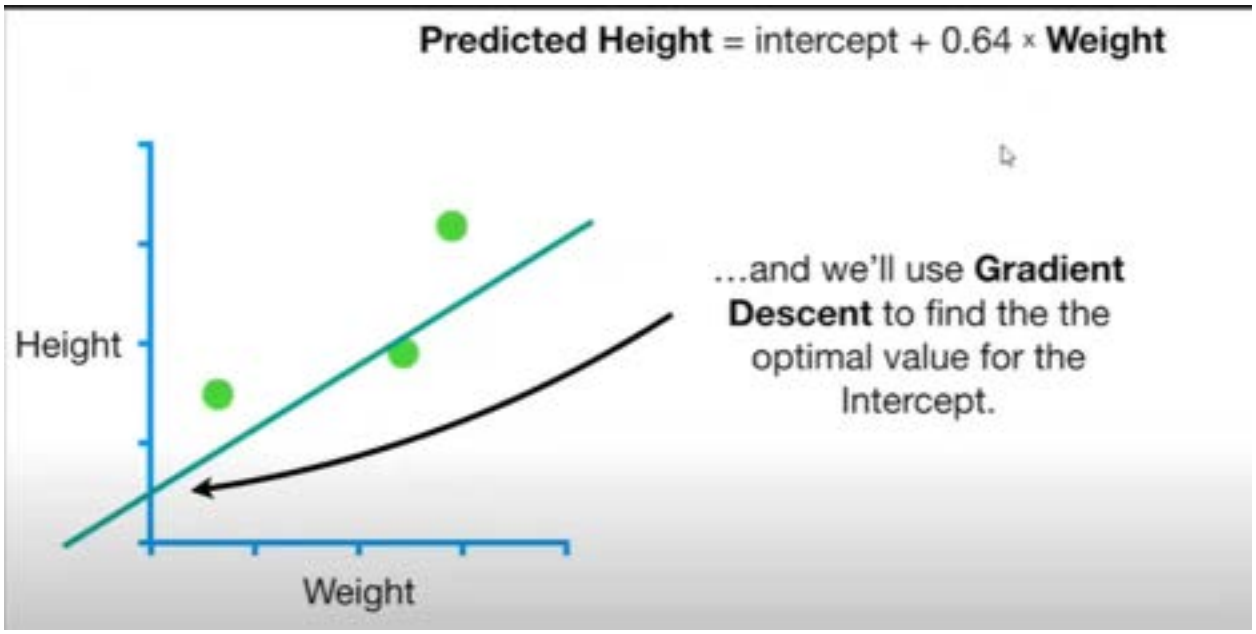
Predicted Height = intercept + slope × **Weight**



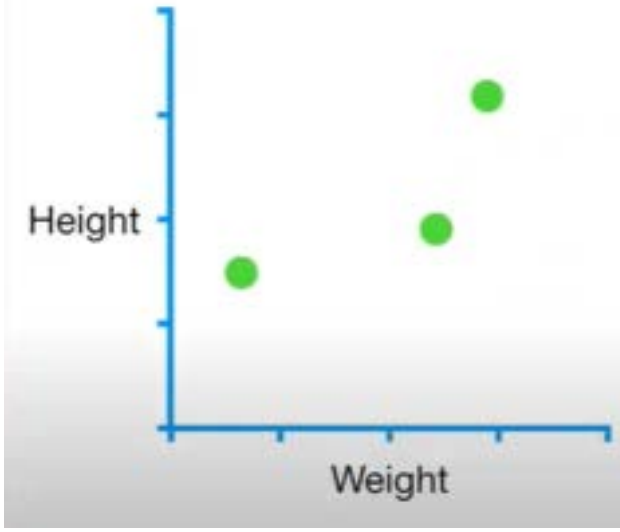
So for now, let's just plug in the **Least Squares** estimate for the **Slope**, **0.64**.



The slope we calcul;ated is 0.64



Predicted Height = intercept + 0.64 × **Weight**

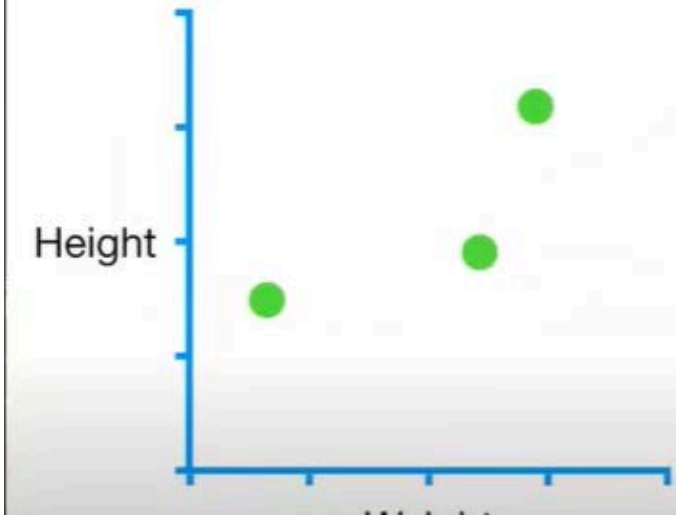


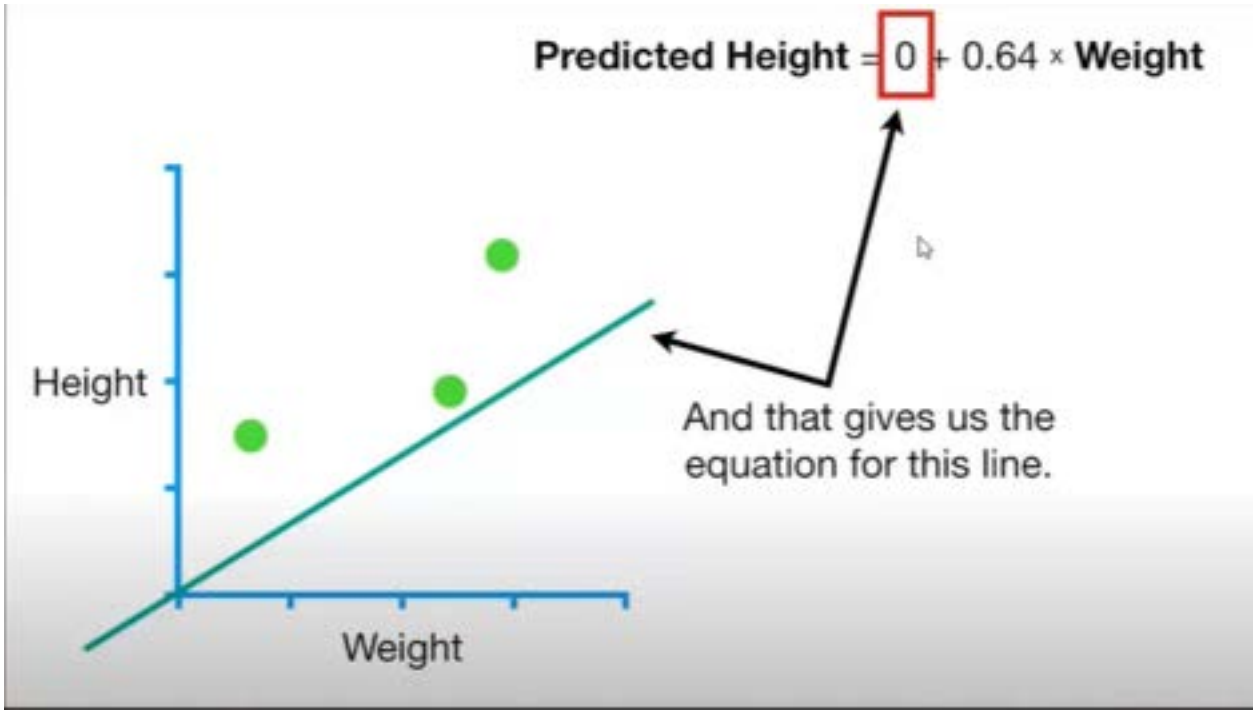
The first thing we do is pick a random value for the **Intercept**.

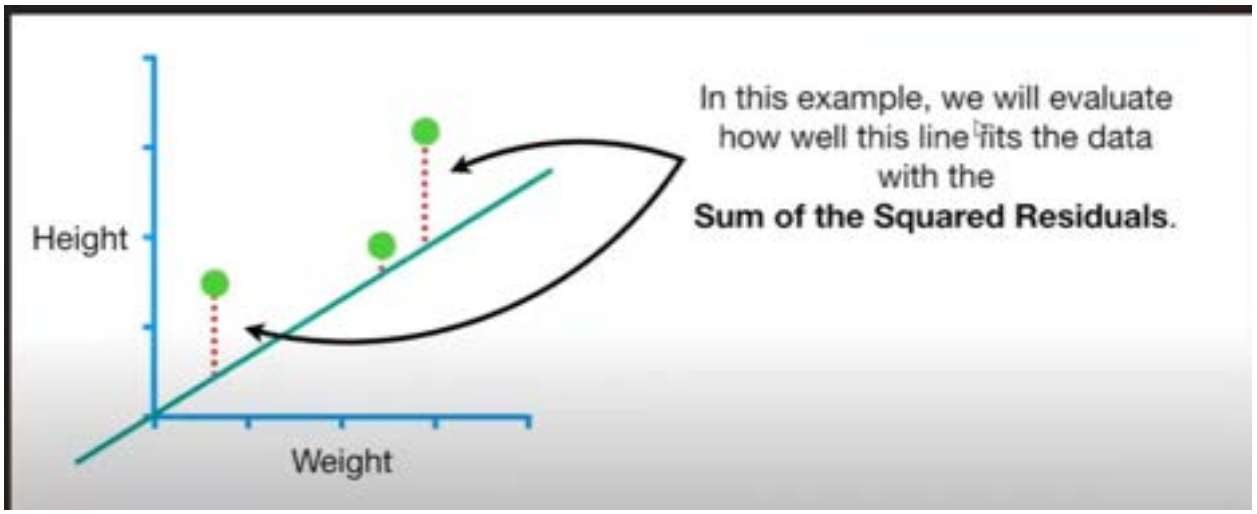
This is just an initial guess that gives **Gradient Descent** something to improve upon.

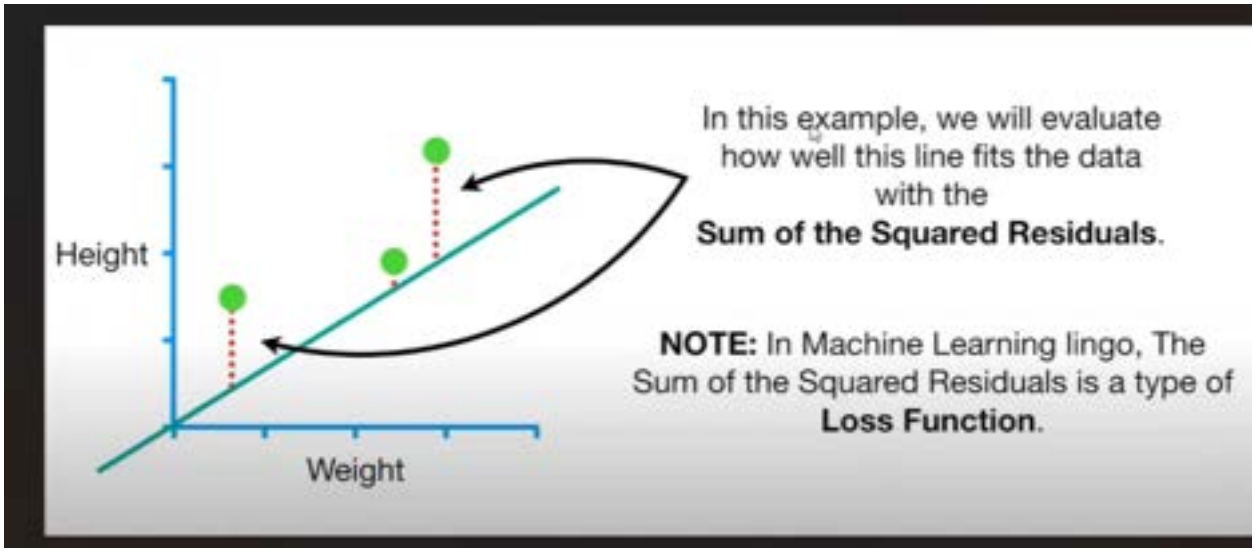
Predicted Height = 0 + 0.64 × **Weigl**

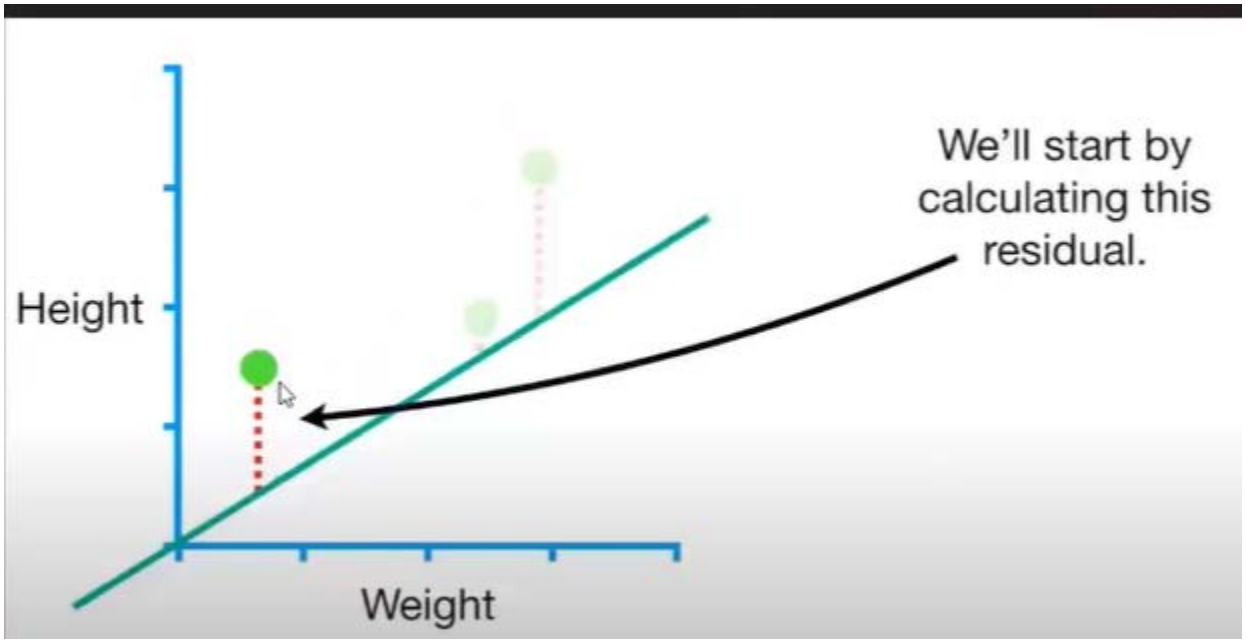
In this case, we'll use **0**, but any number will do.

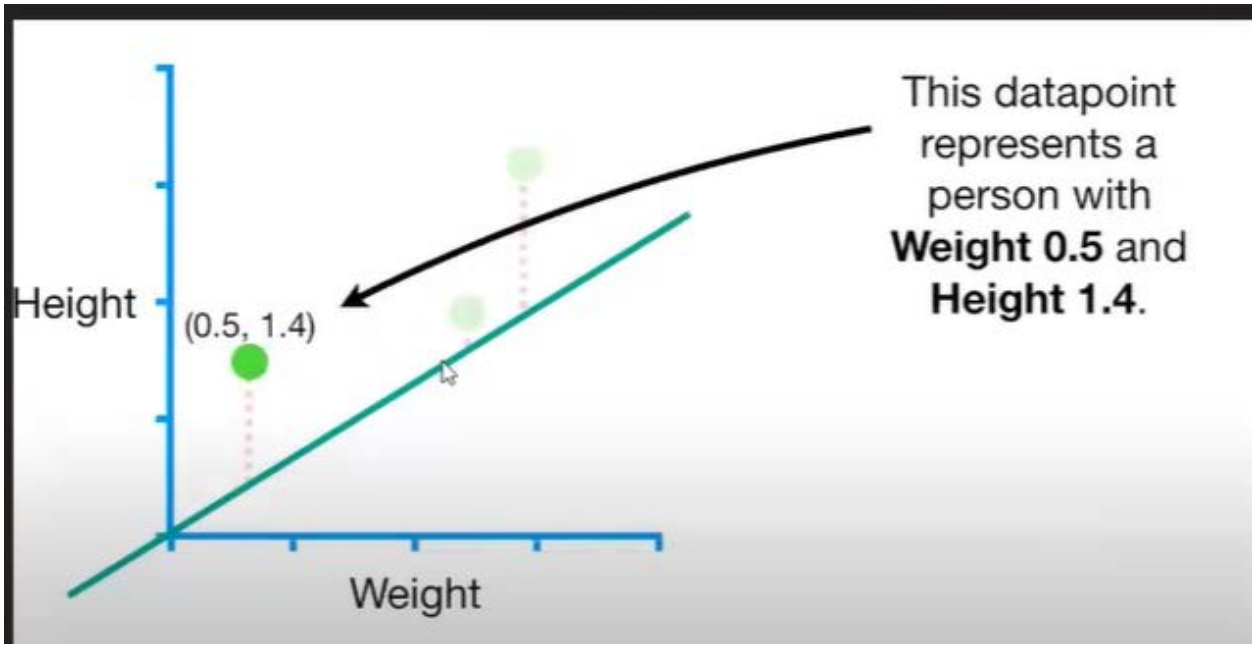


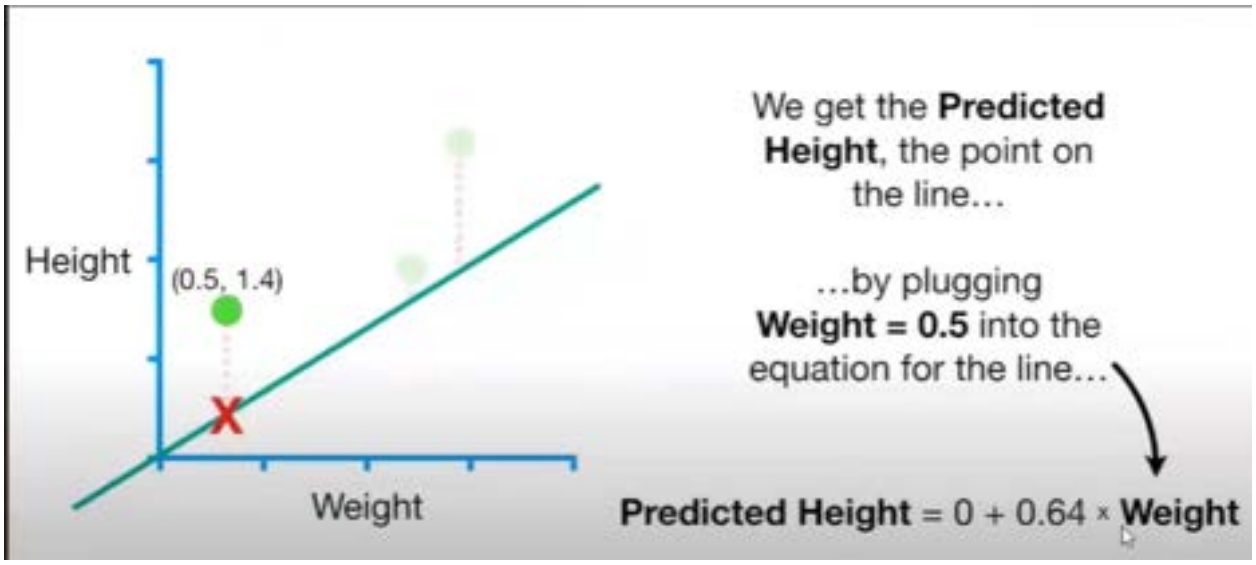

Predicted Height = 0 + 0.64 × Weight
Height
Weight
And that gives us the equation for this line.

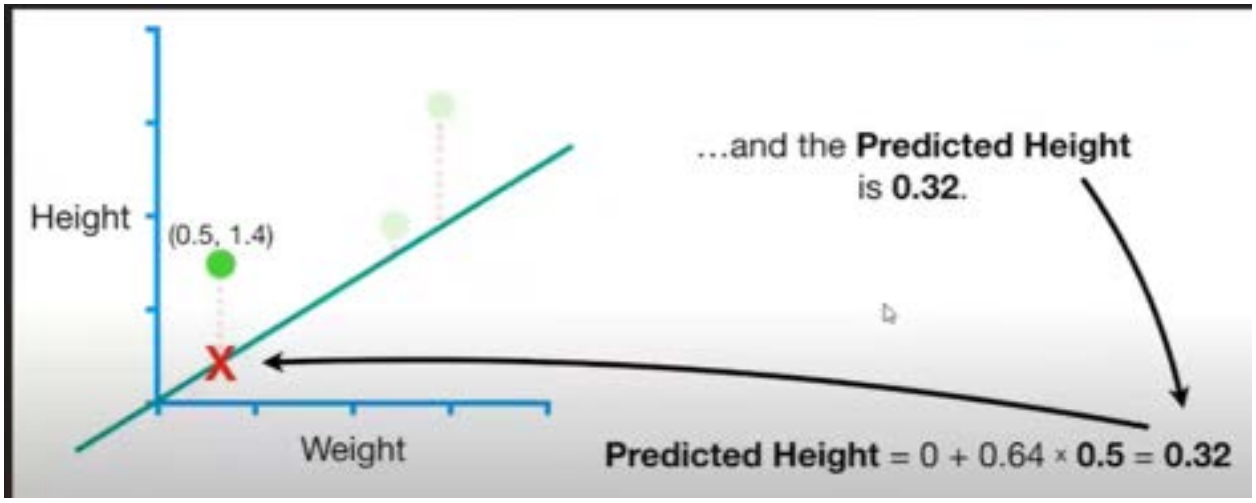

Height
Weight
In this example, we will evaluate how well this line fits the data with the
Sum of the Squared Residuals.

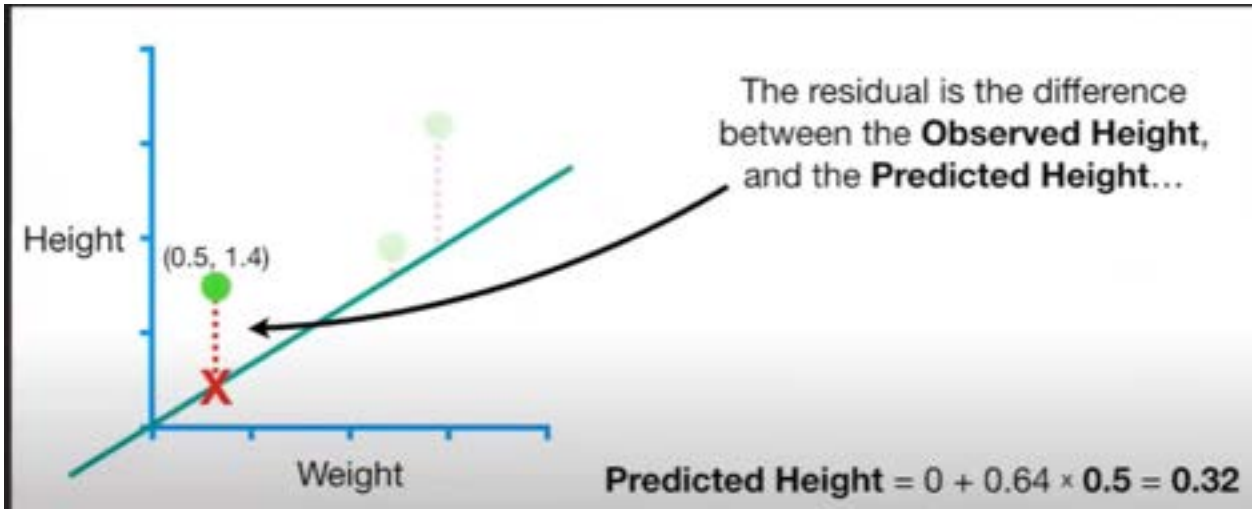

Height
Weight
In this example, we will evaluate how well this line fits the data with the **Sum of the Squared Residuals**.
NOTE: In Machine Learning lingo, The Sum of the Squared Residuals is a type of **Loss Function**.

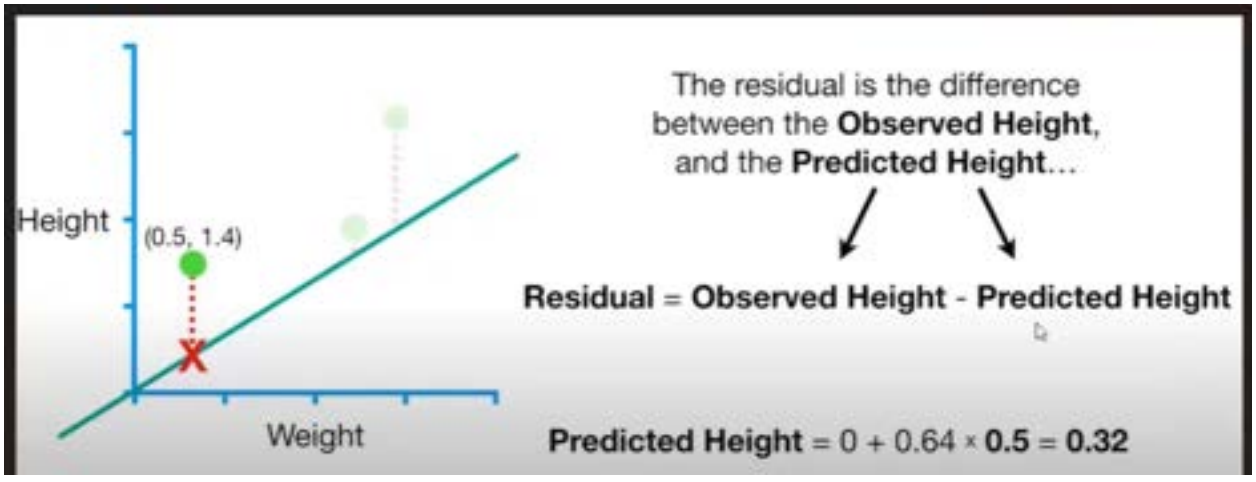

Height
Weight
We'll start by calculating this residual.

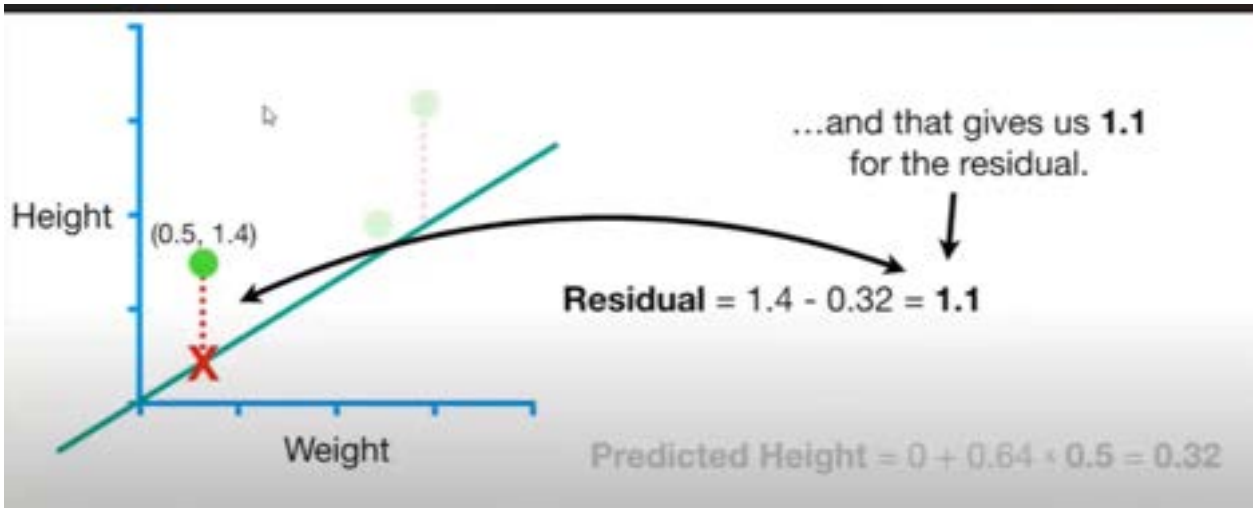

Height
(0.5, 1.4)
Weight
This datapoint represents a person with **Weight 0.5** and **Height 1.4**.

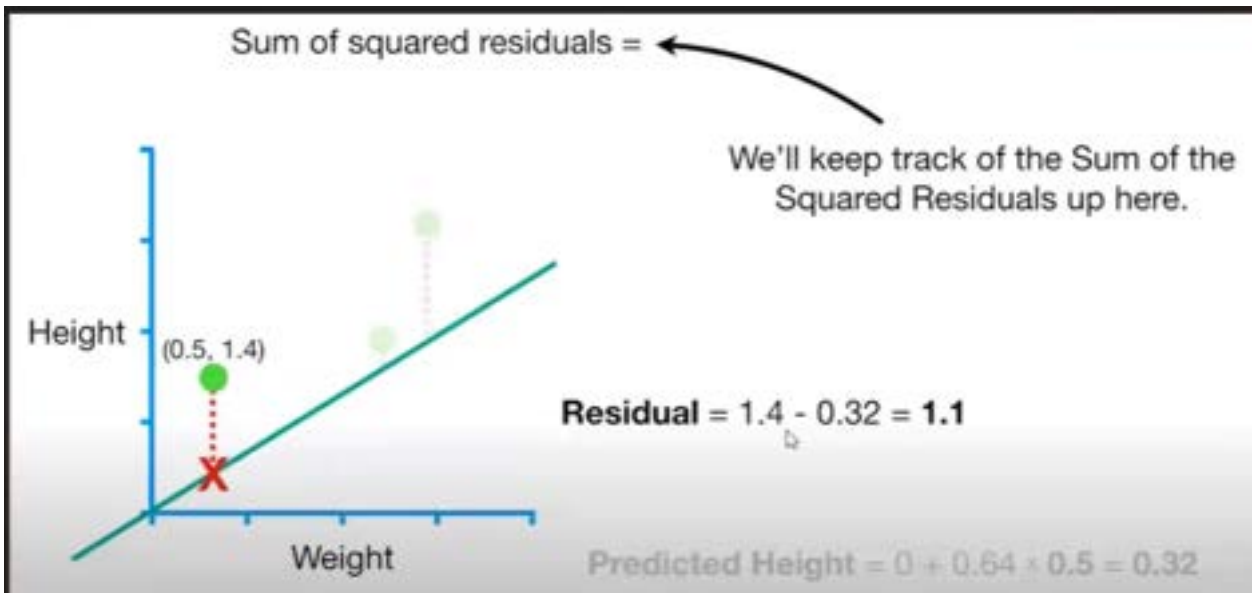

Height
(0.5, 1.4)
X
Weight
We get the **Predicted Height**, the point on the line…
…by plugging **Weight = 0.5** into the equation for the line…
Predicted Height = 0 + 0.64 × **Weight**

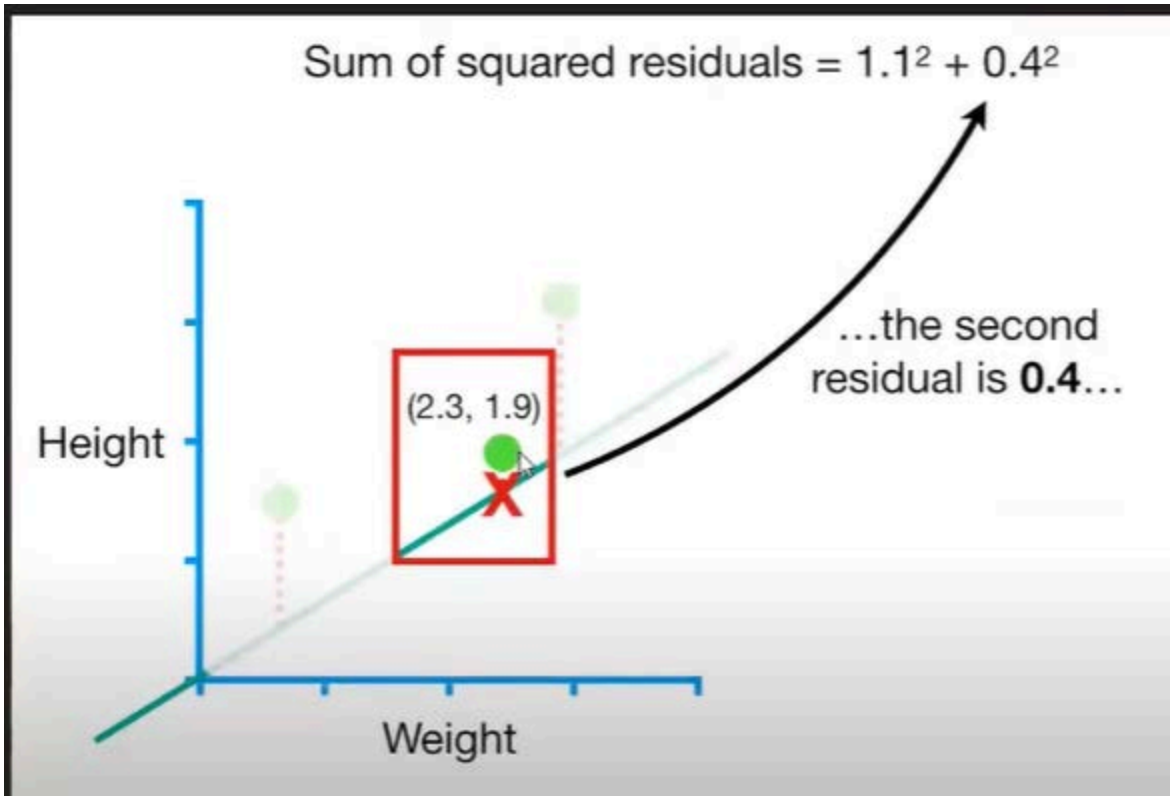

Height
(0.5, 1.4)
Weight
...and the **Predicted Height** is **0.32**.
Predicted Height = 0 + 0.64 × **0.5** = **0.32**

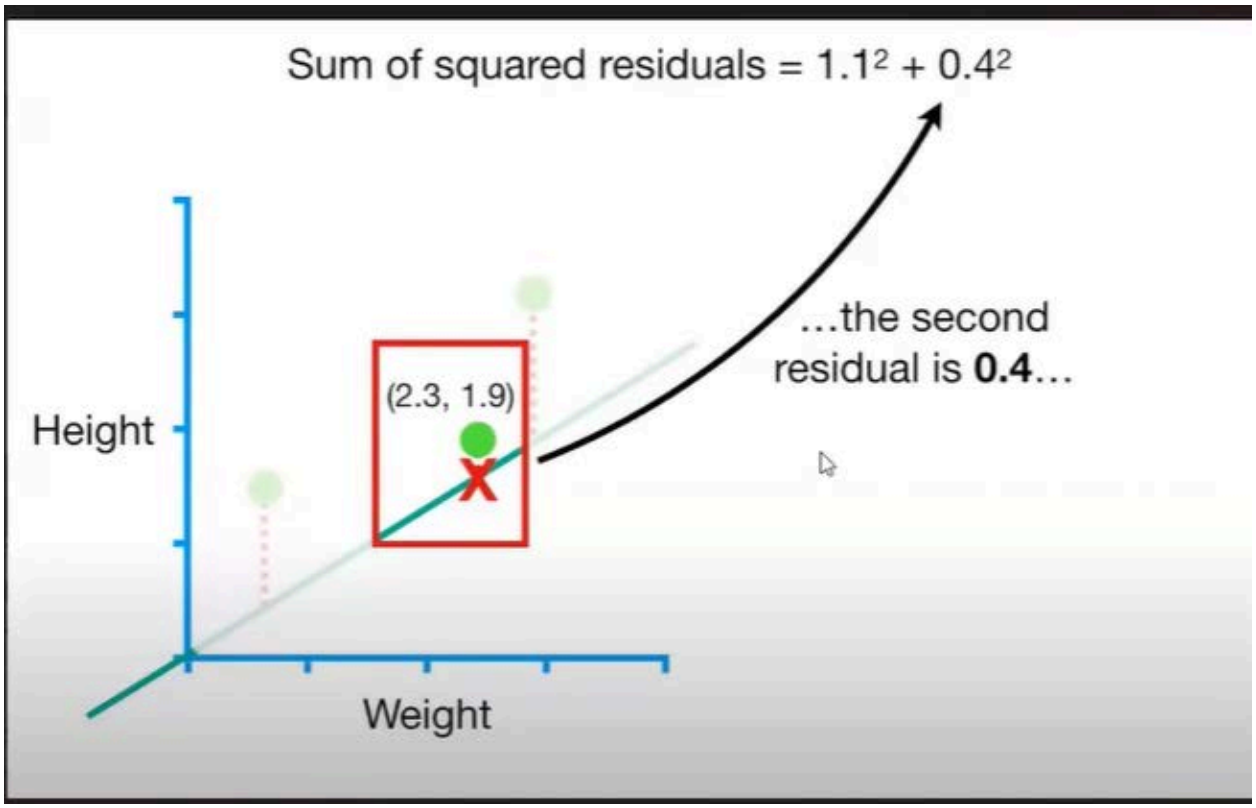

Height
(0.5, 1.4)
Weight
The residual is the difference between the **Observed Height**, and the **Predicted Height**...
Predicted Height = 0 + 0.64 × **0.5** = **0.32**

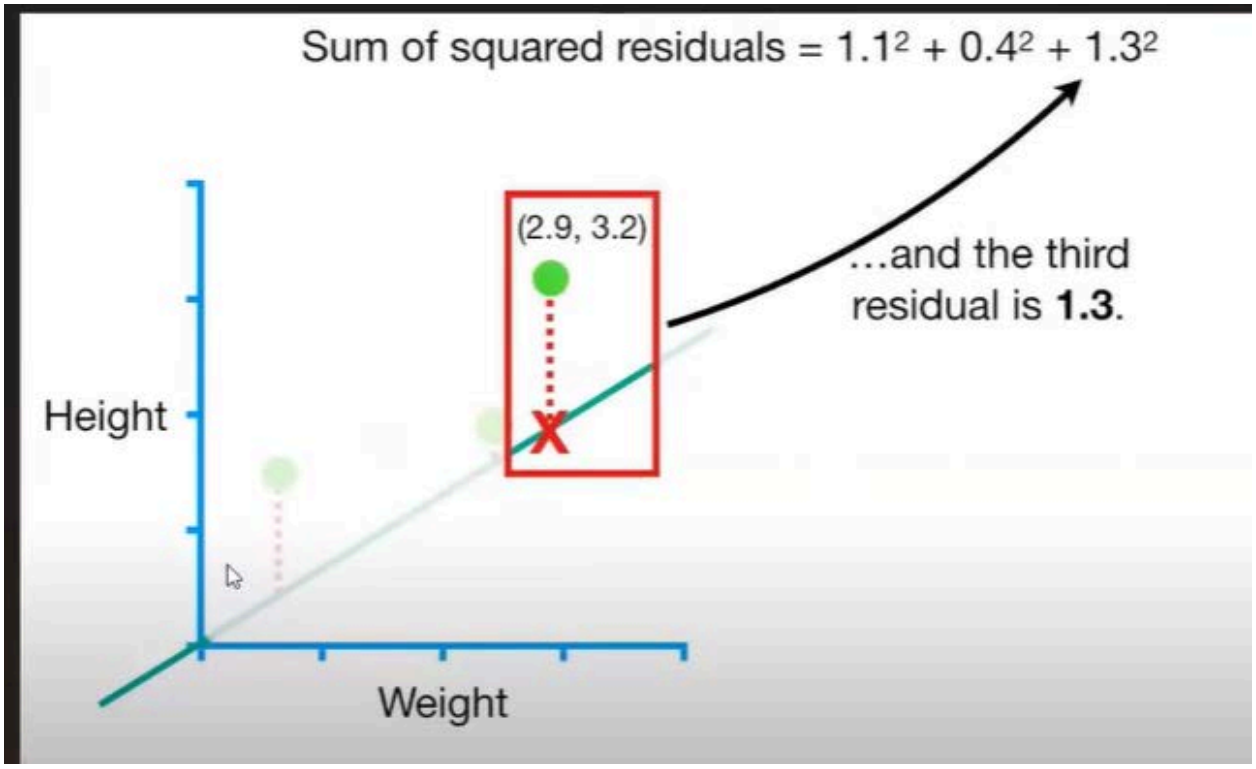

Height
(0.5, 1.4)
Weight
The residual is the difference between the **Observed Height**, and the **Predicted Height**...
Residual = **Observed Height** - **Predicted Height**
Predicted Height = 0 + 0.64 × **0.5** = **0.32**

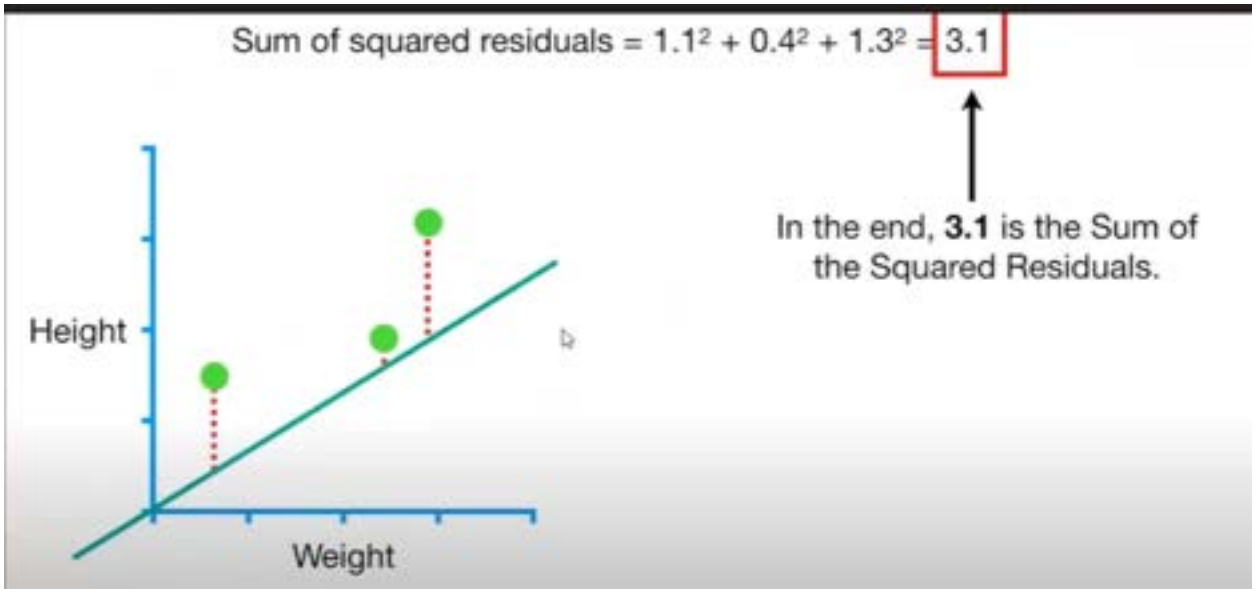

...and that gives us **1.1**
for the residual.
Height
(0.5, 1.4)
Residual = 1.4 - 0.32 = **1.1**
Weight
Predicted Height = 0 + 0.64 × 0.5 = 0.32

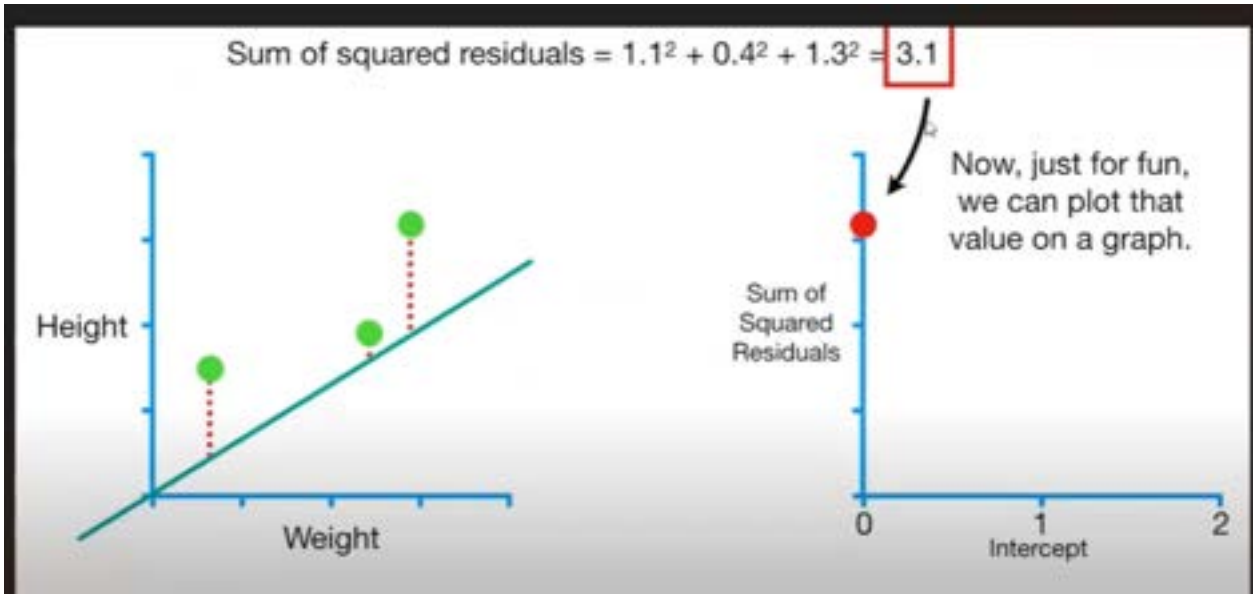

Sum of squared residuals =
We'll keep track of the Sum of the
Squared Residuals up here.
Height
(0.5, 1.4)
Residual = 1.4 - 0.32 = **1.1**
Weight
Predicted Height = 0 + 0.64 × 0.5 = 0.32

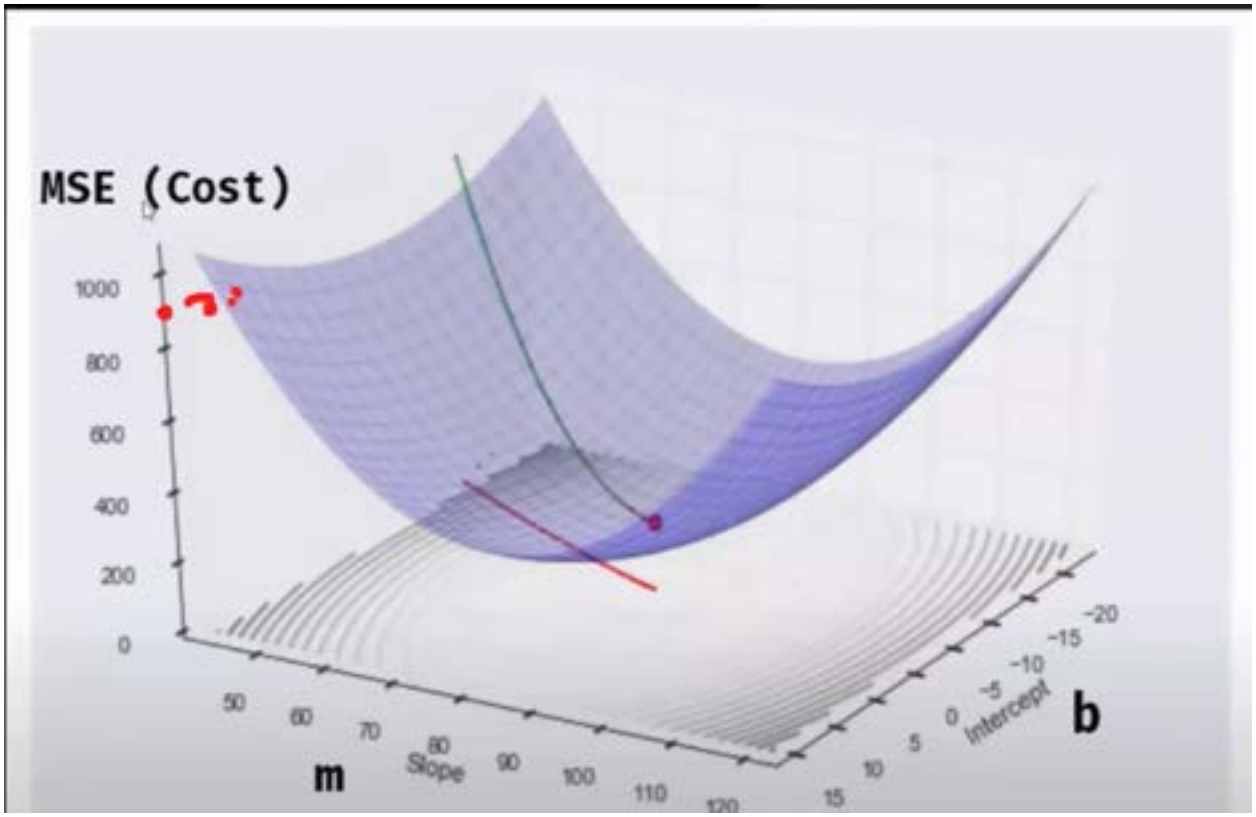

Sum of squared residuals = $1.1^2 + 0.4^2$
…the second residual is **0.4**…
(2.3, 1.9)
Height
Weight


Sum of squared residuals = $1.1^2 + 0.4^2$
…the second residual is **0.4**…
(2.3, 1.9)
Height
Weight

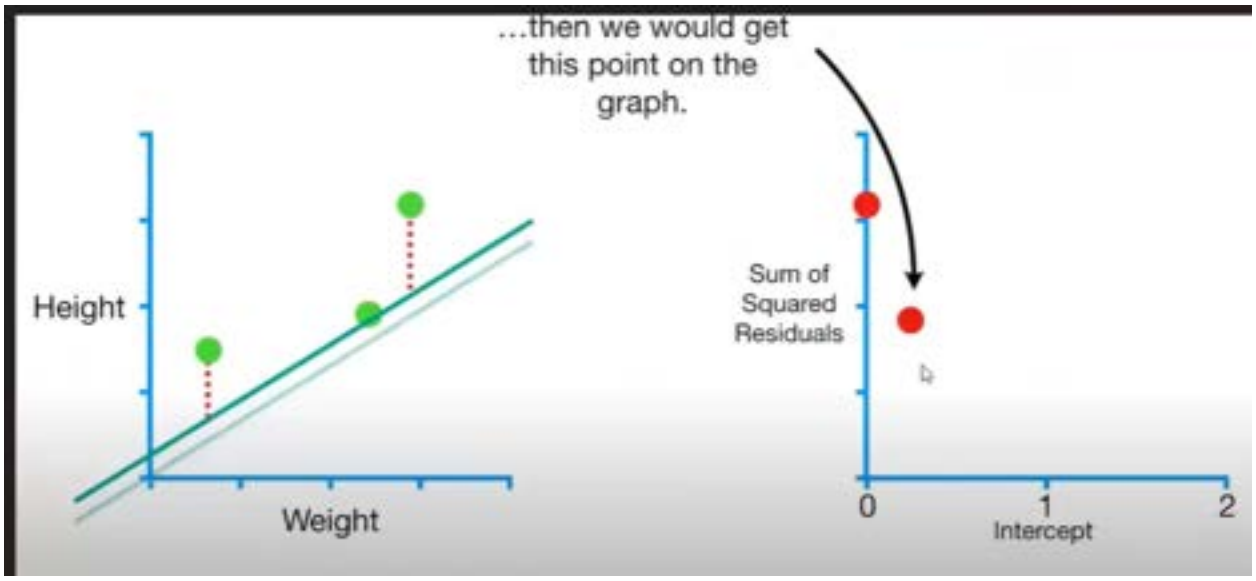

Sum of squared residuals = $1.1^2 + 0.4^2 + 1.3^2$
(2.9, 3.2)
...and the third residual is **1.3**.
Height
Weight

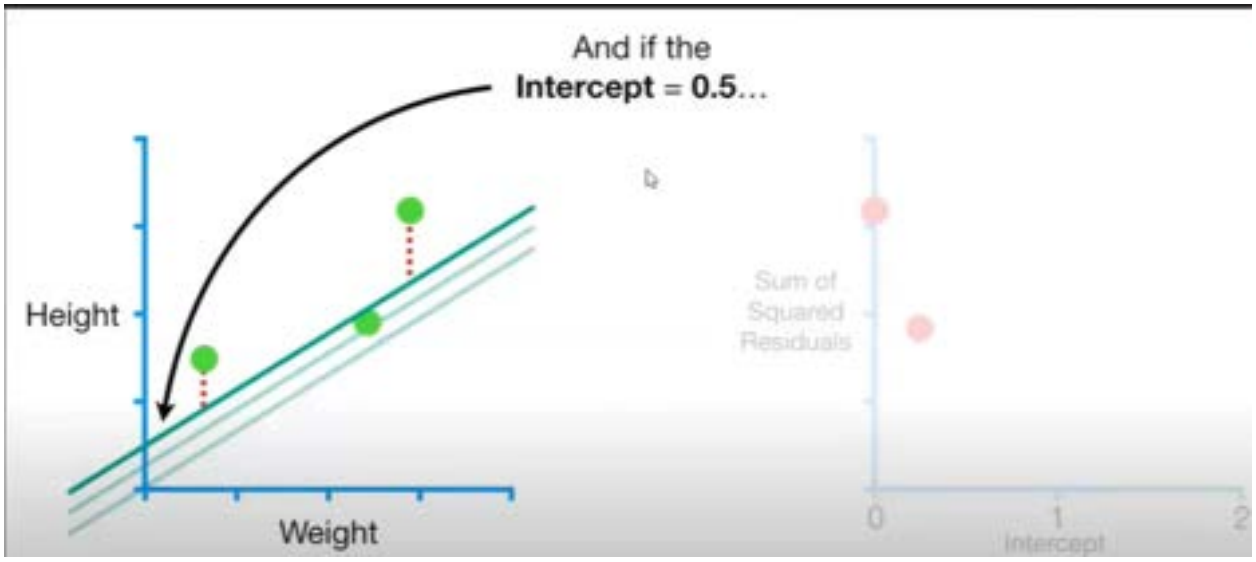

Sum of squared residuals = $1.1^2 + 0.4^2 + 1.3^2$ = 3.1
In the end, **3.1** is the Sum of the Squared Residuals.
Height
Weight

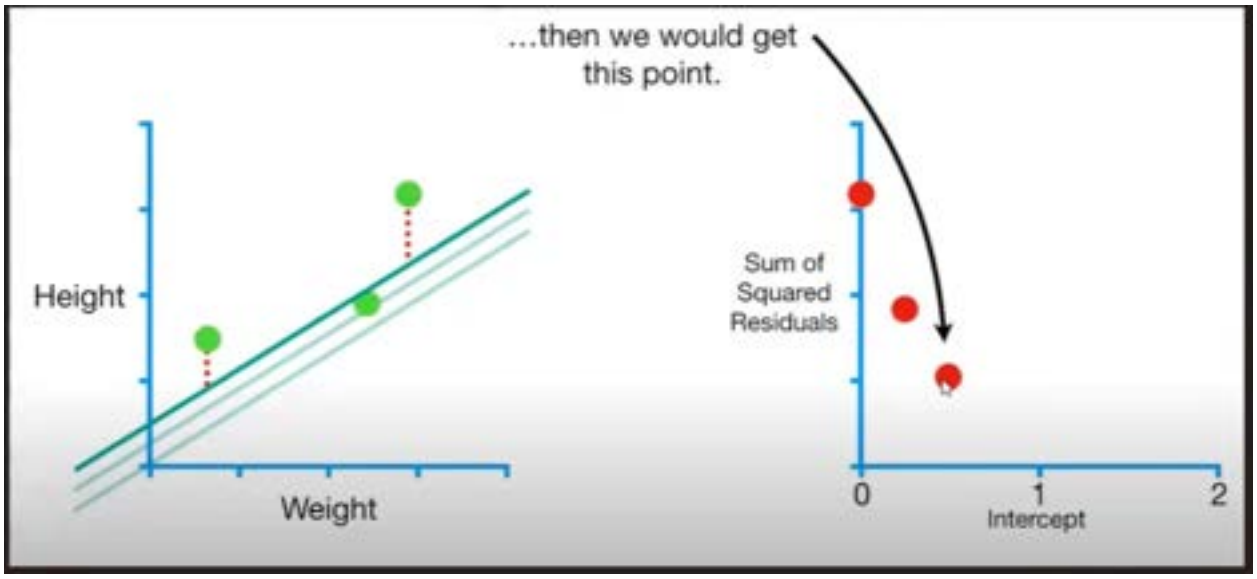


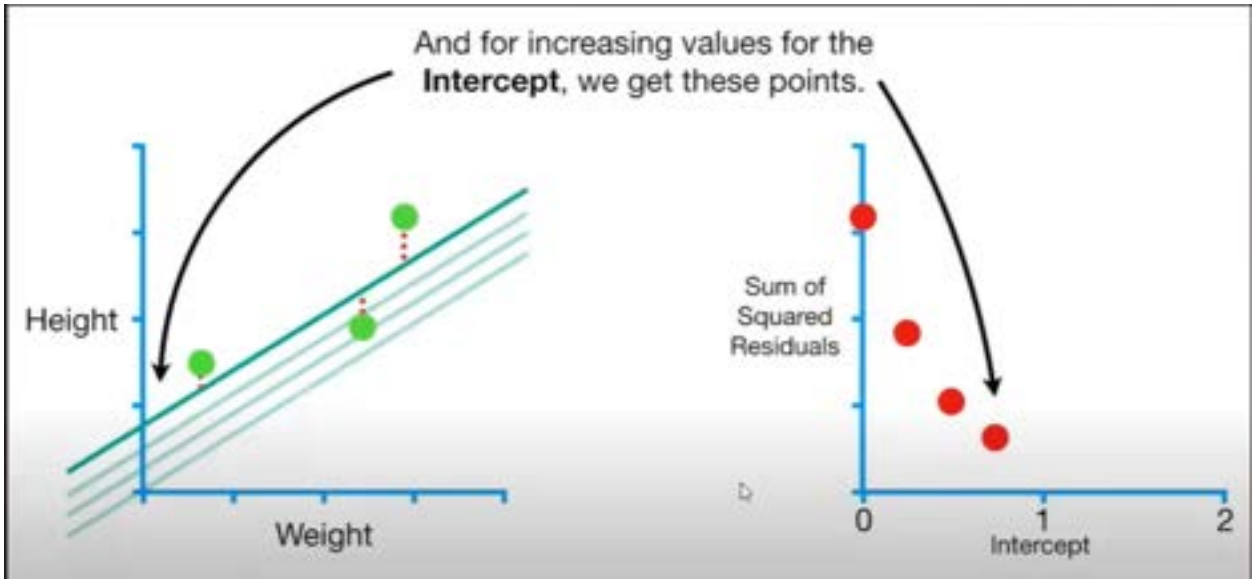


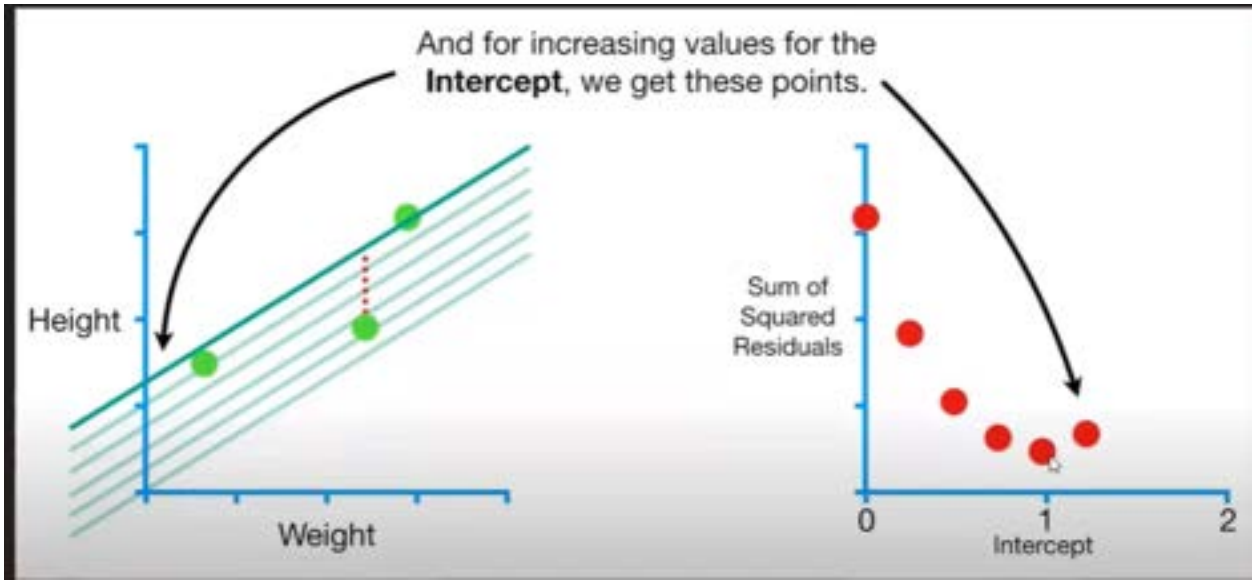
In this graph we got cost function 3.1
If we change our best fit line . the cost will change

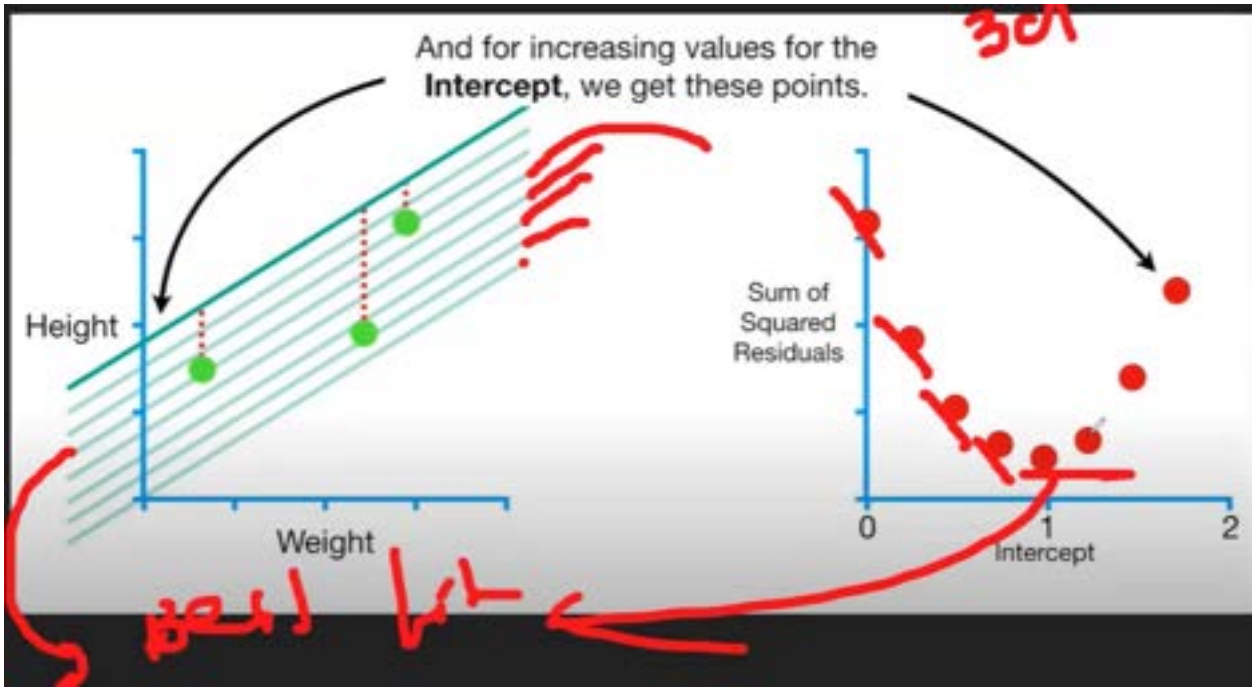

...then we would get this point on the graph.
Height
Weight
Sum of Squared Residuals
0
1
2
Intercept


And if the Intercept = 0.5...
Height
Weight
Sum of Squared Residuals
0
1
2
Intercept

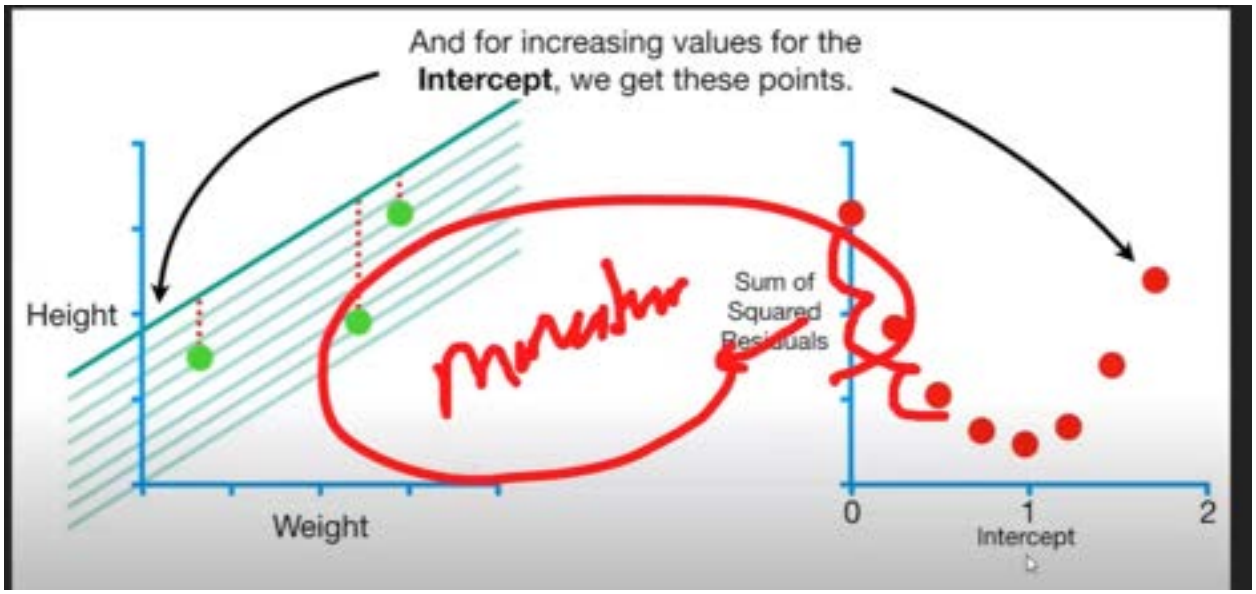

...then we would get
this point.
Height
Weight
Sum of
Squared
Residuals
0
1
2
Intercept


And for increasing values for the
Intercept, we get these points.
Height
Weight
Sum of
Squared
Residuals
0
1
2
Intercept





The line that touches the global minimum that is best fit line

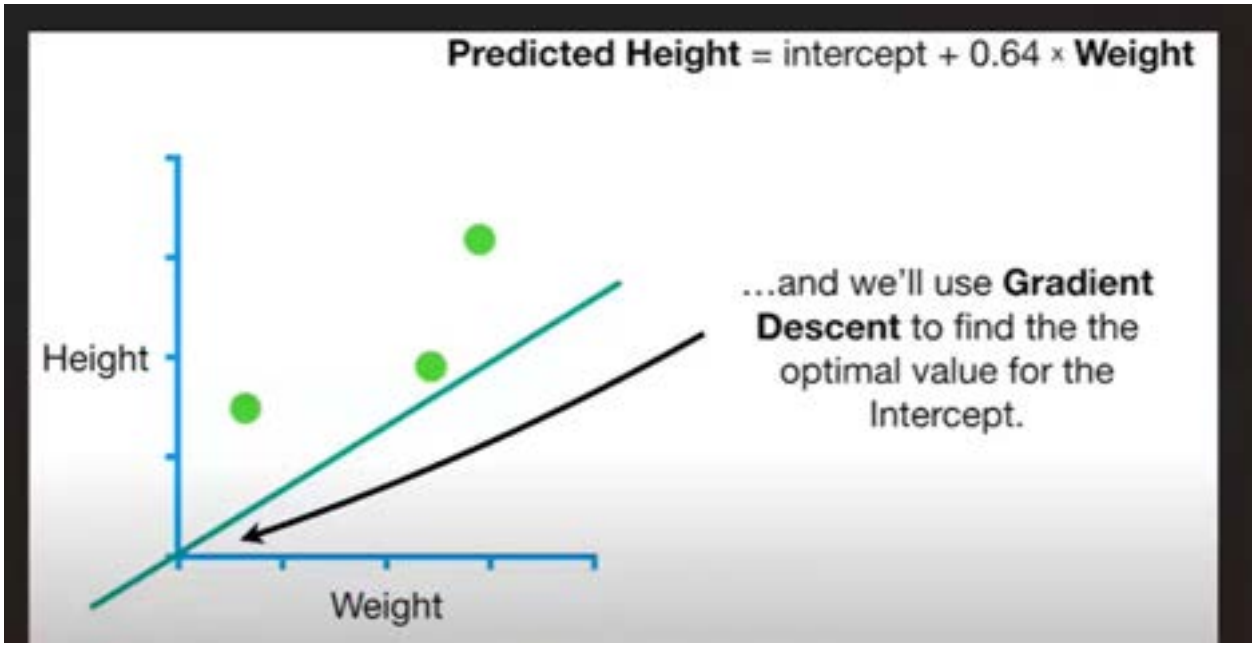


This is called momentum or learning rate
The learning rate is one of the techniques implemented in gradient descent to reh global minima by reducing the step count

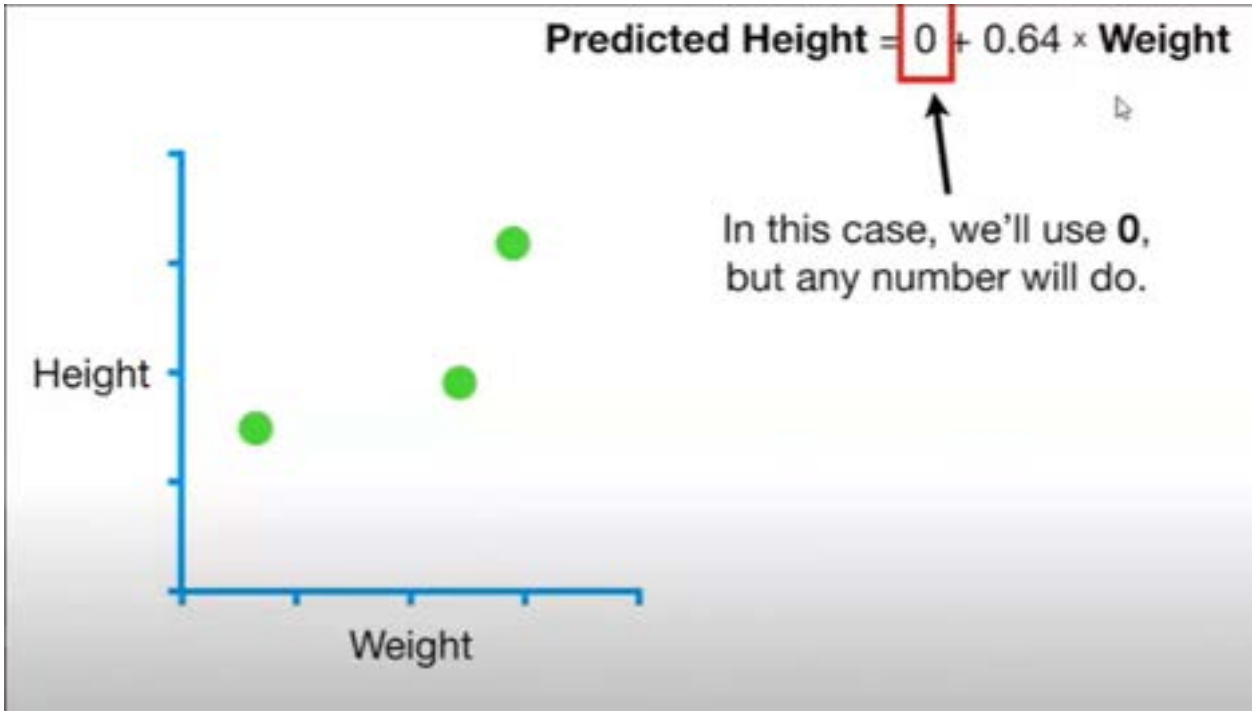
> **gradient descend is an optimization algorithm to find the best fit line from training data**
>
> **learning rate is one of the techniqu apply in graident descend to reach global minima by minimal step**

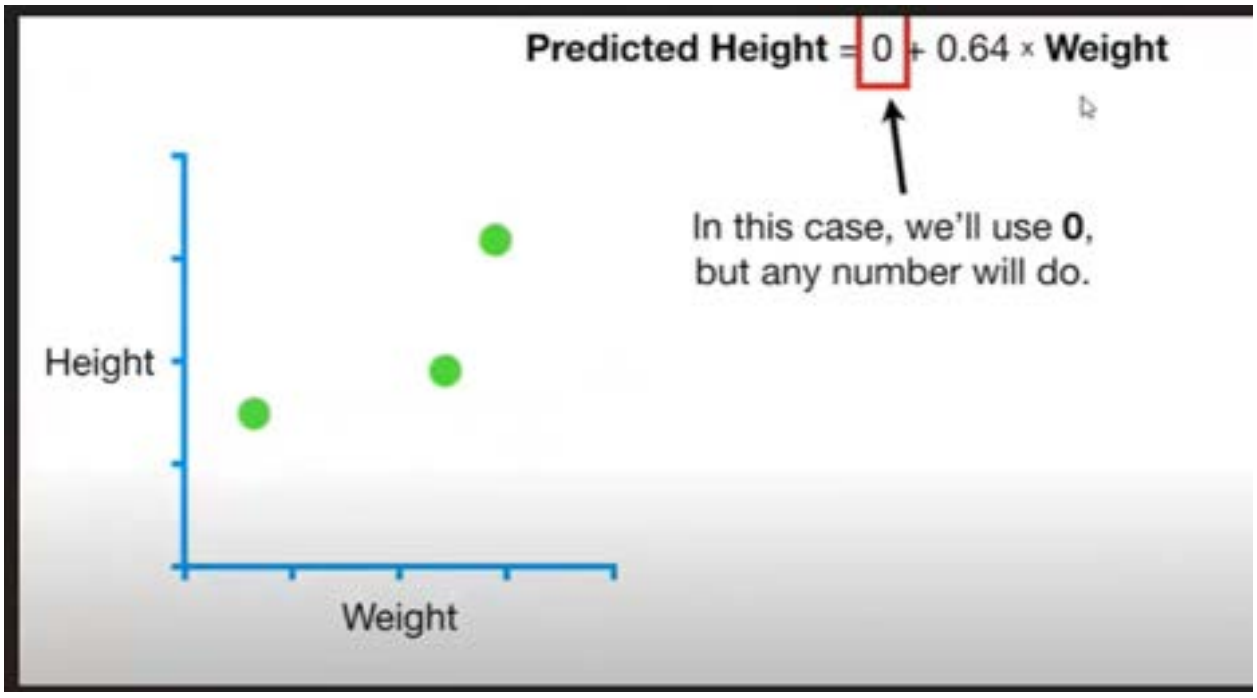
Learning rate always defined to be 0.01 and 0.001
This learning rate introduce in lgbm or XGboost algorithm

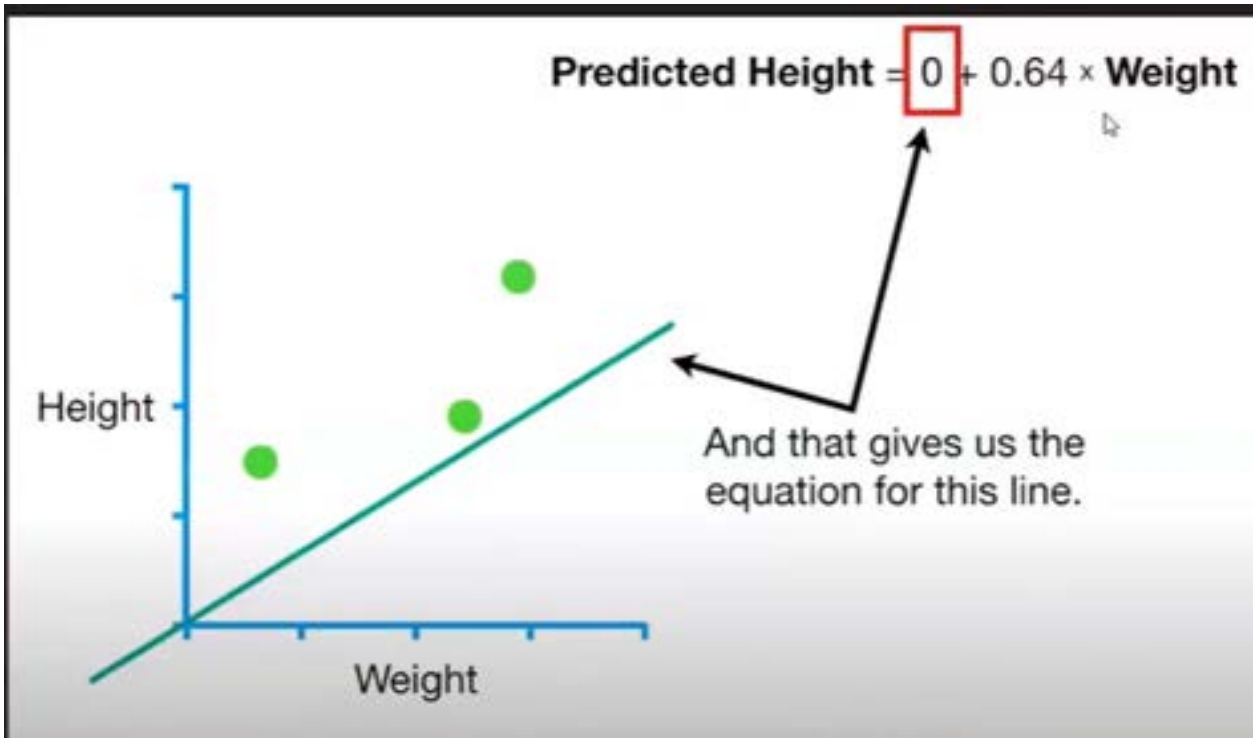
We have some data points

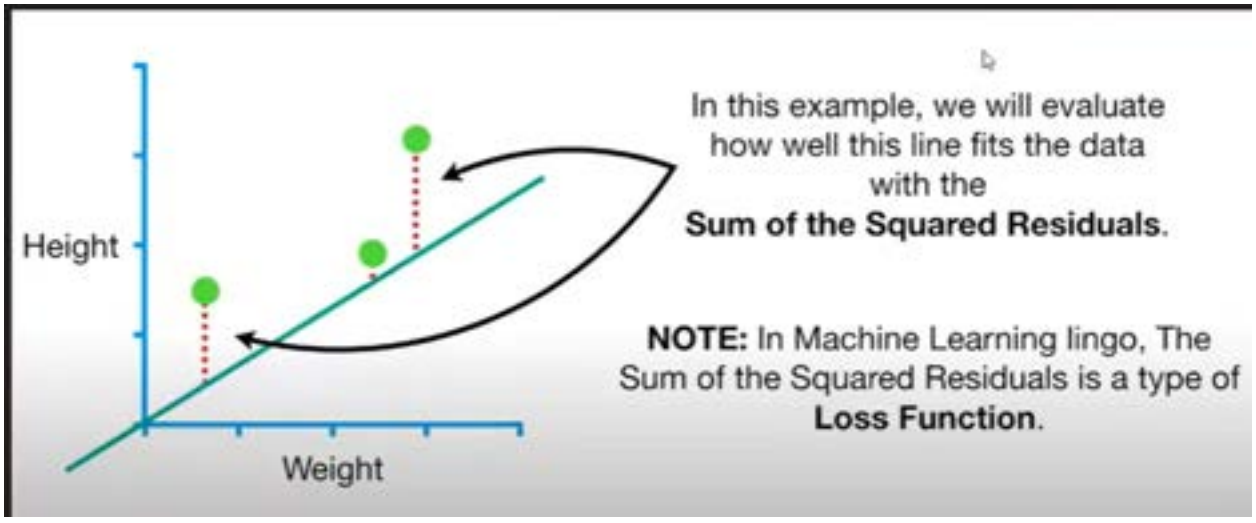


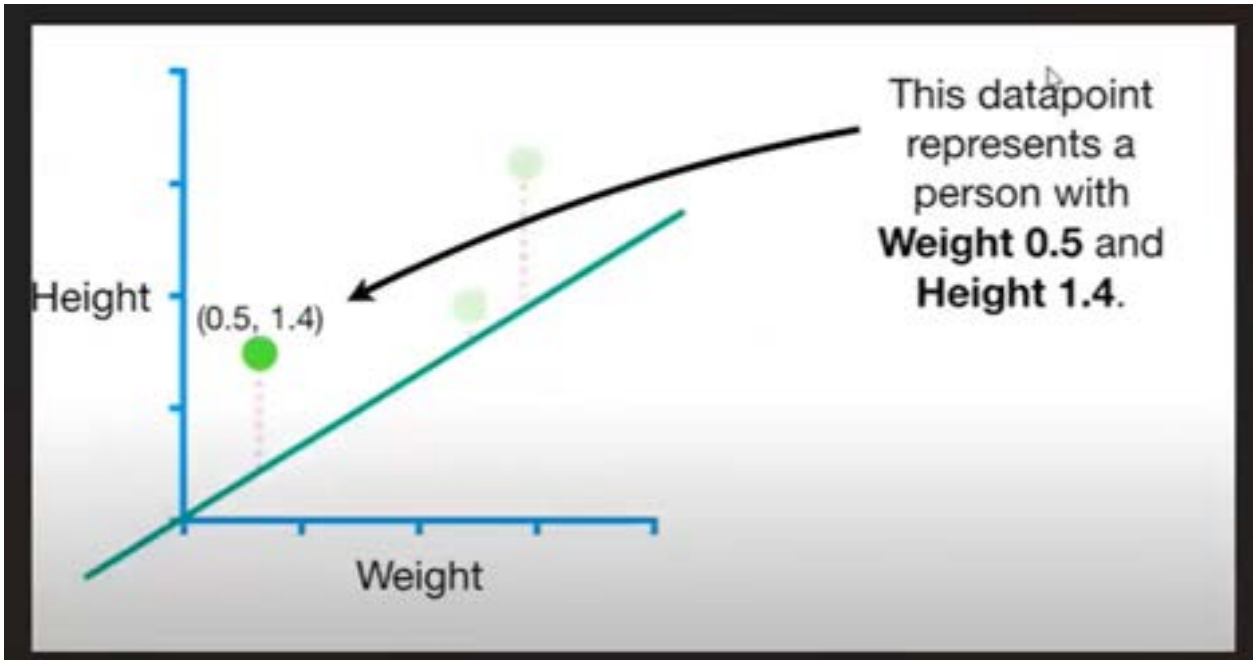
This are all data points kid height and kid weight we plot the best fit line when we plot best fit line here intercept is zero because the line touch the horizon and consider slope is 0.64 for explanation we are giving slope 0.64

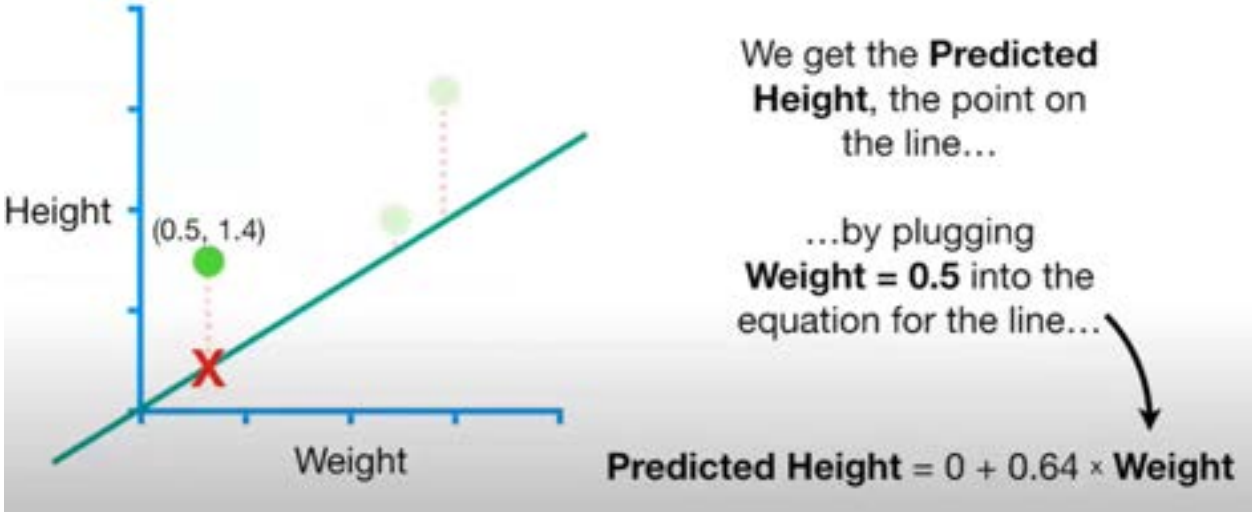


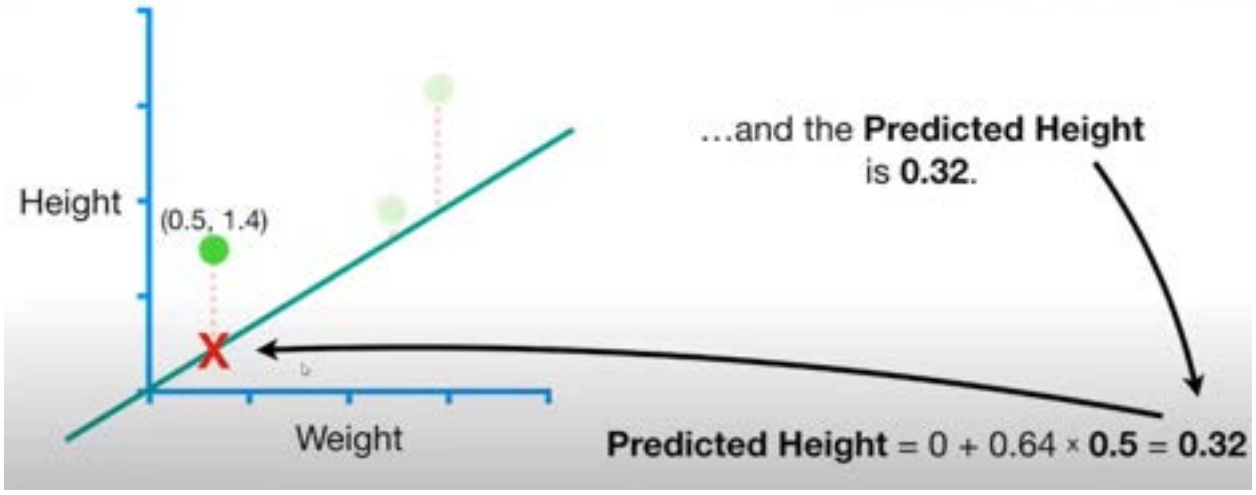


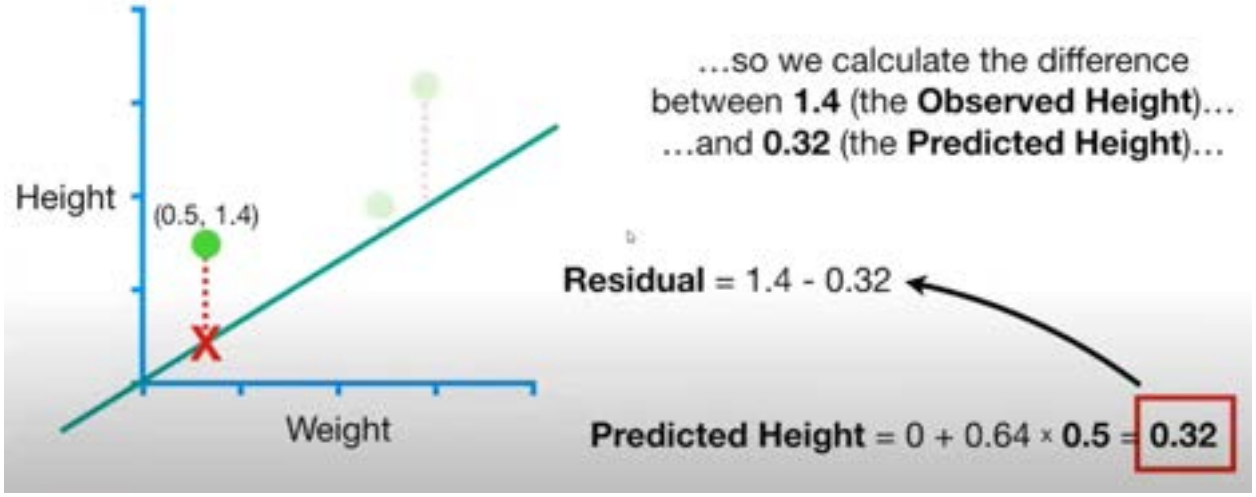

Predicted Height = 0 + 0.64 × Weight
Height
Weight
And that gives us the equation for this line.

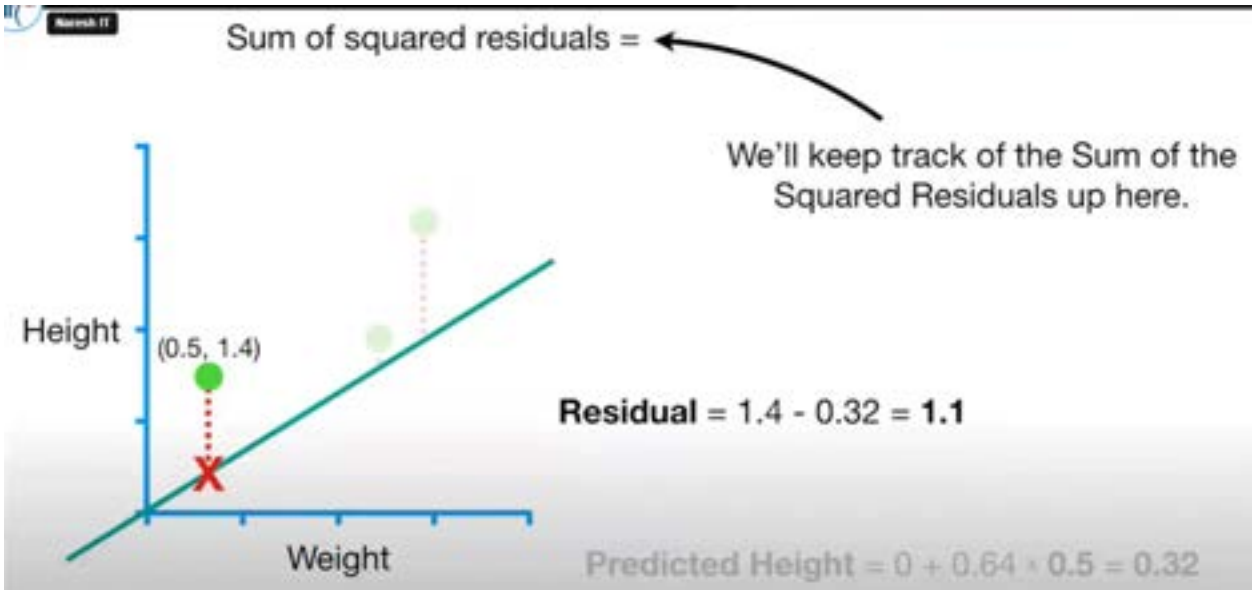

Height
Weight
In this example, we will evaluate how well this line fits the data with the Sum of the Squared Residuals.
NOTE: In Machine Learning lingo, The Sum of the Squared Residuals is a type of Loss Function.

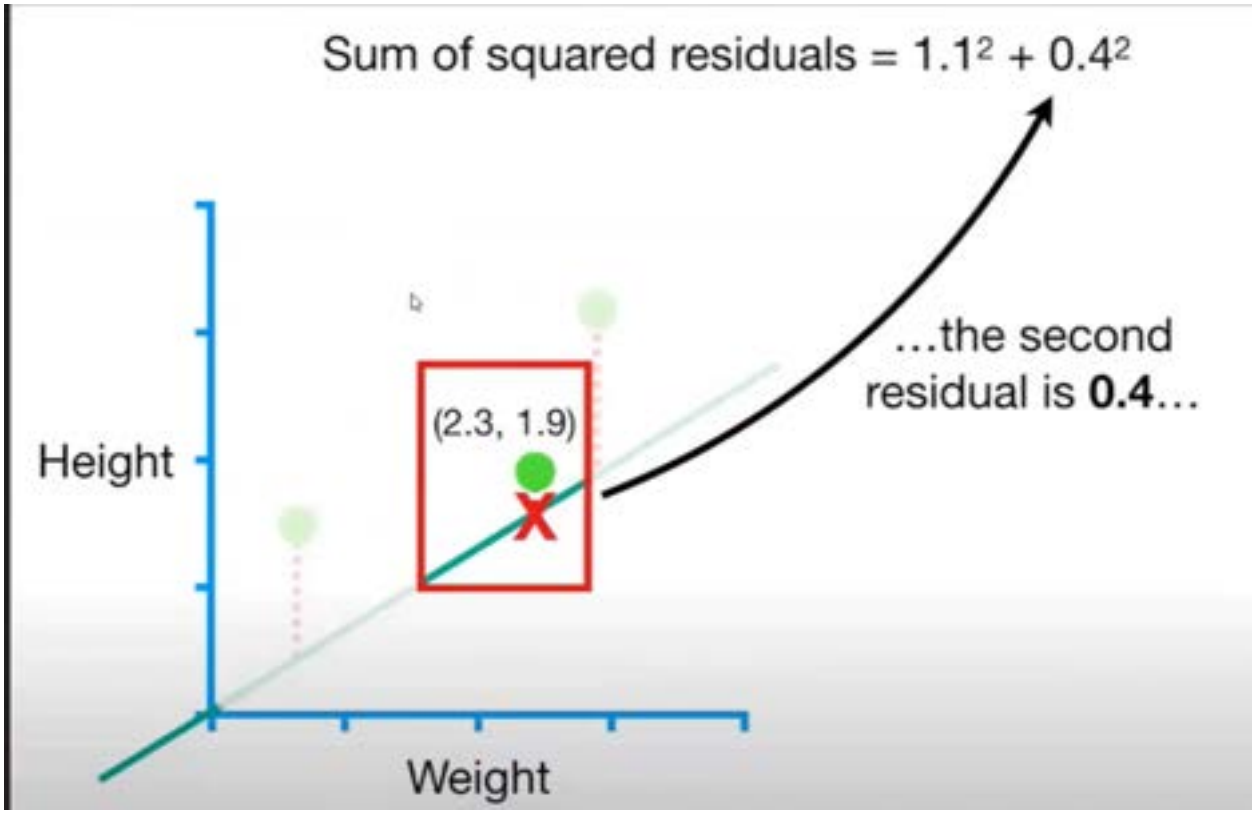

Height
(0.5, 1.4)
Weight
This datapoint represents a person with **Weight 0.5** and **Height 1.4**.

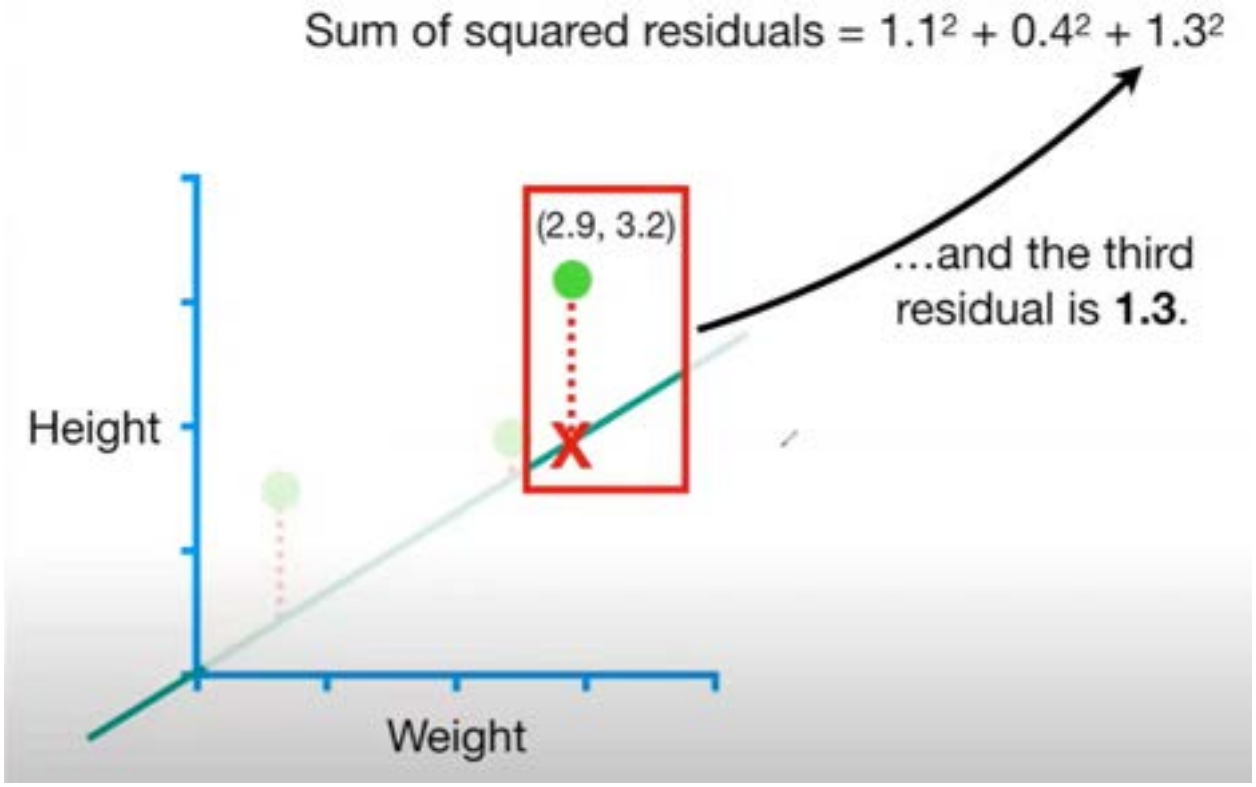

Height
(0.5, 1.4)
Weight
We get the **Predicted Height**, the point on the line…
…by plugging **Weight = 0.5** into the equation for the line…
Predicted Height = 0 + 0.64 × **Weight**

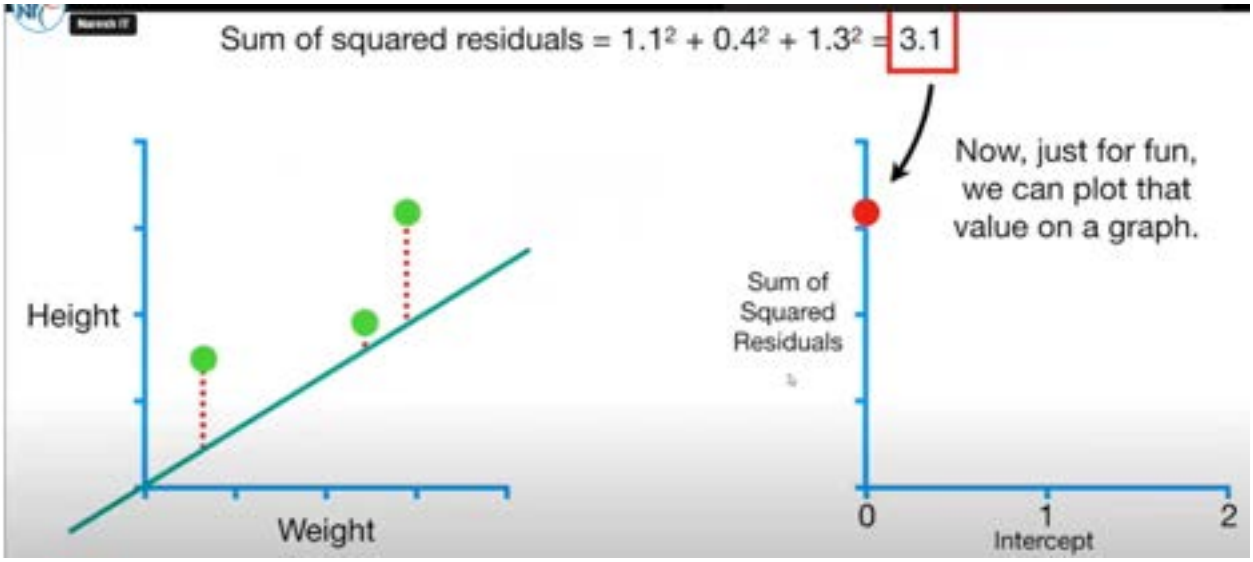

Height
(0.5, 1.4)
Weight
...and the **Predicted Height** is **0.32**.
Predicted Height = 0 + 0.64 × **0.5** = **0.32**

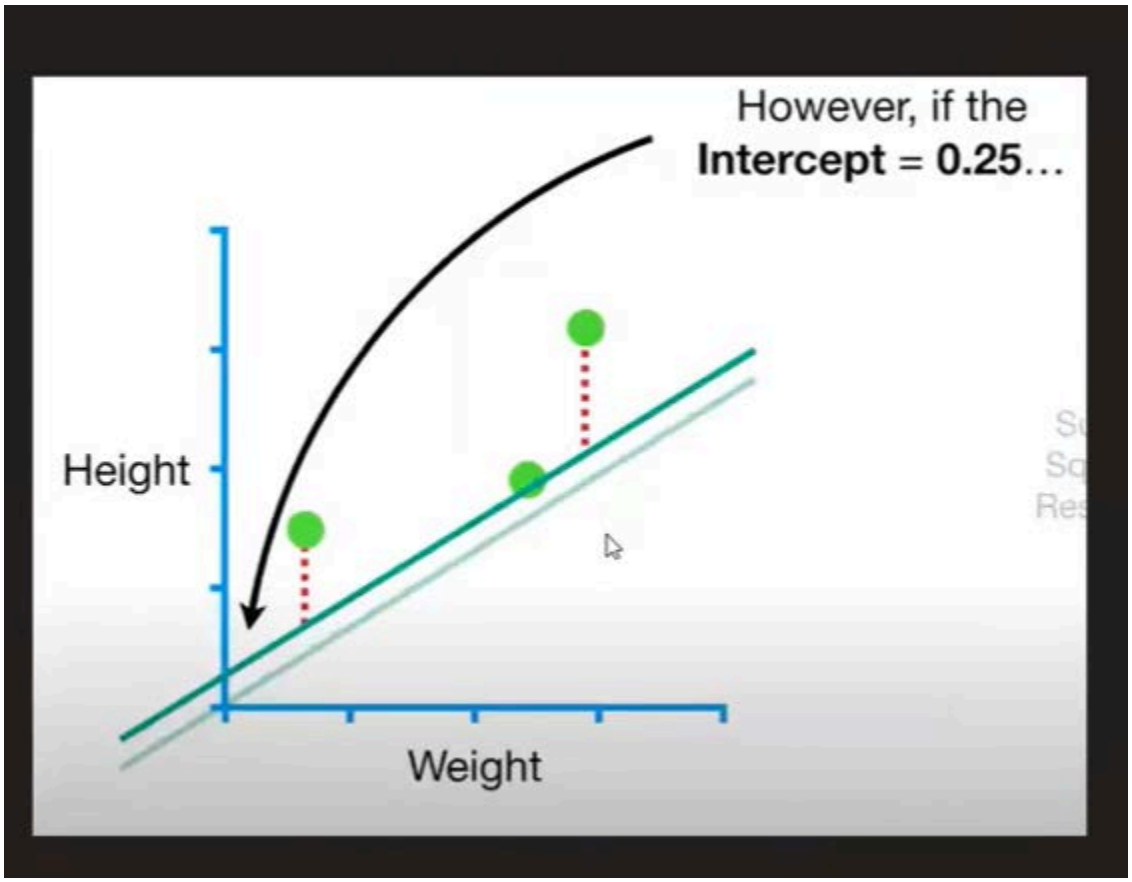

Height
(0.5, 1.4)
Weight
...so we calculate the difference between **1.4** (the **Observed Height**)...
...and **0.32** (the **Predicted Height**)...
Residual = 1.4 - 0.32
Predicted Height = 0 + 0.64 × **0.5** = **0.32**

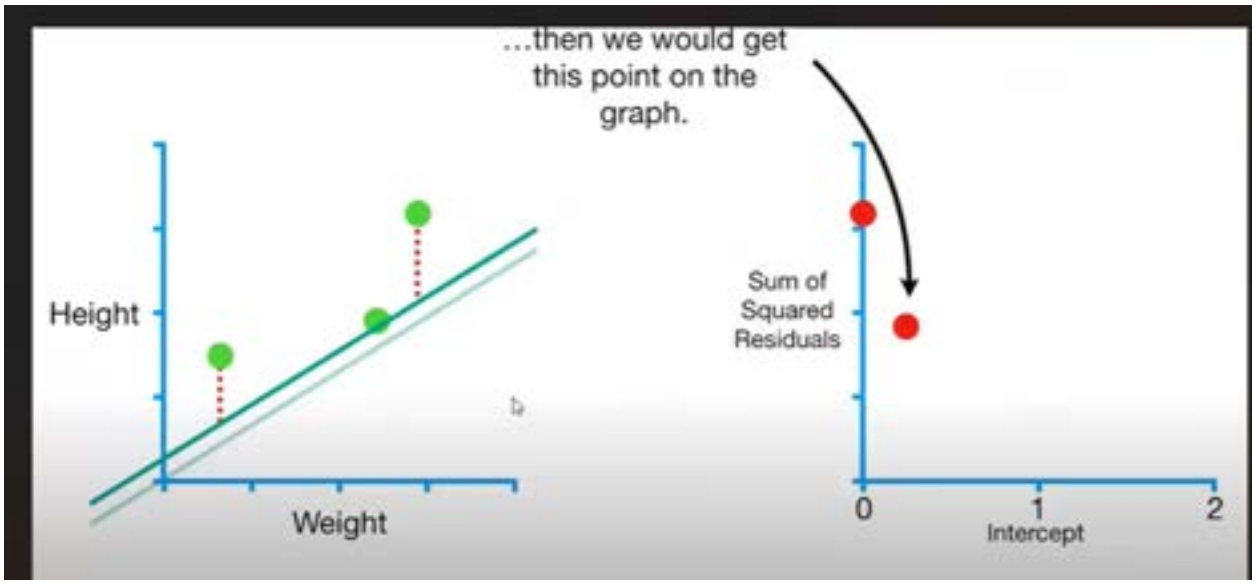

Sum of squared residuals =
We'll keep track of the Sum of the Squared Residuals up here.
Height
(0.5, 1.4)
Weight
Residual = 1.4 - 0.32 = **1.1**
Predicted Height = 0 + 0.64 × 0.5 = 0.32

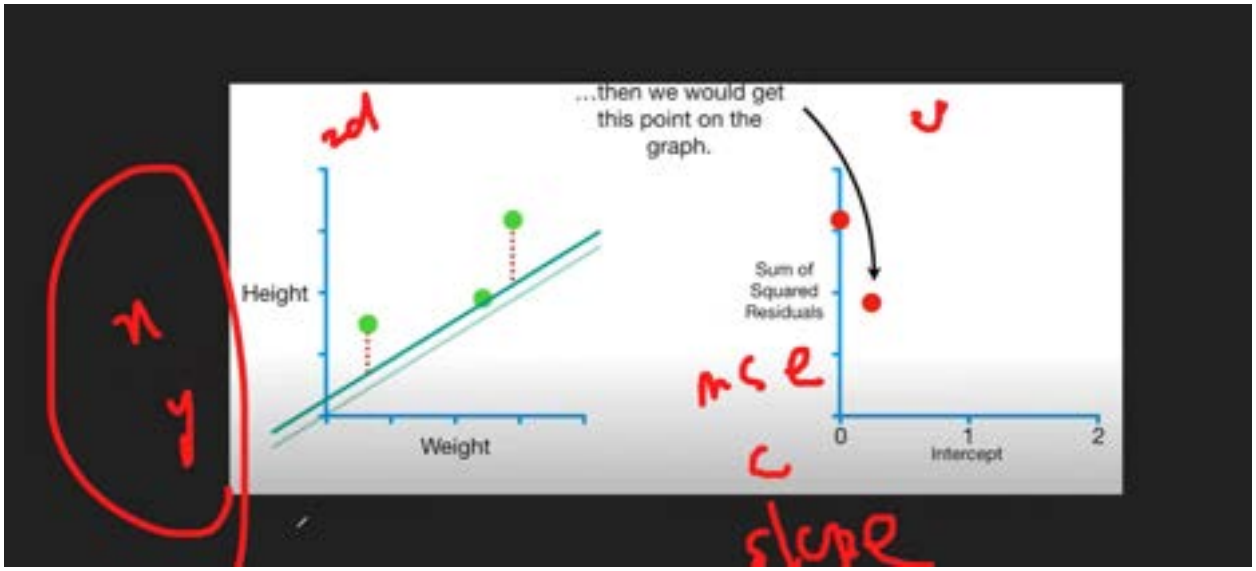

Sum of squared residuals = $1.1^2 + 0.4^2$
…the second residual is **0.4**…
(2.3, 1.9)
Height
Weight


Sum of squared residuals = $1.1^2 + 0.4^2 + 1.3^2$
…and the third residual is **1.3**.
(2.9, 3.2)
Height
Weight


Sum of squared residuals = $1.1^2 + 0.4^2 + 1.3^2 = 3.1$
Now, just for fun, we can plot that value on a graph.
Height
Weight
Sum of Squared Residuals
0
1
2
Intercept


However, if the **Intercept = 0.25**...
Height
Weight

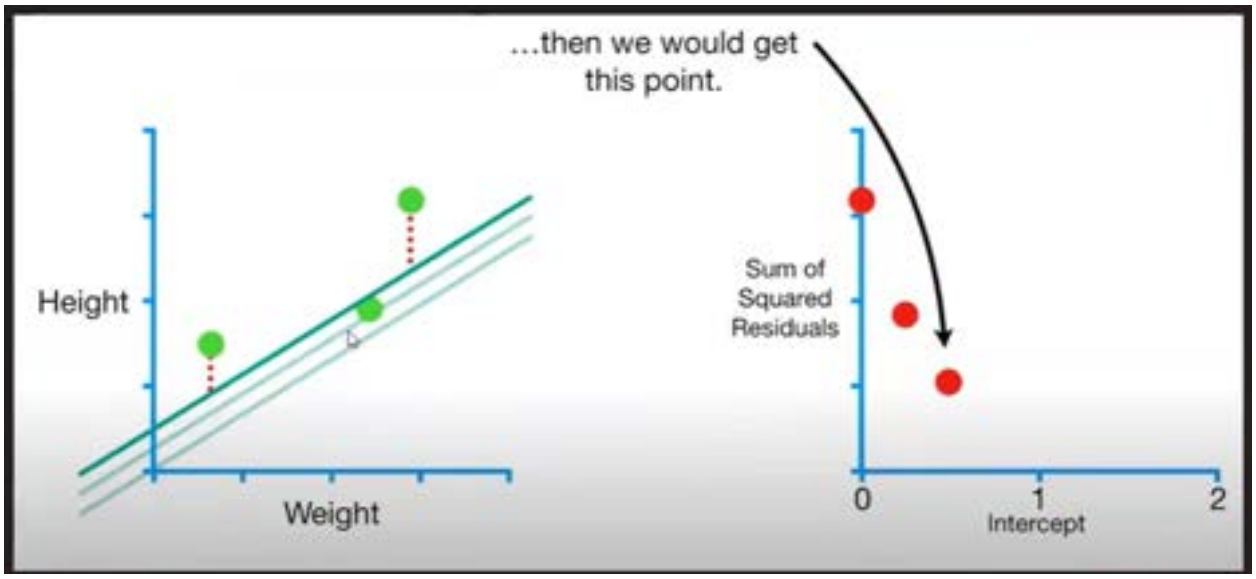


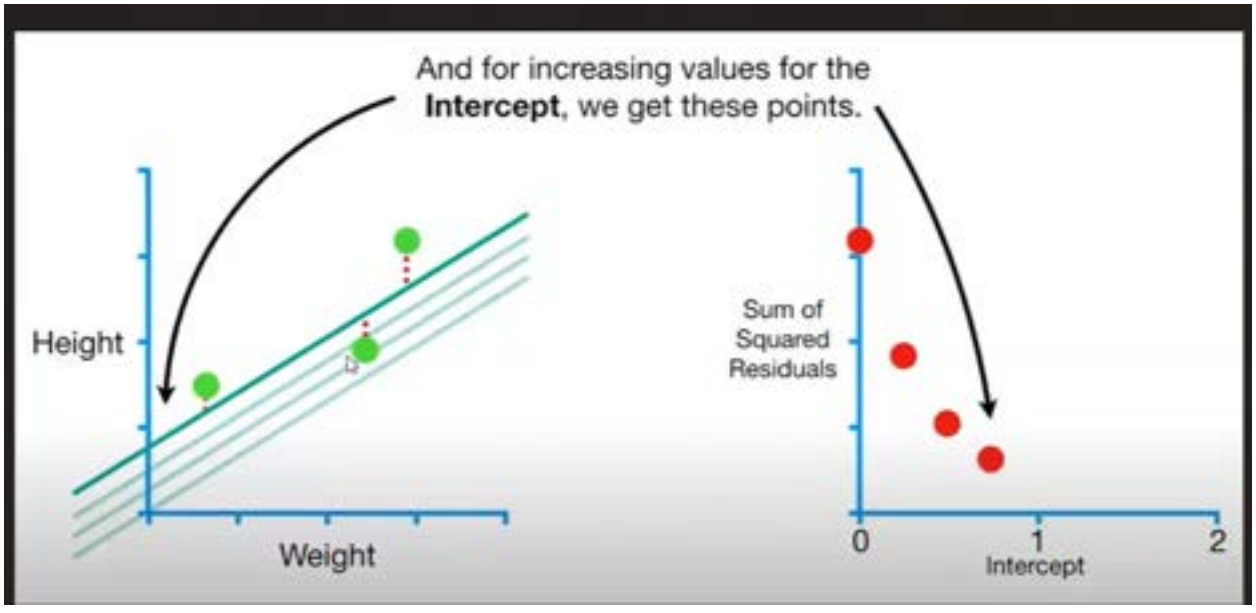


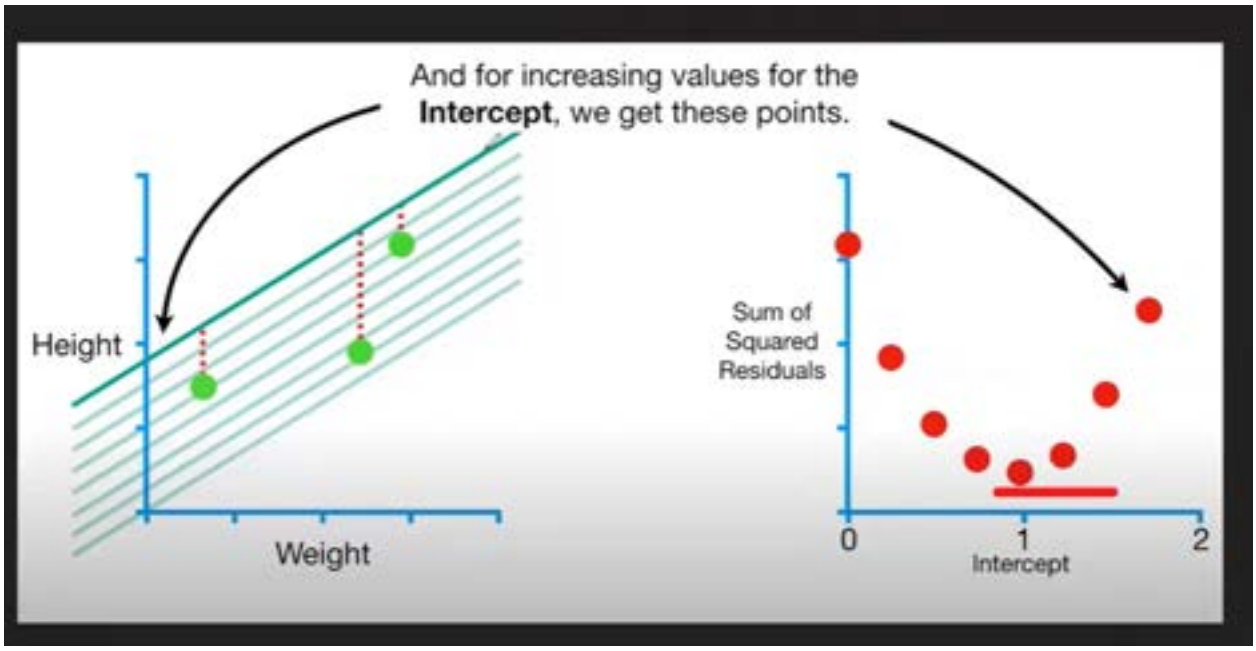
One is 2d graph and another one is 3d graph 2d graph build on x and Y and 3d graph build on cost function , intercept and slope

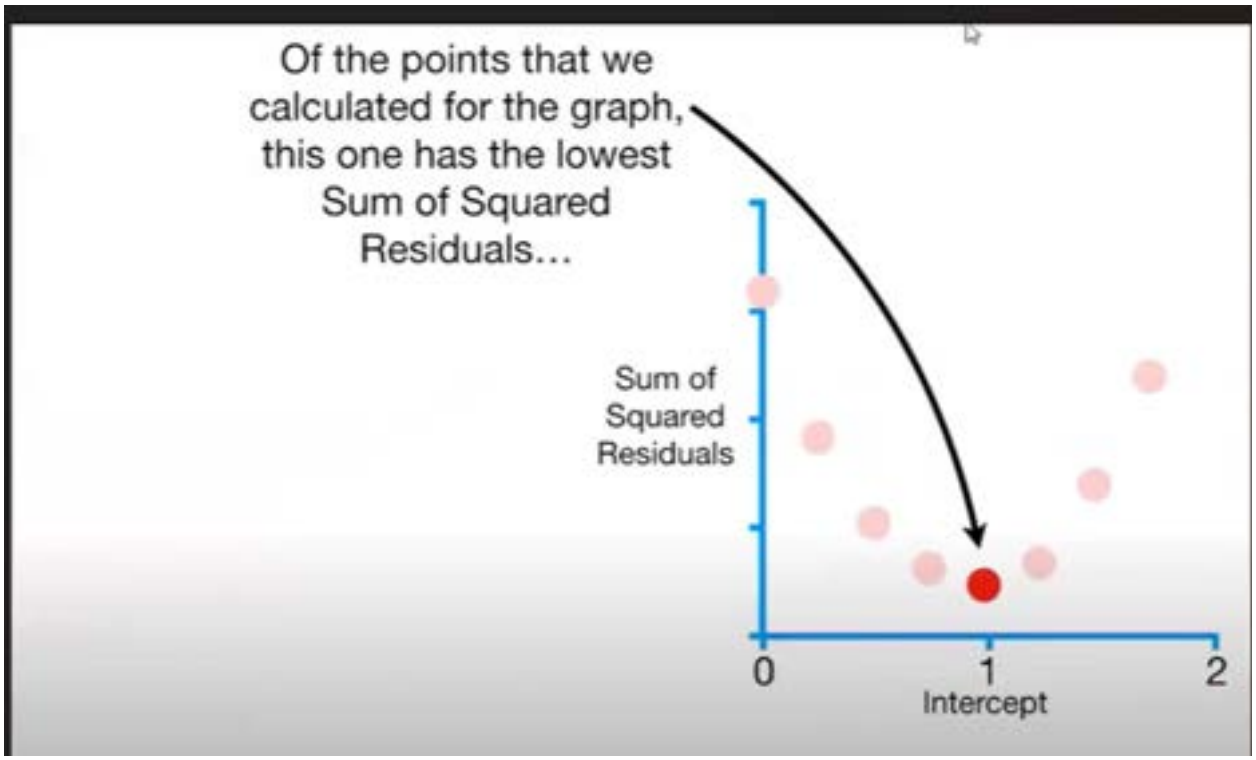
Gradient decent : it is a optimization algorithm to implement on slope and intercept by calculate cost function to decide the best fit line

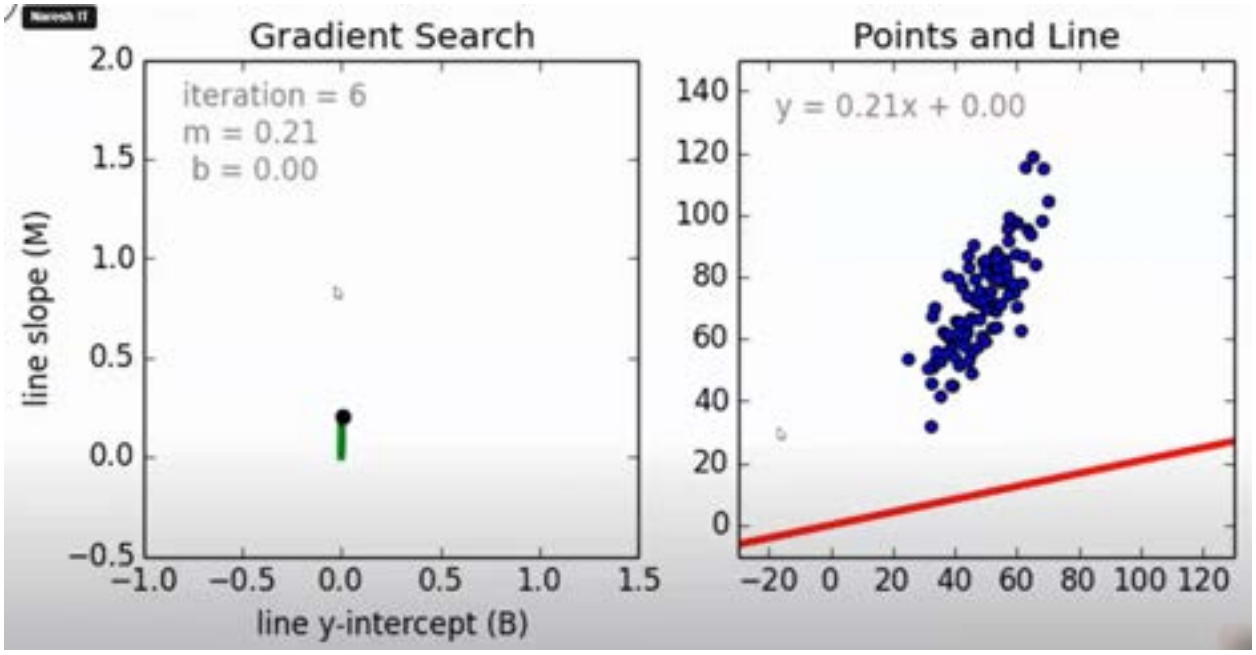
If we change intercept to 0.5 then we got this point

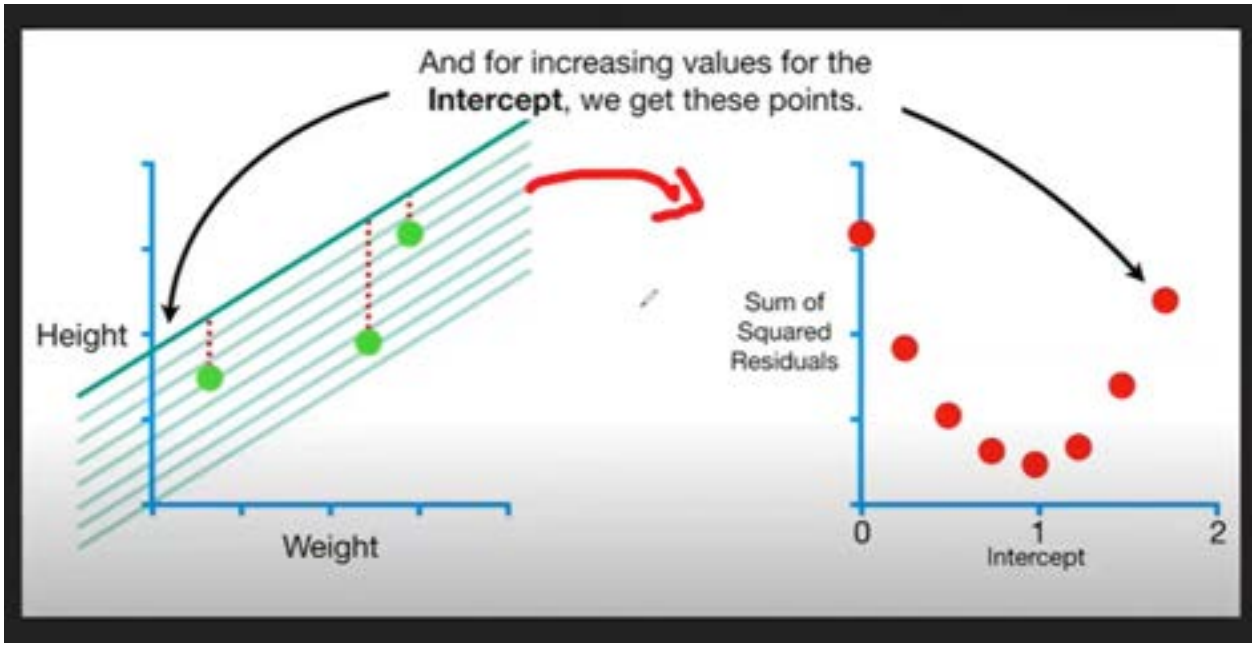

...then we would get
this point.
Height
Weight
Sum of
Squared
Residuals
0
1
2
Intercept


And for increasing values for the
Intercept, we get these points.
Height
Weight
Sum of
Squared
Residuals
0
1
2
Intercept

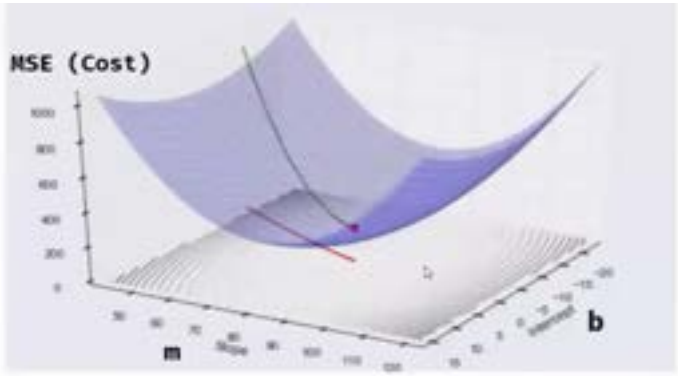

And for increasing values for the **Intercept**, we get these points.
Height
Weight
Sum of Squared Residuals
0
1
2
Intercept

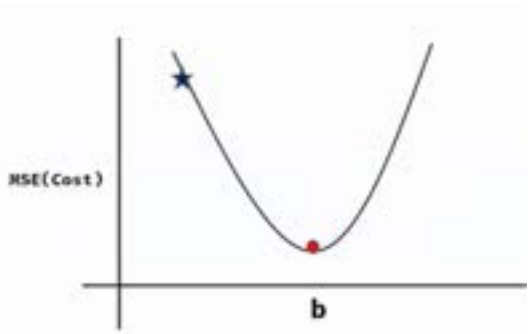

Of the points that we calculated for the graph, this one has the lowest Sum of Squared Residuals...
Sum of Squared Residuals
0
1
2
Intercept



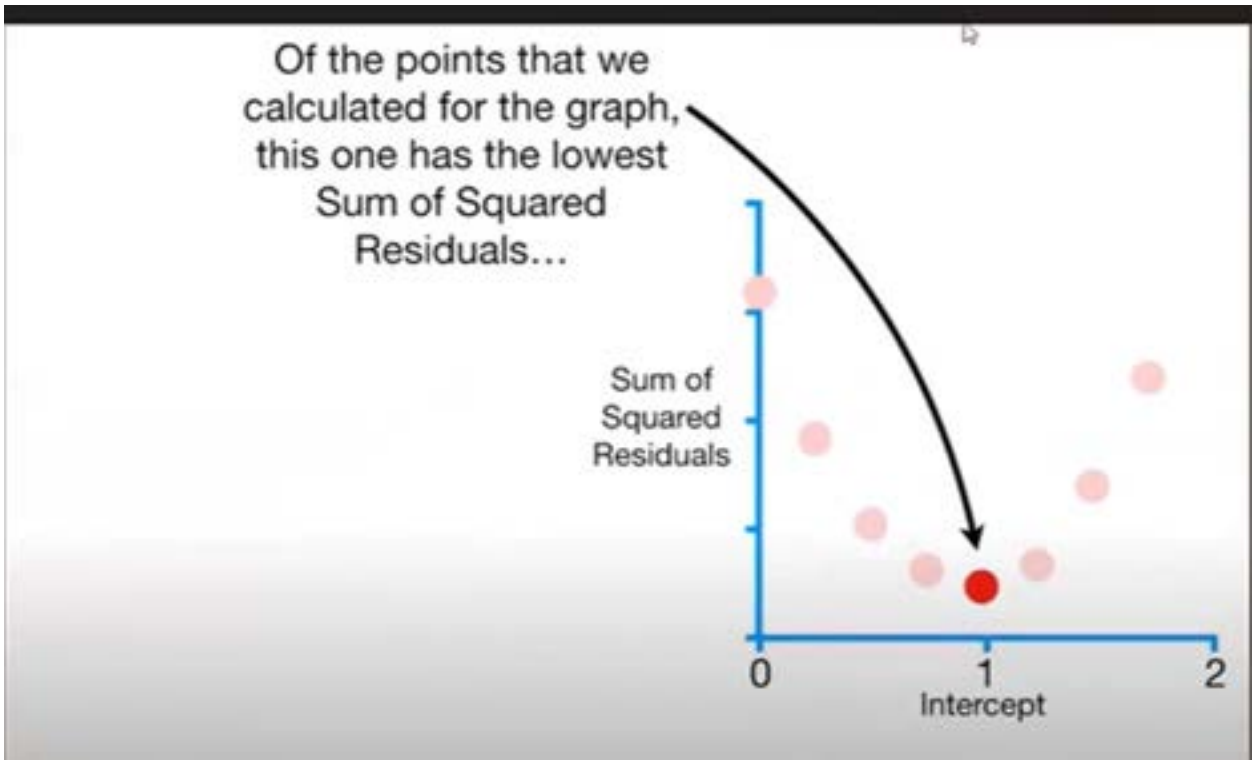


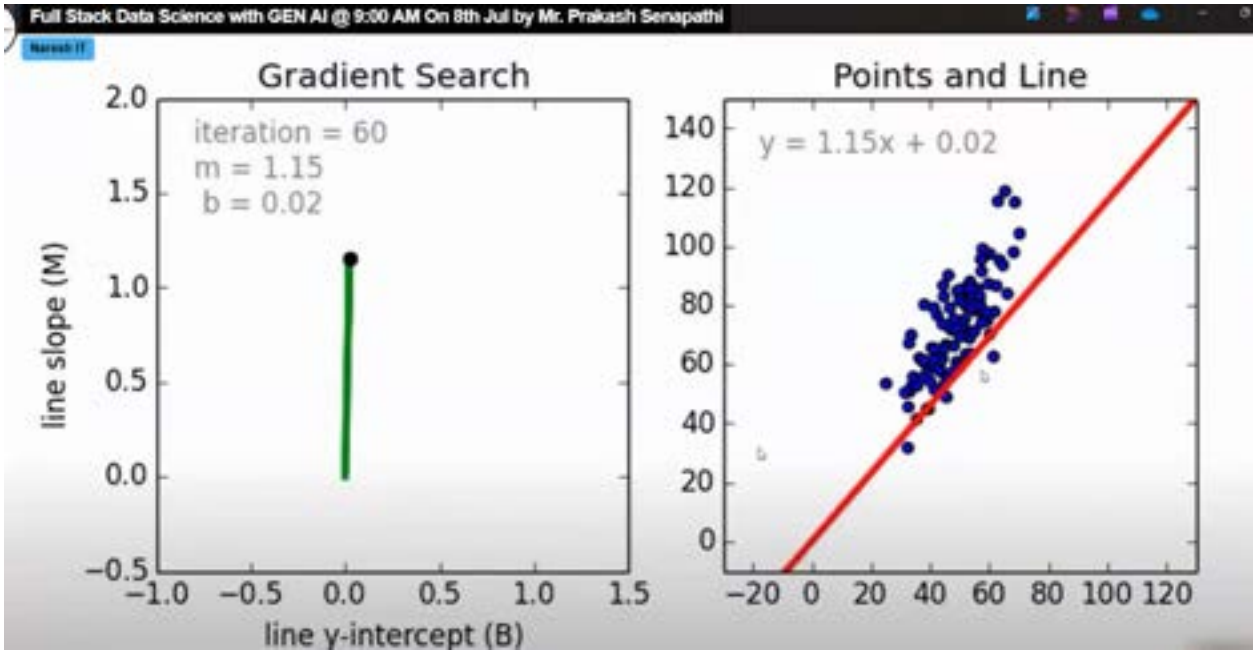
The moment we change the intercept this lines will change


MSE (Cost)
m
b

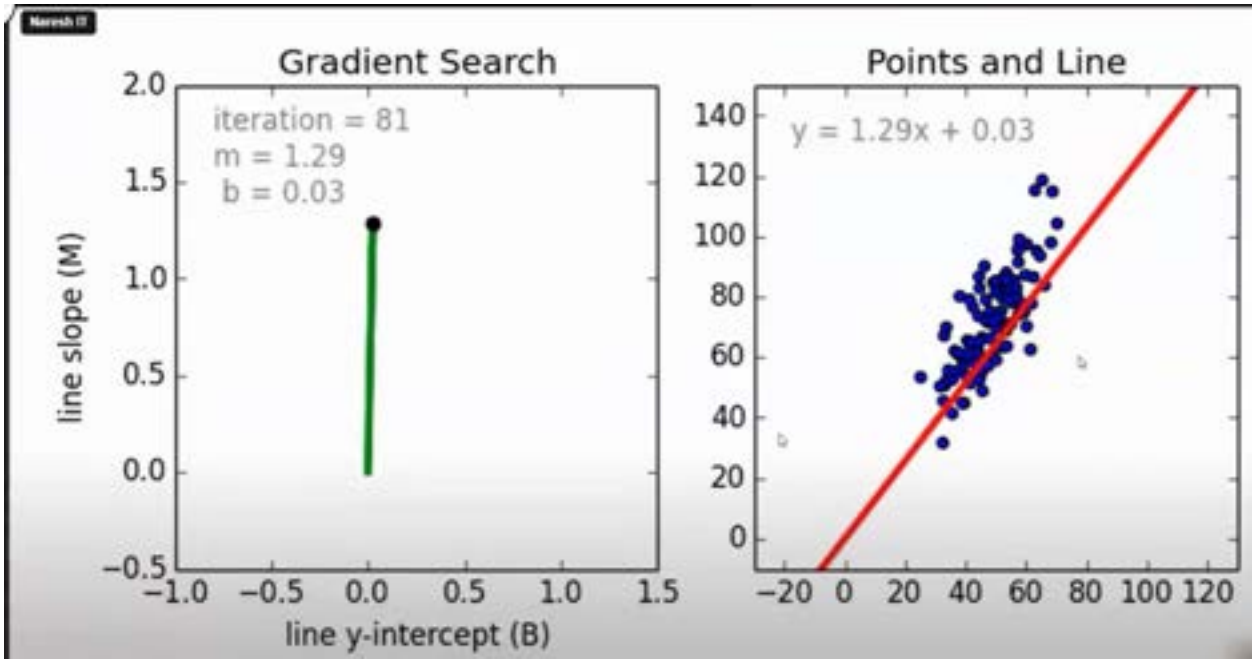

MSE(Cost)
b

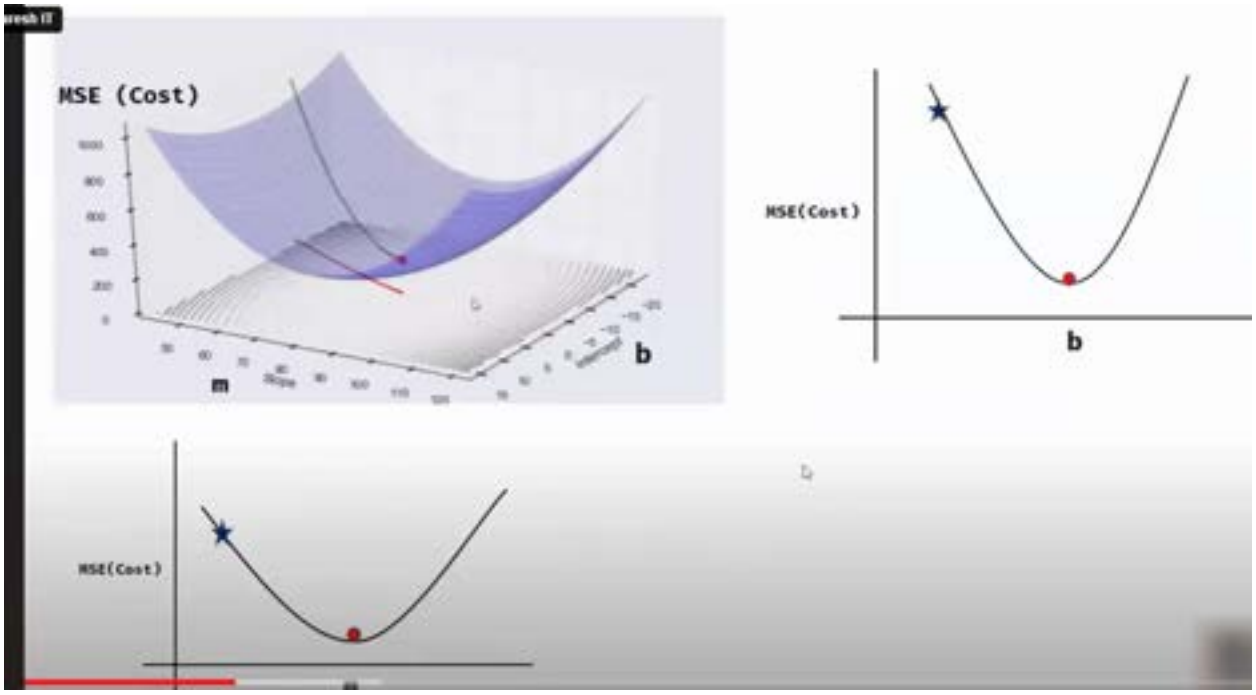

MSE(Cost)
m

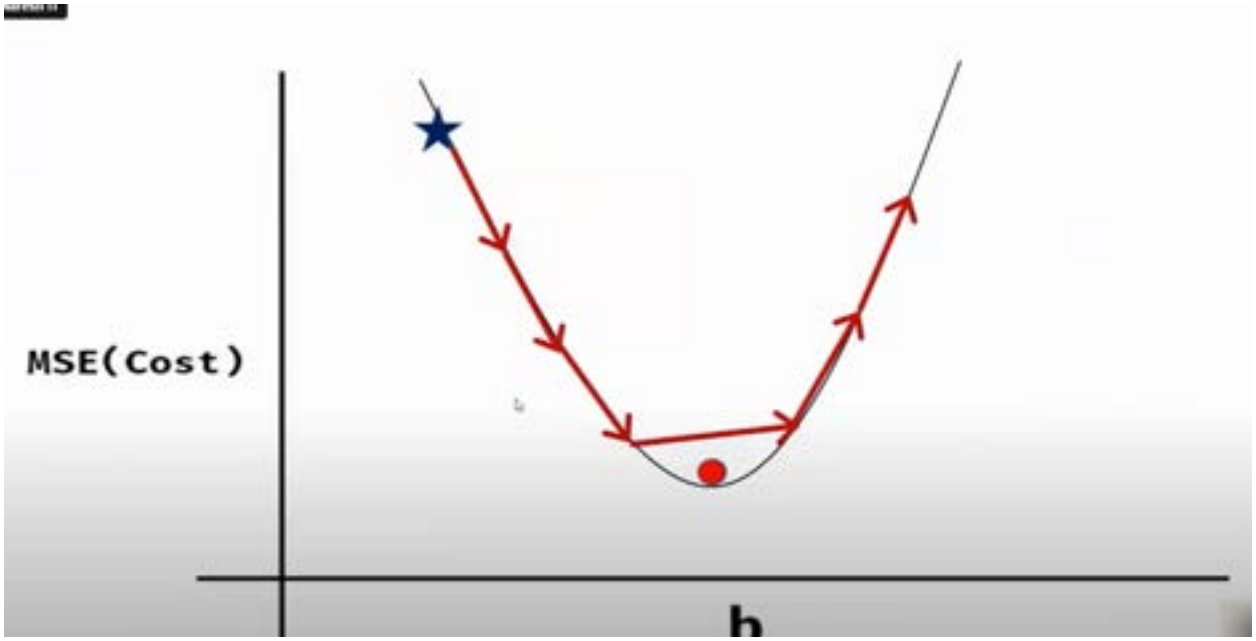

Of the points that we calculated for the graph, this one has the lowest Sum of Squared Residuals...
Sum of Squared Residuals
0
1
2
Intercept



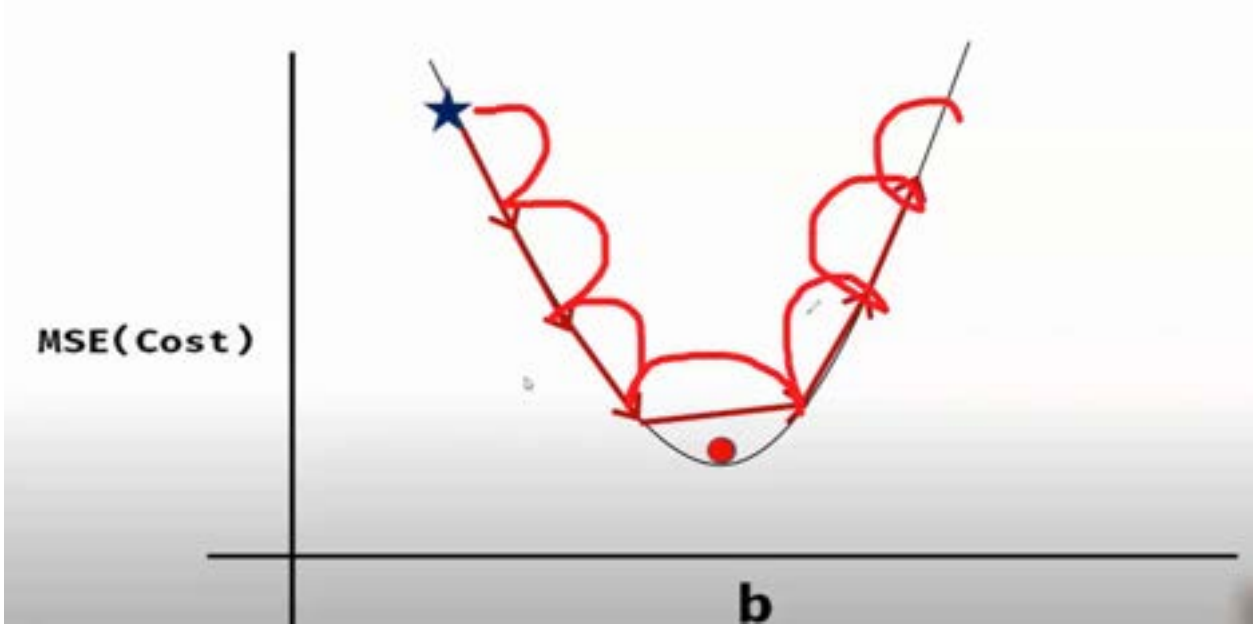
The moment when we change intercept this line will keep on changing

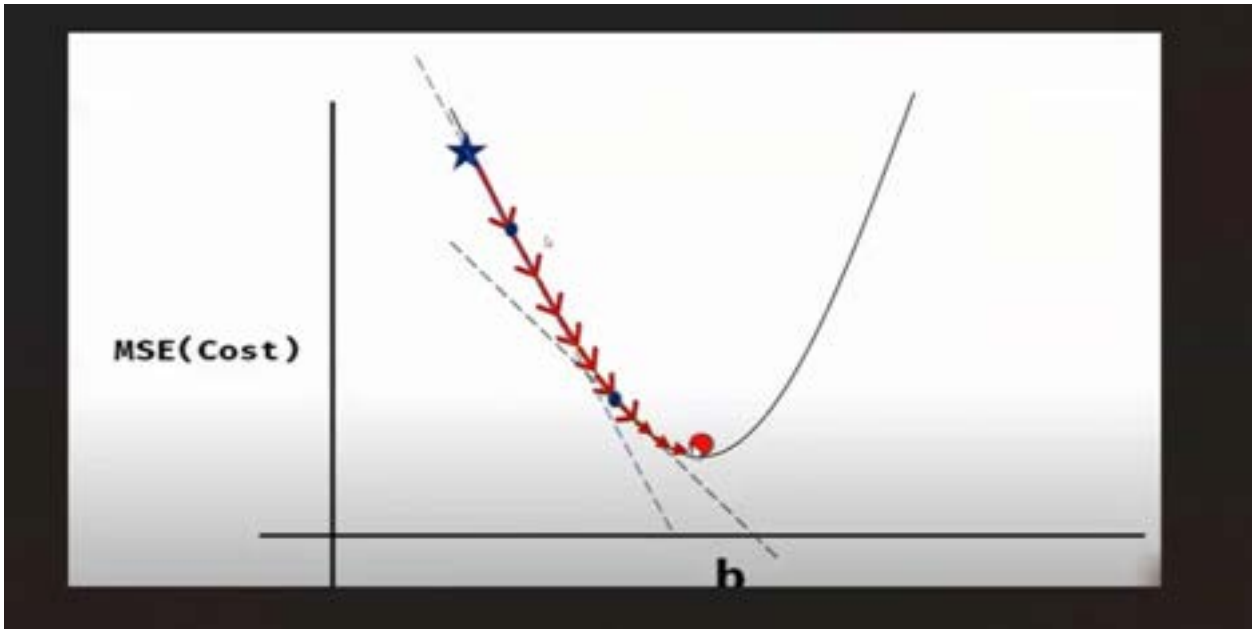






If the step count not baby steps it mysterious global minima





This is called minimum step count baby steps to reach global minima this line is best fit line

Note : in Deep Learning in ANN,CNN, RNN we are using this parameter as optimizer the optimizer algorithm is gradient descend, stochastic gradient descend, batch gradient descend, adam, adagrad, adamax ,rmsprop

DEEP LEARNING ANN, CNN, RNN

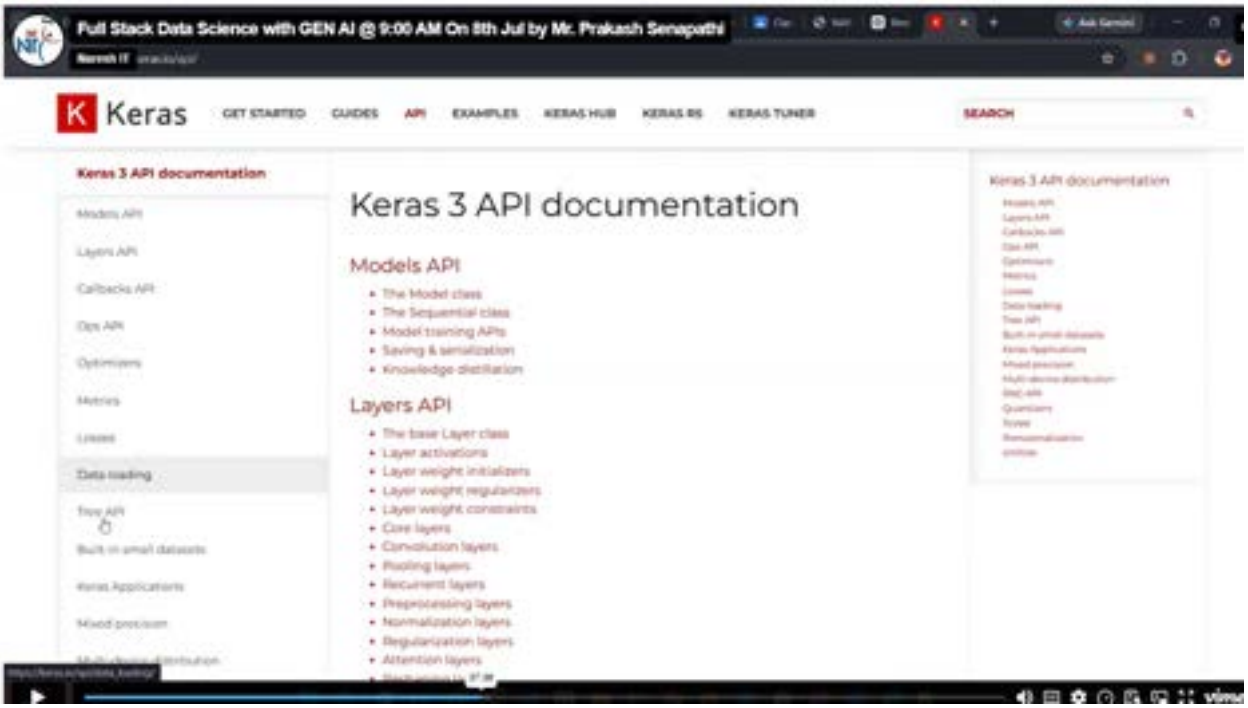
we are using this parameter as optimizer

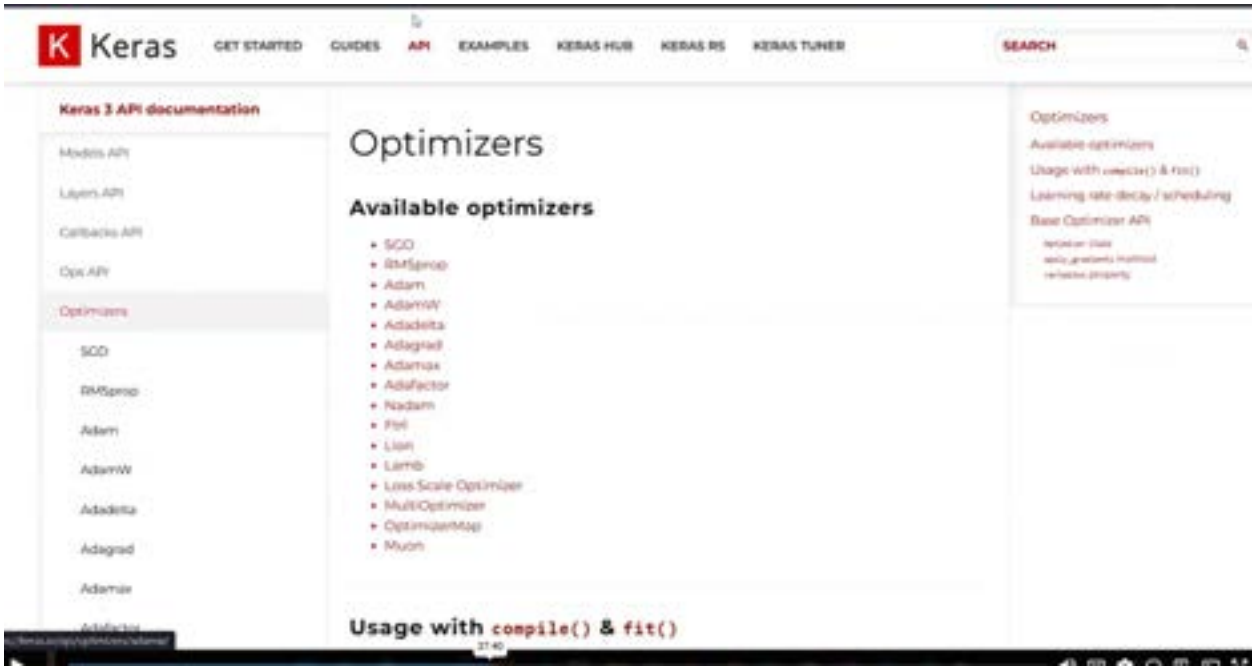
optimizer --

- gradient descend
- stocastic gradient descend
- batch gradient descend
- adam
- adagrad
- adamax
- rmsprop

we can found these optimize using keras library which deep learning framework

We can found this optimizers by using keras libraries which is deep learning framework

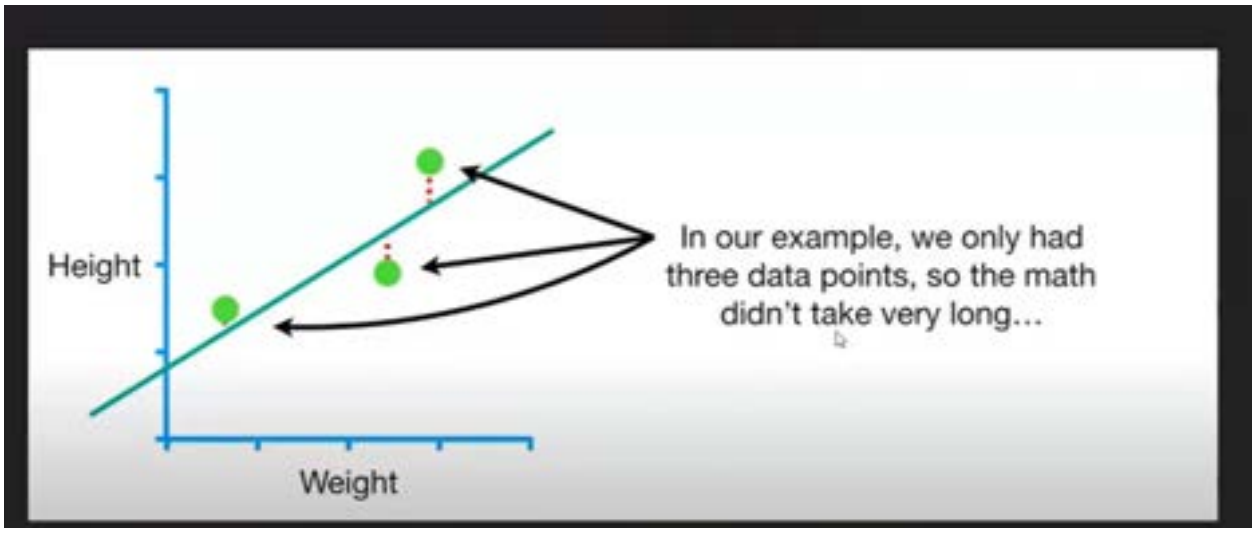




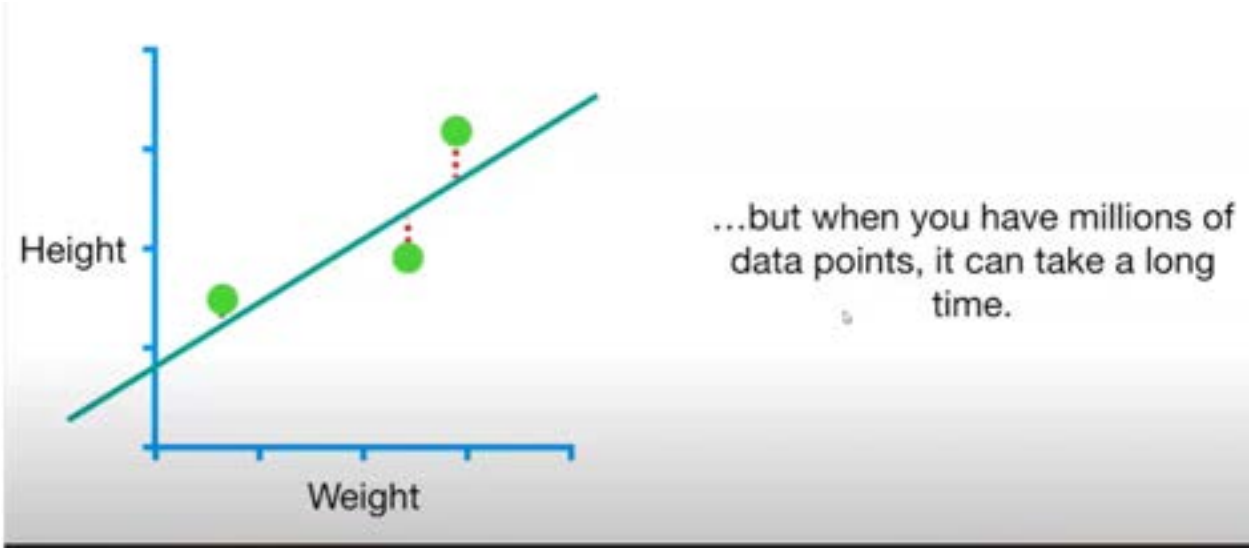
Gradient descend is a optimized algorithm to reach global minima by calculating each and every data point while training the data

> **gradiend descent is an optimized algorithm to reach global minima by calculation each and everydatapoint while traing the data**

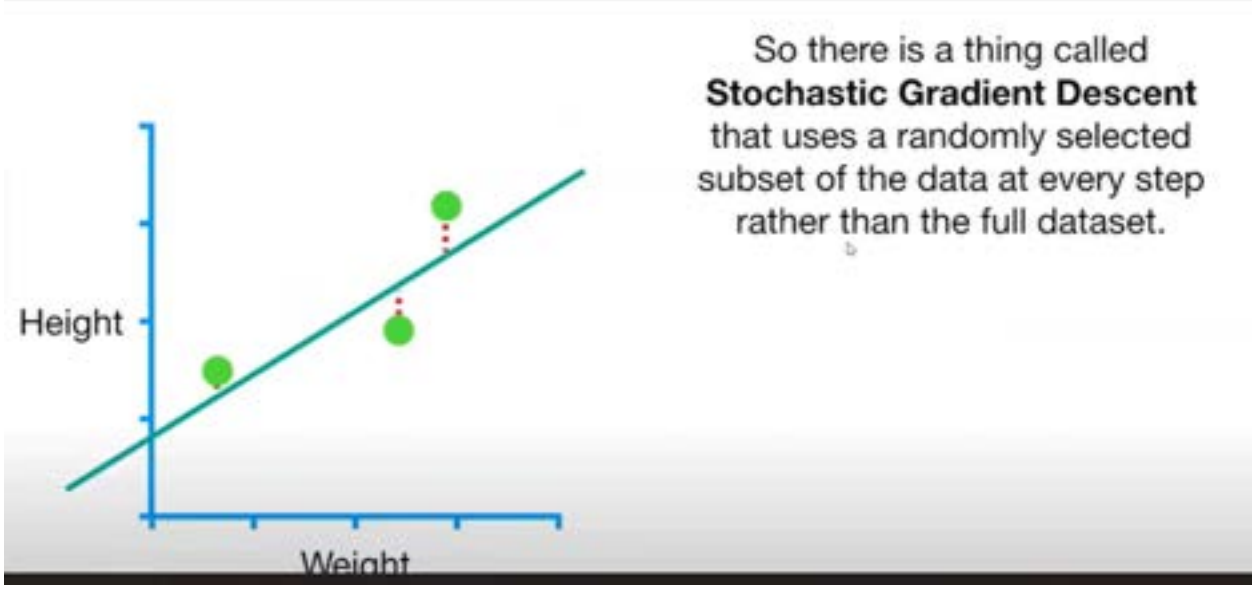
While we build ML and deep Learning we not work only with 3 points we will work with 30K points example it need to take 30K times



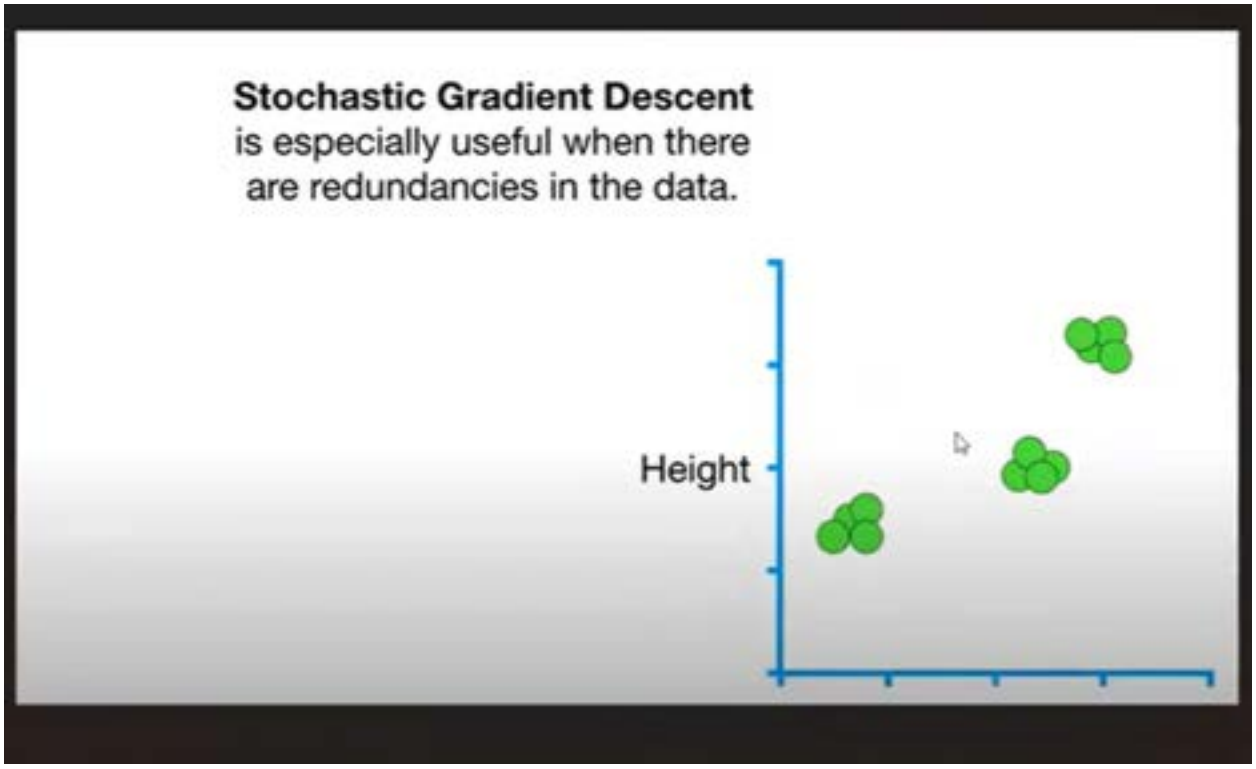
In this we have only 3 points the math will not take long we did not calculate math long time if we have million of data points we cannot calculate



That is why there is a thing called stochastic gradient descend



The stochastic gradient descend is faster than gradient descend



Here we have 12 points are there if we want to calculate gradient descend then we need to calculate 12 times if we want to calculate stochastic gradient descend

How stochastic gradient descent works.
A) stochastic gradient descent out of 12 points it will create a cluster of random points example below is the



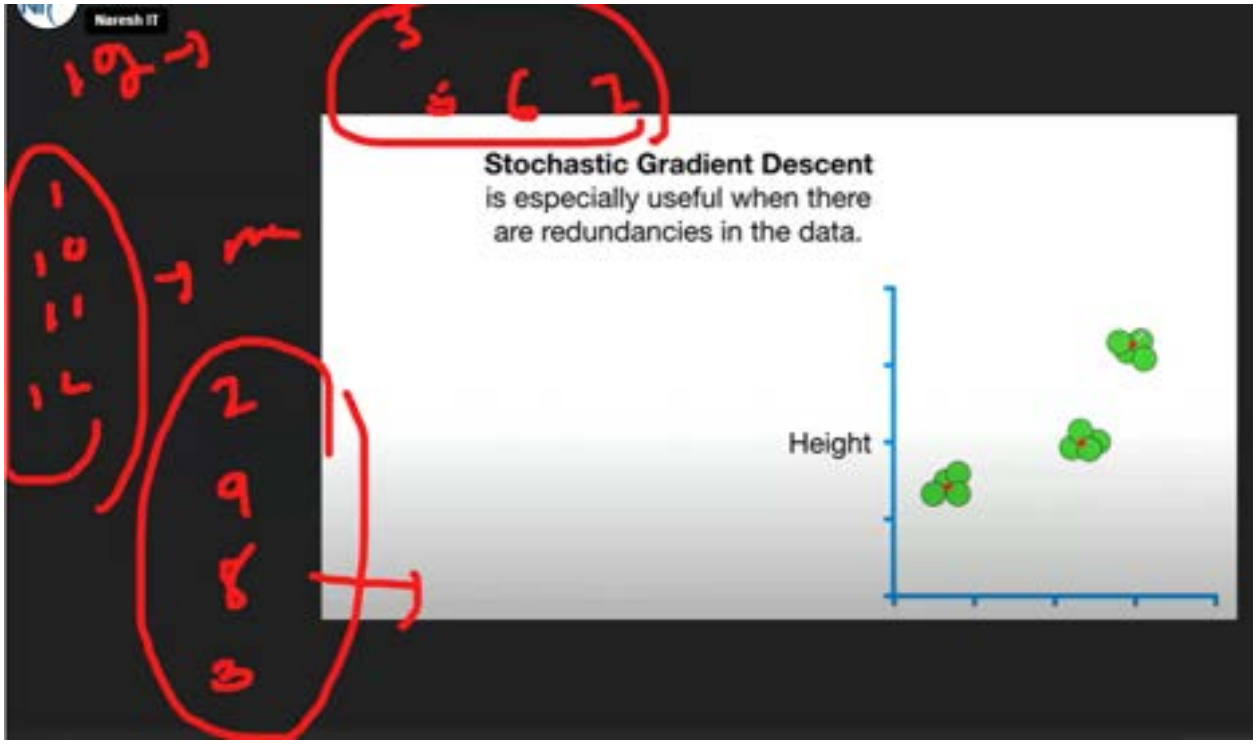
one of the group amd calculate the mean



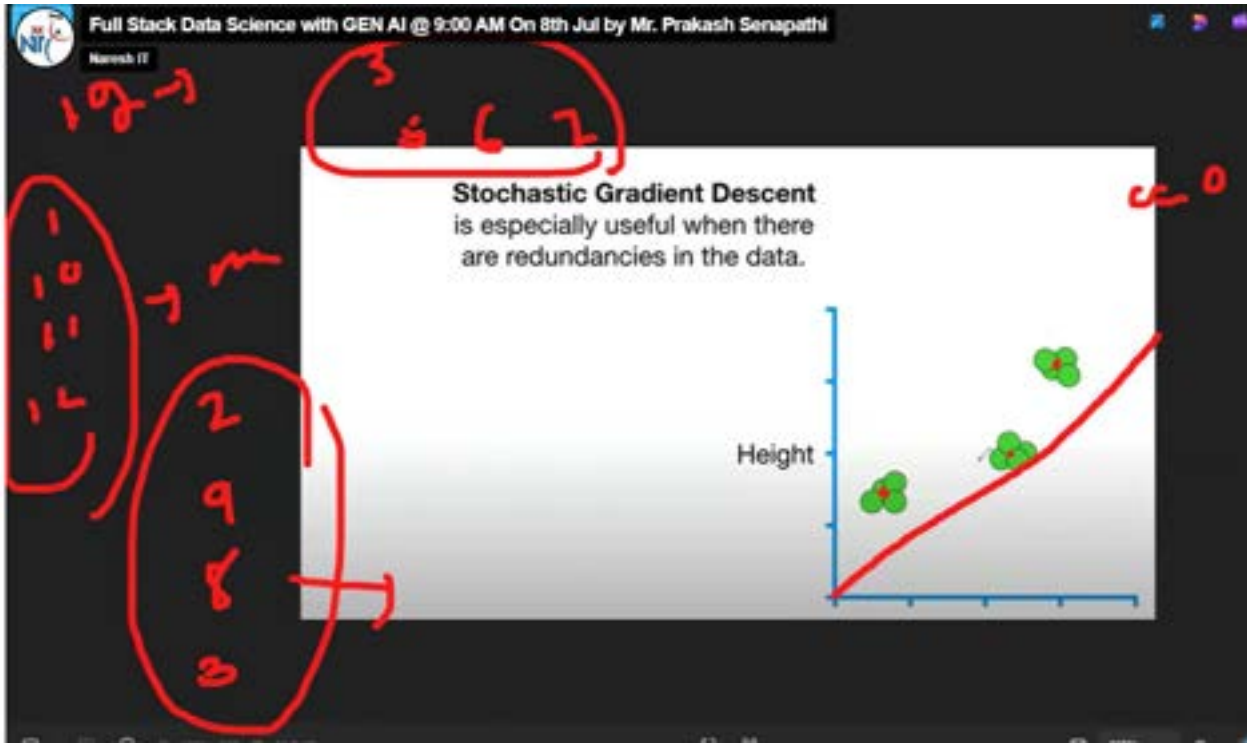
This is another group then calculate the mean rest of points



calculate the mean

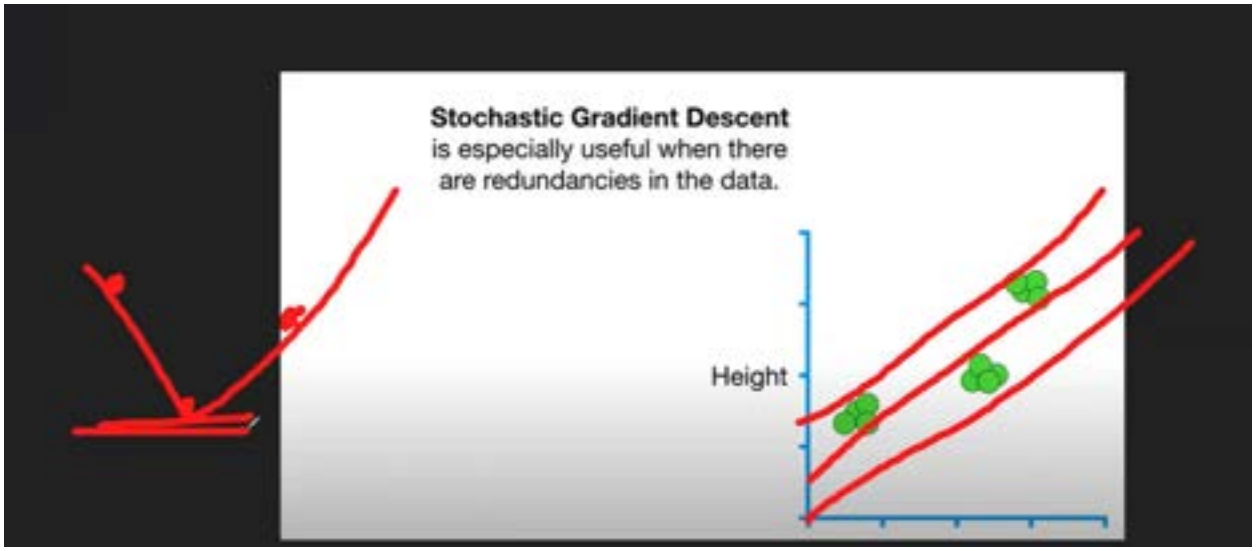


After calculating mean we got the center point after getting the center point now we try to calculate with these intercept red line c = 0 now the math gets calculate 3 times

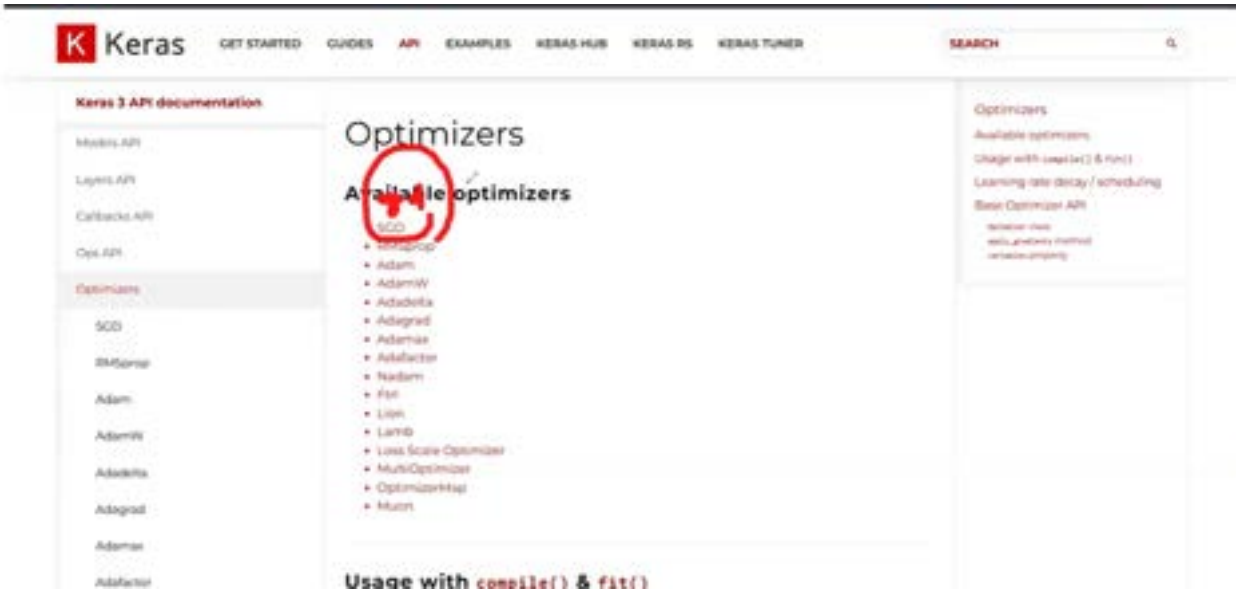


In the case of gradient descend we need t calculate 12 times for 12 points now for this stochastic gradient descend we need calculate 3 times so the complexity is reduced

The stochastic gradient descend is another algorithm calculate each and every data point it takes cluster of the random points take mean then it calculates the cost function



Here we get the global minima for this reason gradient descend eliminated we cannot see gradient descend in the below SGD means stochastic gradient descend we cannot see the gradient descend



Batch gradient descend :

A) SGD and batch both work same but only small difference that is

In SGD—--> Group a random points and take mean of it

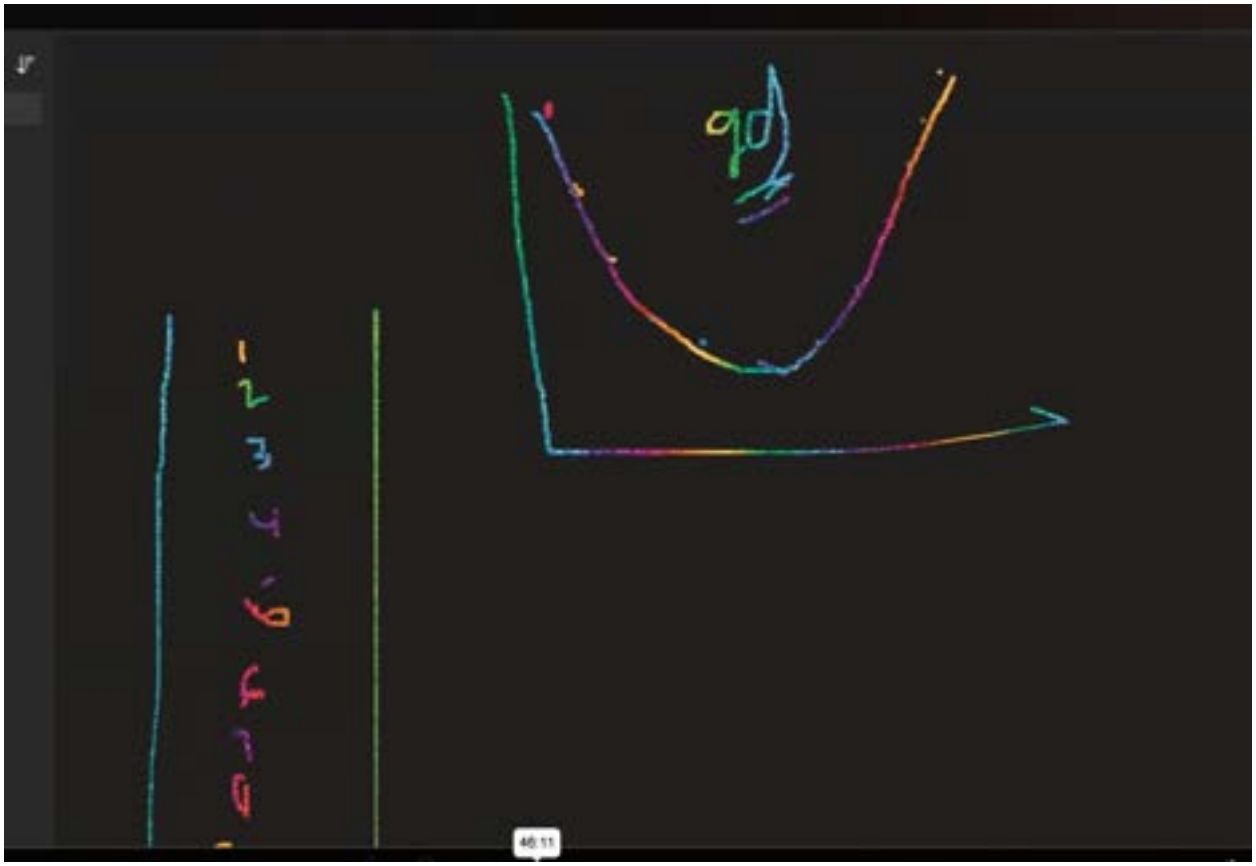
In batch —->It take a group of continue points

Example below

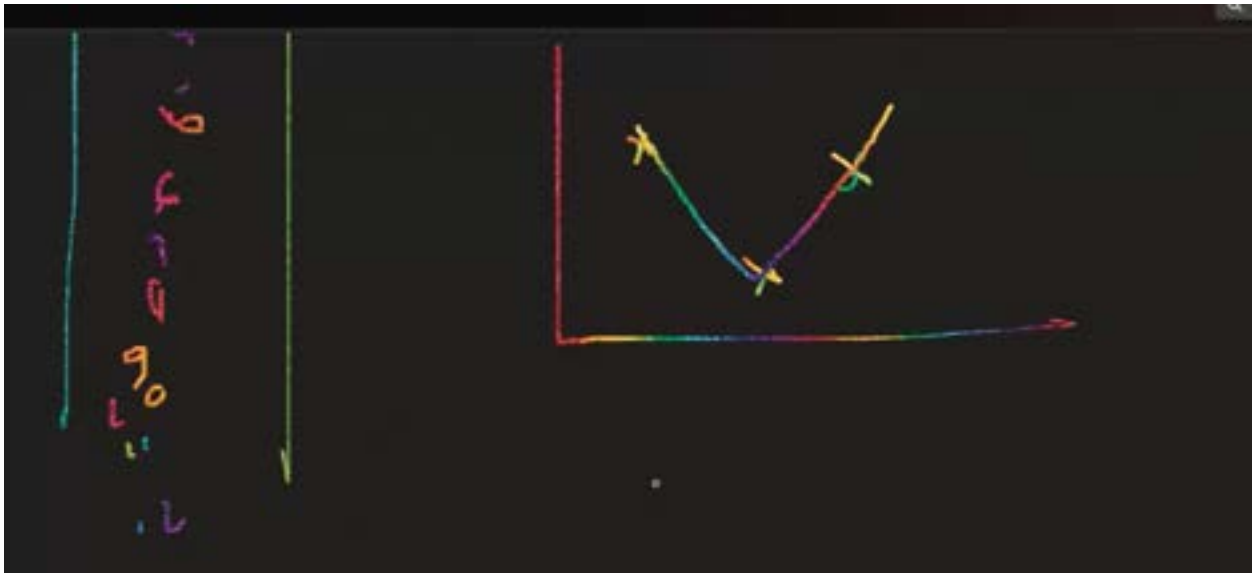
Here 12 data points for this 12 points if we want to calculate the math we need to calculate 12 times



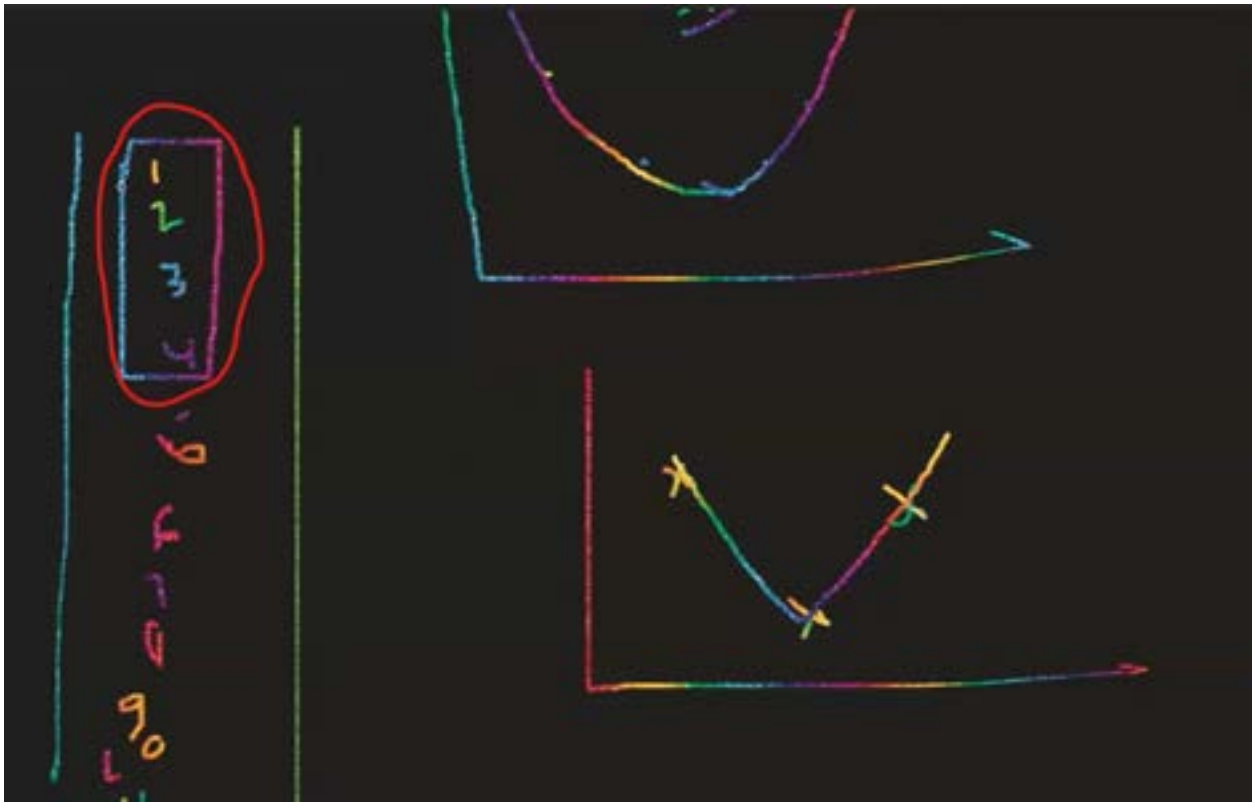
Here we can see 12 points



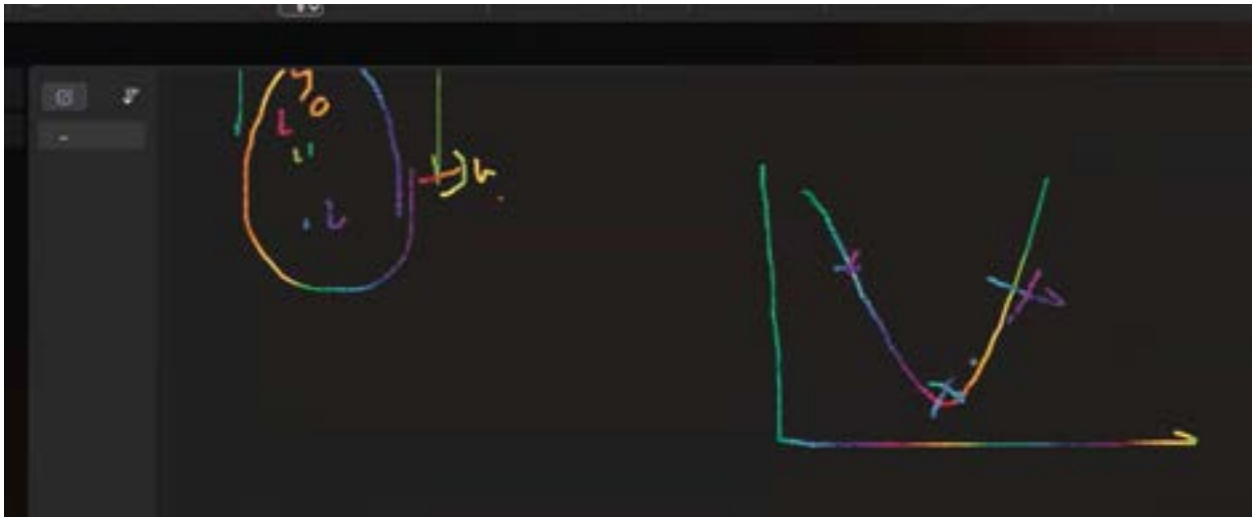
This is gradient descend in below cluster we can calculate 3 times it is a cluster of random points



Here we calculate 3 times
Batch when we do batch here 1234 is one batch



Here 1,2,3,4 is a batch and it is like a mean and the next batch is 5,6,7,8 is another batch here the data points are sequential it is not random we can take a mean another batch 9,10,11,12



we can found these optimize using keras library which deep learning framework

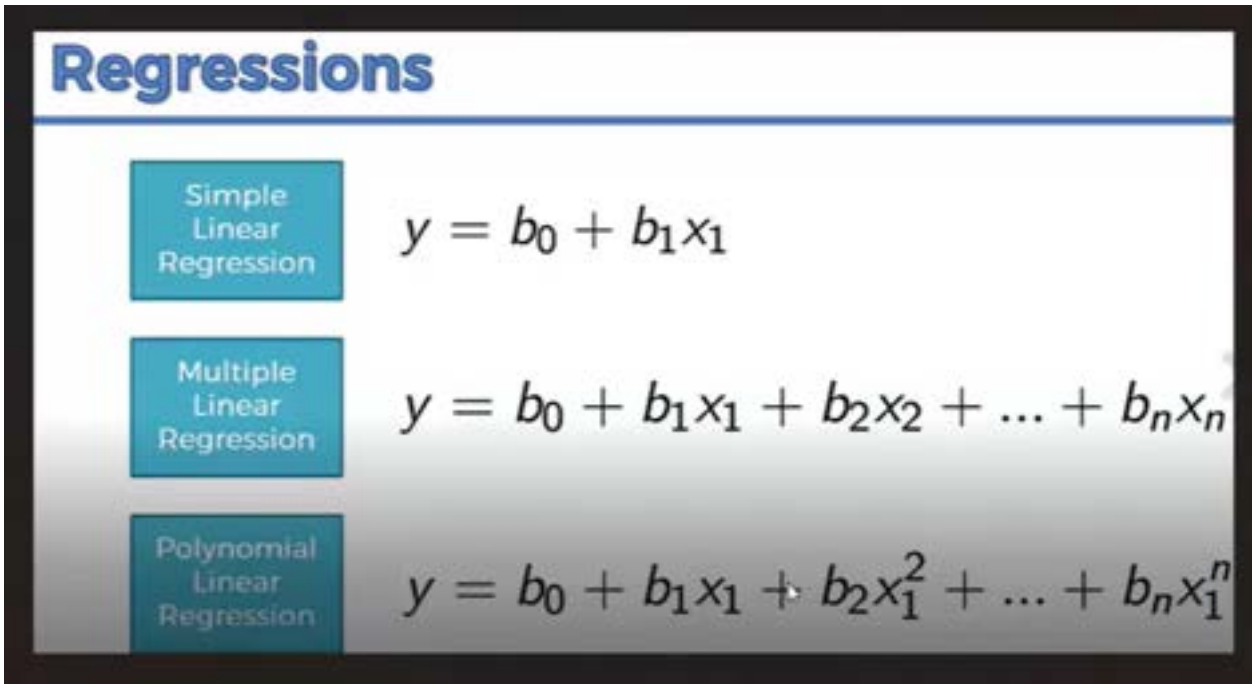
===
gradiend descent is an optimized algorithm to reach global minima by less learning rate and it will calculation each and every datapoint while traing the data
===
stocastic gradiend descent is an optimized algorithm to reach global minima by less learning rate and it will calculation random of datapoint, create cluster with mean while traing the data
===
batch gradiend descent is an optimized algorithm to reach global minima by less learning rate and it will calculation batchd sequential of datapoint, create cluster with mean while traing the da

stocastic gradiend descent is an optimized algorithm to reach global minima by less learning rate and it will calculation random of datapoint, create cluster with mean while traing the data
===
batch gradiend descent is an optimized algorithm to reach global minima by less learning rate and it will calculation batchd sequential of datapoint, create cluster with mean while traing the da
== gd vs sgd vs bgd

batch gradiend descent is an optimized algorithm to reach global minima by less learning rate and it will calculation batchd sequential of datapoint, create cluster with mean while traing the da
== gd vs sgd vs bgd

slr, mlr, gd, sgd, bgd (practicle it will com ann, cnn, rnn)
l1, l2

Polynomial Linear regression:

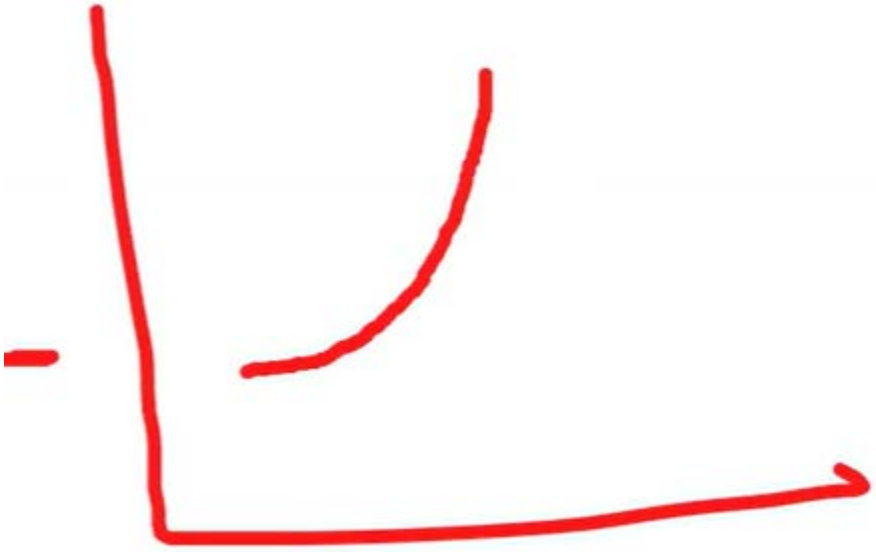


In simple linear regression we have 2 variables 1.Independent 2. Dependent

In Multilinear regression We have one dependent and many independent
In Polynomial we have decree1,decree2,decree3,decree4,decree5

Q)What is linear and what is non linear

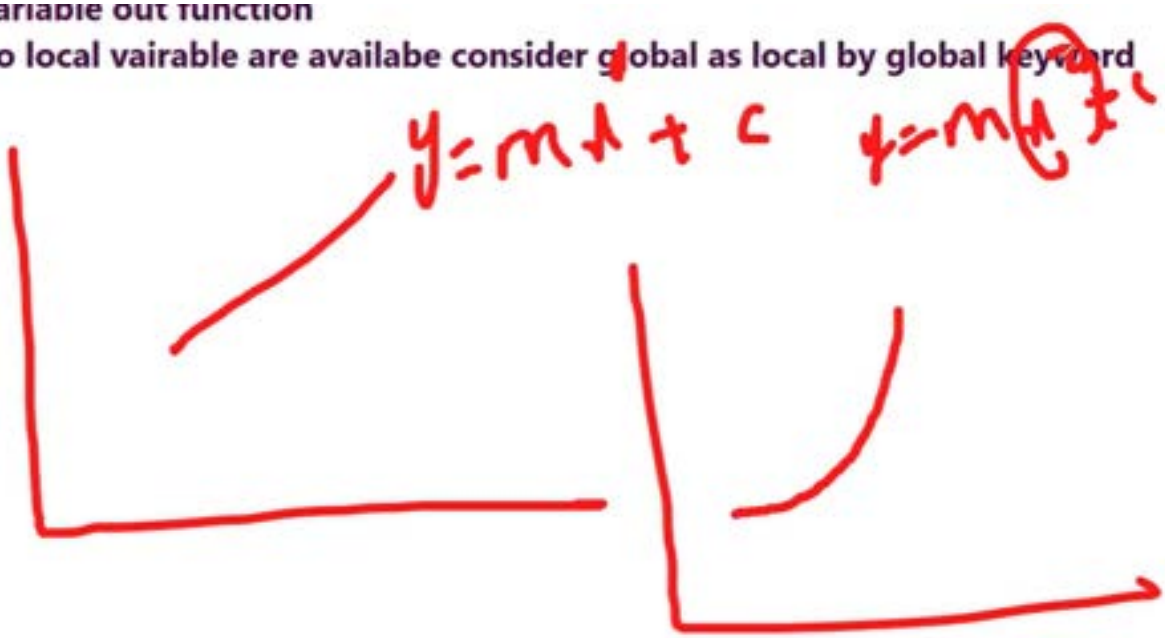
- **A) For linear regression Y = mx+c**
- **B) For Polynomial Linear regression Y = mx^2+c and it is non linear**



C) Polynomial is not linear

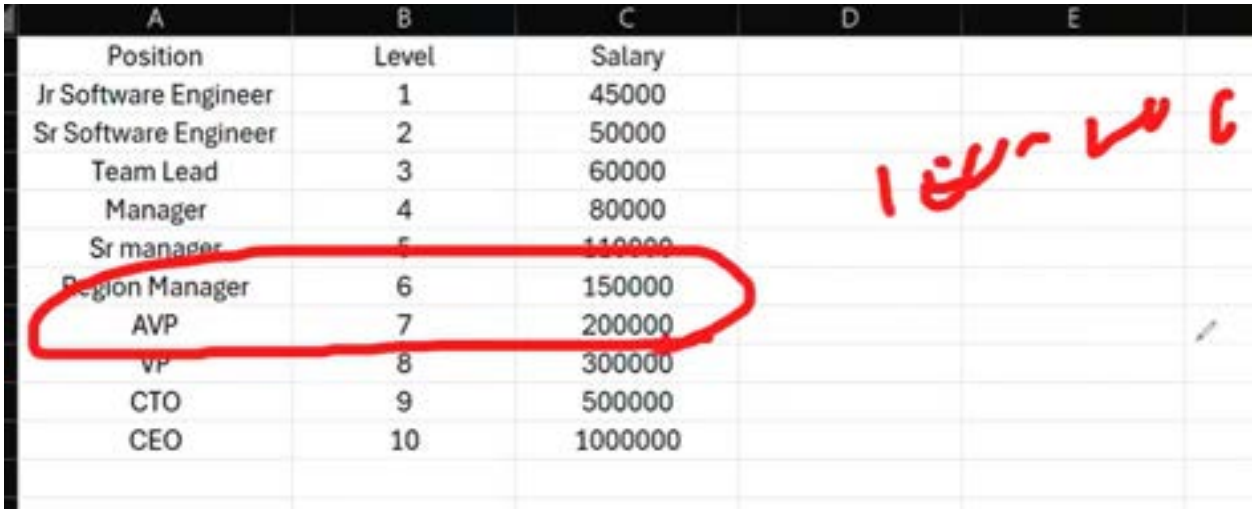
if variable out function

if no local vairable are availabe consider global as local by global keyword



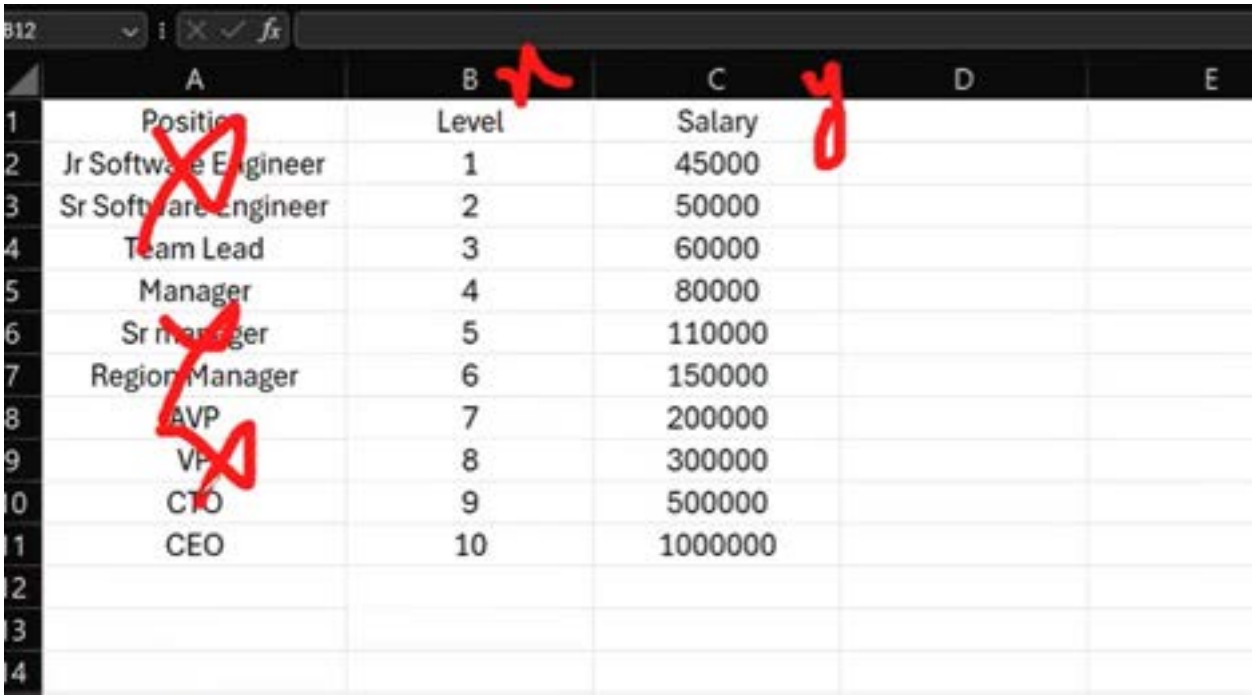
	A	B	C
1	Position	Level	Salary
2	Jr Software Engineer	1	45000
3	Sr Software Engineer	2	50000
4	Team Lead	3	60000
5	Manager	4	80000
6	Sr manager	5	110000
7	Region Manager	6	150000
8	AVP	7	200000
9	VP	8	300000
10	CTO	9	500000
11	CEO	10	1000000
12			
13			
14			

If regional manager put down the paper company needs to hire to another person his experience is 6.5years he is more experience to sr manager and less experience to AVP what should be package the company should give that is between 150 to 200



We need to build a model how much we need to give if tomorrow if CEOresigns we need to predict how much we need to give package here we are ignoring positions and taking

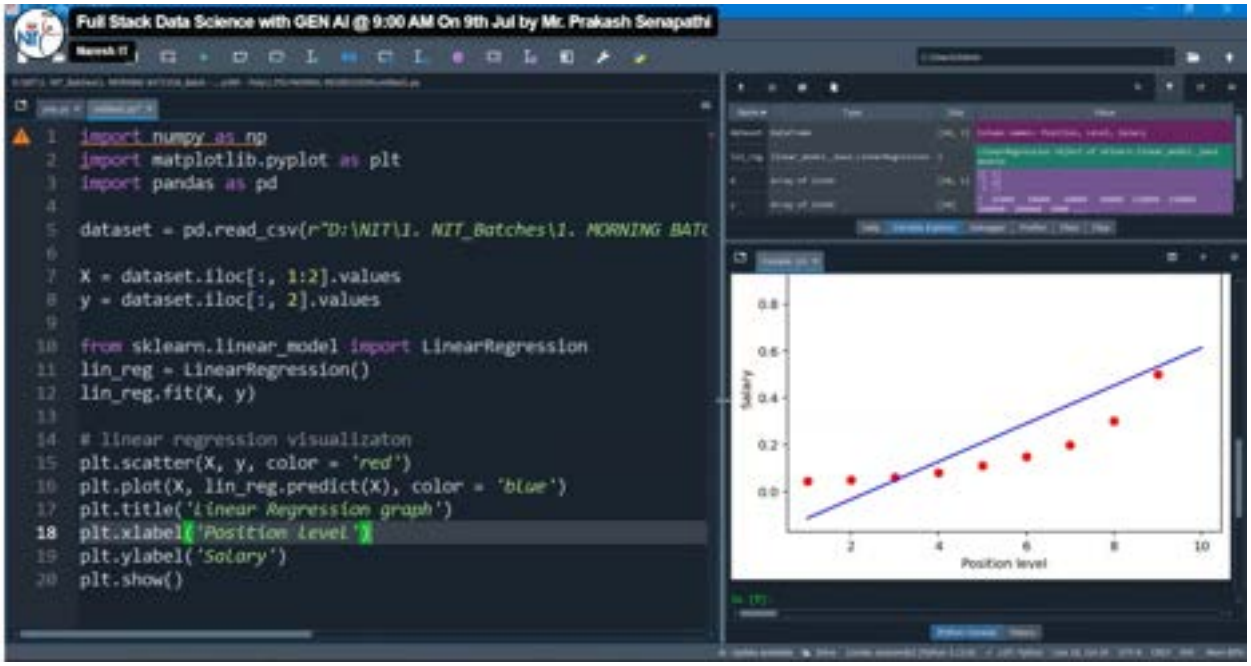
x and y considerations



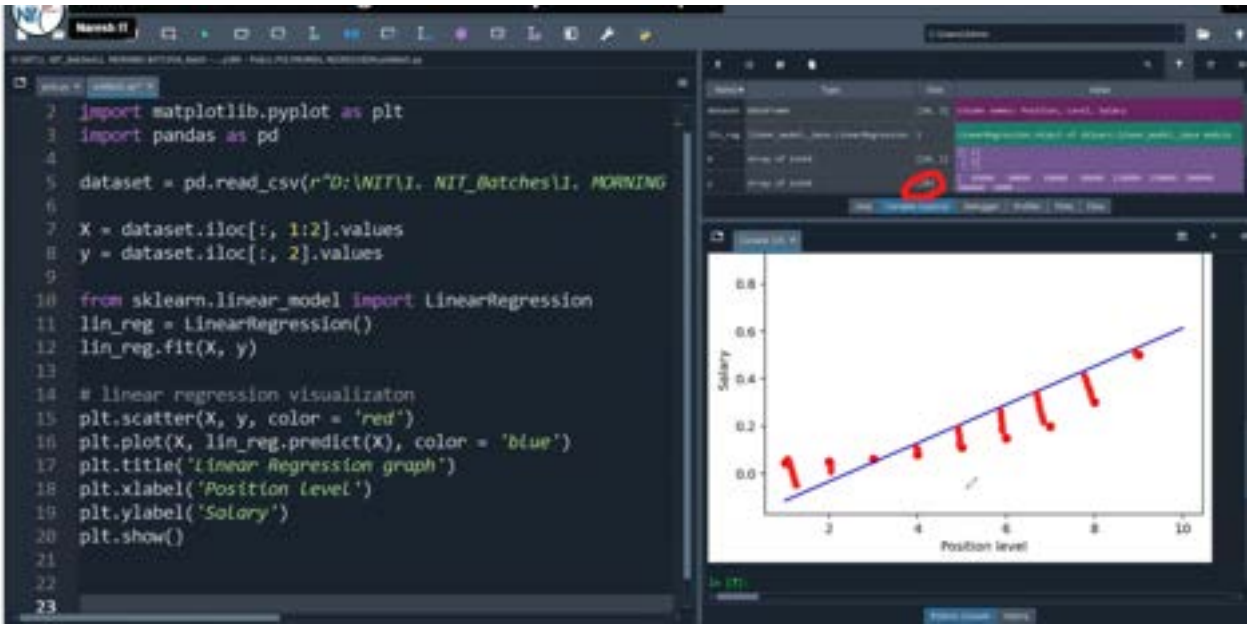
	A	B	C	D	E
1	Positio	Level	Salary		
2	Jr Softwa e E gineer	1	45000		
3	Sr Soft are Engineer	2	50000		
4	Team Lead	3	60000		
5	Manager	4	80000		
6	Sr m ger	5	110000		
7	Regio Manager	6	150000		
8	AVP	7	200000		
9	VP	8	300000		
10	CTO	9	500000		
11	CEO	10	1000000		
12					
13					
14					

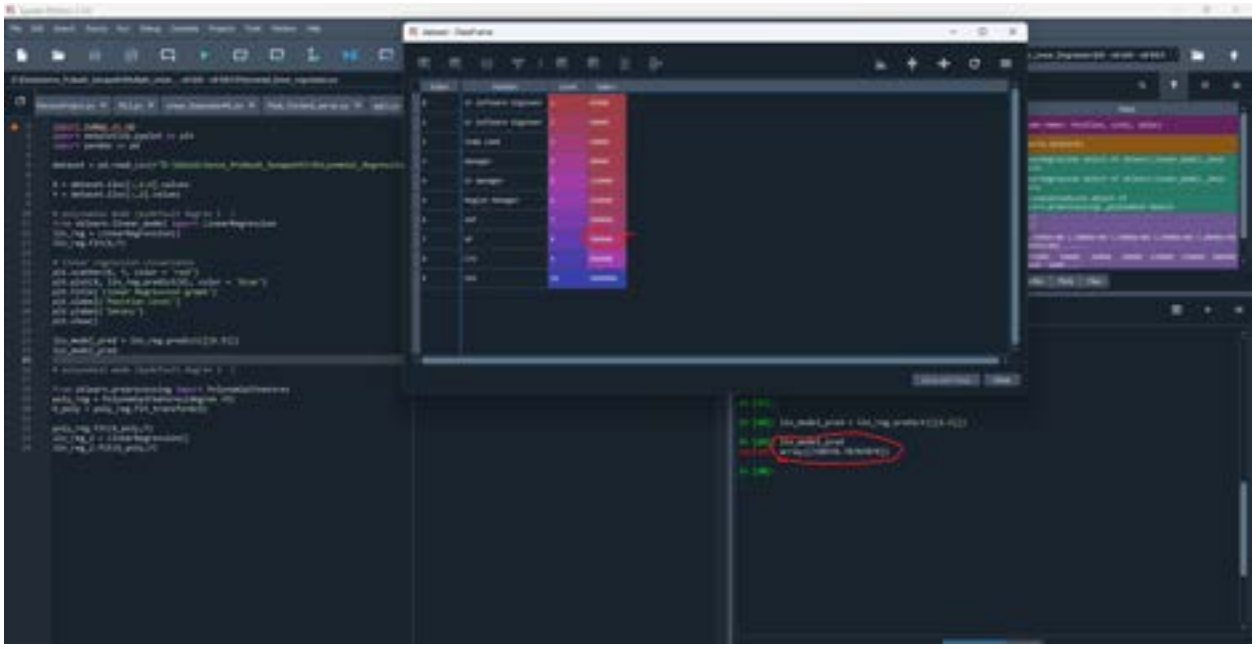
	A	B	C
	Position	Level	Salary
2	Jr Software Engineer	1	45000
3	Sr Software Engineer	2	50000
4	Team Lead	3	60000
5	Manager	4	80000
6	Sr manager	5	110000
7	Region Manager	6	150000
8	AVP	7	200000
9	VP	8	300000
10	CTO	9	500000
11	CEO	10	1000000
12			

In session 59 cracked 11IPA to 55IPA please watch video for resume purpose

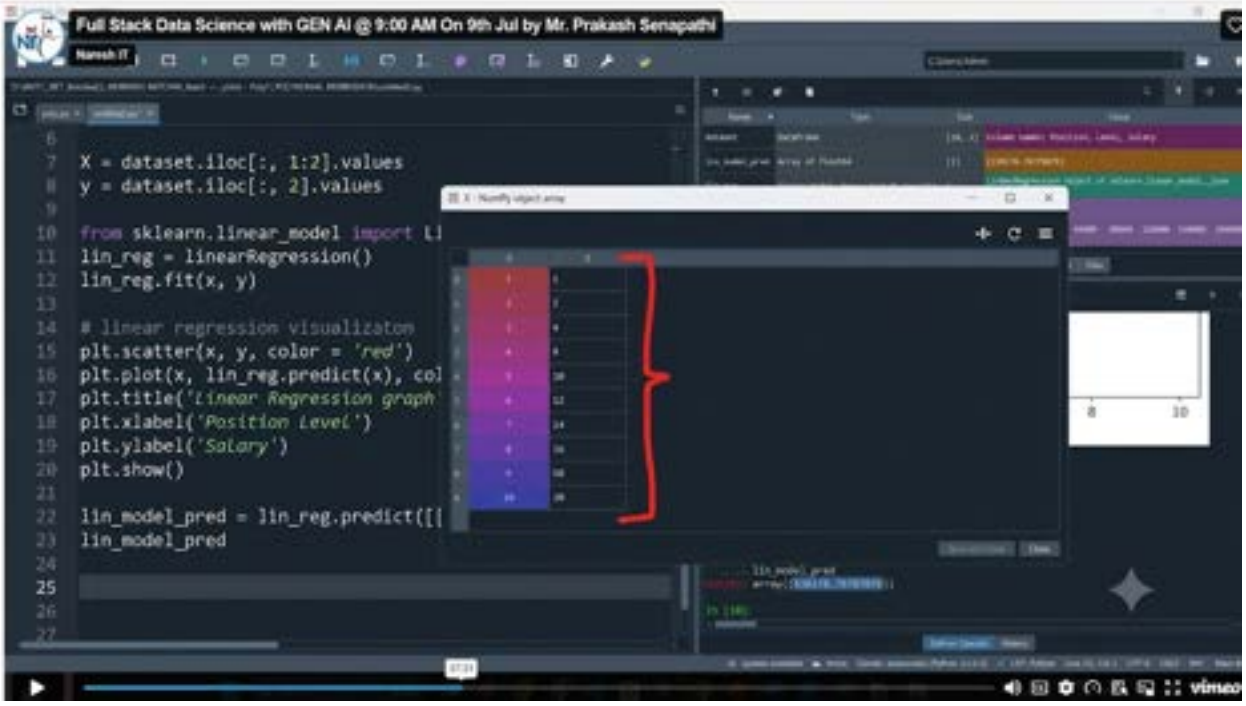


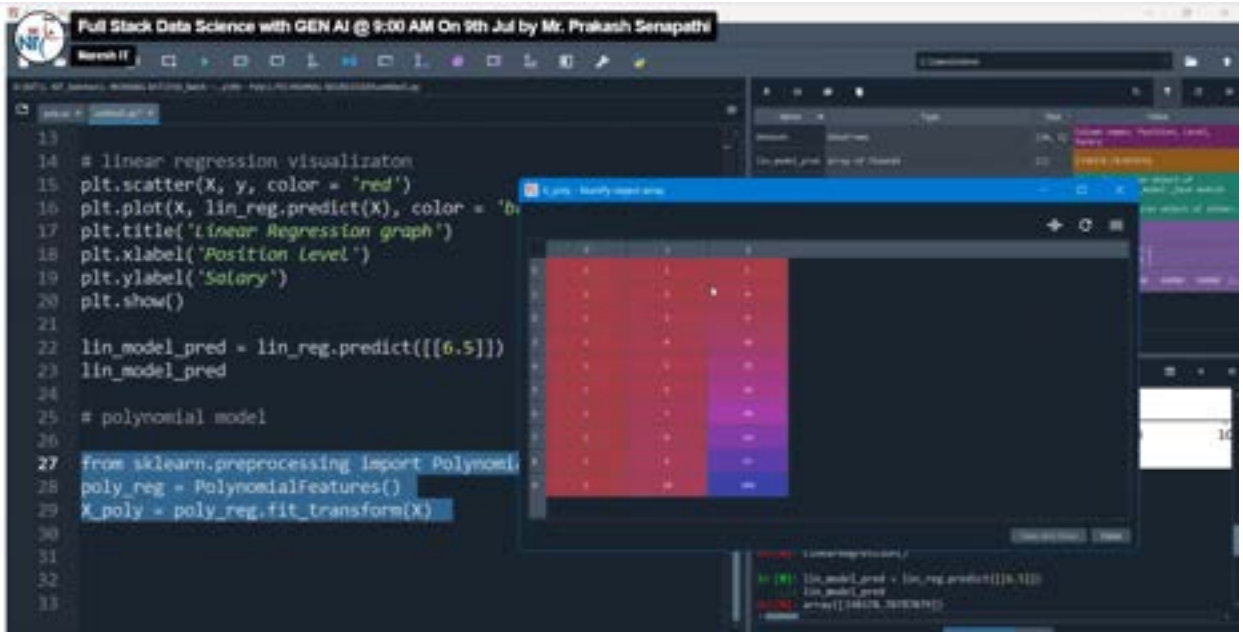
This graph is not best fit because out of 10 points only 2 points touching linear there is a high error

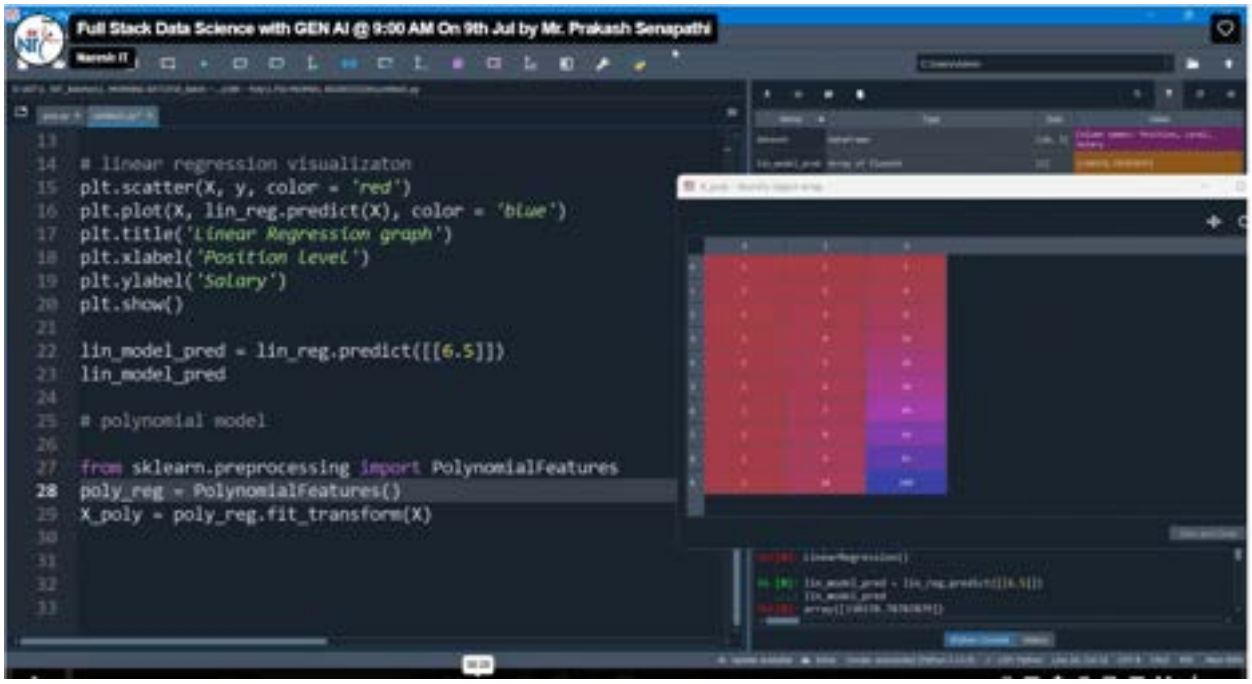


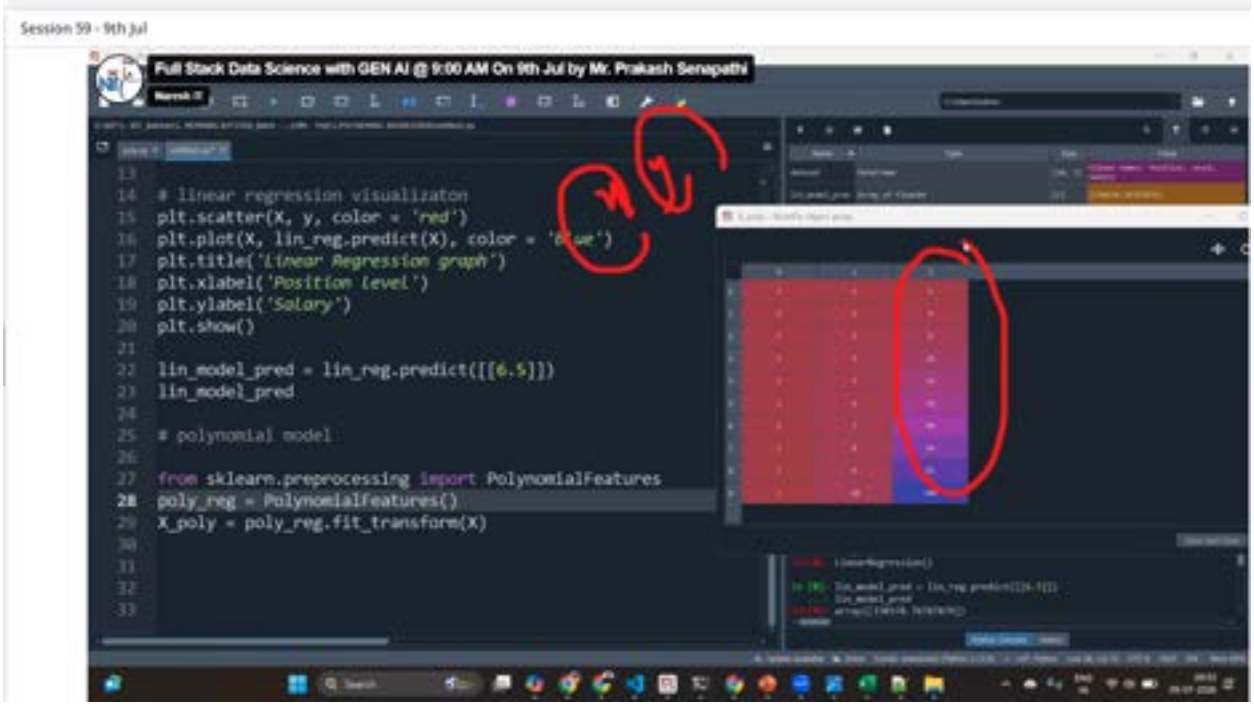


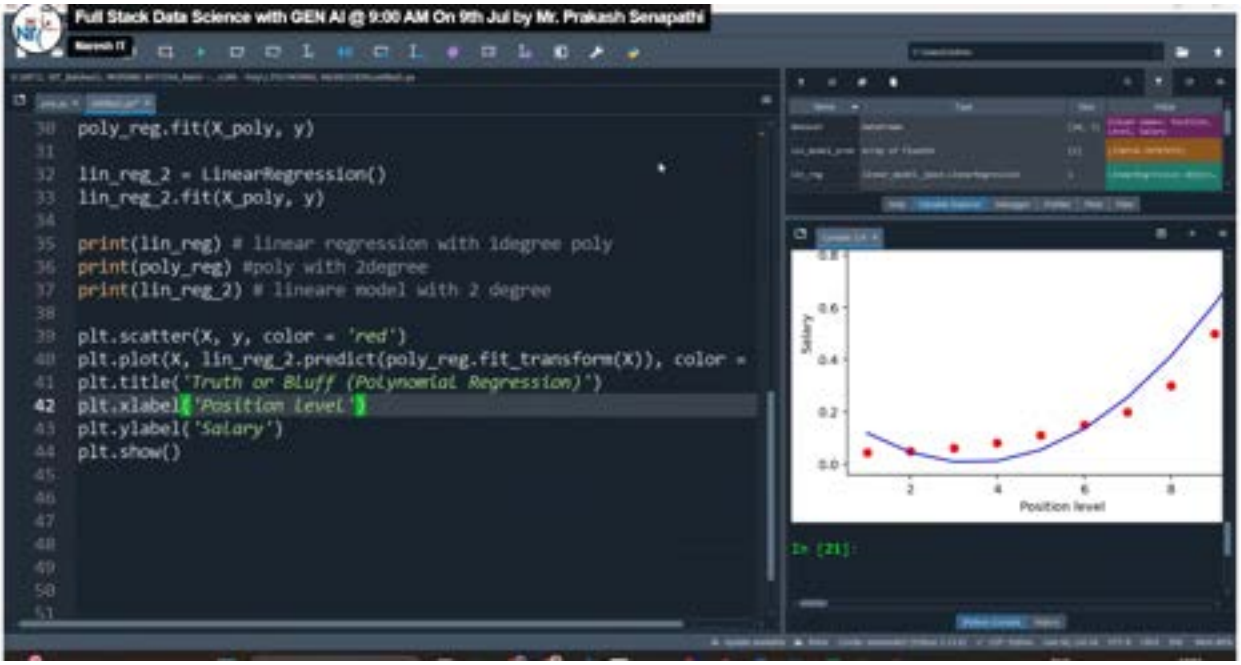
It predicted the VP salary for 6.5 years but it is the VP salary So it is not a good model the model should predict salary between 6 and 7



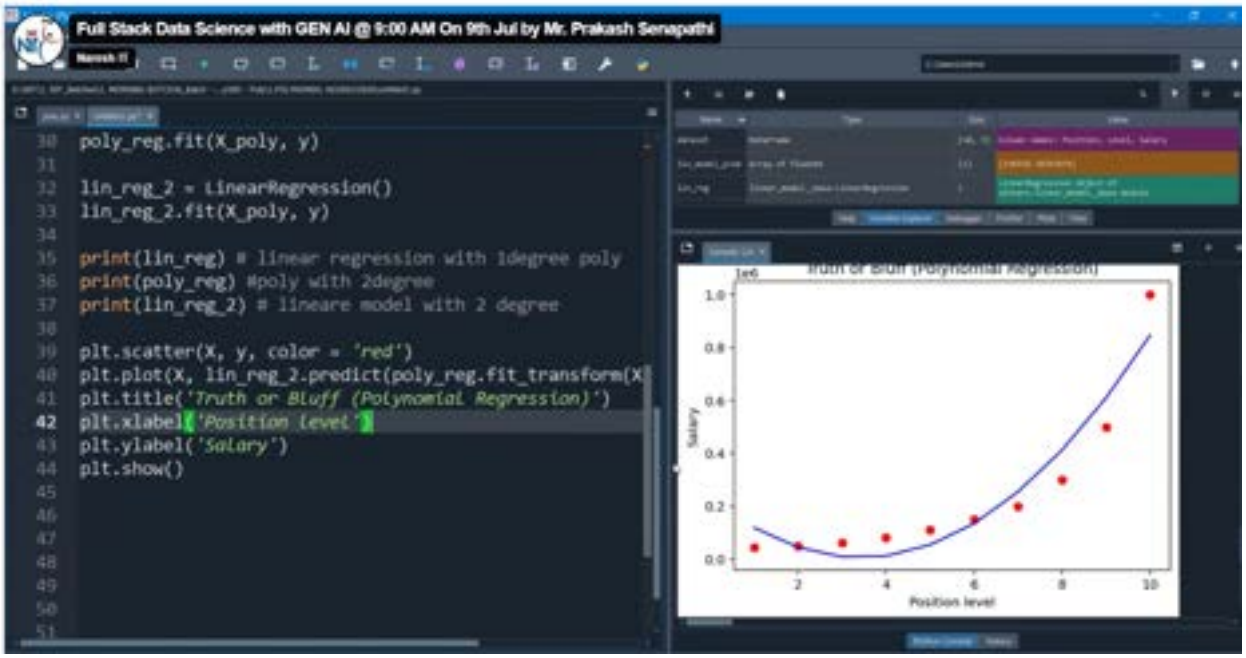

Full Stack Data Science with GEN AI @ 9:00 AM On 9th Jul by Mr. Prakash Senapathi
linear regression visualizaton
plt.scatter(X, y, color = 'red')
plt.plot(X, lin_reg.predict(X), color = 'b
plt.title('Linear Regression graph')
plt.xlabel('Position Level')
plt.ylabel('Salary')
plt.show()
lin_model_pred = lin_reg.predict([[6.5]])
lin_model_pred
polynomial model
from sklearn.preprocessing import Polynomi
poly_reg = PolynomialFeatures()
X_poly = poly_reg.fit_transform(X)


Full Stack Data Science with GEN AI @ 9:00 AM On 9th Jul by Mr. Prakash Senapathi
linear regression visualizaton
plt.scatter(X, y, color = 'red')
plt.plot(X, lin_reg.predict(X), color = 'blue')
plt.title('Linear Regression graph')
plt.xlabel('Position Level')
plt.ylabel('Salary')
plt.show()
lin_model_pred = lin_reg.predict([[6.5]])
lin_model_pred
polynomial model
from sklearn.preprocessing import PolynomialFeatures
poly_reg = PolynomialFeatures()
X_poly = poly_reg.fit_transform(X)

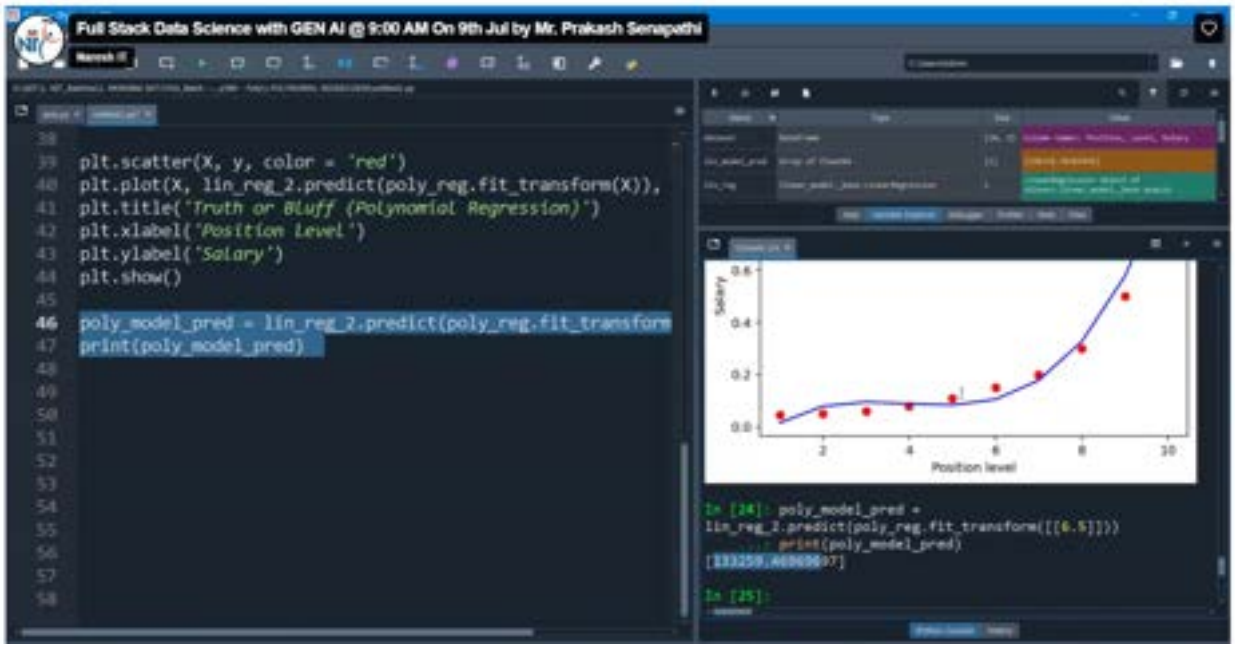




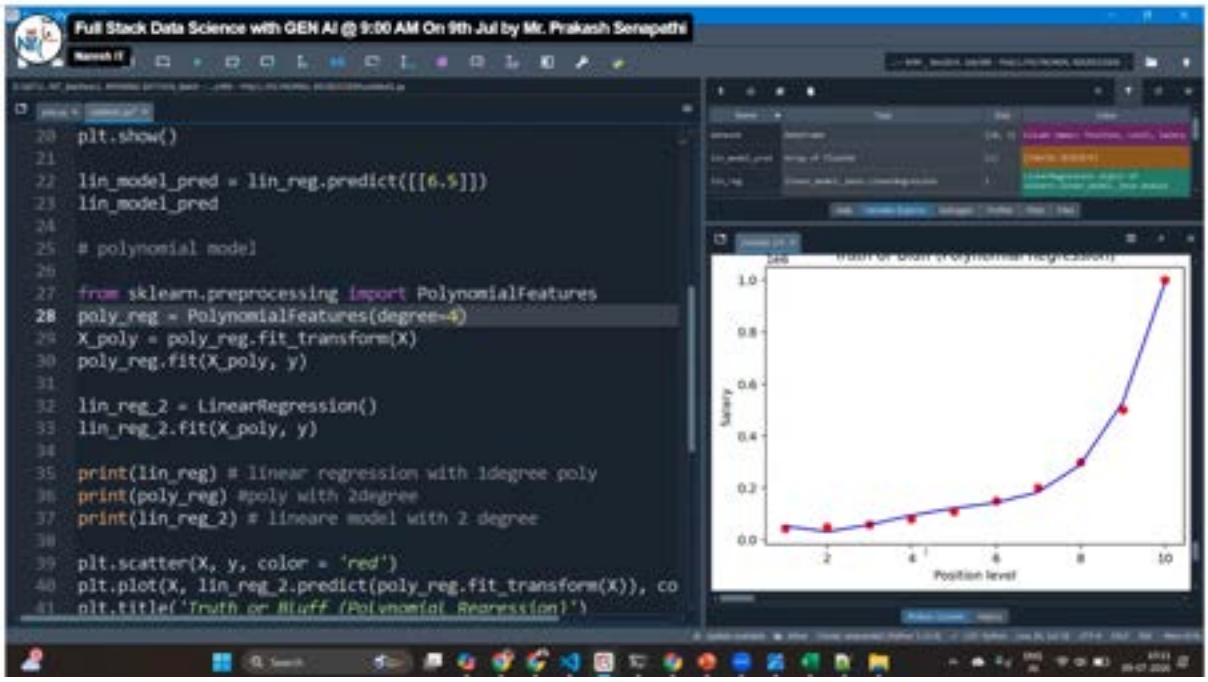
This is non linear graph

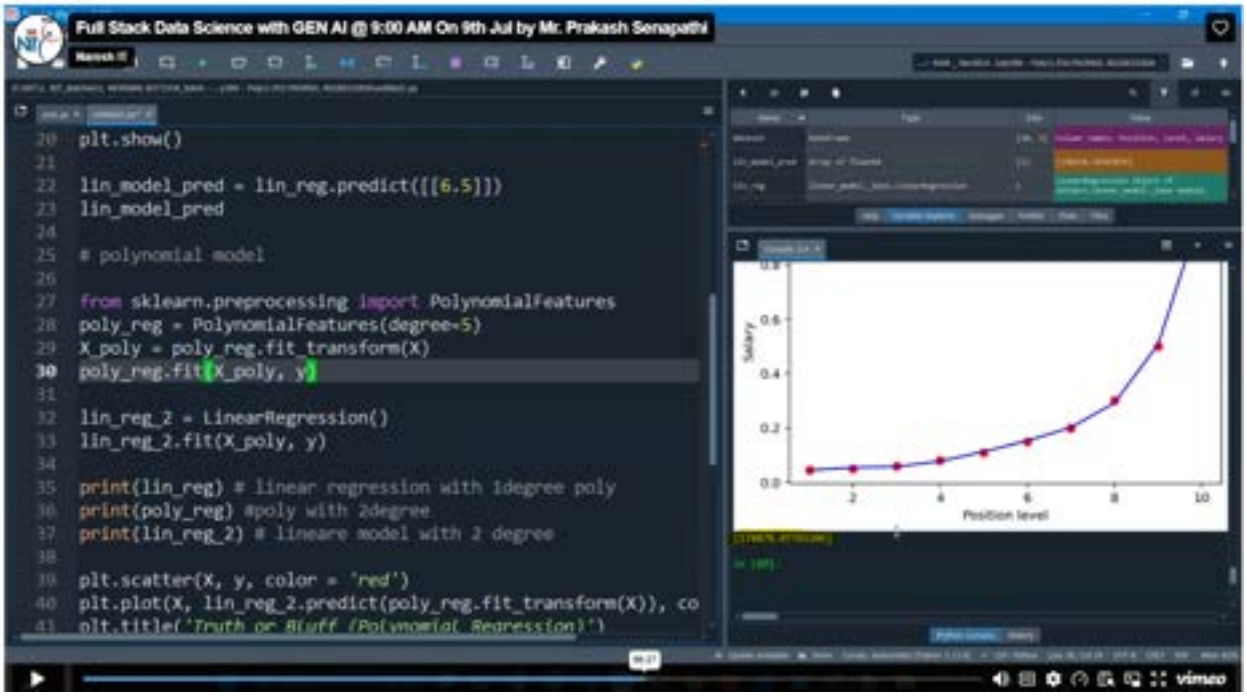

Full Stack Data Science with GEN AI @ 9:00 AM On 9th Jul by Mr. Prakash Senapathi
poly_reg.fit(X_poly, y)
lin_reg_2 = LinearRegression()
lin_reg_2.fit(X_poly, y)
print(lin_reg) # linear regression with 1degree poly
print(poly_reg) #poly with 2degree
print(lin_reg_2) # lineare model with 2 degree
plt.scatter(X, y, color = 'red')
plt.plot(X, lin_reg_2.predict(poly_reg.fit_transform(X
plt.title('Truth or Bluff (Polynomial Regression)')
plt.xlabel('Position Level')
plt.ylabel('Salary')
plt.show()
Salary
Position level

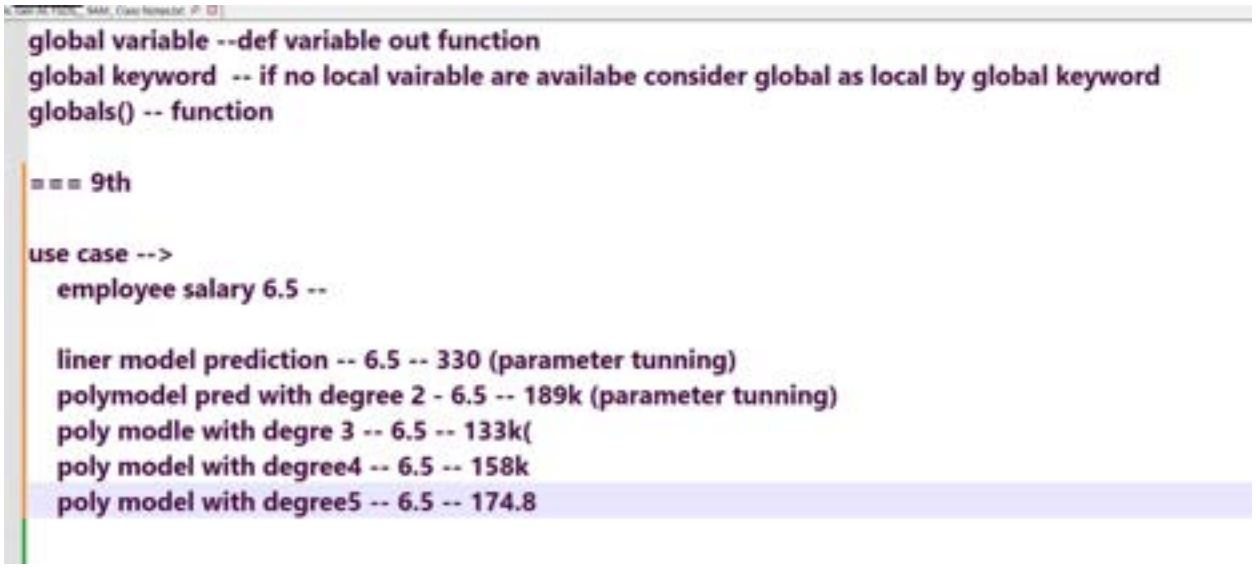

Full Stack Data Science with GEN AI @ 9:00 AM On 9th Jul by Mr. Prakash Senapathi
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
dataset = pd.read_csv(r"D:\NIT\1. NIT_Batches\1. MORNING BATCH\N
X = dataset.iloc[:, 1:2].values
y = dataset.iloc[:, 2].values
from sklearn.linear_model import LinearRegression
lin_reg = LinearRegression()
lin_reg.fit(X, y)
linear regression visualization
plt.scatter(X, y, color = 'red')
plt.plot(X, lin_reg.predict(X), color = 'blue')
plt.title('Linear Regression graph')
plt.xlabel('Position Level')
plt.ylabel('Salary')
plt.show()
Salary
Position level


Full Stack Data Science with GEN AI @ 9:00 AM On 9th Jul by Mr. Prakash Senapathi
plt.scatter(X, y, color = 'red')
plt.plot(X, lin_reg_2.predict(poly_reg.fit_transform(X)),
plt.title('Truth or Bluff (Polynomial Regression)')
plt.xlabel('Position Level')
plt.ylabel('Salary')
plt.show()
poly_model_pred = lin_reg_2.predict(poly_reg.fit_transform
print(poly_model_pred)
Salary
Position level
In [24]: poly_model_pred =
lin_reg_2.predict(poly_reg.fit_transform([[6.5]]))
...: print(poly_model_pred)
In [25]:

n 59 - 9th Jul


Full Stack Data Science with GEN AI @ 9:00 AM On 9th Jul by Mr. Prakash Senapathi
plt.show()
lin_model_pred = lin_reg.predict([[6.5]])
lin_model_pred
polynomial model
from sklearn.preprocessing import PolynomialFeatures
poly_reg = PolynomialFeatures(degree=4)
X_poly = poly_reg.fit_transform(X)
poly_reg.fit(X_poly, y)
lin_reg_2 = LinearRegression()
lin_reg_2.fit(X_poly, y)
print(lin_reg) # linear regression with 1degree poly
print(poly_reg) #poly with 2degree
print(lin_reg_2) # lineare model with 2 degree
plt.scatter(X, y, color = 'red')
plt.plot(X, lin_reg_2.predict(poly_reg.fit_transform(X)), co
plt.title('Truth or Bluff (Polynomial Regression)')
Salary
Position level


Full Stack Data Science with GEN AI @ 9:00 AM On 9th Jul by Mr. Prakash Senapathi
plt.show()
lin_model_pred = lin_reg.predict([[6.5]])
lin_model_pred
polynomial model
from sklearn.preprocessing import PolynomialFeatures
poly_reg = PolynomialFeatures(degree=5)
X_poly = poly_reg.fit_transform(X)
poly_reg.fit(X_poly, y)
lin_reg_2 = LinearRegression()
lin_reg_2.fit(X_poly, y)
print(lin_reg) # linear regression with 1degree poly
print(poly_reg) #poly with 2degree
print(lin_reg_2) # lineare model with 2 degree
plt.scatter(X, y, color = 'red')
plt.plot(X, lin_reg_2.predict(poly_reg.fit_transform(X)), co
Salary
Position level
vimeo



global variable --def variable out function
global keyword -- if no local vairable are availabe consider global as local by global keyword
globals() -- function
=== 9th
use case -->
employee salary 6.5 --
liner model prediction -- 6.5 -- 330 (parameter tunning)
polymodel pred with degree 2 - 6.5 -- 189k (parameter tunning)
poly modle with degre 3 -- 6.5 -- 133k(
poly model with degree4 -- 6.5 -- 158k
poly model with degree5 -- 6.5 -- 174.8
poly model with degree6 - 6.5 -- 174.1

Here we are divided data in to x tain and y train overfitting not comes in to picture here

Example : inorder to food tate rating in hyderabad we will rate in below way

```
Hyd -- 10 resturent
   1- 85% -- name
   2- 90%
   3- 95%
```

Exactly in ML one dataset we will build several models and predict score like below

```
exactly in machine learning model
   one dataset build several model

1- poly model -- 174.8
2- svr model -- ?
3- knnr model --?
4- dtr model --?
5- rf model -->
   who ever high accuracy i pickle
   frontend
   depl\oy
   mlops
   cicid
```

Machine Learning Model Evaluation & Deployment Pipeline

In machine learning, a single dataset is often used to build and train several different models so you can compare their performance.

1. Model Comparison & Training

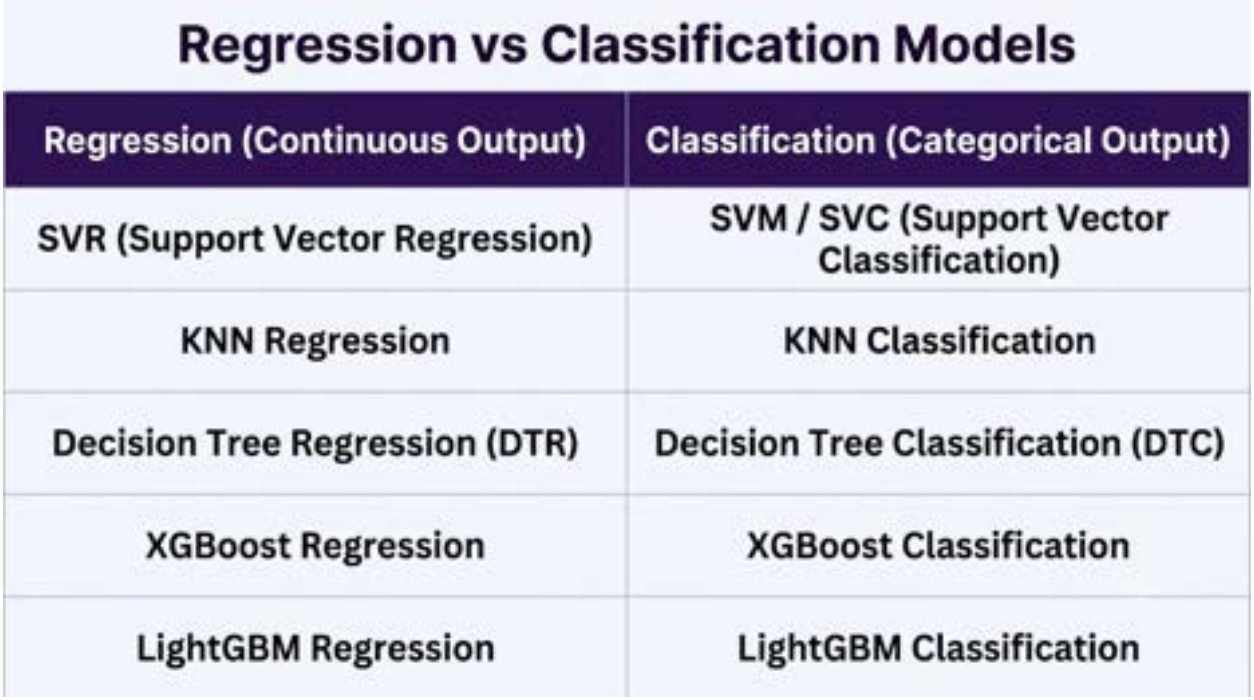
- **Polynomial Regression Model (Poly Model): Evaluated with a score/metric of 174.8.**
- **Support Vector Regression (SVR Model): Pending evaluation.**

- **K-Nearest Neighbors Regression (KNNR Model): Pending evaluation.**
- **Decision Tree Regression (DTR Model): Pending evaluation.**
- **Random Forest Regression (RF Model): Pending evaluation.**

2. Deployment Workflow

Once all models are trained and tested:

- **Model Selection: Pick the model with the highest accuracy and serialize (save) it using Pickle.**
- **Frontend Integration: Build a user interface (frontend) to interact with the trained model.**
- **Deployment: Deploy the web application/model to a live environment.**
- **MLOps & CI/CD: Implement MLOps and automated CI/CD (Continuous Integration / Continuous Deployment) pipelines for monitoring, updates, and maintenance**



Machine Learning: Regression vs Classification Comparison

Algorithm Family	Regression Model (Continuous Output)	Classification Model (Categorical Output)
Support Vector Machine	SVR (Support Vector Regression)	SVM / SVC (Support Vector Classification)

K-Nearest Neighbors	**KNN Regression**	**KNN Classification**
Decision Trees	**DTR (Decision Tree Regression)**	**DTC (Decision Tree Classification)**
XGBoost	**XGBoost Regression**	**XGBoost Classification**
LightGBM	**LightGBM Regression**	**LightGBM Classification**

quantum ml

ml model in quantum machine --v ery faster
same ml model will implet - databricks, aws, axur, gcp everywhere

ml model score will increase using apply HT technique , parameter technique best score
===
SVR model --> 30%
svm model --> 70% at classifcaiton